

▼ Distributed file system

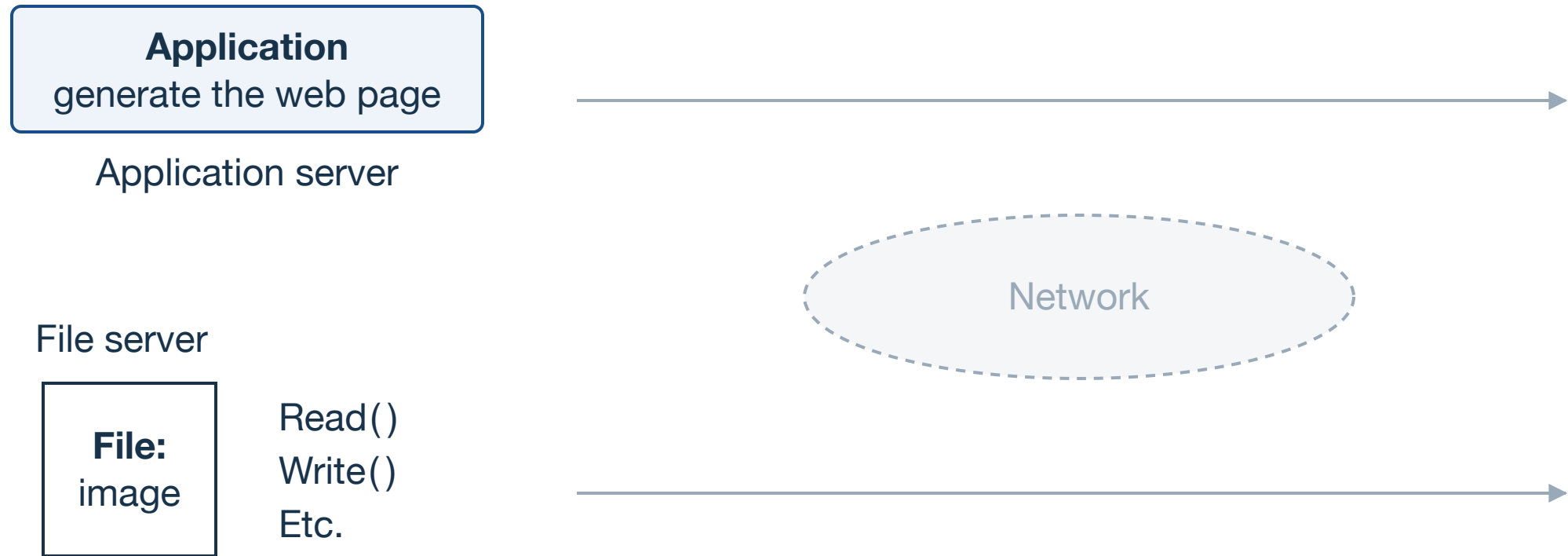
- 1 ♦ Distributed File System (DFS)
- 2 ♦ Case study: File system + RPC
- ▼ Accessing remote files
 - 3 ♦ Ways for accessing remote files
- ▼ NFS with RPC
 - 4 ♦ NFS: Network File System
 - 5 ♦ Recall: the file system API
 - 6 ♦ From the file system API to NFS RPCs
 - 7 ♦ NFS protocols step-by-step: `mount`
 - 8 ♦ NFS protocols: `read` a file
 - 9 ♦ Design choice: no `open` RPC in NFS
 - 10 ♦ Review: `open`
 - 11 ♦ What if we faithfully implement `open` with RPC?
 - 12 ♦ Stateless on the NFS server
 - 13 ♦ Stateless is a common design: e.g., LLM APIs
 - 14 ♦ NFS protocols: `read` a file
 - 15 ♦ What is a file handle?
 - 16 ♦ Case 1: rename after open
 - 17 ♦ What is a file handle?
 - 18 ♦ Case 2: delete after open
 - 19 ♦ Put it all together: file handle
 - 20 ♦ NFS protocols: `read` a file
 - 21 ♦ Problem: performance overheads
 - 22 ♦ Challenge of caching: cache coherence
 - 23 ♦ Handling inconsistency
 - 24 ♦ Improving read performance
 - 25 ♦ Not the end of the story: NFS is continuously improving
 - 26 ♦ Reference materials
- ▼ Case study: GFS
 - 27 ♦ Design assumptions: environments
 - 28 ♦ More design assumptions: file access
 - 29 ♦ GFS design goals
 - 30 ♦ GFS interface
 - 31 ♦ GFS architecture
 - 32 ♦ GFS files
 - 33 ♦ Why large chunks?
 - 34 ♦ Why large chunks? Reuse one TCP connection
 - 35 ♦ Chunks (blocks) and chunkservers
 - 36 ♦ Short summary: GFS vs. naïve file system?
 - 37 ♦ Why does GFS use one master? To keep the design simple
 - 38 ♦ Client-GFS interaction model
 - 39 ♦ Reading a file in GFS
 - 40 ♦ Challenge of writing: the order of writes
 - 41 ♦ Writing a file in GFS: two phases
 - 42 ♦ The primary tells every replica the write order
 - 43 ♦ Writing a file in GFS: two phases
 - 44 ♦ Summary: GFS in Google clusters
- ▼ HDFS
 - 45 ♦ HDFS: another popular (open-source) DFS
 - 46 ♦ Design goals & assumptions of HDFS
 - 47 ♦ Recall GFS files
 - 48 ♦ HDFS files
- ▼ Case study: 3FS
 - 49 ♦ Files behind AI workloads
 - 50 ♦ What has changed since GFS?
 - 51 ♦ 3FS keeps many designs of GFS and NFS
 - 52 ♦ Where does 3FS keep metadata?
 - 53 ♦ What is a transactional key-value store?
 - 54 ♦ Why not a single master?
 - 55 ♦ Which opened files does 3FS track?
 - 56 ♦ Why not track opened files?
 - 57 ♦ How does a 3FS client find a chunk?
 - 58 ♦ Why compute chunk locations?
 - 59 ♦ What does locality-oblivious mean?
 - 60 ♦ Why locality-oblivious?
 - 61 ♦ How do applications access 3FS?
 - 62 ♦ What is FUSE?
 - 63 ♦ Why is FUSE not enough?
 - 64 ♦ NFS vs. GFS vs. 3FS
- ▼ Summary
 - 65 ♦ Summary
- 66 ♦ References

Distributed File System (DFS)

Yubin Xia / Xingda Wei • IPADS, Shanghai Jiao Tong University

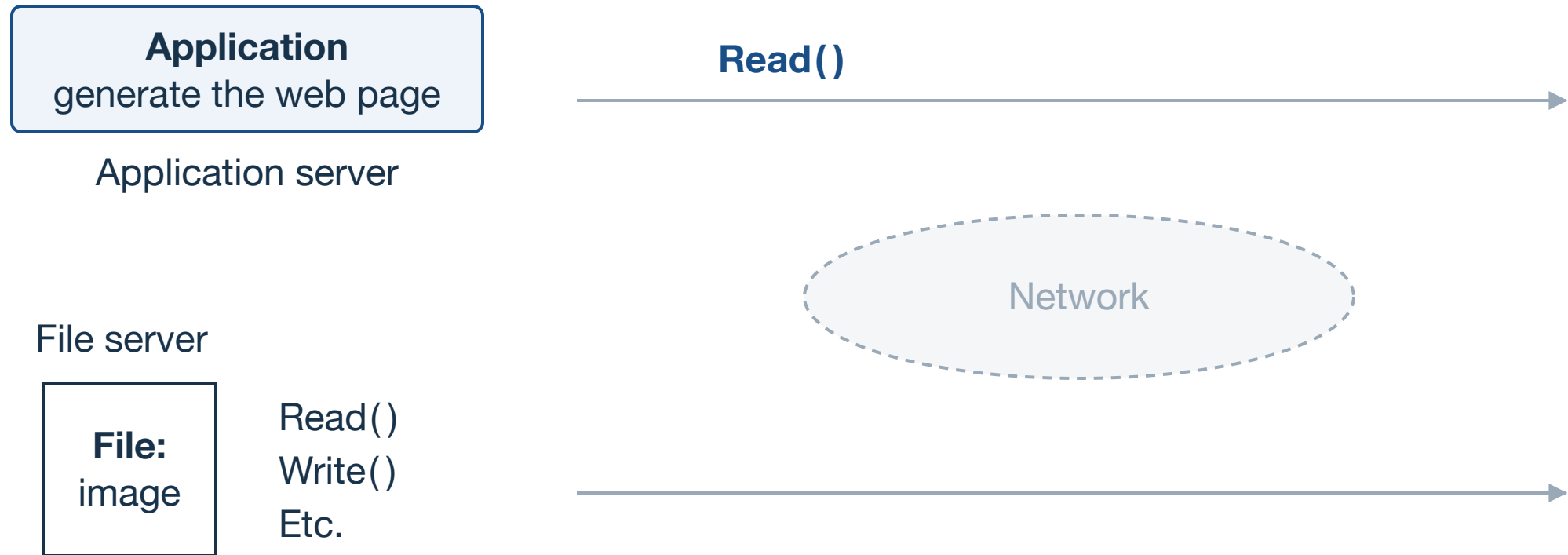
Case study: File system + RPC

Goal: access a file stored at remote



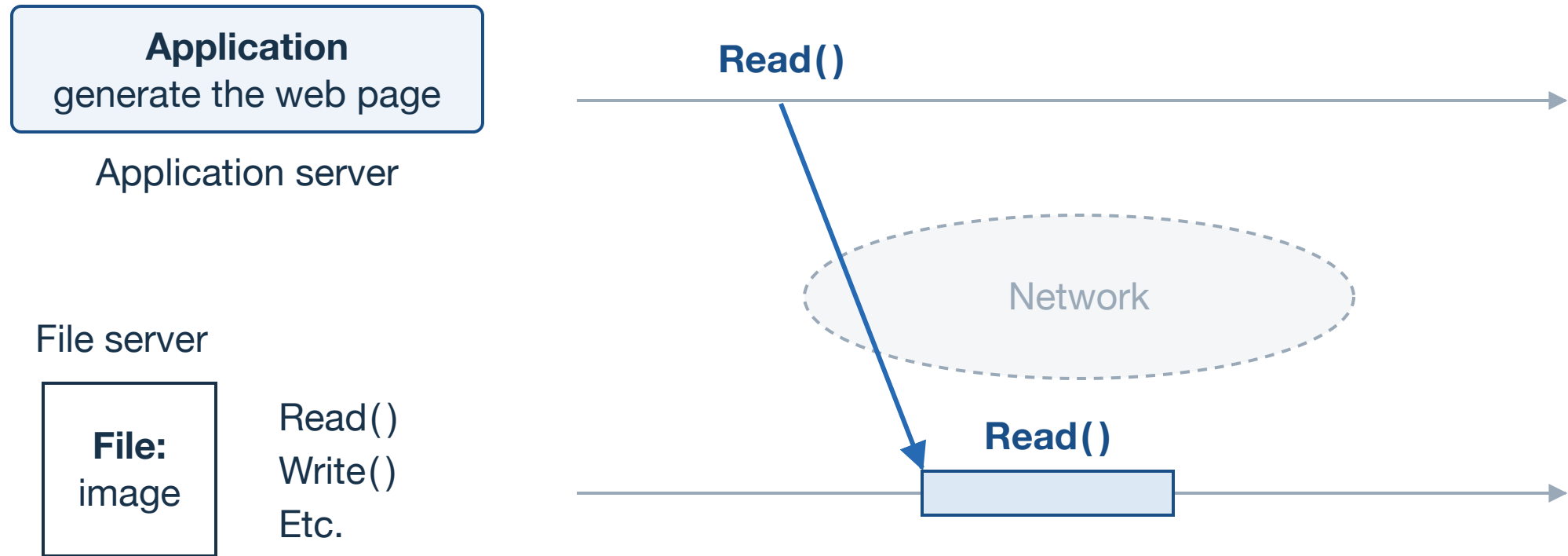
Case study: File system + RPC

Goal: access a file stored at remote



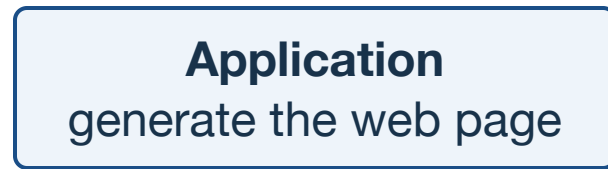
Case study: File system + RPC

Goal: access a file stored at remote



Case study: File system + RPC

Goal: access a file stored at remote

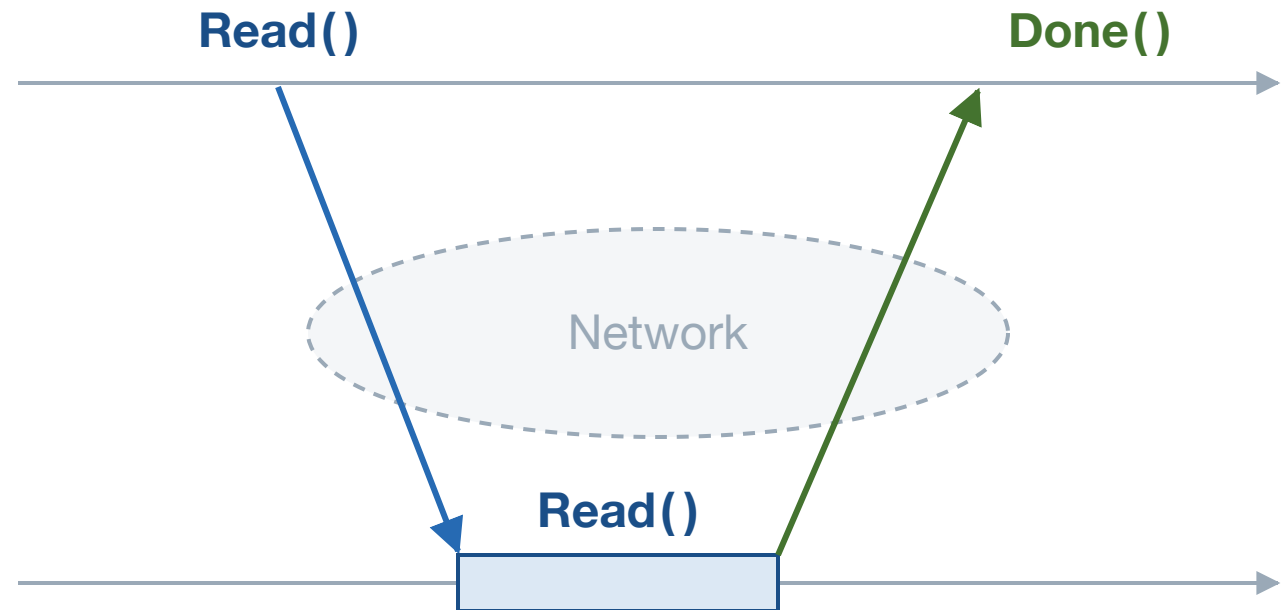


Application server

File server



Read()
Write()
Etc.



Ways for accessing remote files

Many familiar ways: **FTP**, **telnet**, cloud drives (网盘), ...

- Explicit access (knowing it is a distributed file system)
- User-directed connection to access remote resources



There is also a **transparent** approach

- E.g., NFS, GFS, etc.
- Applications access the remote files like local files

NFS with RPC

NFS: Network File System

Design goals (by Sun, 1980s, designed for workstations)

- Any machine can be a client or a server
- Support **diskless** workstations
- Support **heterogeneous** deployment
 - Different HW, OS, underlying file system
- Access **transparency**
 - Use the remote access model
- Recovery from **failure**
 - Stateless, UDP, client retries
- High **performance**
 - Use caching and read-ahead

Question: can we simply replace the APIs (e.g., open, read/write) of the inode file system with RPCs for a DFS?

Recall: the file system API

API

- CHDIR, MKDIR
- CREAT, LINK, UNLINK, RENAME
- SYMLINK
- MOUNT, UNMOUNT
- OPEN, READ, WRITE, APPEND, CLOSE
- SYNC

Idea: Implemented as ~~system calls~~ remote procedure calls to user applications

From the file system API to NFS RPCs

inode file system API	NFS RPC
OPEN (path, mode)	LOOKUP (dirfh, name)
READ (fd, buf, n)	READ (fh, offset, count)
WRITE (fd, buf, n)	WRITE (fh, offset, count, data)
SEEK (fd, offset)	—
CLOSE (fd)	—
STAT (path)	GETATTR (fh)
MKDIR (path)	MKDIR (dirfh, name, attr)
UNLINK (path)	REMOVE (dirfh, name)
RENAME (from, to)	RENAME (dirfh, name, tofh, toname)

From the file system API to NFS RPCs

inode file system API	NFS RPC
OPEN (path, mode)	LOOKUP (dirfh, name)
READ (fd, buf, n)	READ (fh, offset, count)
WRITE (fd, buf, n)	WRITE (fh, offset, count, data)
SEEK (fd, offset)	—
CLOSE (fd)	—
STAT (path)	GETATTR (fh)
MKDIR (path)	MKDIR (dirfh, name, attr)
UNLINK (path)	REMOVE (dirfh, name)
RENAME (from, to)	RENAME (dirfh, name, tofh, toname)

Green: unchanged

From the file system API to NFS RPCs

inode file system API	NFS RPC	What changes
OPEN (path , mode)	LOOKUP (dirfh , name)	
READ (fd , buf , n)	READ (fh , offset , count)	fd → fh, + offset
WRITE (fd , buf , n)	WRITE (fh , offset , count , data)	fd → fh, + offset
SEEK (fd , offset)	—	gone
CLOSE (fd)	—	gone
STAT (path)	GETATTR (fh)	
MKDIR (path)	MKDIR (dirfh , name , attr)	
UNLINK (path)	REMOVE (dirfh , name)	
RENAME (from , to)	RENAME (dirfh , name , tofh , toname)	

Green: unchanged

Orange: changed

Grey: gone in NFS

From the file system API to NFS RPCs

inode file system API	NFS RPC	What changes
<code>OPEN (path , mode)</code>	<code>LOOKUP (dirfh , name)</code>	gone: only LOOKUP
<code>READ (fd , buf , n)</code>	<code>READ (fh , offset , count)</code>	fd → fh, + offset
<code>WRITE (fd , buf , n)</code>	<code>WRITE (fh , offset , count , data)</code>	fd → fh, + offset
<code>SEEK (fd , offset)</code>	—	gone
<code>CLOSE (fd)</code>	—	gone
<code>STAT (path)</code>	<code>GETATTR (fh)</code>	path → fh
<code>MKDIR (path)</code>	<code>MKDIR (dirfh , name , attr)</code>	path → dirfh, name
<code>UNLINK (path)</code>	<code>REMOVE (dirfh , name)</code>	path → dirfh, name
<code>RENAME (from , to)</code>	<code>RENAME (dirfh , name , tofh , toname)</code>	path → dirfh, name

Green: unchanged Orange: changed Grey: gone in NFS

NFS protocols step-by-step: `mount`

Static mounting

- A mount request contacts the server

```
# Server: add the list of shared directories to /etc/exports  
# Client:  
mount 192.168.1.100:/users/paul /home/paul
```

NFS protocols step-by-step: mount

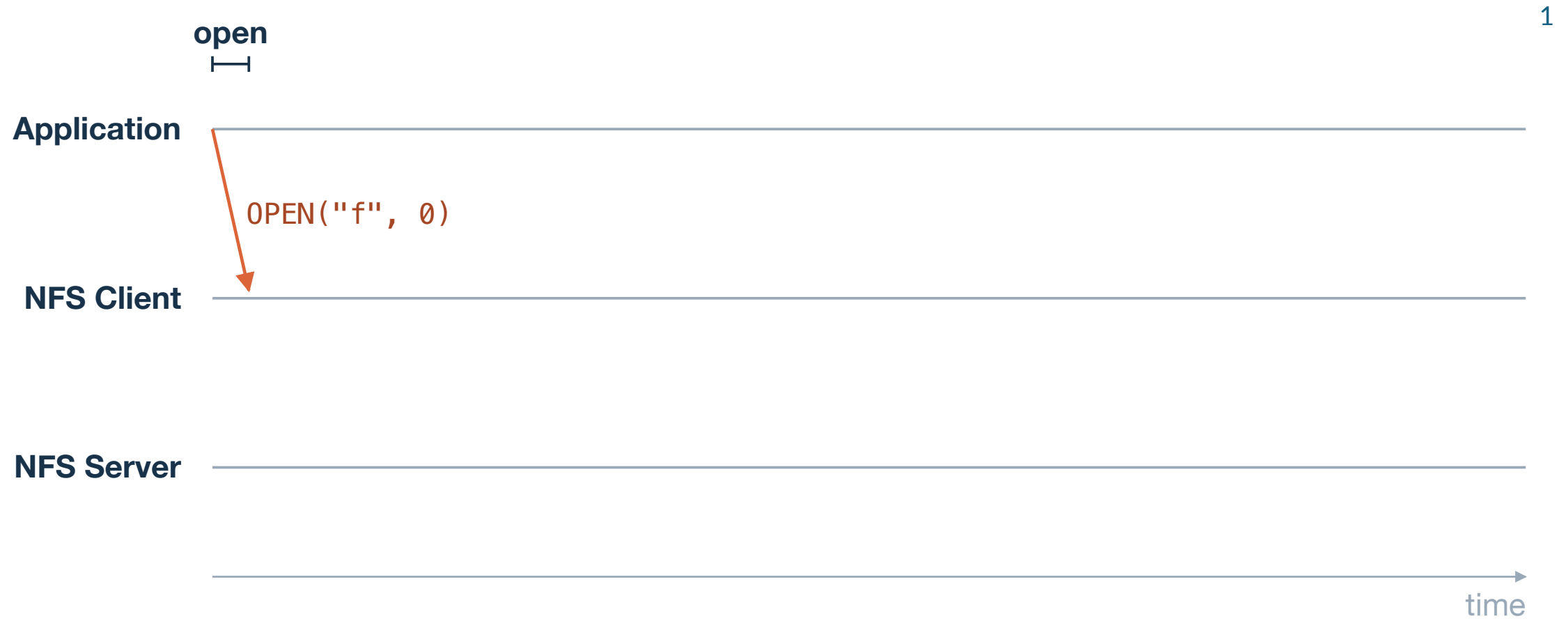
Protocol

- Request access to the exported directory tree
 - Requests **permission** to access the contents
- **Client:** parses the **pathname**, contacts the server for the file **handle**
- **Server:** a standard inode file system that implements the NFS RPC callbacks
- The server returns the **file handle (fh)**, the name of the remote file

Client: creates an in-memory VFS inode (vnode) at the mount point that internally points to the remote file

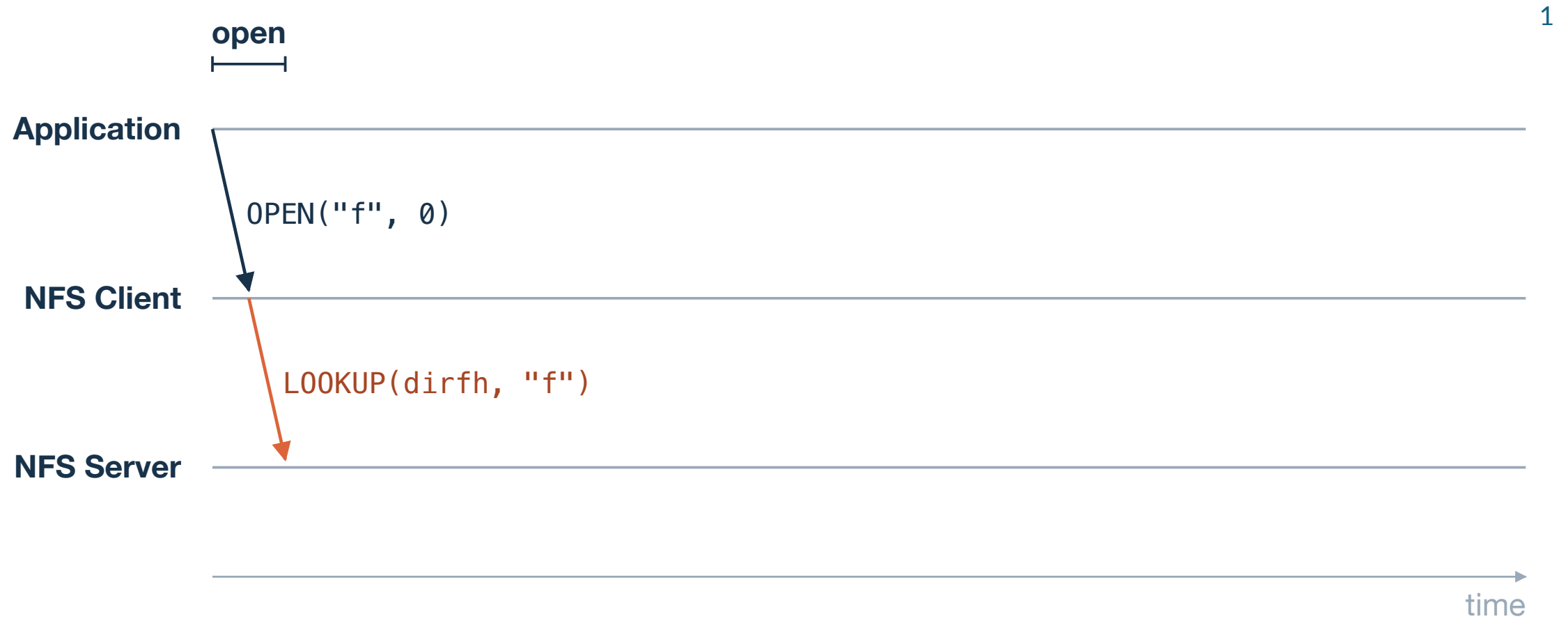
- Executes the NFS RPC calls to realize the file system APIs
- The NFS must behave as a transparent file system to the applications

NFS protocols: read a file



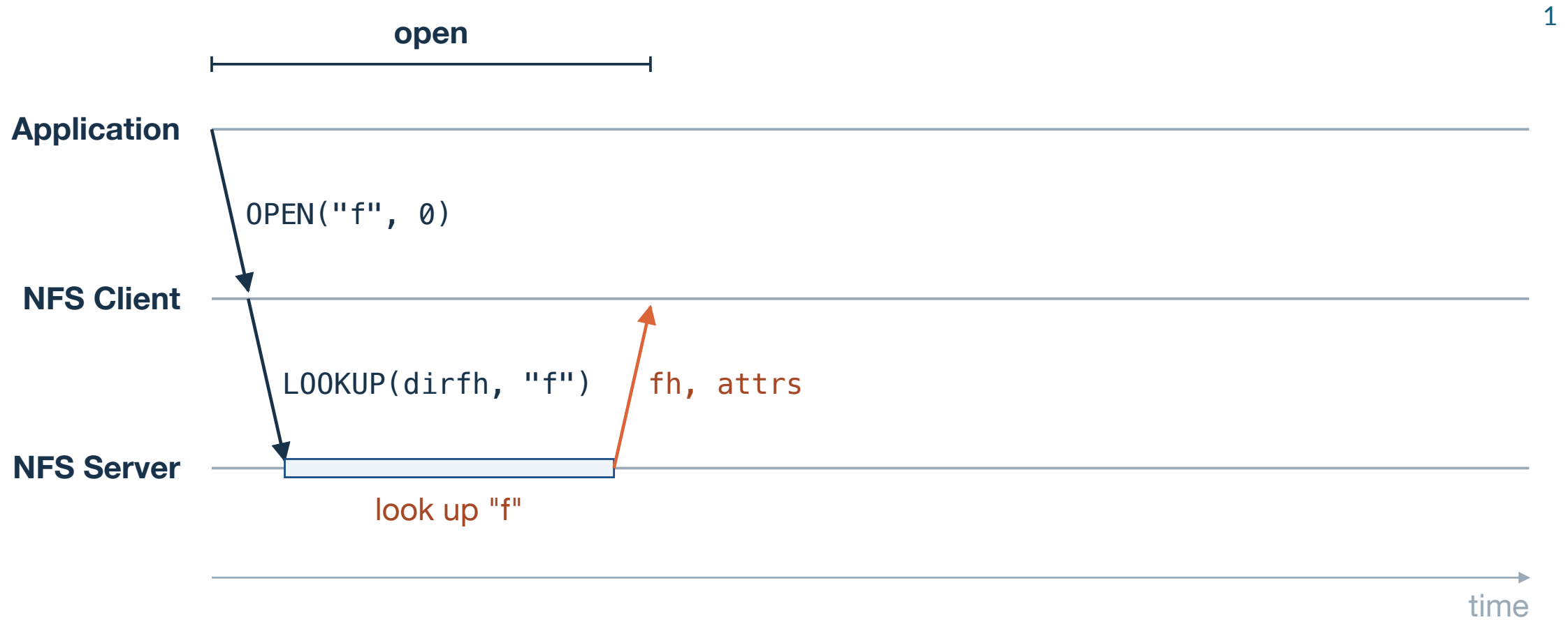
¹ Source: Saltzer & Kaashoek, Principles of Computer System Design, Chapter 4

NFS protocols: **read** a file

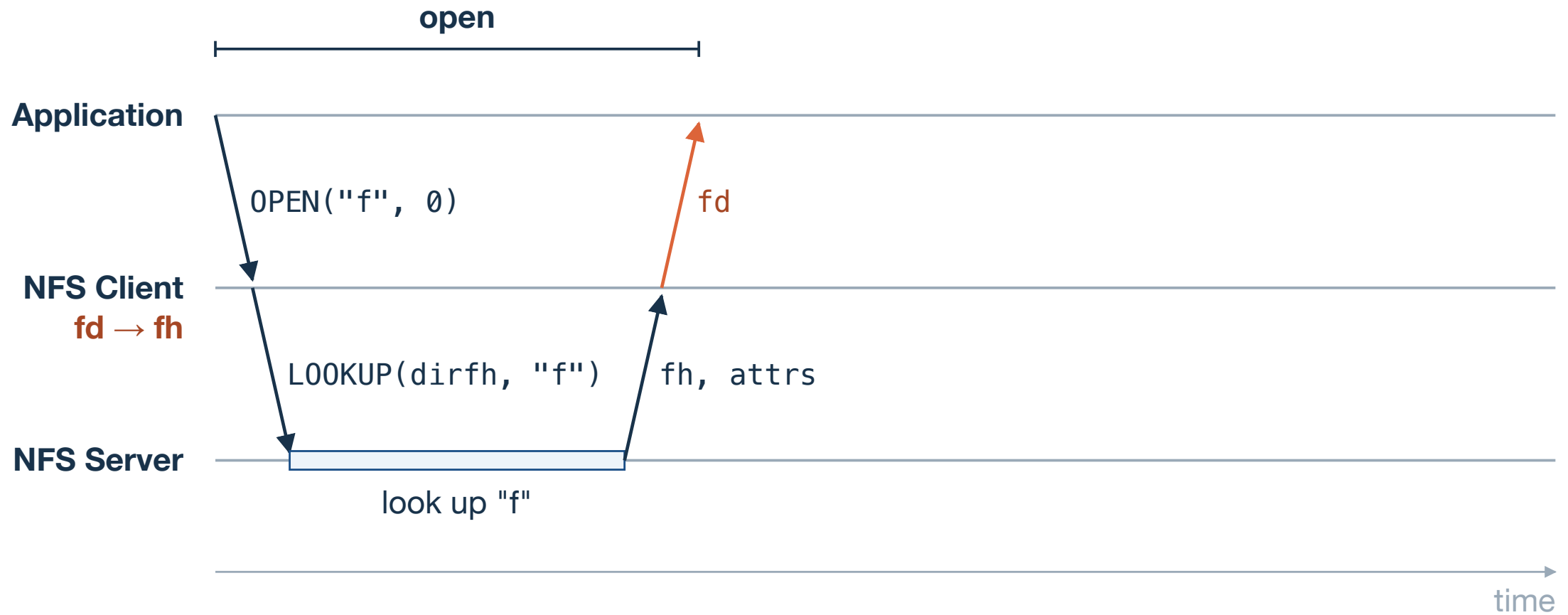


¹ Source: Saltzer & Kaashoek, Principles of Computer System Design, Chapter 4

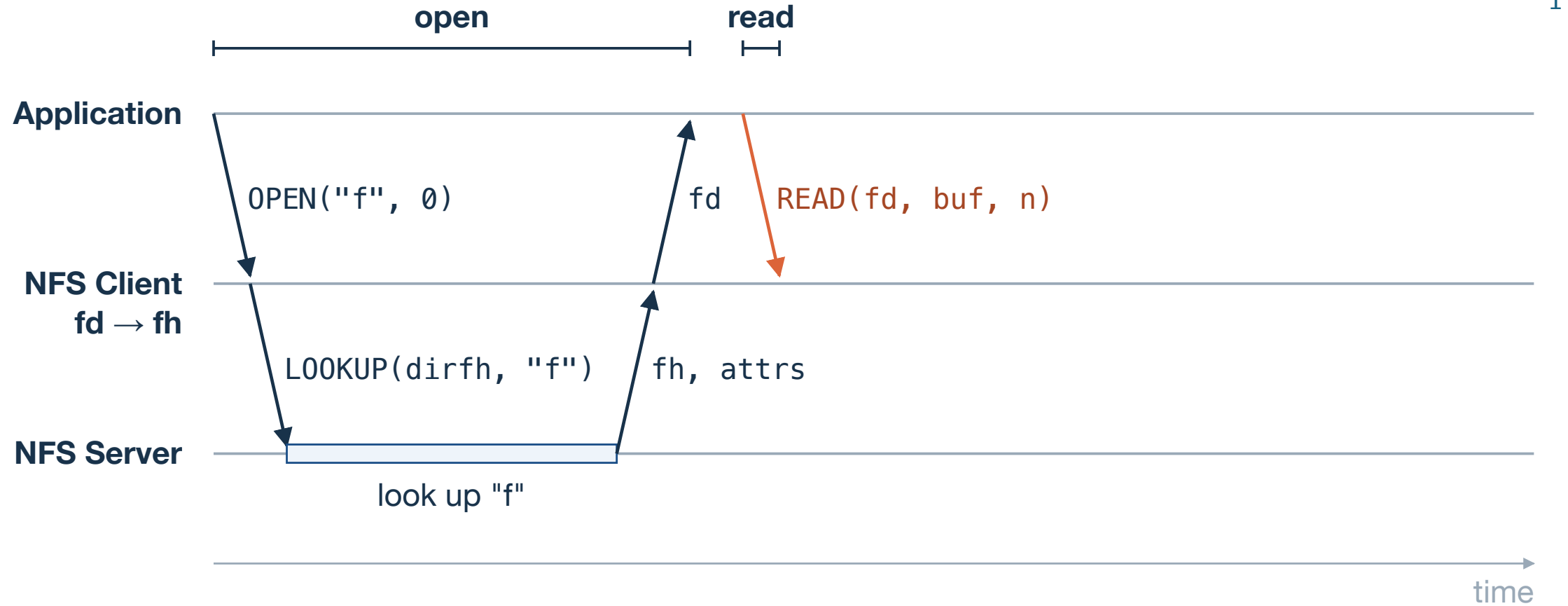
NFS protocols: **read** a file



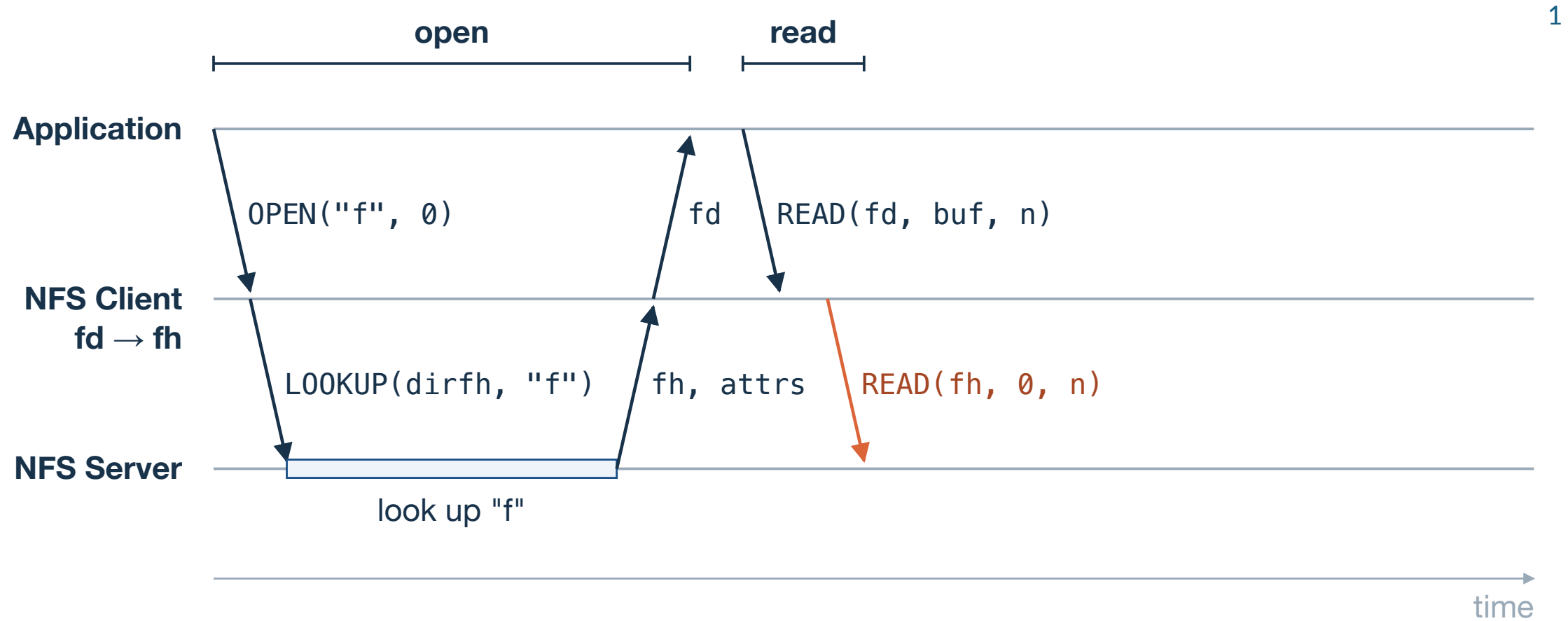
NFS protocols: **read** a file



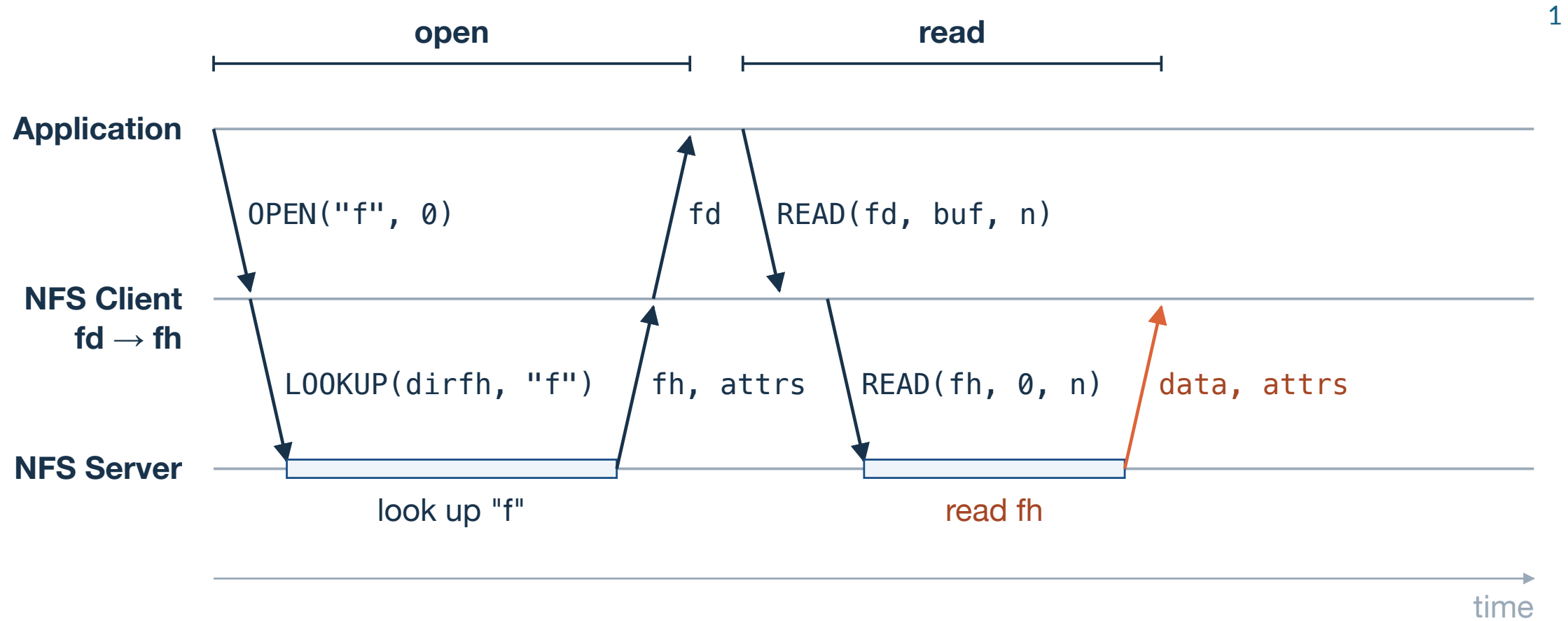
NFS protocols: **read** a file



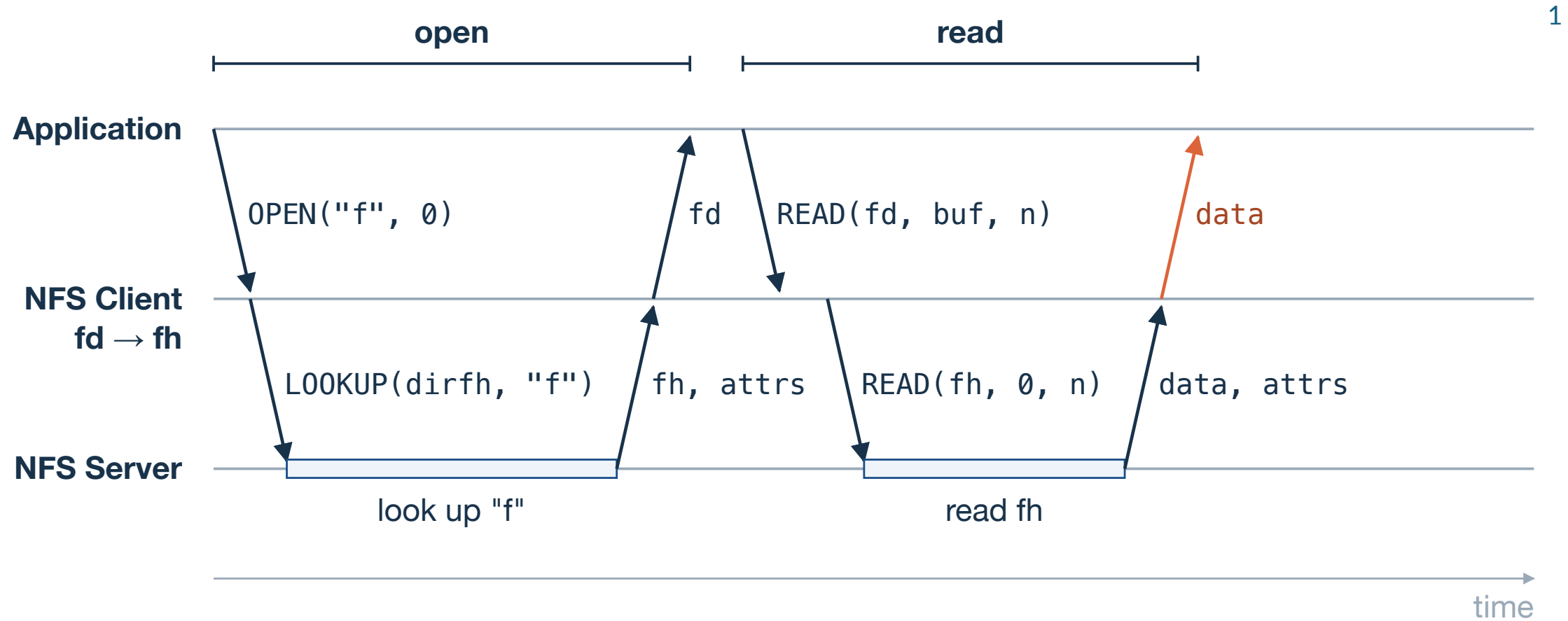
NFS protocols: **read** a file



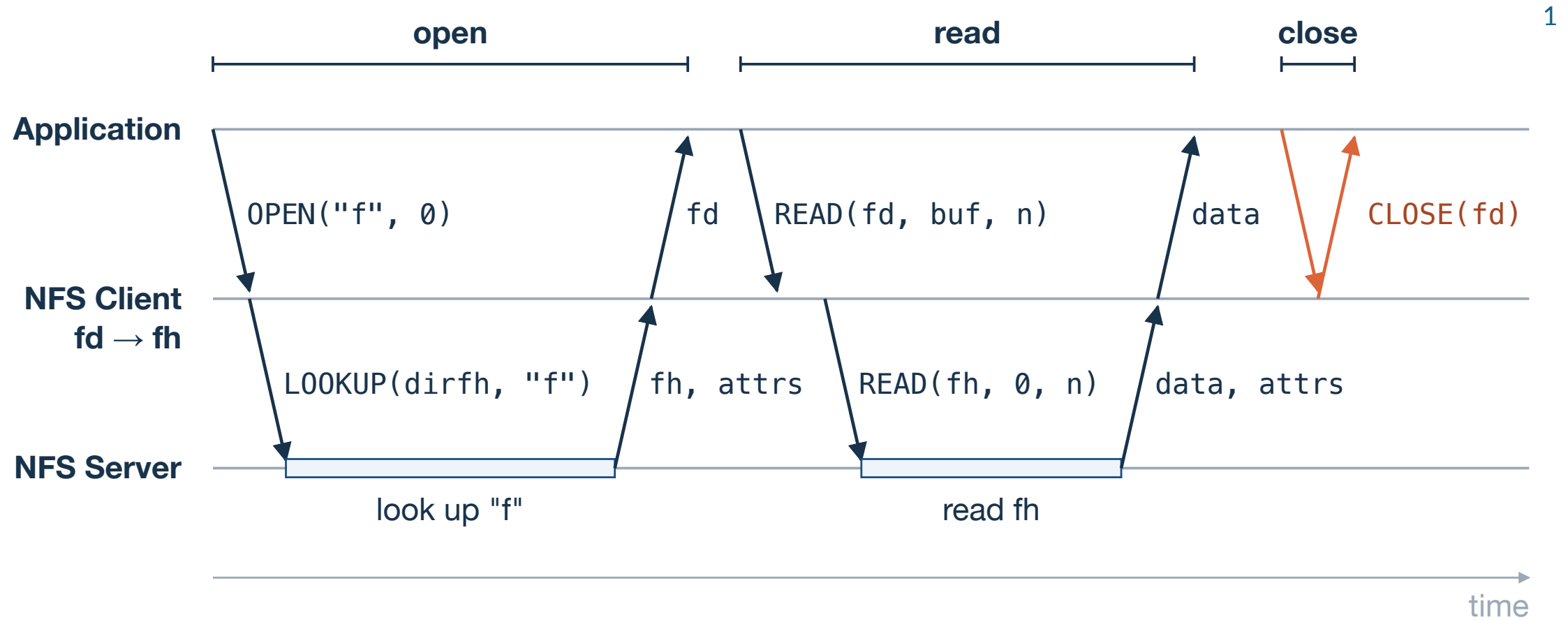
NFS protocols: **read** a file



NFS protocols: **read** a file



NFS protocols: **read** a file



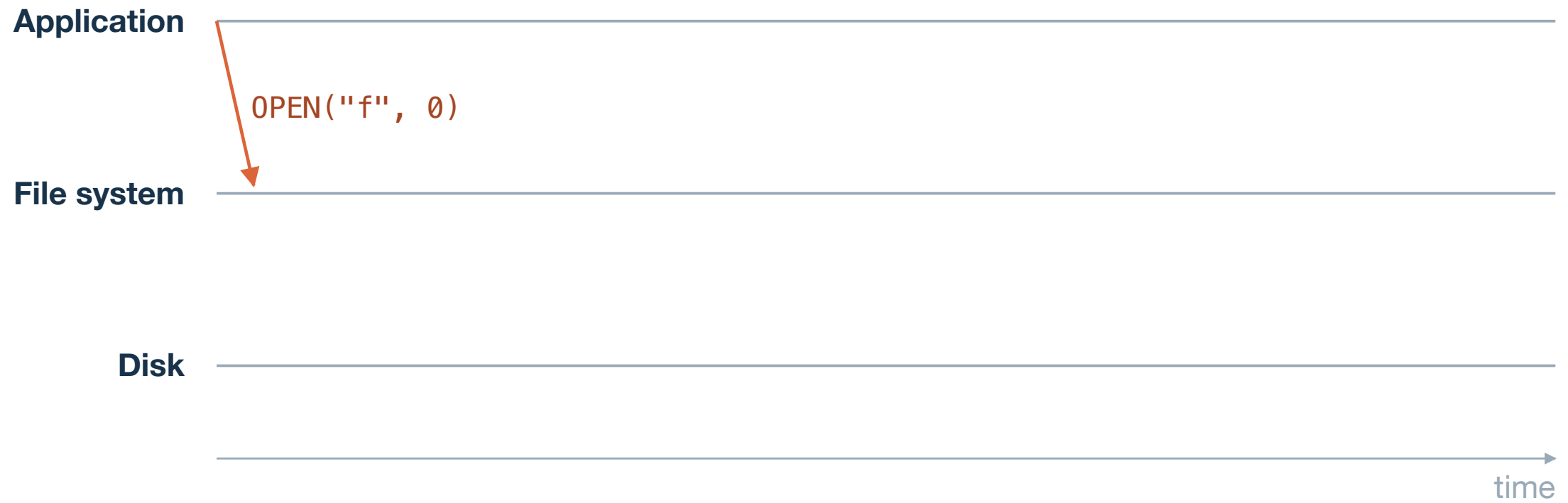
Design choice: no `open` RPC in NFS

The NFS client library executes `open` locally

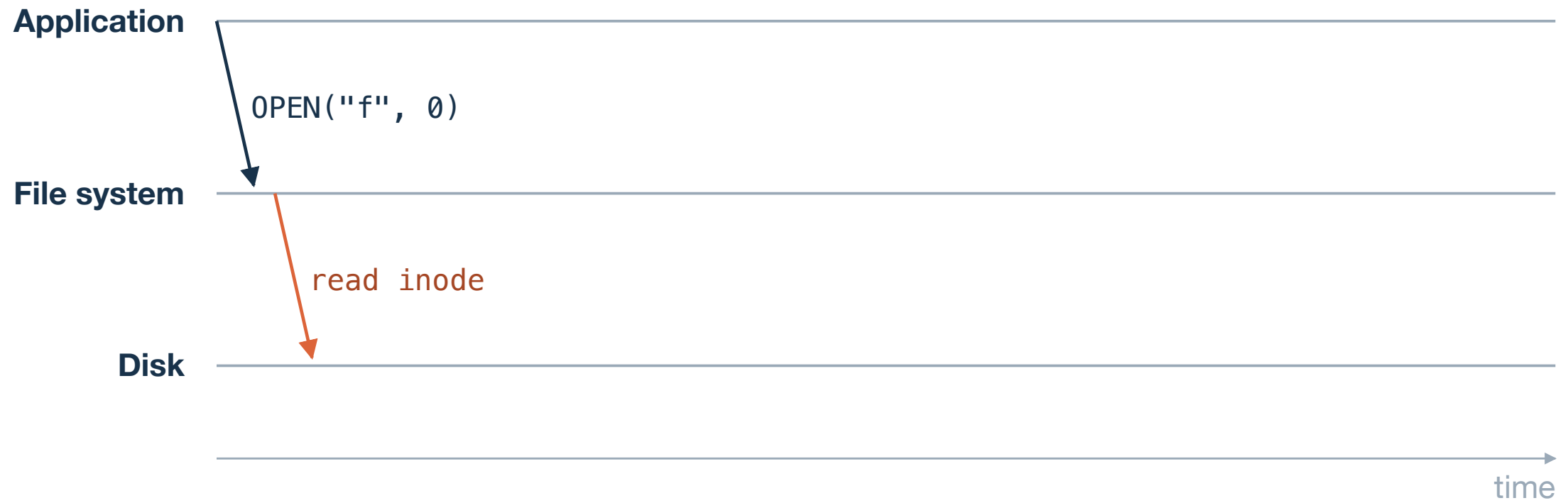
- By issuing the `lookup` API instead. Why?

To answer this, we need to recall what `open` does

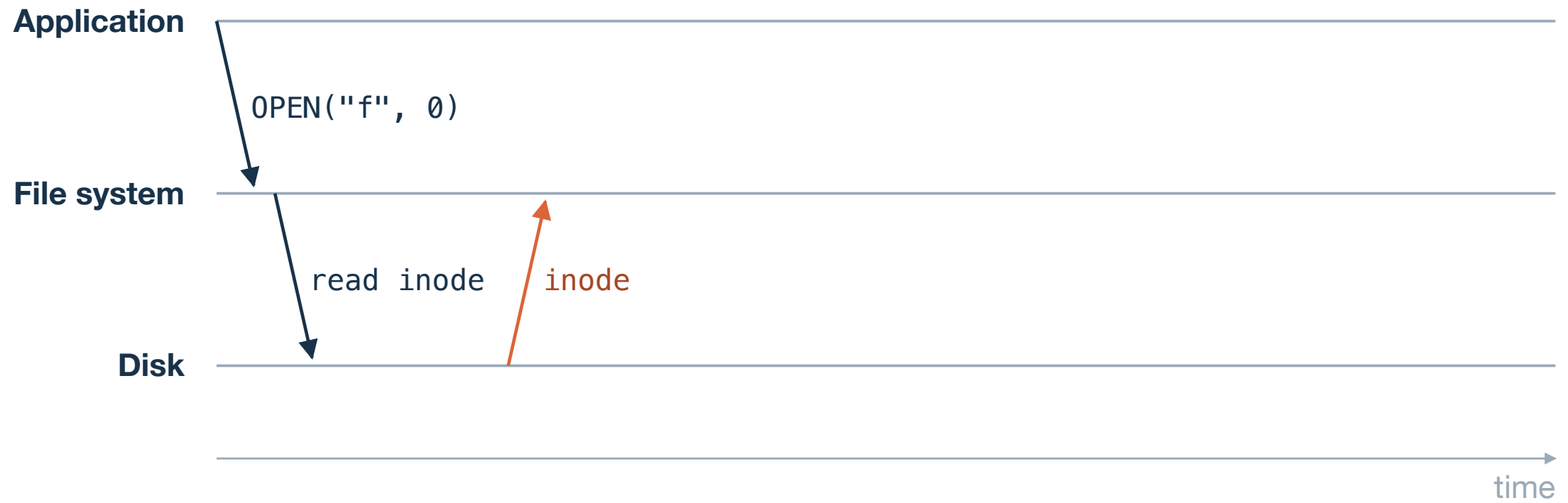
Review: open



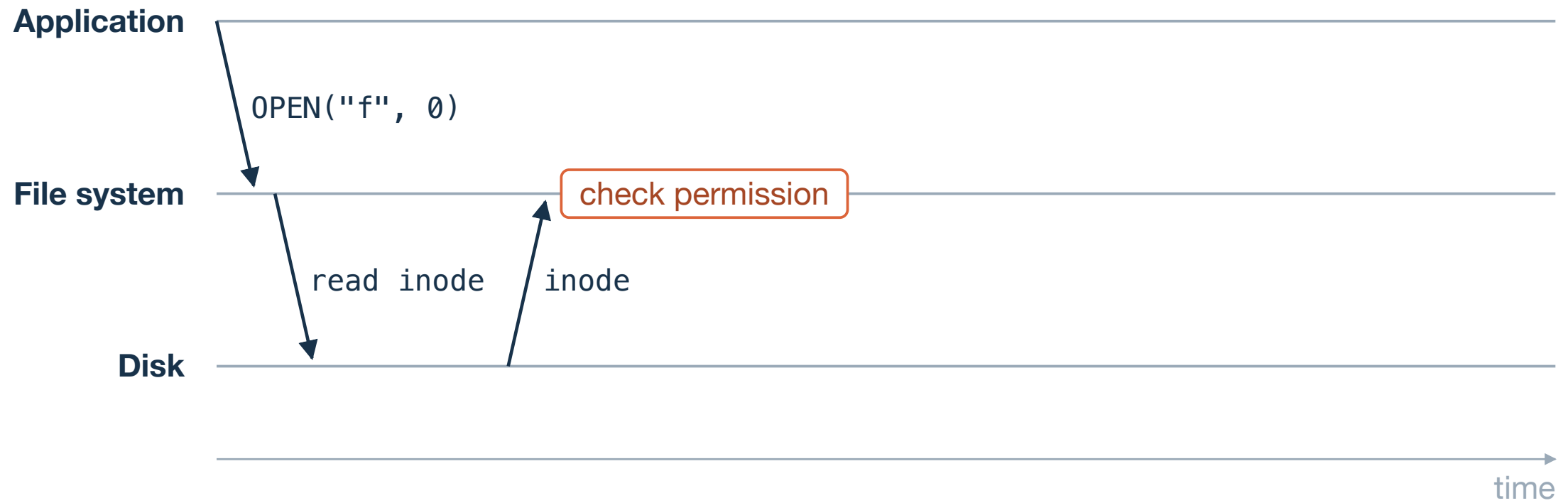
Review: [open](#)



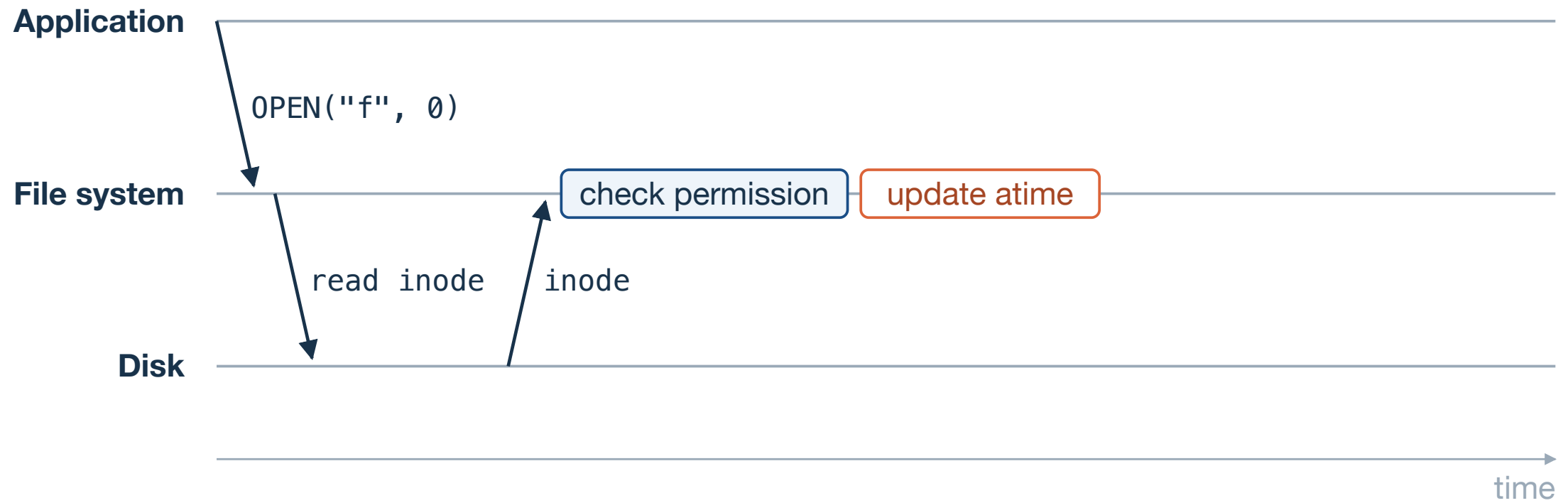
Review: open



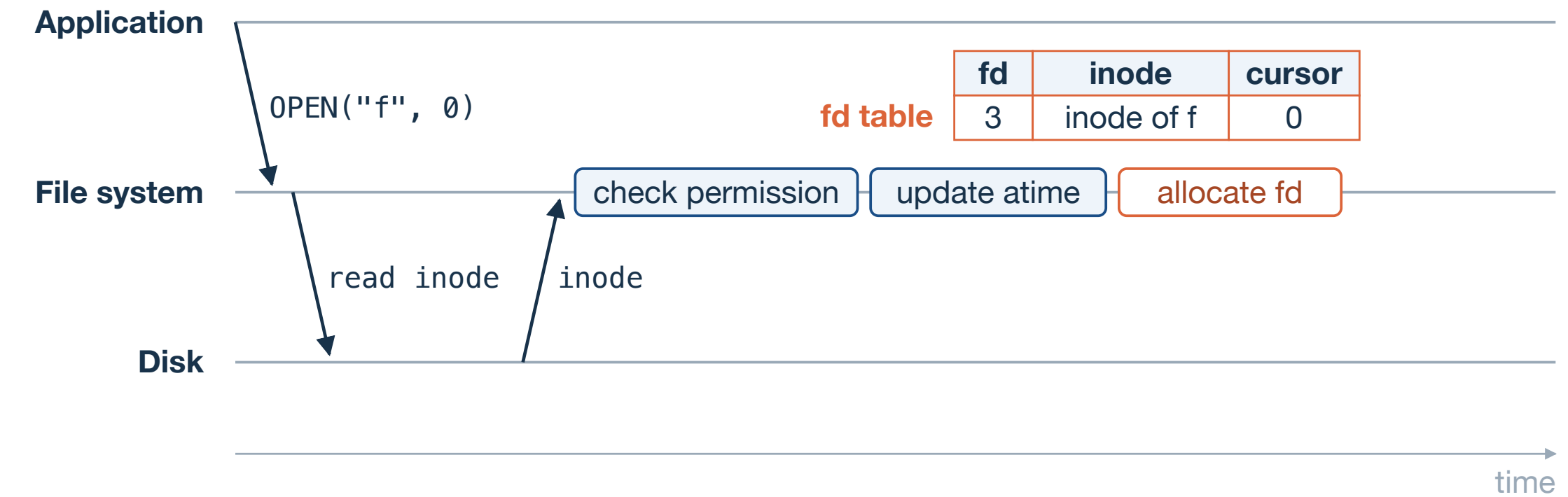
Review: open



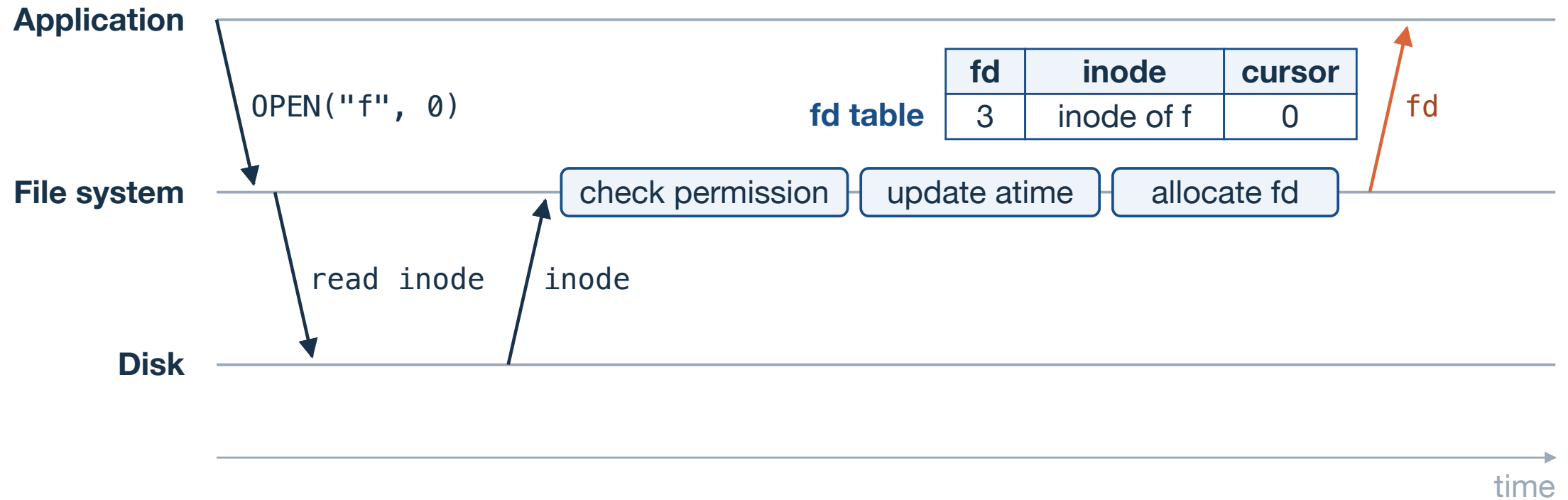
Review: open



Review: open

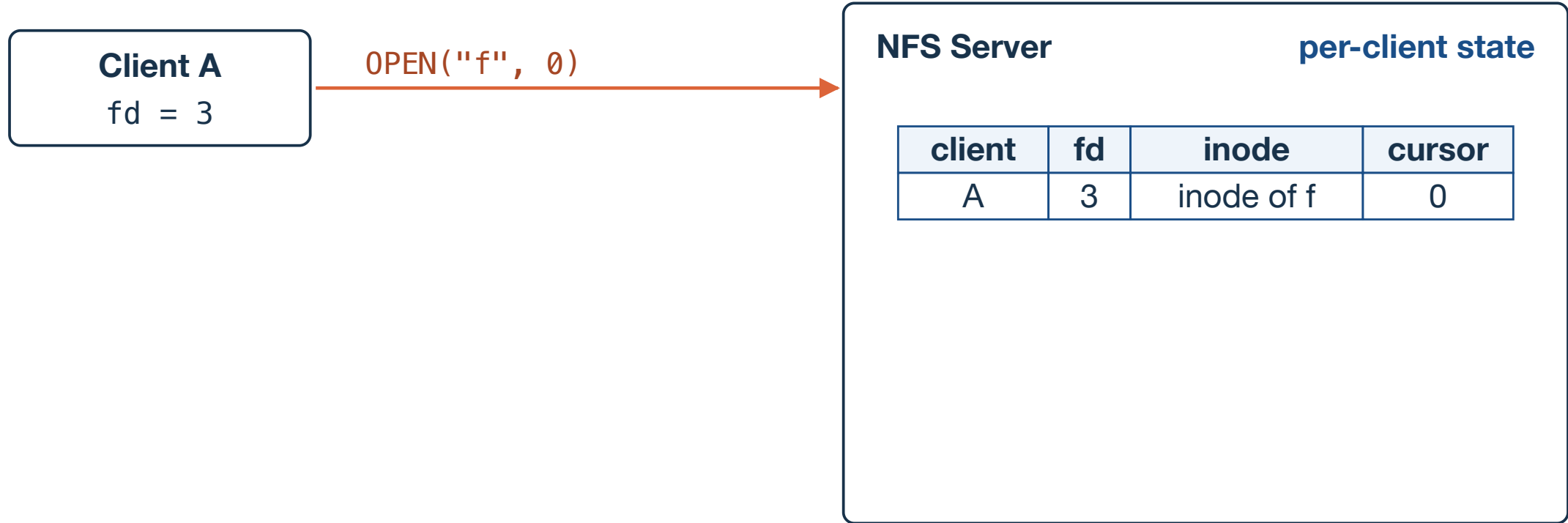


Review: `open`

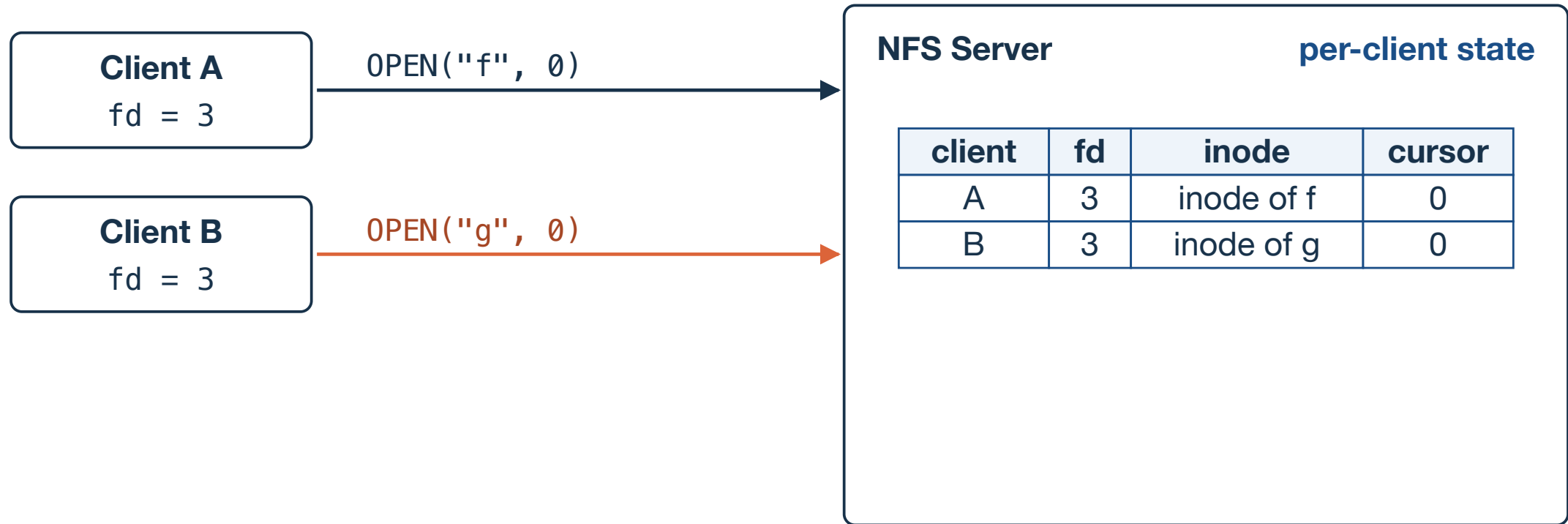


The fd is then used by `READ`, `WRITE`, `CLOSE`, etc.

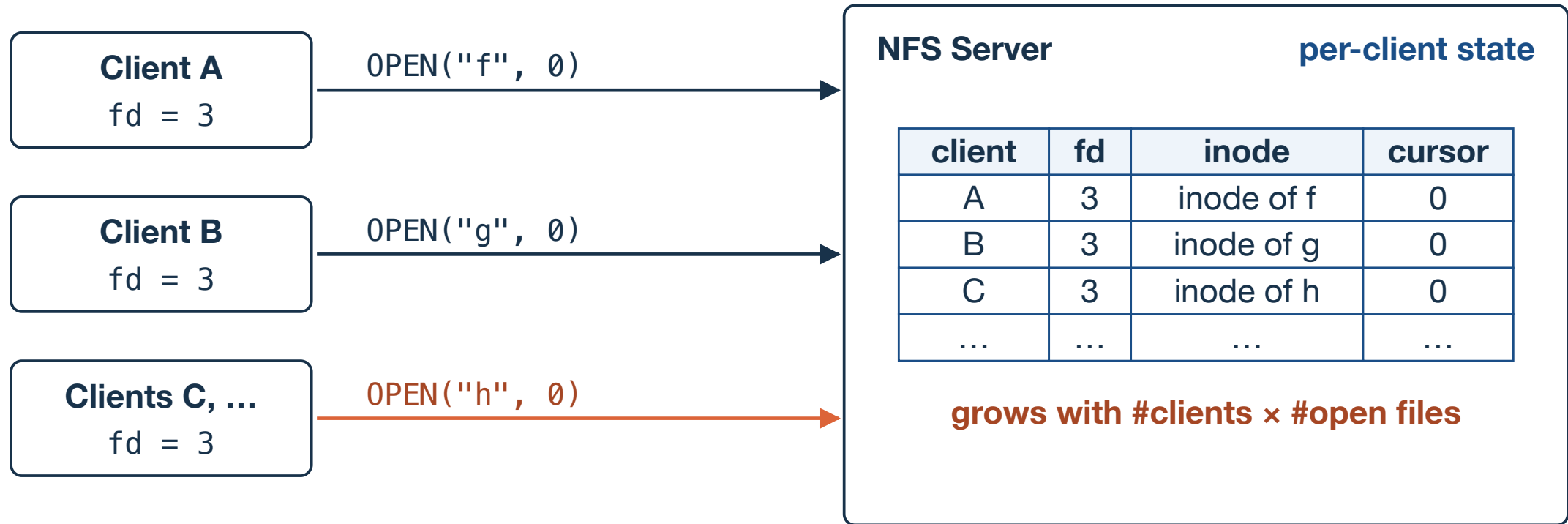
What if we faithfully implement `open` with RPC?



What if we faithfully implement `open` with RPC?



What if we faithfully implement `open` with RPC?



What if we faithfully implement `open` with RPC?

Client A

fd = 3

Client B

fd = 3

Clients C, ...

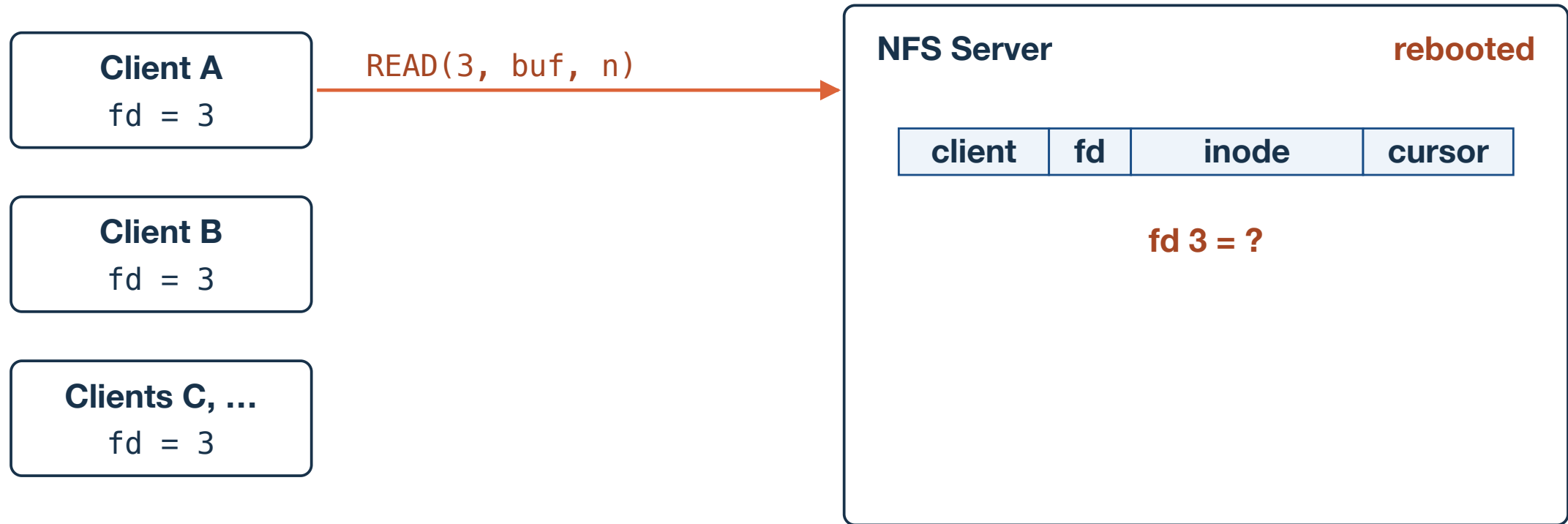
fd = 3

NFS Server

crash

client	fd	inode	cursor
A	3	inode of f	0
B	3	inode of g	0
C	3	inode of h	0
...	...	...	...

What if we faithfully implement `open` with RPC?



What if we faithfully implement **open** with RPC?

Client A

fd = 3

Client B

fd = 3

Clients C, ...

fd = 3

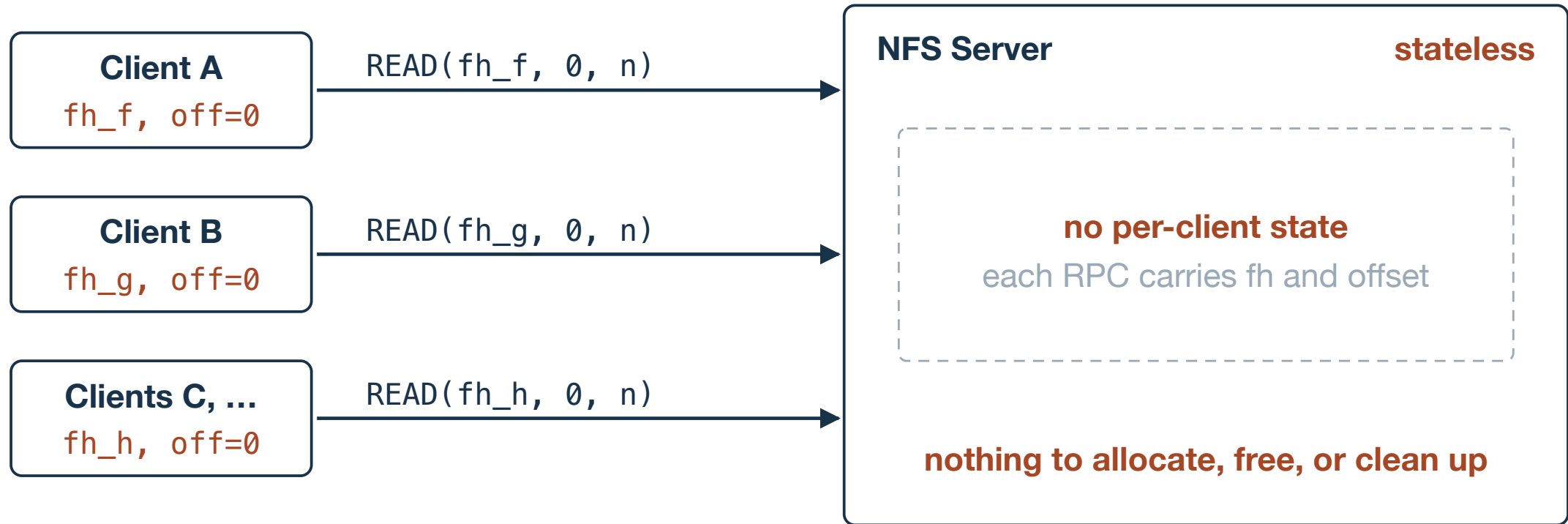
NFS Server

per-client state

client	fd	inode	cursor
A	3	inode of f	0
B	3	inode of g	0
C	3	inode of h	0
...	...	...	...

- allocate a row on every **OPEN**
- free the row on **CLOSE**
- clean up rows of crashed clients

What if we faithfully implement `open` with RPC?



NFS chooses a simple design: no `open`, and the RPCs are **stateless**

- For better fault tolerance & scalability, and a simpler server

Stateless on the NFS server

Stateless on the NFS server

- Each RPC contains all the information

Q: What about state like the file cursor?

- The client maintains the state, including the file cursor

Stateless simplifies failure handling

- The client can repeat a request until it receives a reply (at least once)

But, naïve retry of stateless operations is not always good

- The server may execute the same request twice
- Solution: each RPC is tagged with a transaction ID (xid), and the server keeps some "soft" state: a reply cache (recall at-most-once RPC)

Stateless is a common design: e.g., LLM APIs

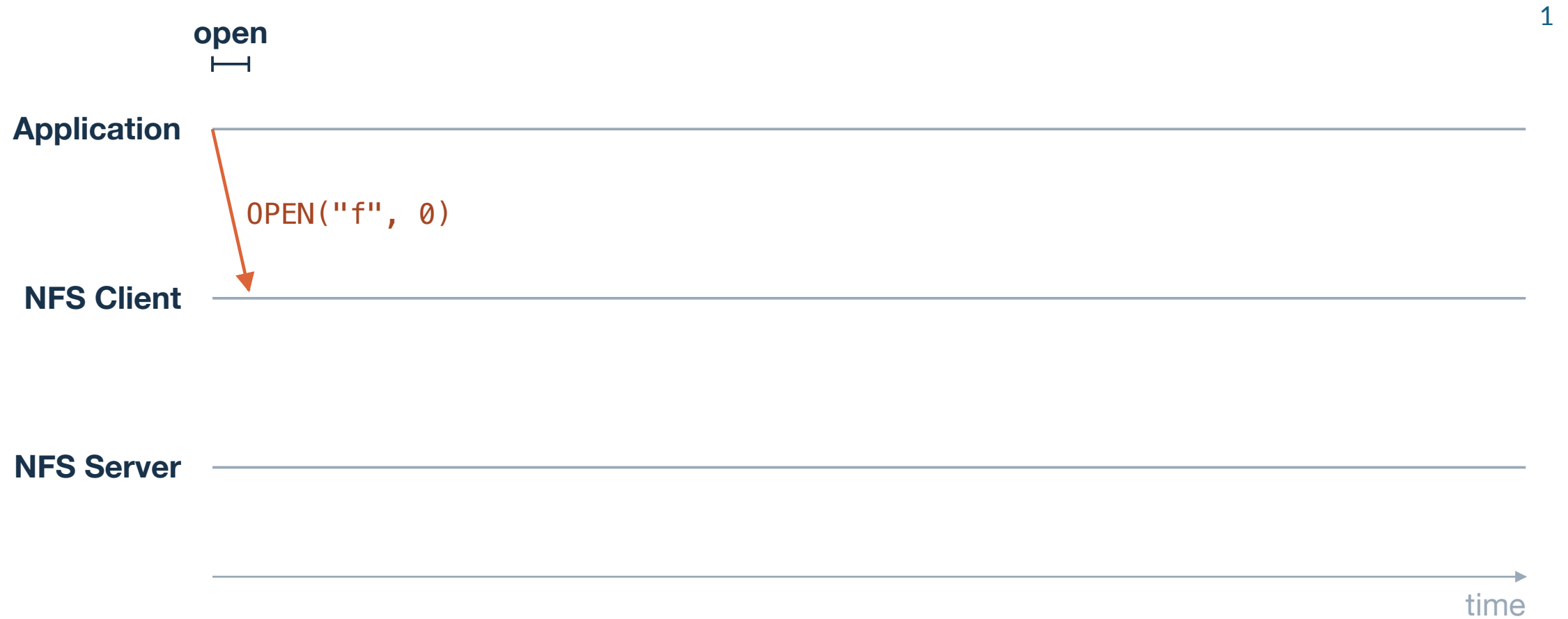
OpenAI API: "each text generation request is **independent and stateless**"

```
const response = await openai.responses.create({
  model: "gpt-6-astra",
  input: [
    { role: "user", content: "knock knock." },
    { role: "assistant", content: "Who's there?" },
    { role: "user", content: "Orange." },
  ],
});
```

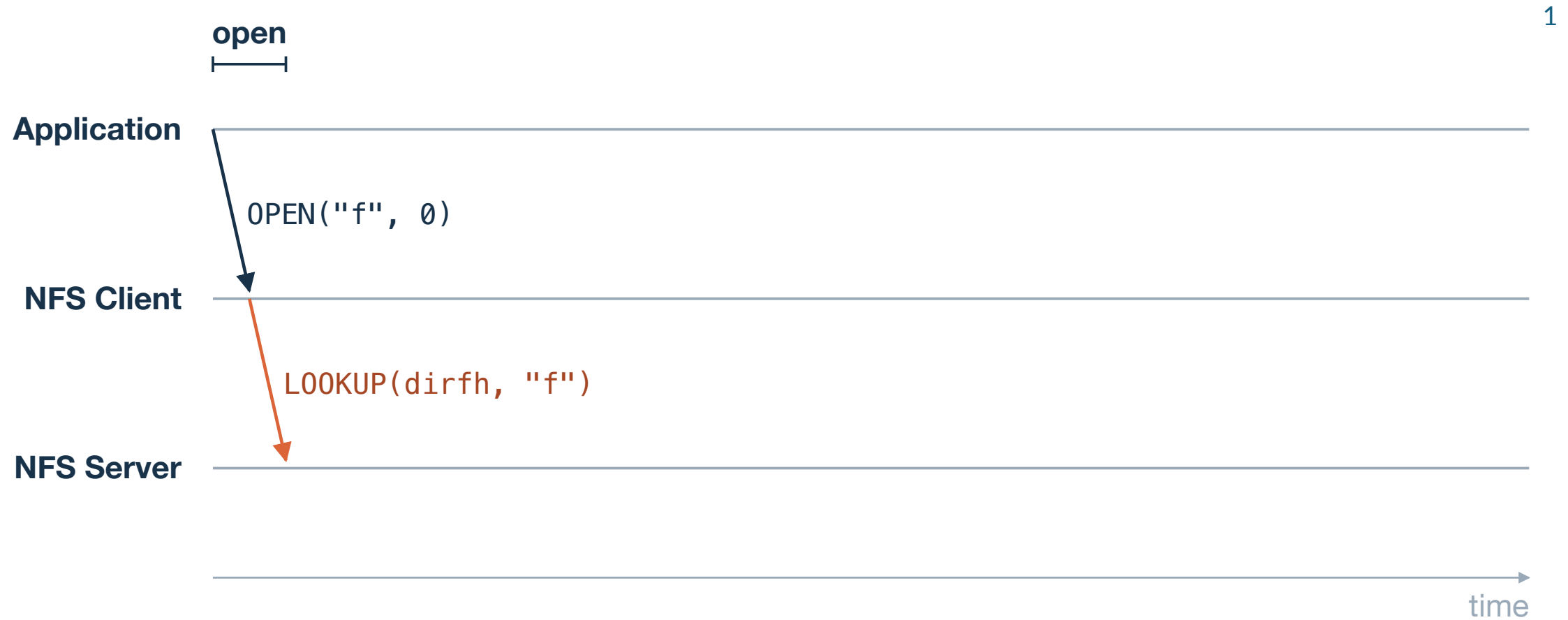
	NFS	OpenAI API
Each request carries	fh, offset, count	the whole message history
The client keeps	the file cursor	the conversation

Stateful option: `previous_response_id`, like `open` returning an fd

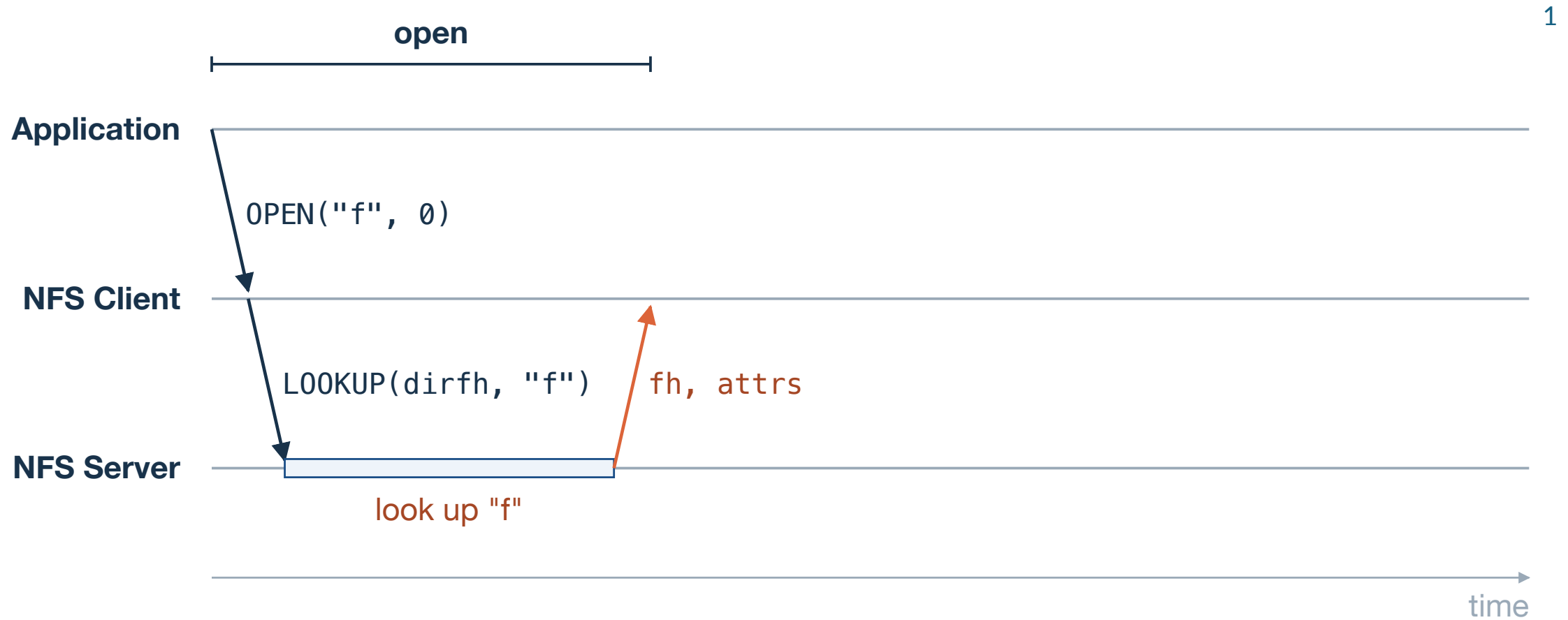
NFS protocols: read a file



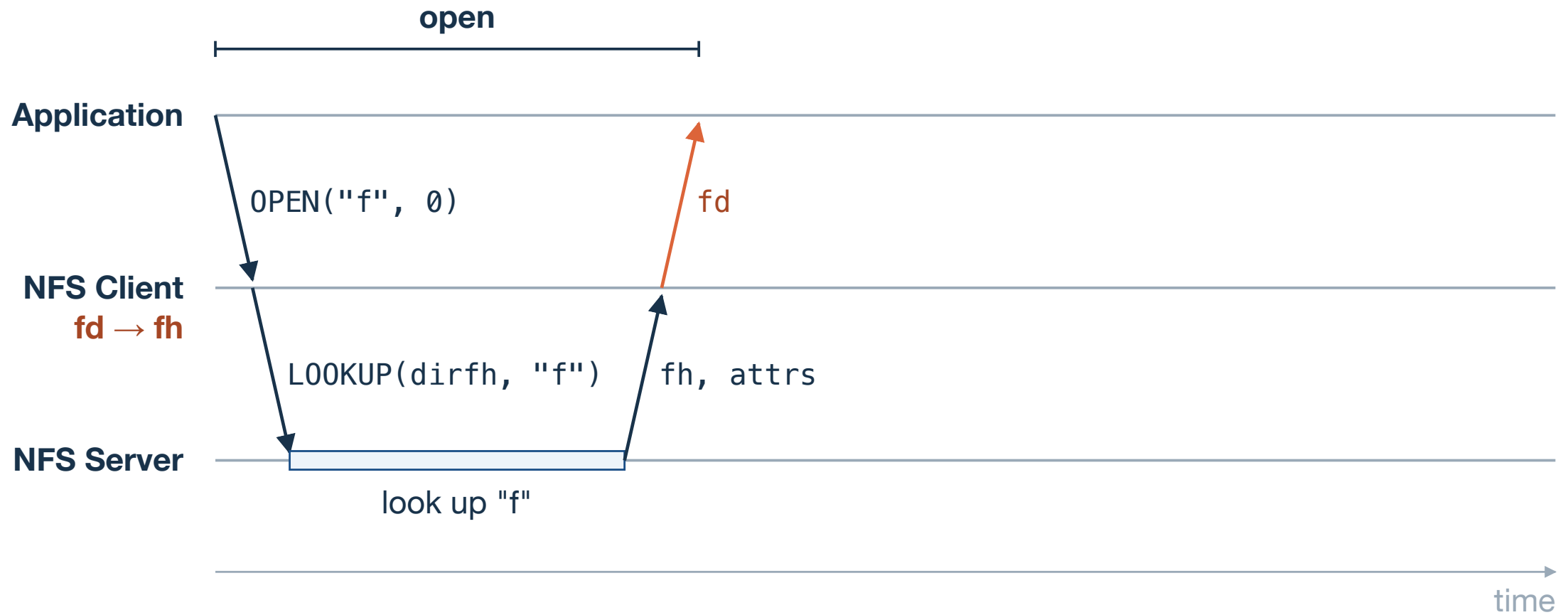
NFS protocols: **read** a file



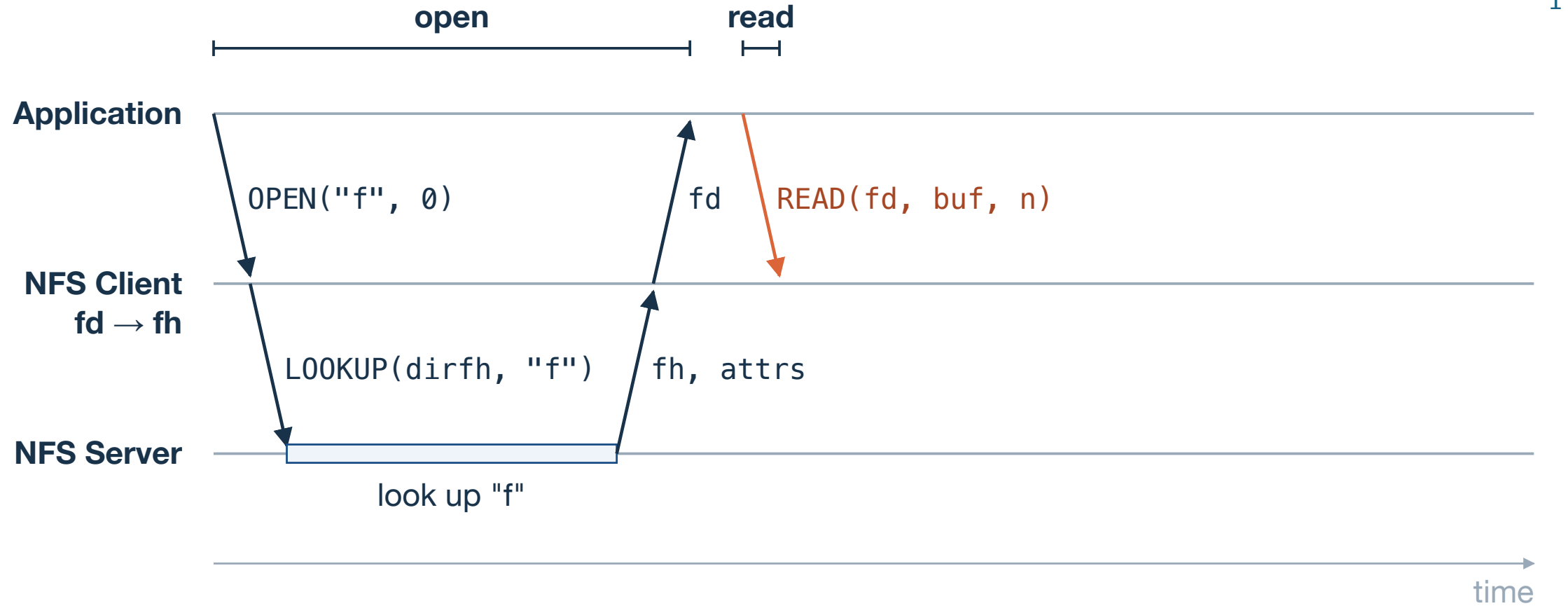
NFS protocols: **read** a file



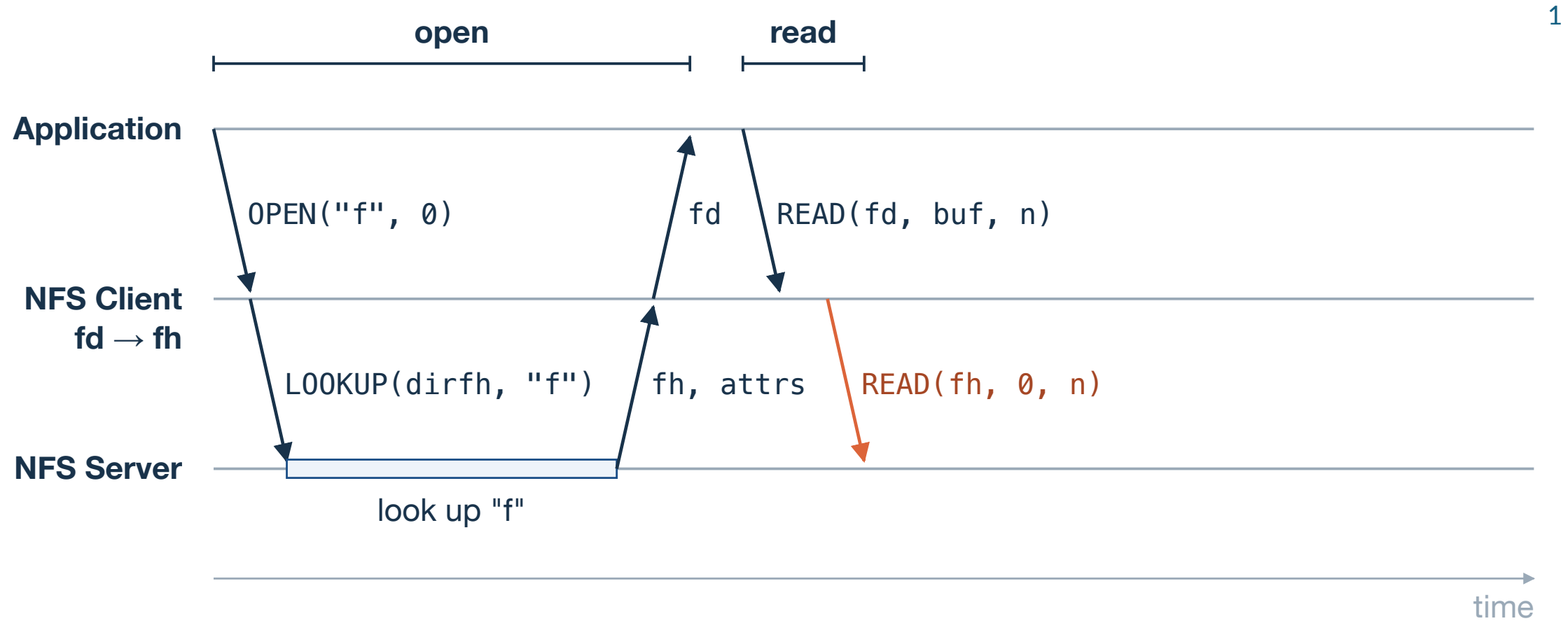
NFS protocols: **read** a file



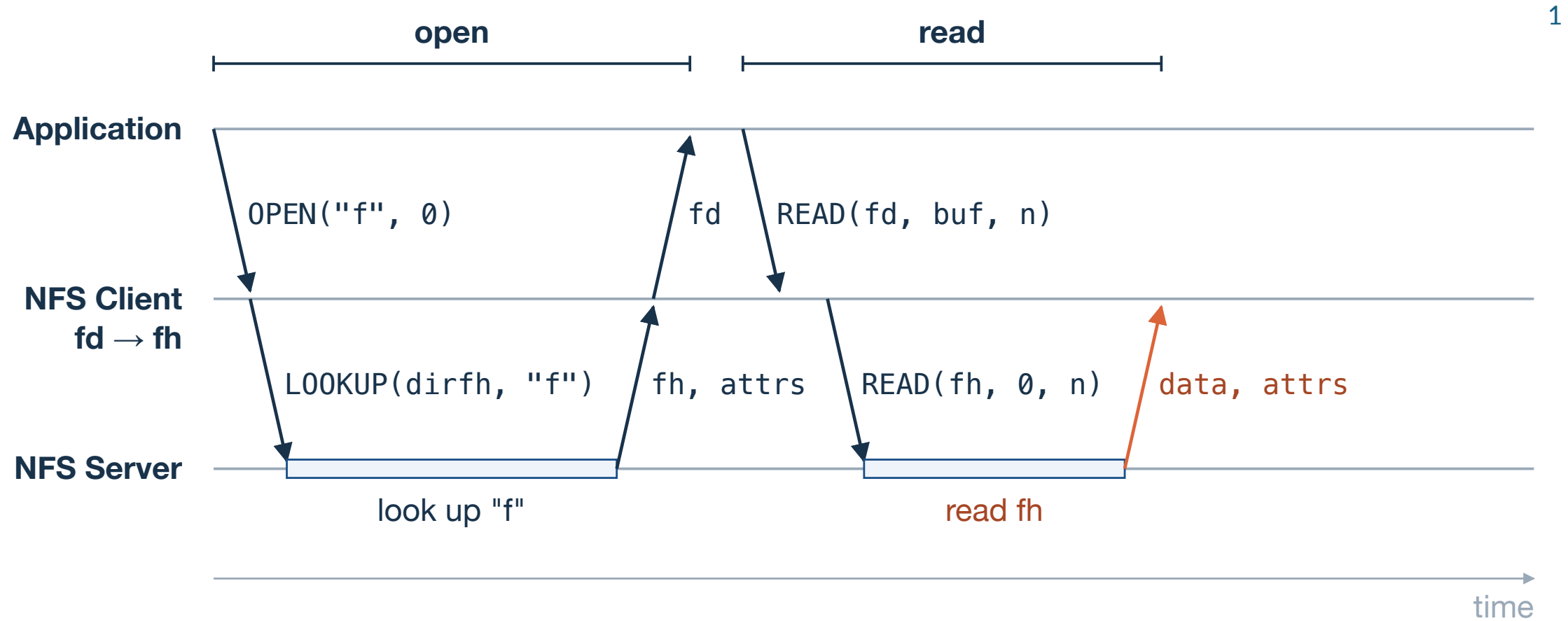
NFS protocols: **read** a file



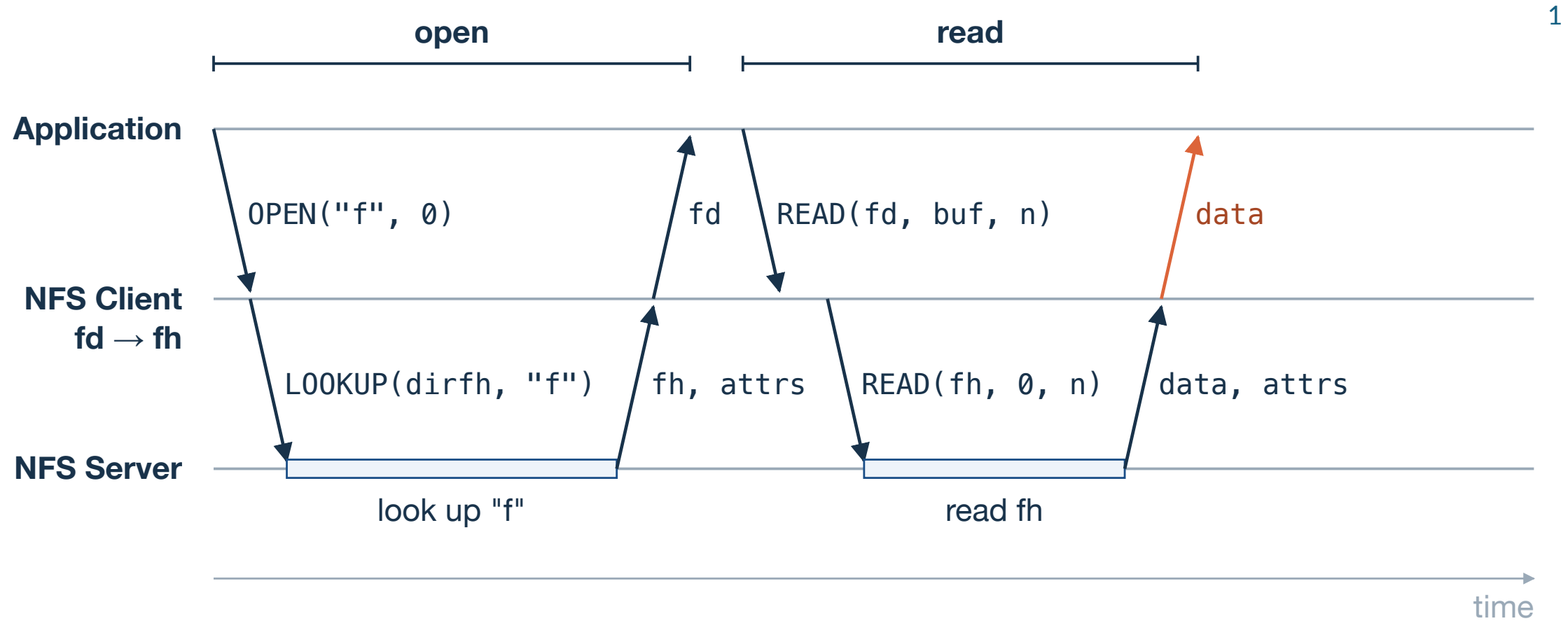
NFS protocols: **read** a file



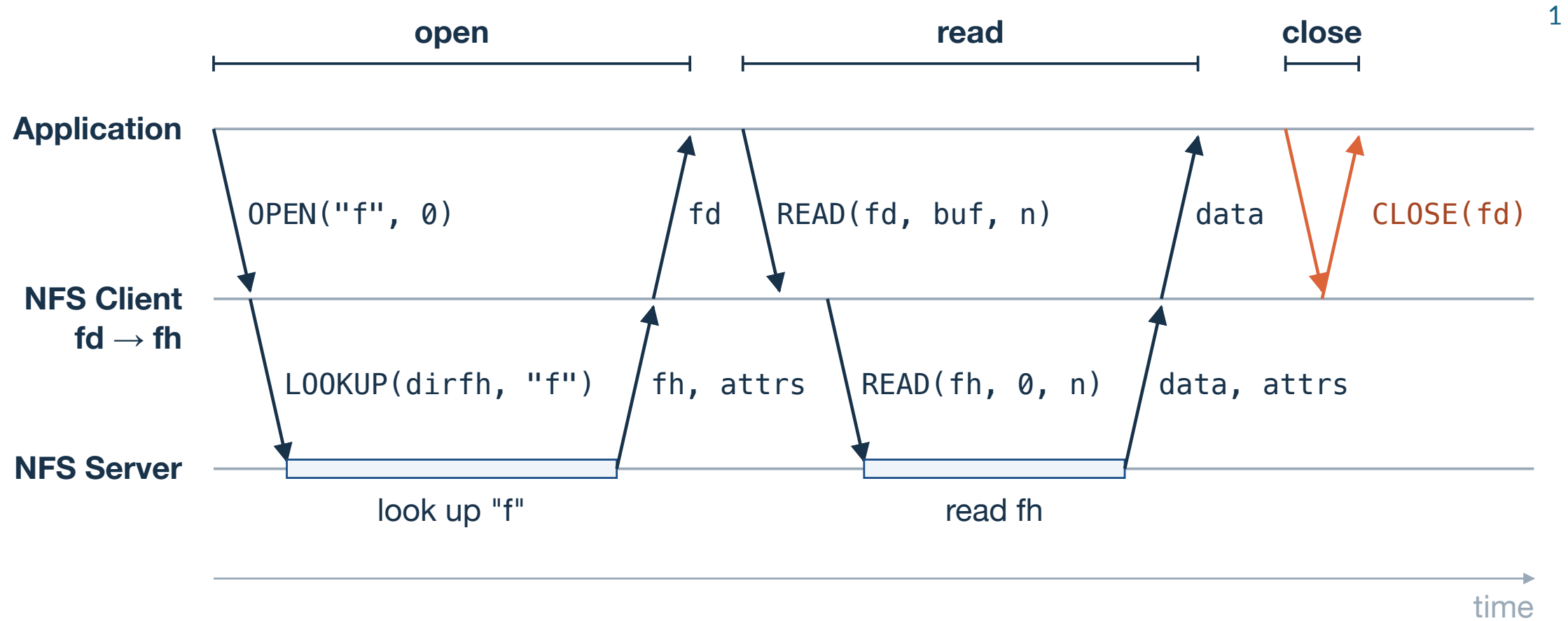
NFS protocols: **read** a file



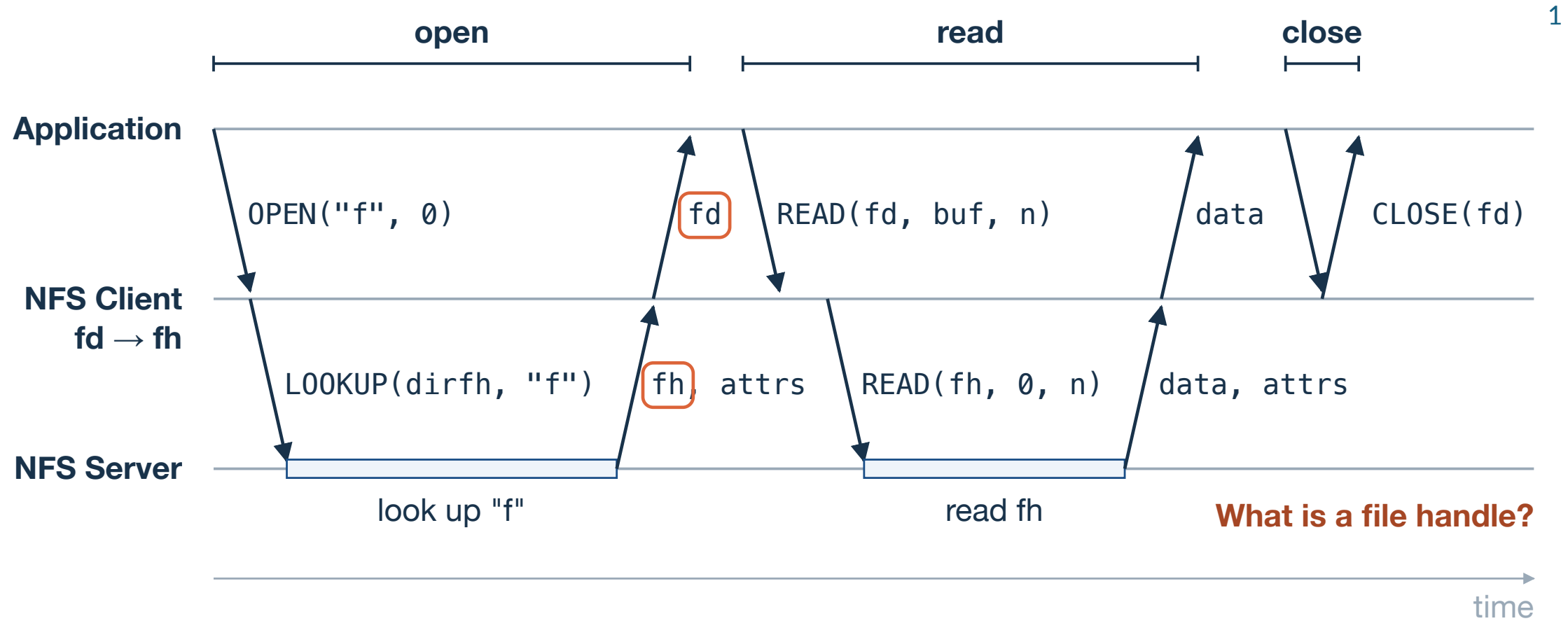
NFS protocols: **read** a file



NFS protocols: **read** a file



NFS protocols: **read** a file



What is a file handle?

Basically, it is used to name the remote file

- I.e., identifying which file to access on the server
- Question: how to implement it?

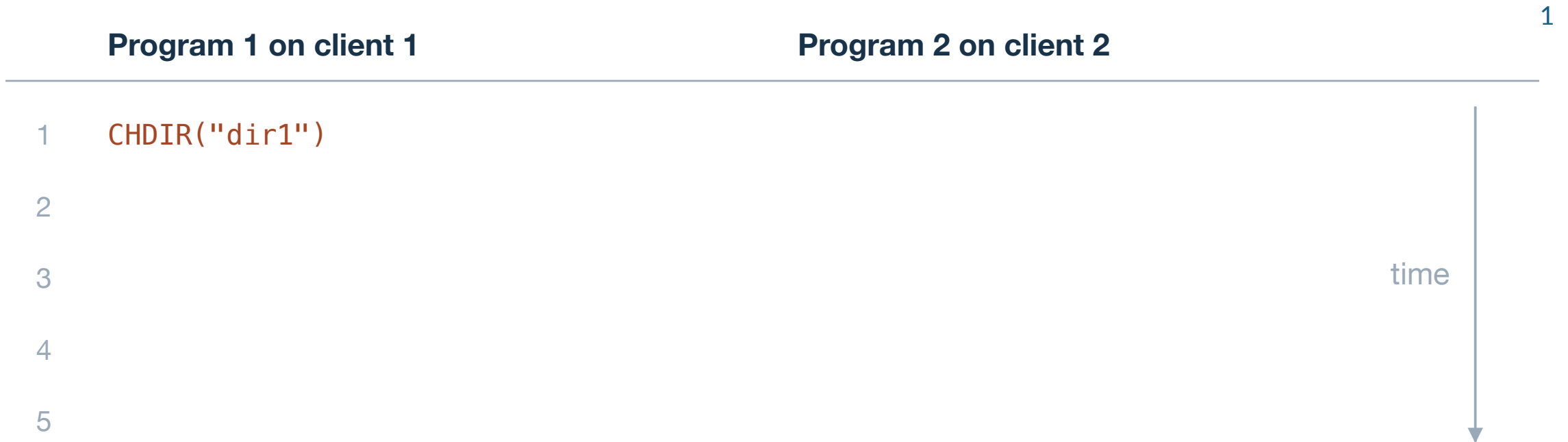
We cannot use the fd, because the server has no file table

What are the other options?

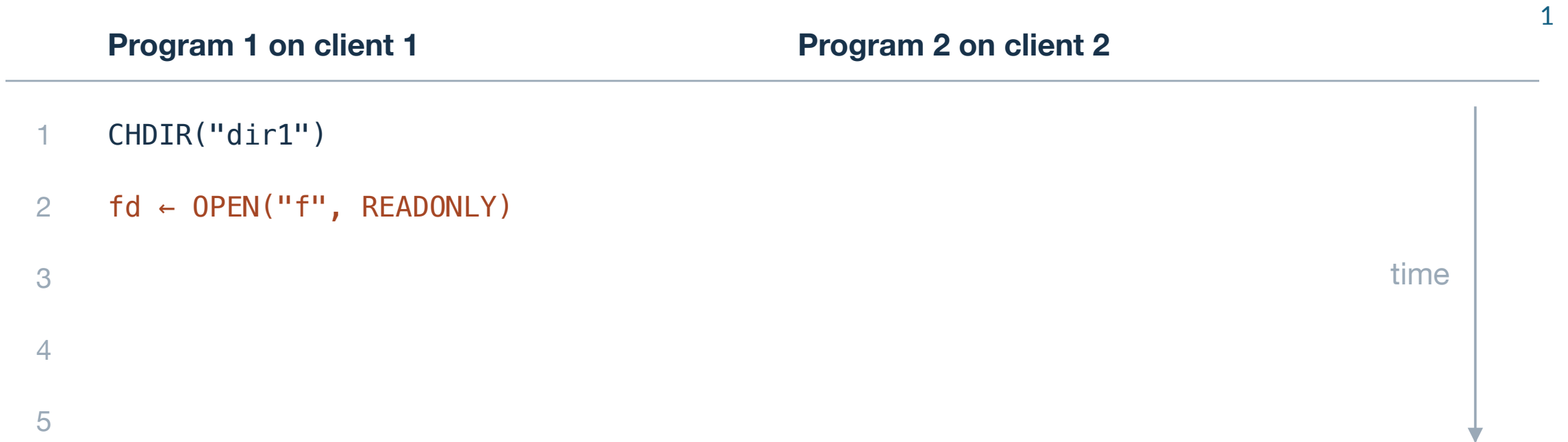
- Path name
- Inode number

Q: Can we use the path name in the handle?

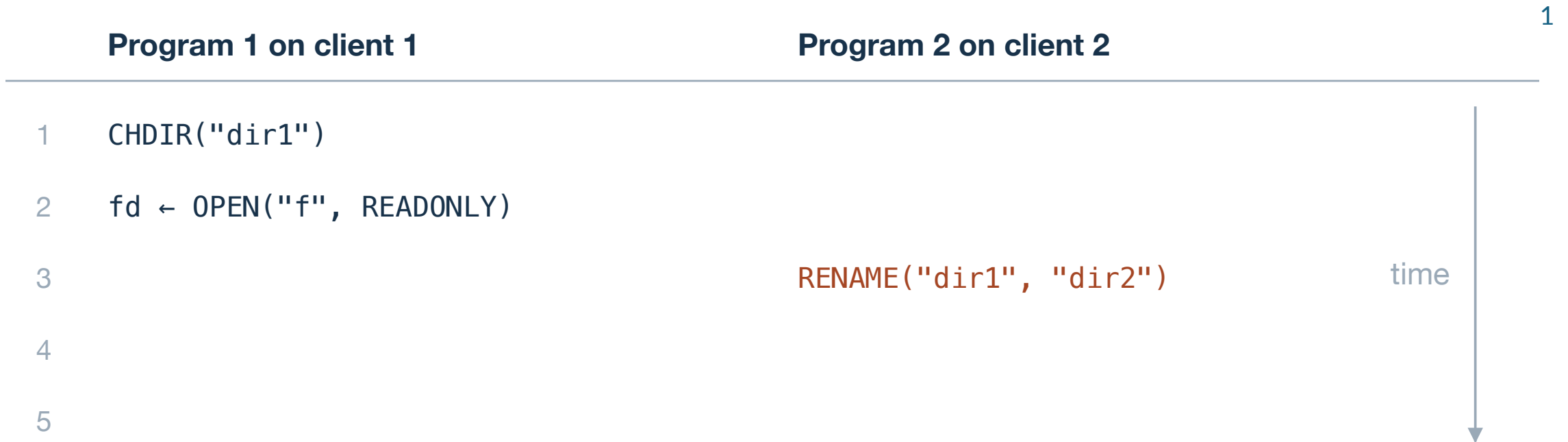
Case 1: rename after open



Case 1: rename after open



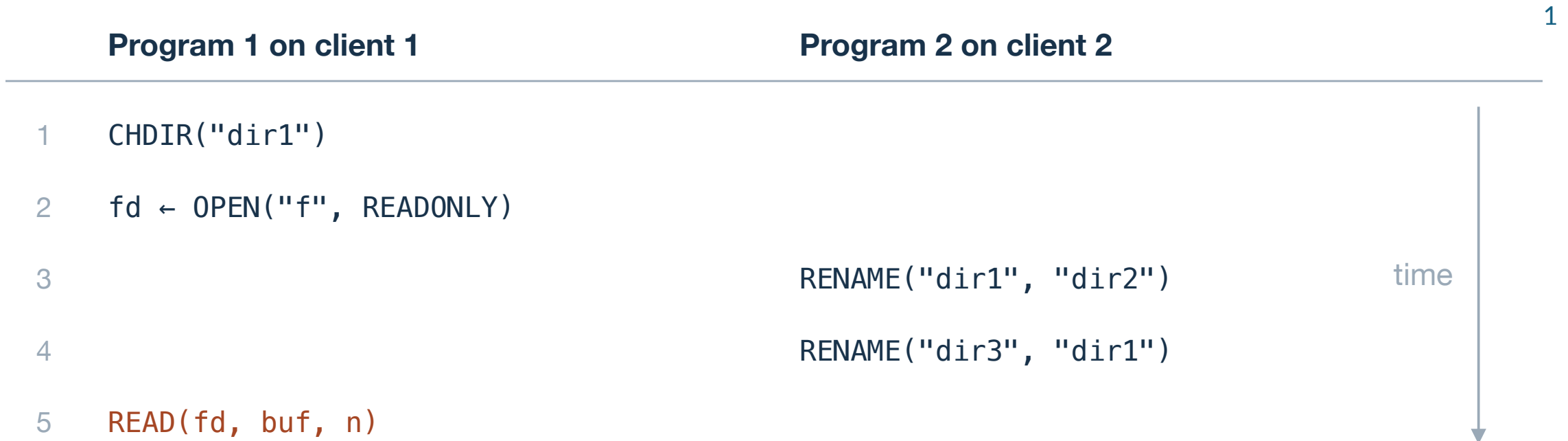
Case 1: rename after open



Case 1: rename after open



Case 1: rename after open



UNIX spec

- Program 1 should read "dir2/f"
- NFS should keep the spec, but using the path name is insufficient

What is a file handle?

Basically, it is a name

- I.e., which file to access on the server

We cannot use the fd, because the server has no file table

What are the other options?

- Path name
- Inode number

Q: Can we directly use the inode number?

Case 2: delete after open



Remedy: let program 2 read the previous file, or return an error

- Cannot read the wrong file!

Case 2: delete after open

Program 1 on client 1	Program 2 on client 2	NFS server
1	<code>fd ← OPEN("f", READONLY)</code>	<code>LOOKUP → fh (7, gen 1)</code>
2 <code>UNLINK("f")</code>		
3 <code>fd ← OPEN("f", CREATE)</code>		
4	<code>READ(fd, buf, n)</code>	

Case 2: delete after open

		1	
	Program 1 on client 1	Program 2 on client 2	NFS server
1		<code>fd ← OPEN("f", READONLY)</code>	<code>LOOKUP → fh (7, gen 1)</code>
2	<code>UNLINK("f")</code>		<code>REMOVE → free inode 7</code>
3	<code>fd ← OPEN("f", CREATE)</code>		
4		<code>READ(fd, buf, n)</code>	

Case 2: delete after open

			1
Program 1 on client 1	Program 2 on client 2	NFS server	
1	fd ← OPEN("f", READONLY)	LOOKUP → fh (7, gen 1)	
2 UNLINK("f")		REMOVE → free inode 7	
3 fd ← OPEN("f", CREATE)		CREATE → fh (7, gen 2)	
4	READ(fd, buf, n)		

Case 2: delete after open

1			
Program 1 on client 1	Program 2 on client 2	NFS server	
1	fd ← OPEN("f", READONLY)	LOOKUP → fh (7, gen 1)	
2 UNLINK("f")		REMOVE → free inode 7	
3 fd ← OPEN("f", CREATE)		CREATE → fh (7, gen 2)	
4	READ(fd, buf, n)	READ fh (7, gen 1) gen 1 ≠ 2 → error	

How to avoid program 2 reading the wrong file?

- **Generation number**: a number incremented upon assigning an inode number to a new file
- The fh contains both the inode number & the generation number
- If the `read`'s fh does not match the current generation, then abort

Case 2: delete after open

			1
Program 1 on client 1	Program 2 on client 2	NFS server	
1	<code>fd ← OPEN("f", READONLY)</code>	LOOKUP → fh (7, gen 1)	
2 <code>UNLINK("f")</code>		REMOVE → free inode 7	
3 <code>fd ← OPEN("f", CREATE)</code>		CREATE → fh (7, gen 2)	
4	<code>READ(fd, buf, n)</code>	READ fh (7, gen 1) gen 1 ≠ 2 → error	

How to avoid program 2 reading the new file?

- Generation number: a number incremented upon assigning an inode number to a new file

UNIX spec

- On a local FS, program 2 will read the old file

Not the same as the UNIX spec! It's a **tradeoff**

Put it all together: file handle

Basically, it is a name

- I.e., which file to access on the server

We cannot use the fd, because the server has no file table

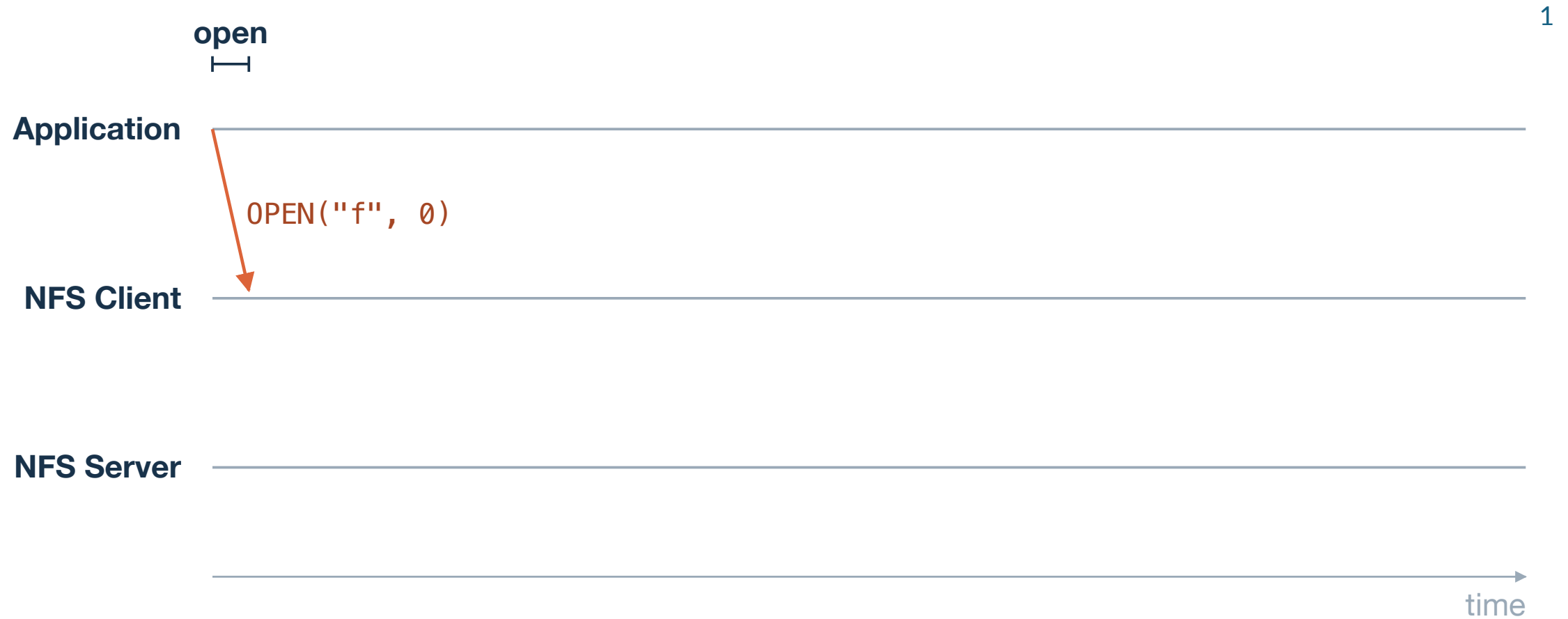
What are the other options?

- ~~Path name~~
- ~~Inode number~~
- Inode number + generation number

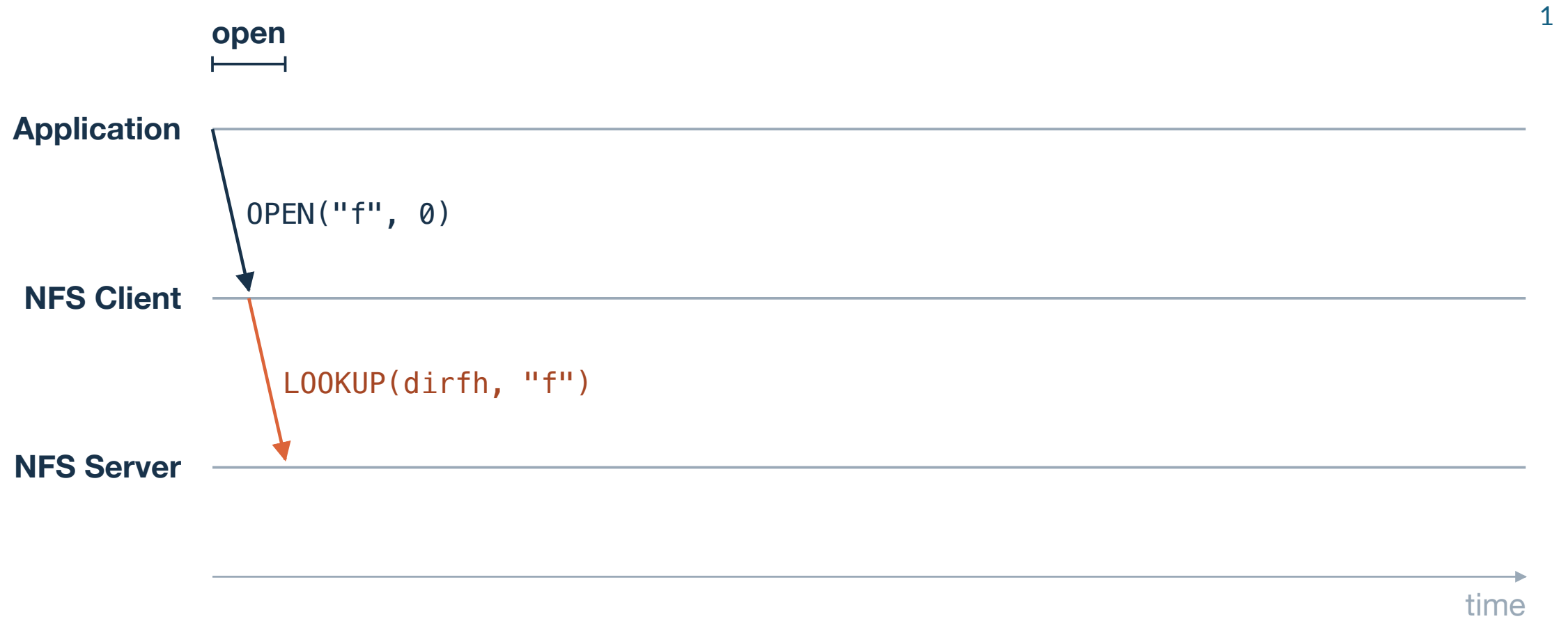
The file handle contains two main parts

- Inode number: for the server to locate the file
- Generation number: for the server to maintain the consistency of a file
- Some other less important information, e.g., the file system type

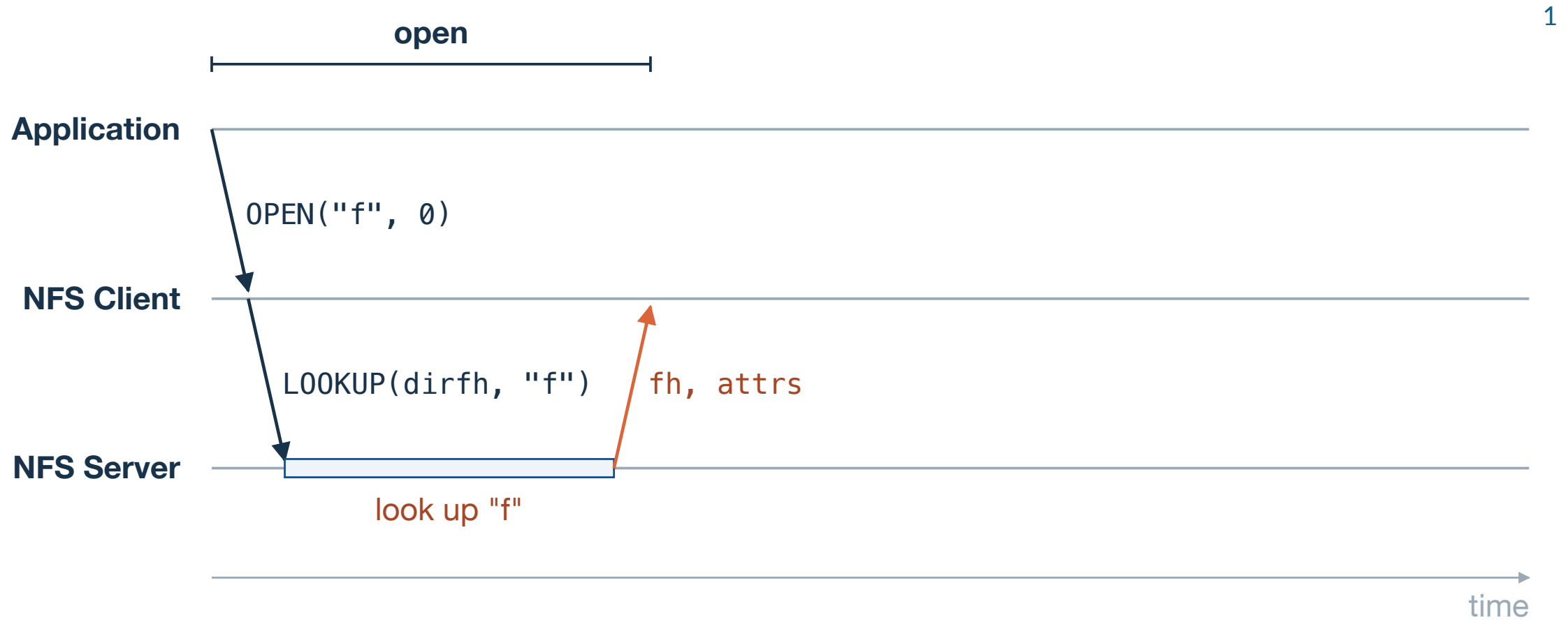
NFS protocols: read a file



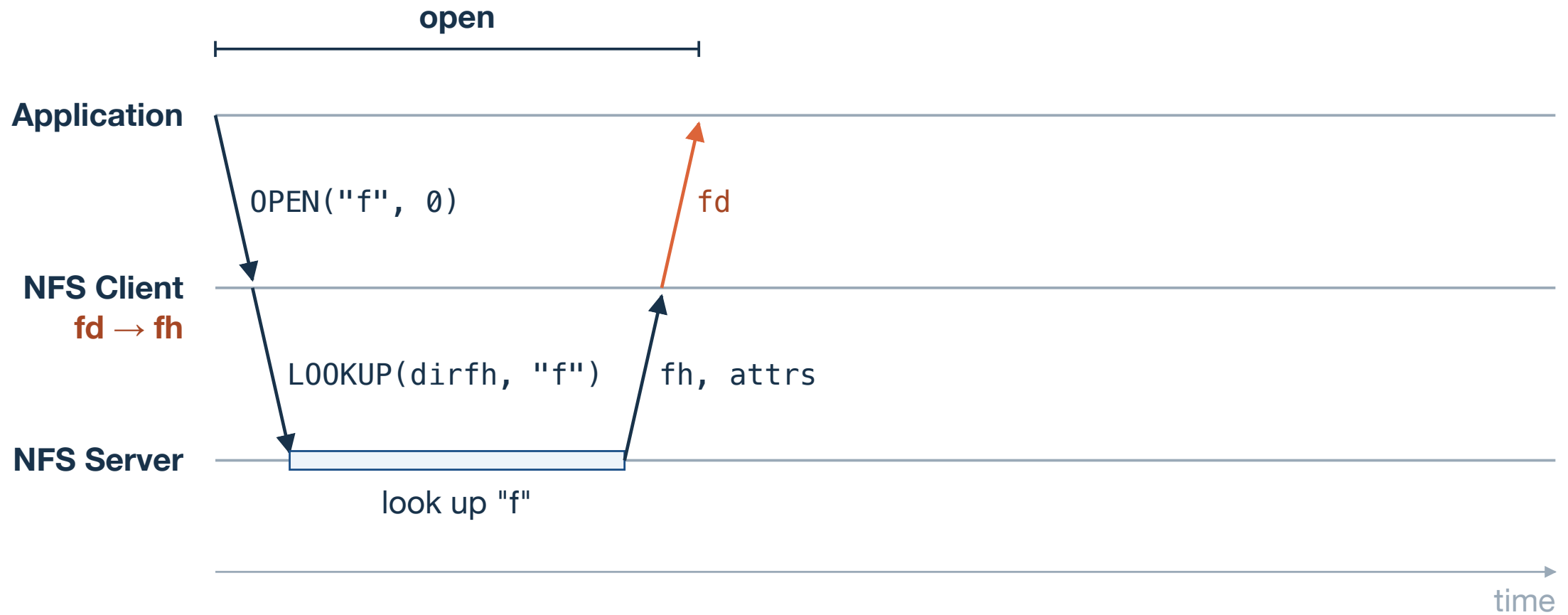
NFS protocols: **read** a file



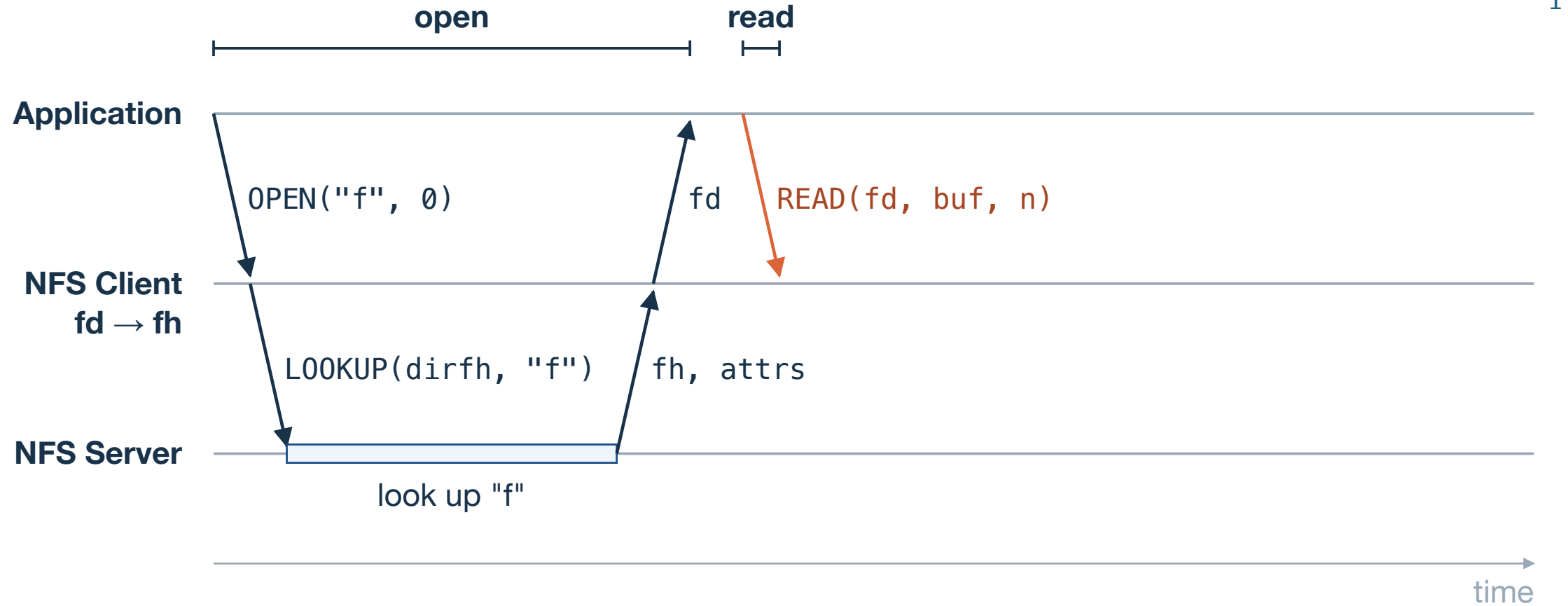
NFS protocols: **read** a file



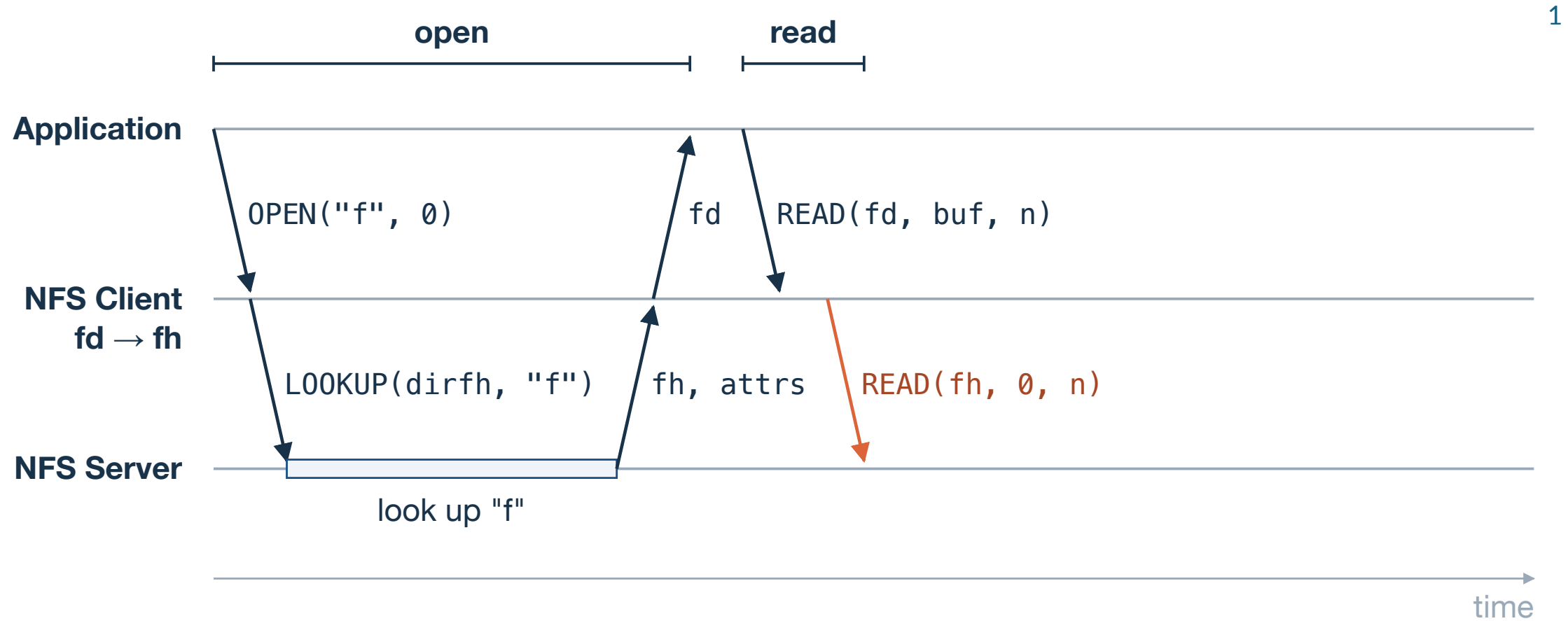
NFS protocols: **read** a file



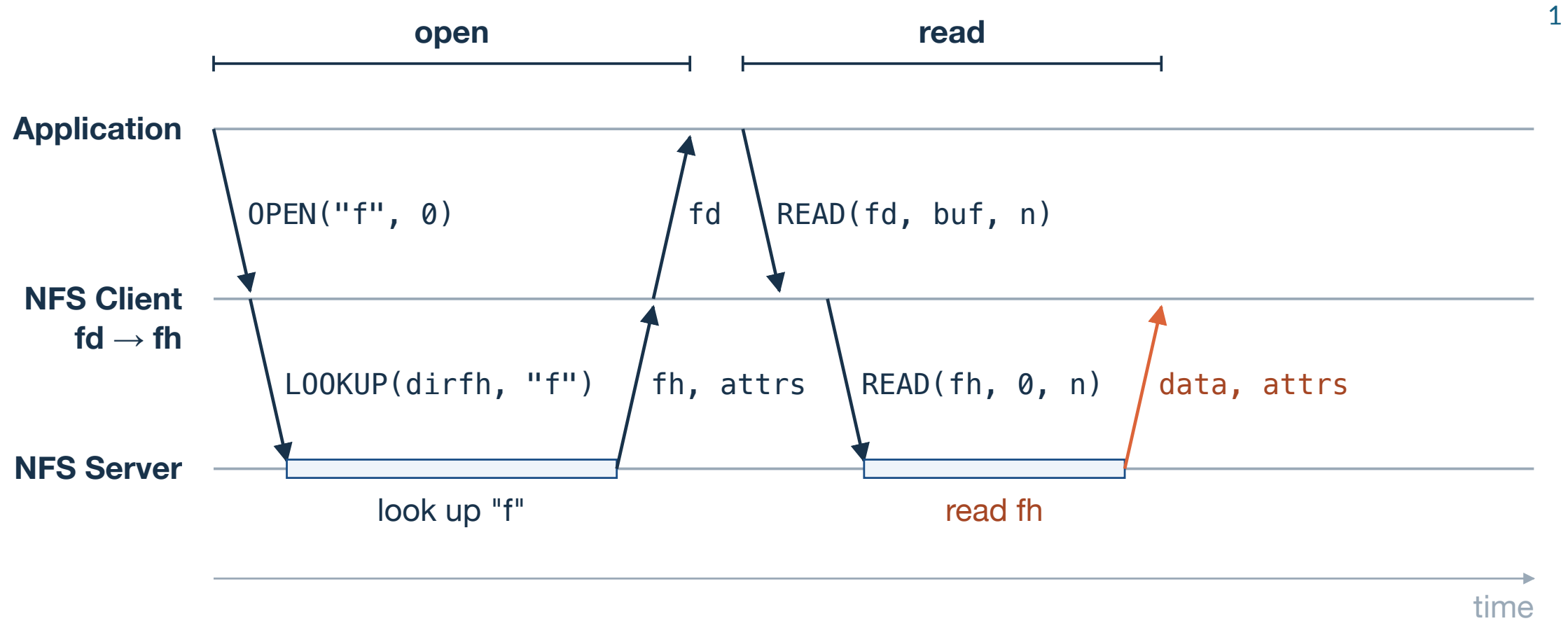
NFS protocols: **read** a file



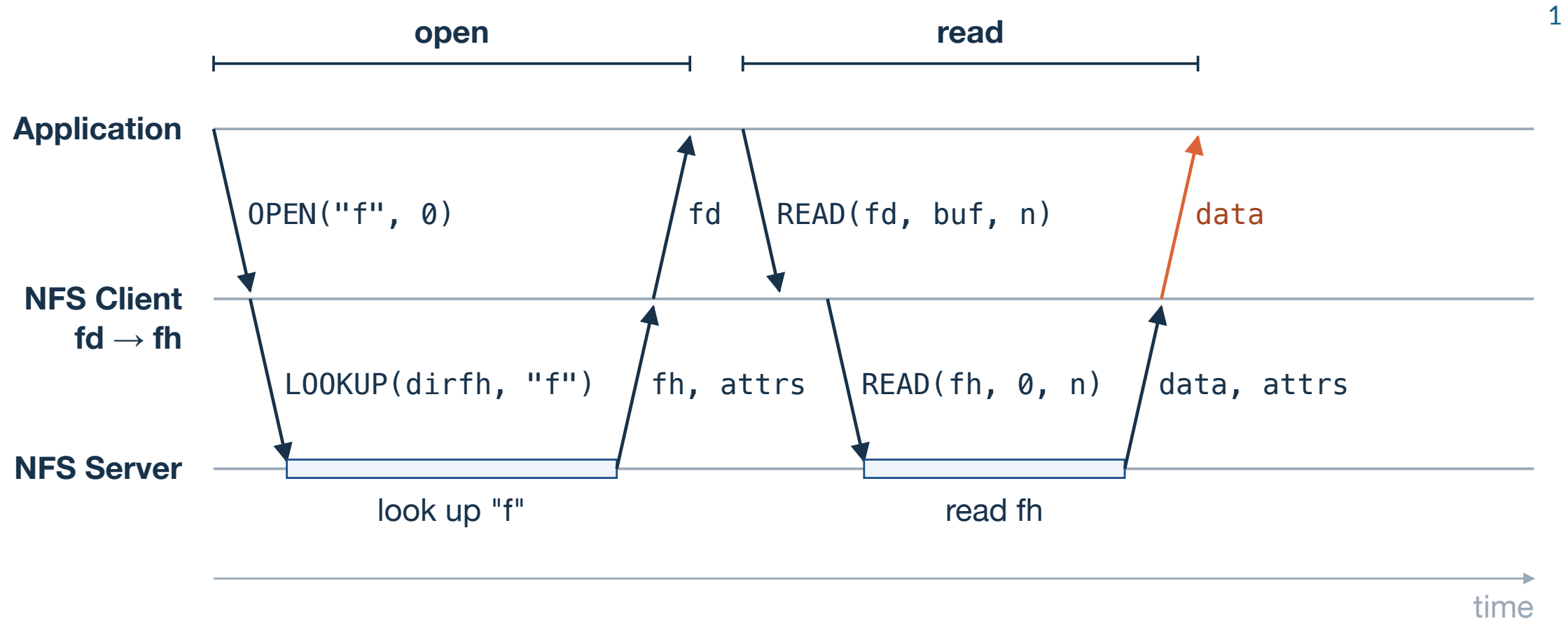
NFS protocols: **read** a file



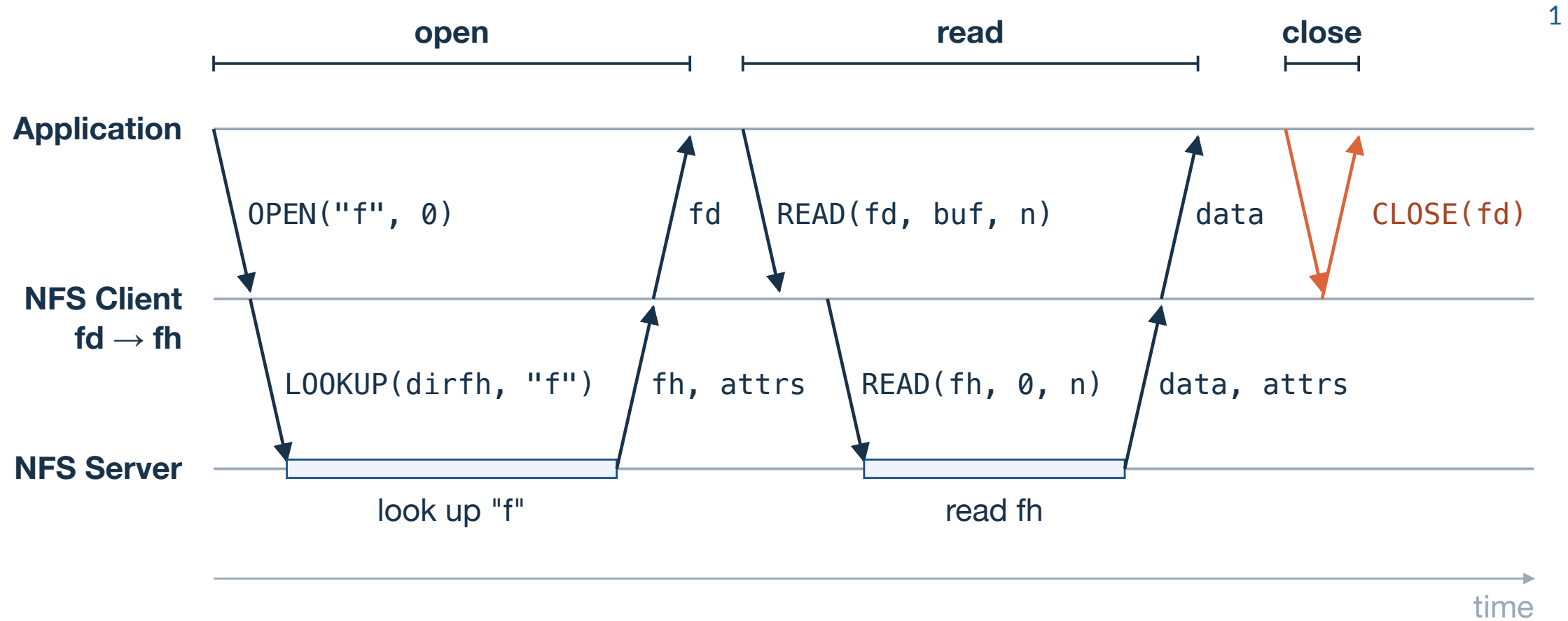
NFS protocols: **read** a file



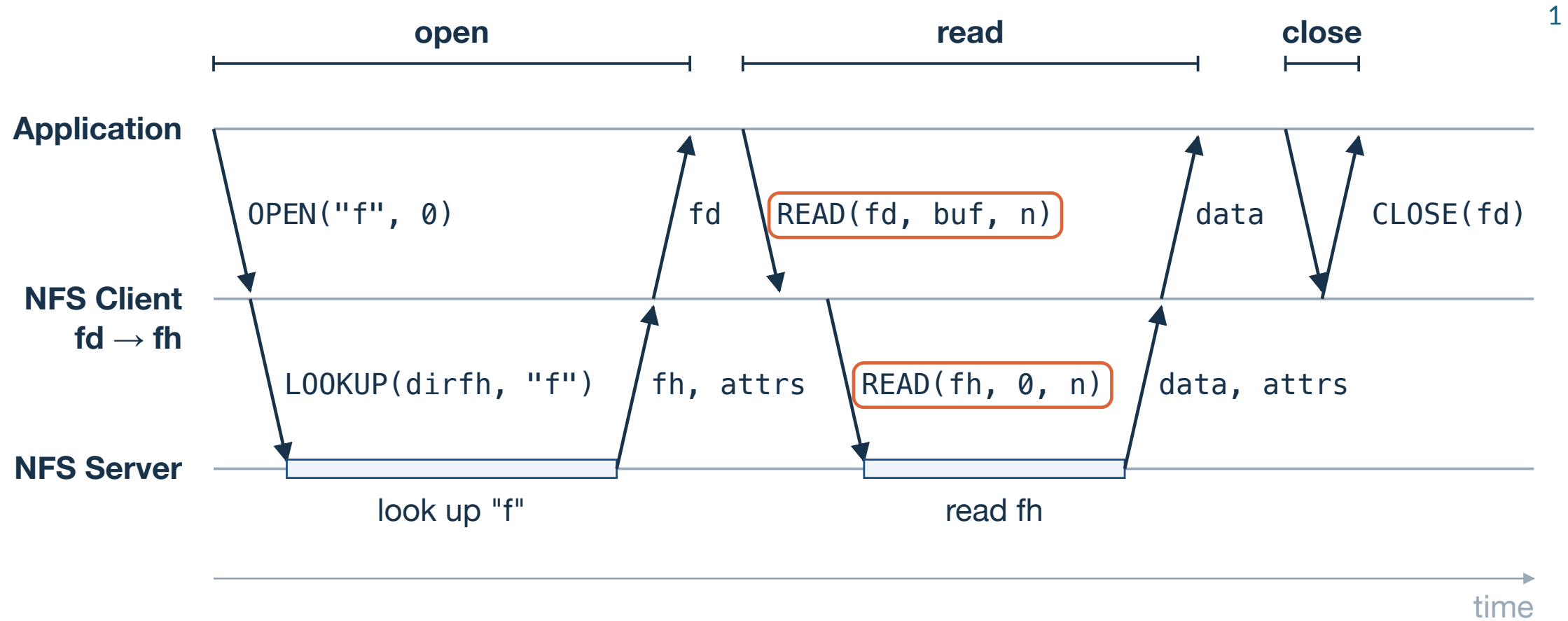
NFS protocols: **read** a file



NFS protocols: **read** a file



NFS protocols: **read** a file



Problem: performance overheads

Usually **slower** than local, but not always

- Question: depends on what?

→ File server performance & network speed

Optimization: **caching** at the client

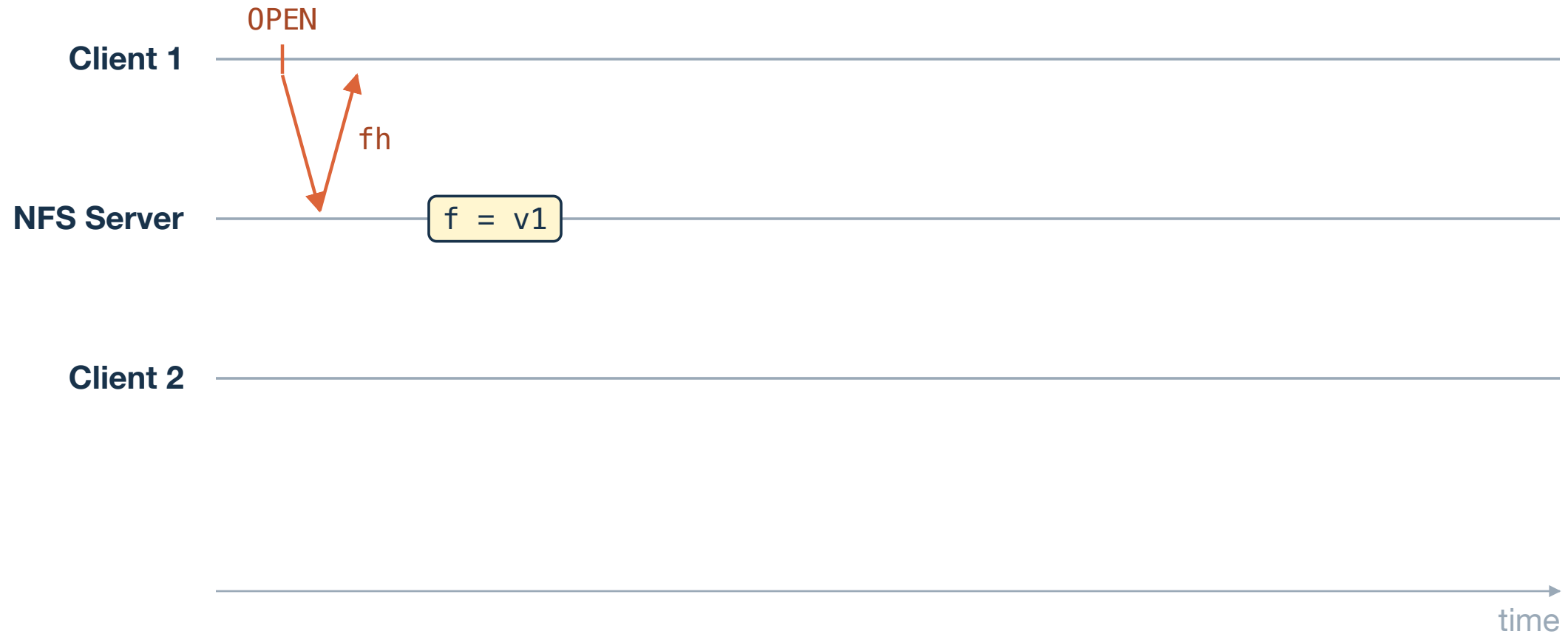
- Goal: reduce the number of remote ops
- Caching: `read`, `readlink`, `getattr`, `lookup`, `readdir`
 1. Cache file data at the client (buffer cache)
 2. Cache file attribute information at the client
 3. Cache pathname bindings for faster lookup

Server side

- Caching is "automatic" via the buffer cache

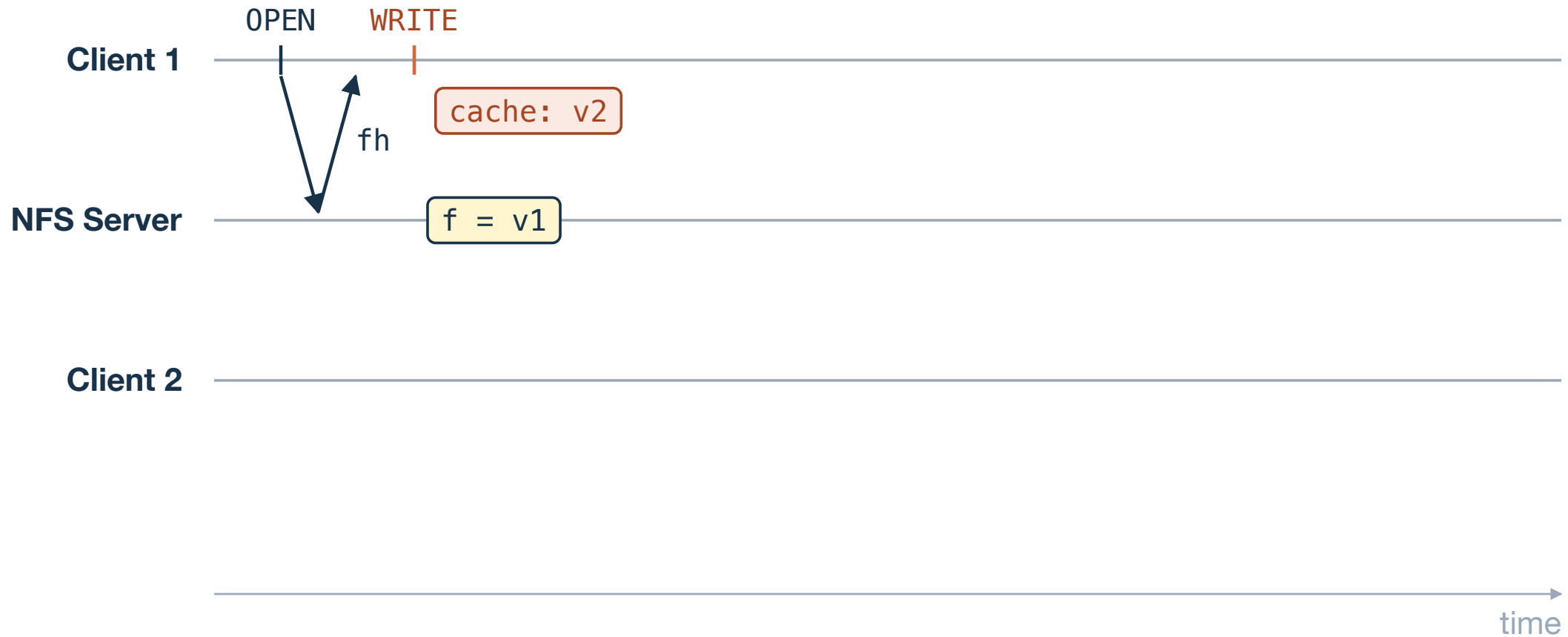
Challenge of caching: cache coherence

1



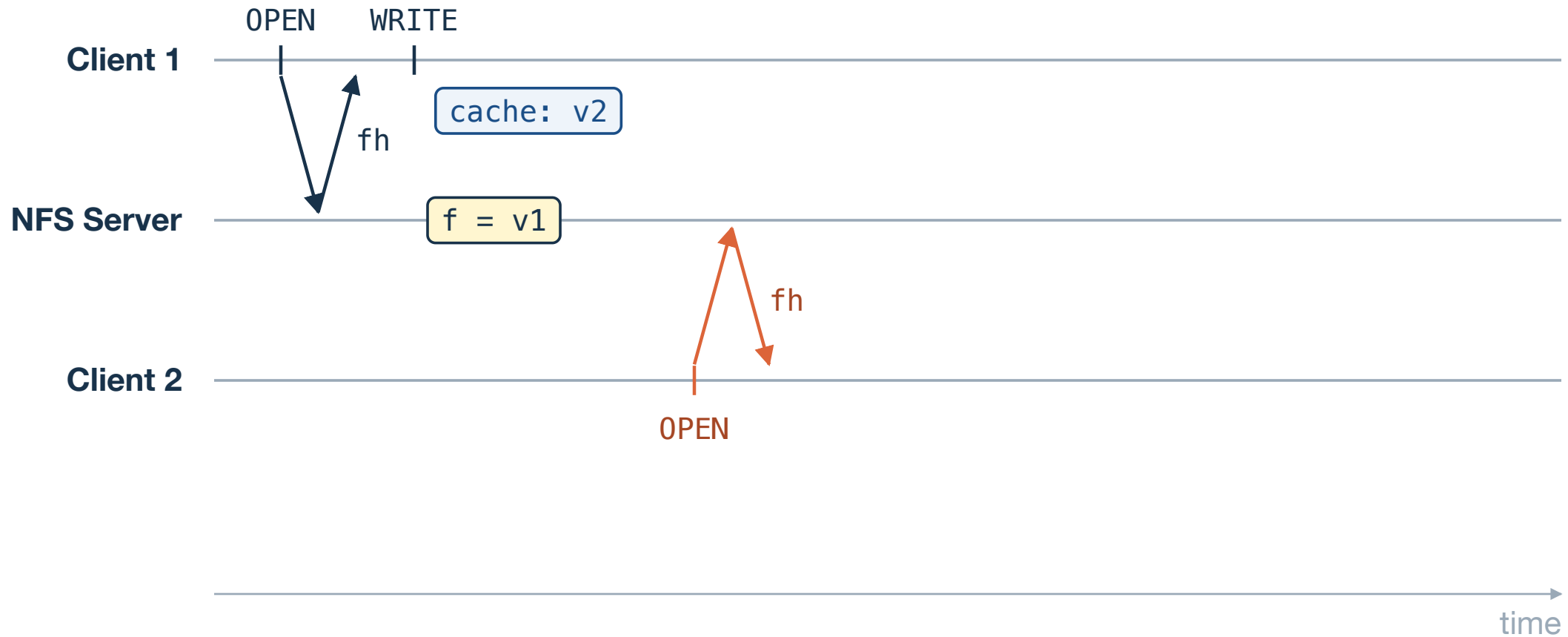
Challenge of caching: cache coherence

1



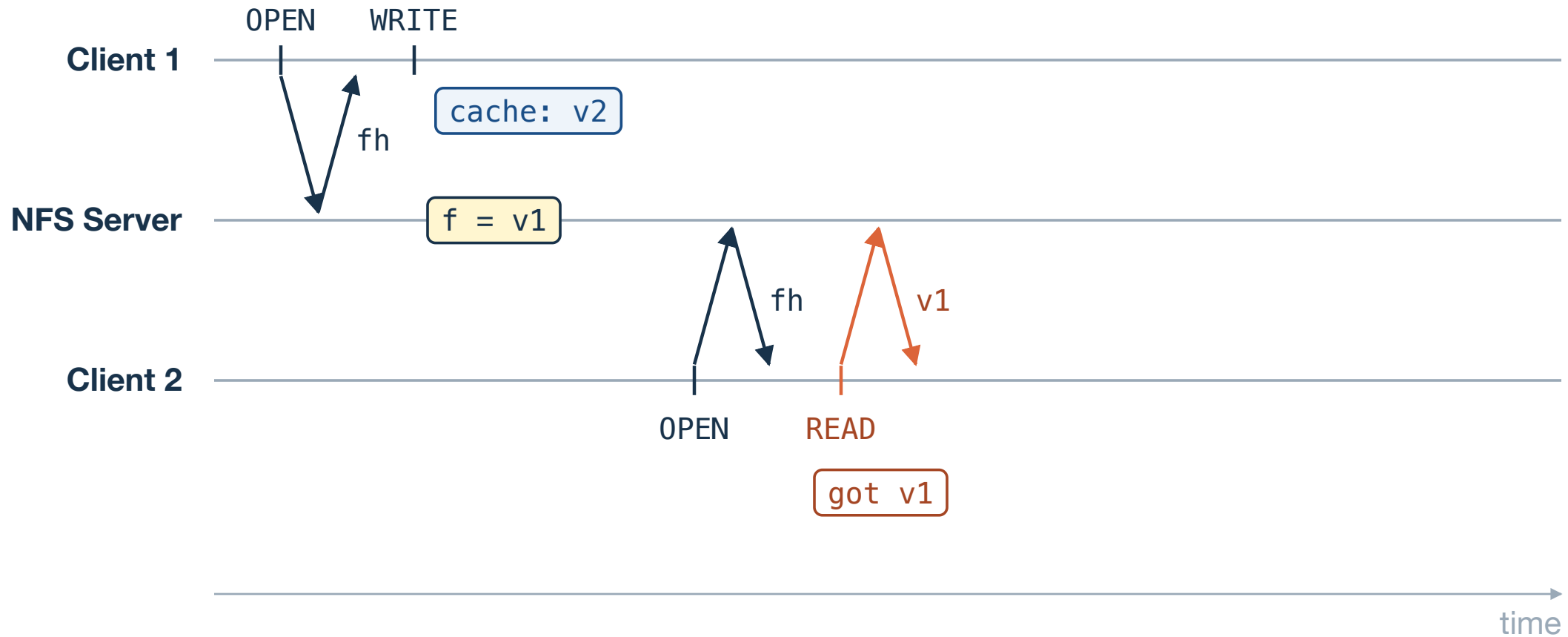
Challenge of caching: cache coherence

1



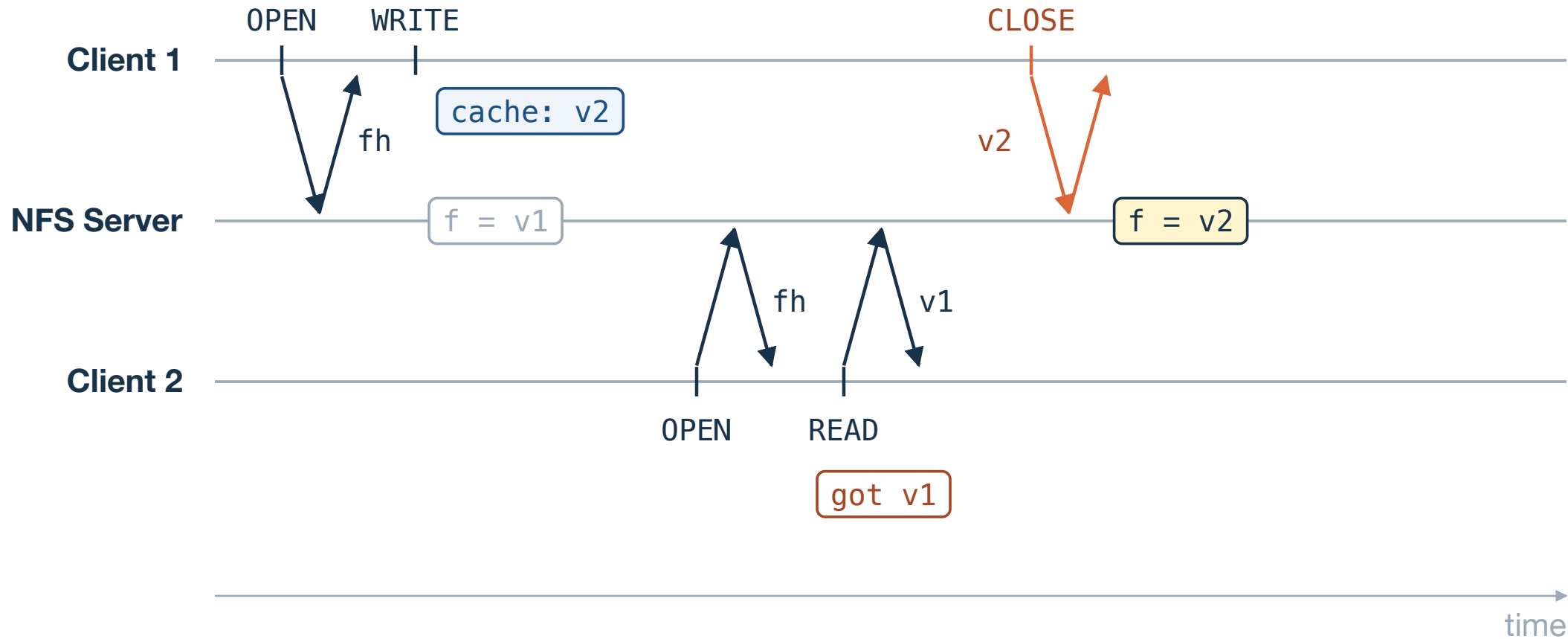
Challenge of caching: cache coherence

1



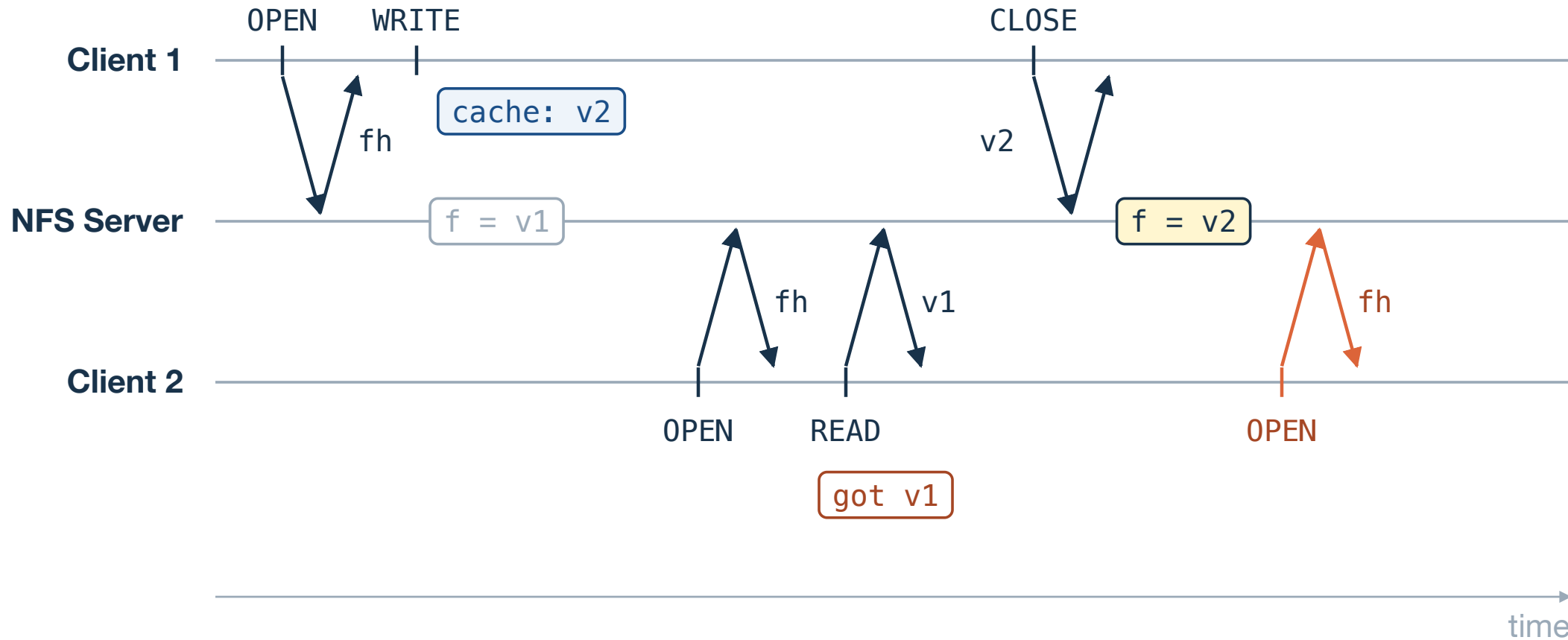
Challenge of caching: cache coherence

1



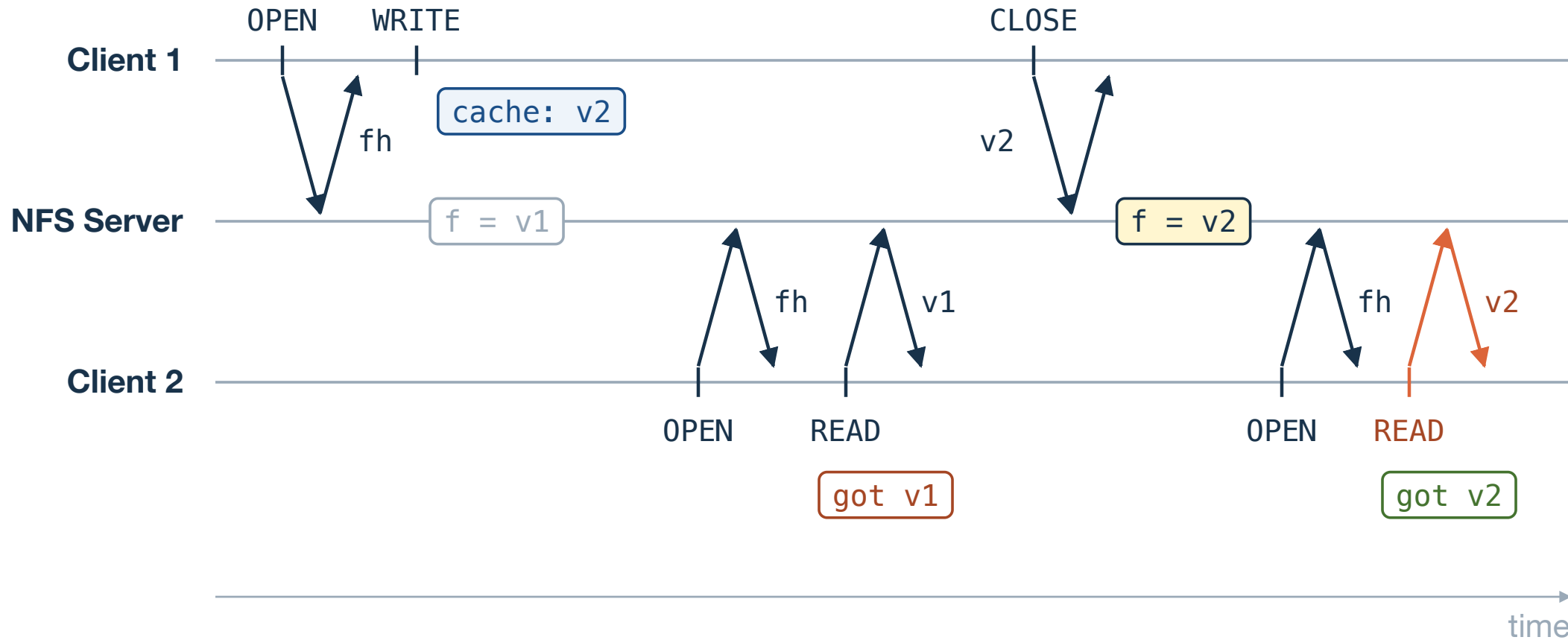
Challenge of caching: cache coherence

1



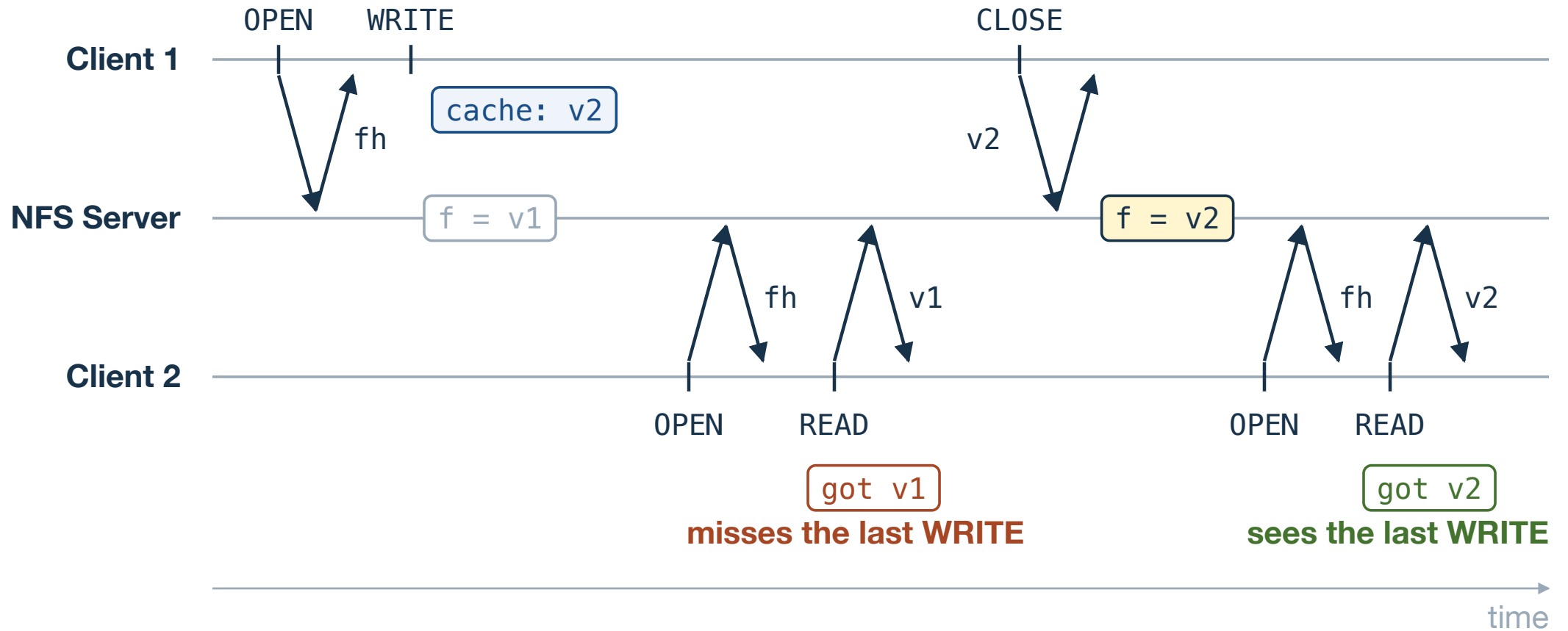
Challenge of caching: cache coherence

1



Challenge of caching: cache coherence

1



Challenge of caching: cache coherence

Type 1: close-to-open consistency

- Higher data rate
- `GETATTR` on `OPEN`, to get the last modification time
- Compare the time with its cache
- On `CLOSE`, send cached writes to the server

Type 2: read/write coherence

- On a local file system, `READ` gets the newest data
- On NFS, the client has a cache
- NFS could guarantee read/write coherence for every operation, or just for certain operations

Handling inconsistency

Inconsistencies may arise in NFS

For close-to-open consistency

- Flush the cache upon `close`
- If a data block is modified, it is marked `dirty`, then flushed on file `close`

For read/write coherence

- Resolve `inconsistencies` with `validation`
- Both the server and the client save `timestamps` of files
- When a file is opened or the server is contacted for new data
 - Compare the last modification time
 - If the remote one is more recent, invalidate the cached data
- Always `invalidate` data after some time: open files (3 s), directories (30 s)

Improving read performance

Transfer data in large chunks

- 8 KB by default

Read-ahead

- Optimize for sequential file access
- Send requests to read disk blocks before the applications request them

Not the end of the story: NFS is continuously improving

Version 3	Version 4
User-level lock manager	More state: control of caching, notification of file changes
NVRAM support	Server exports a single name space (pseudo file system)
Adjust RPC retries dynamically	Compound RPC support
Client-side disk caching	Extended attributes and ACLs
Support 64-bit file sizes	Negotiate the security mechanism on mount
TCP support and large-block transfers	...
Commit operation	
...	

Reference materials

RFC 1831: RPC specification

- <https://www.rfc-editor.org/rfc/rfc1831>

RFC 1832: XDR specification

- <https://www.rfc-editor.org/rfc/rfc1832>

Case study: GFS

The Google File System (SOSP 2003)

Design assumptions: environments

Assumptions for **conventional** file systems **don't work**

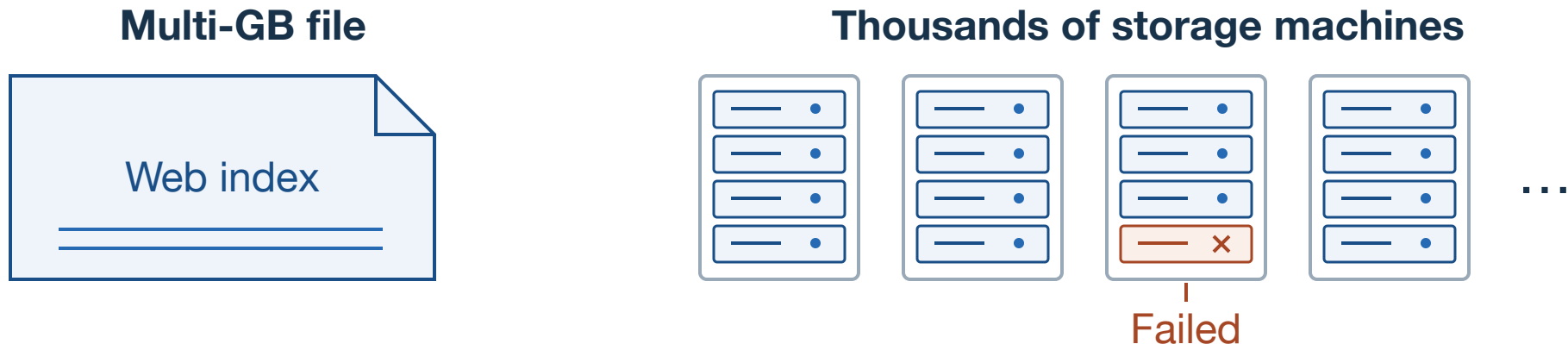
- E.g., "most files are small", "short lifetimes"

Files are huge: multi-GB files are common, e.g., a large web index

- I/O ops and block size choices are affected

Component **failures** are the **norm**, not an exception

- File system = thousands of storage machines
- Some % not working at any given time (recall "A story of scaling")



More design assumptions: file access

Most files are **appended**, not overwritten (why? E.g., add new web pages to Google's global store)

- **Random** writes within a file are **rare**
 - Means we need to provide efficient **appends**

Workload

- Mostly reads: large streaming reads
 - Once created, files are **mostly read**; often **sequential** (e.g., read a previously archived page)

We should design the FS API together with the apps: it benefits the system

GFS design goals

Scalable distributed file system

- E.g., can NFS store a very large file?

Designed for large data-intensive applications

- Essential for distributed computing frameworks, e.g., MapReduce

Fault-tolerant; runs on commodity hardware

- E.g., what if a server crashes, in the case of NFS?

Delivers high performance to a large number of clients

GFS interface

GFS does **not** have a **standard OS-level API**

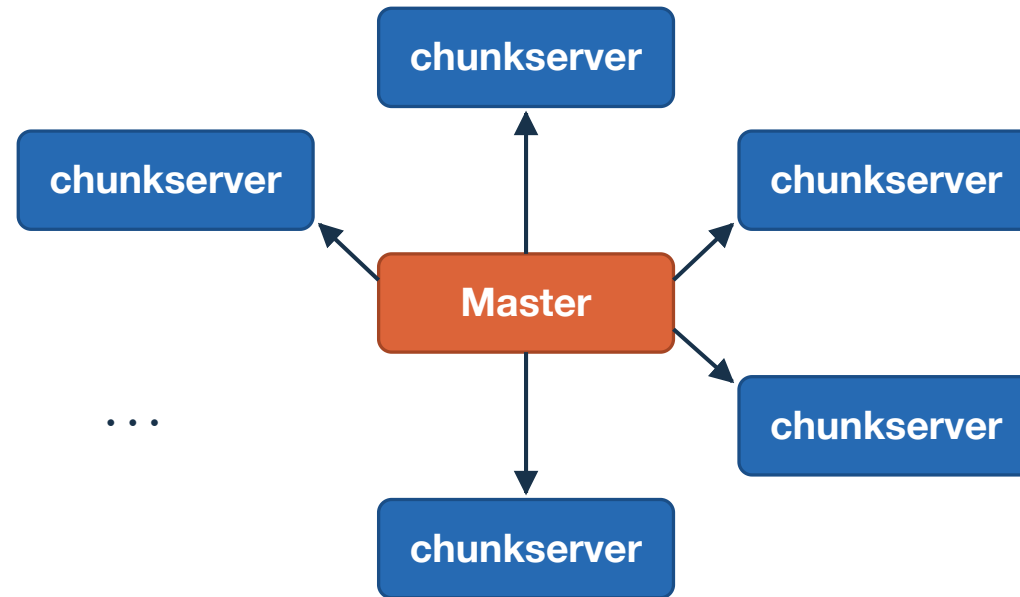
- No full POSIX API (though it can be implemented)
- It provides a user-level API (simpler to implement)

Operations

- Basic ops: `create/delete/open/close/read/write`
- Additional ops: `snapshot/append`
- Not supported ops: `link`, `symlink`, `rename`

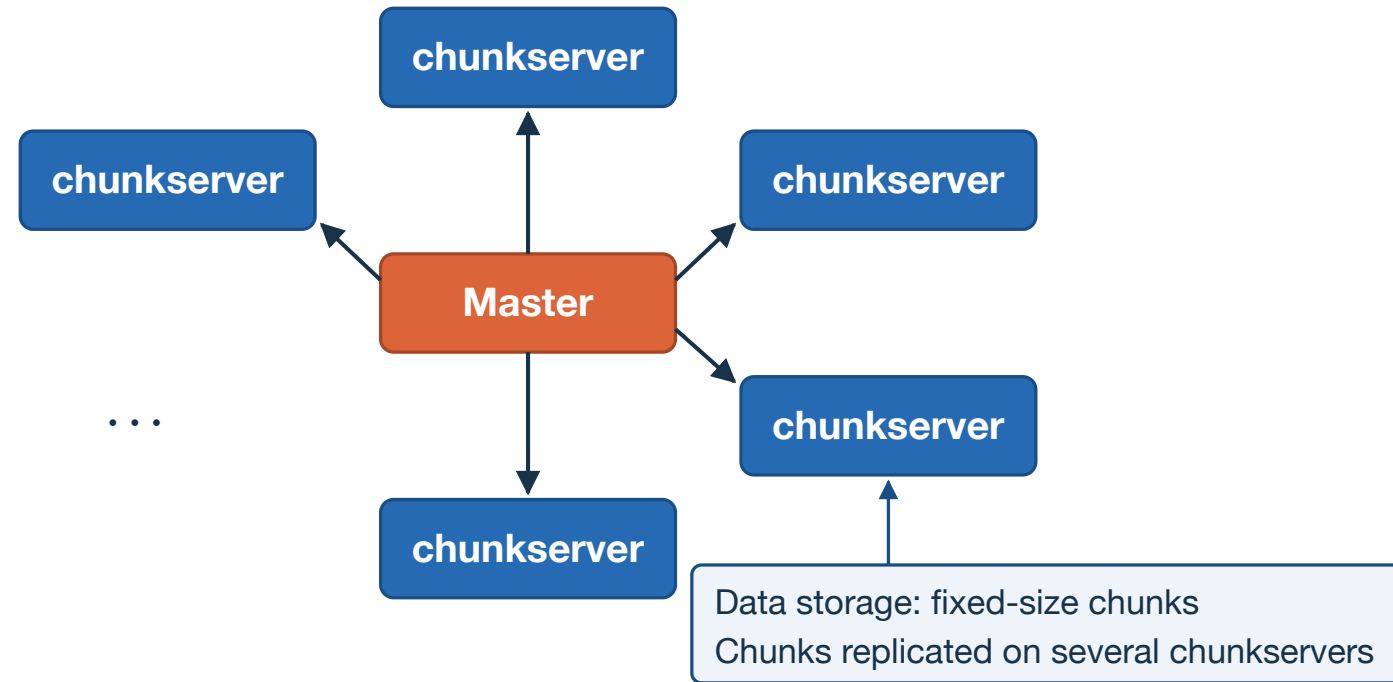
GFS architecture

A GFS cluster = **1 master** (distributed inode layer) + **N chunkservers** (distributed block layer + replication)



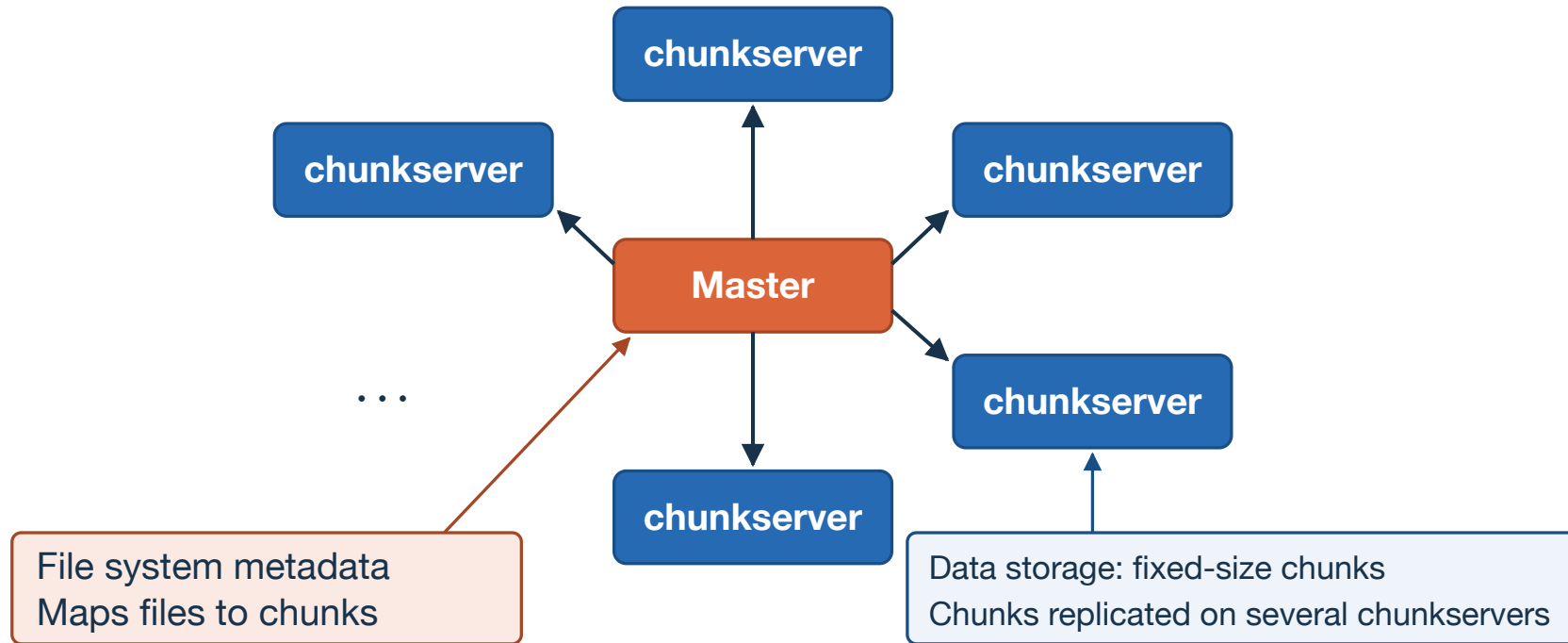
GFS architecture

A GFS cluster = **1 master** (distributed inode layer) + **N chunkservers** (distributed block layer + replication)



GFS architecture

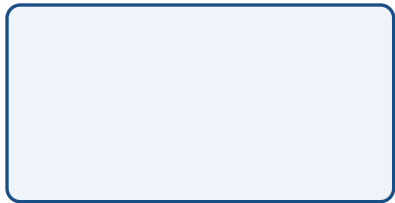
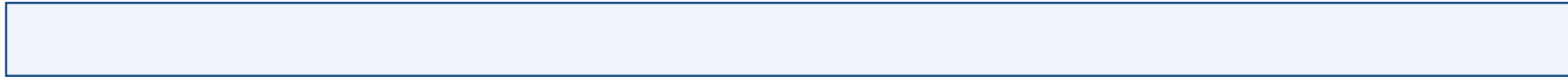
A GFS cluster = **1 master** (distributed inode layer) + **N chunkservers** (distributed block layer + replication)



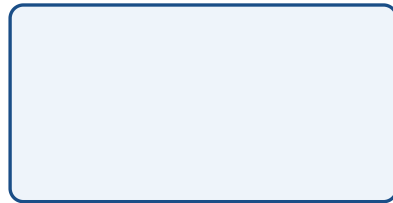
GFS files

A file is split into 64 MB chunks; each chunk is replicated on several chunkservers, and the master records where each chunk lives

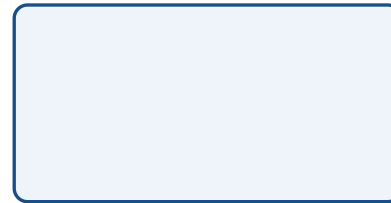
A huge file (e.g., 1 TB)



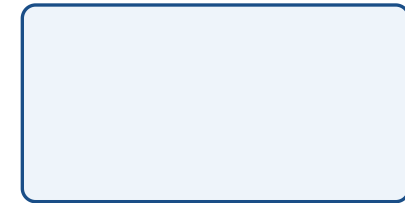
chunkserver 1



chunkserver 2

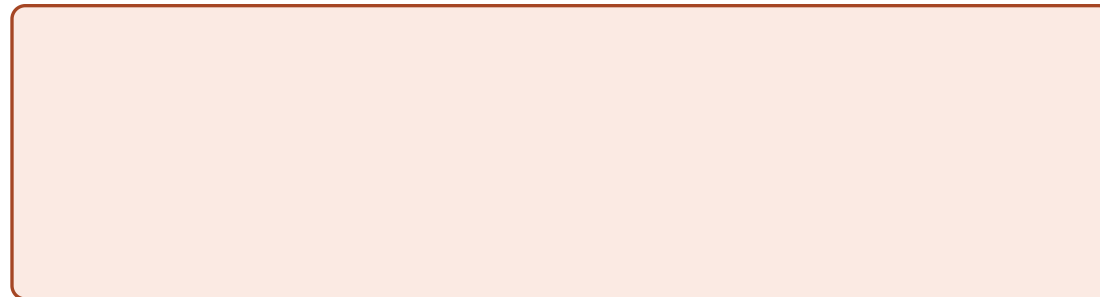


chunkserver 3



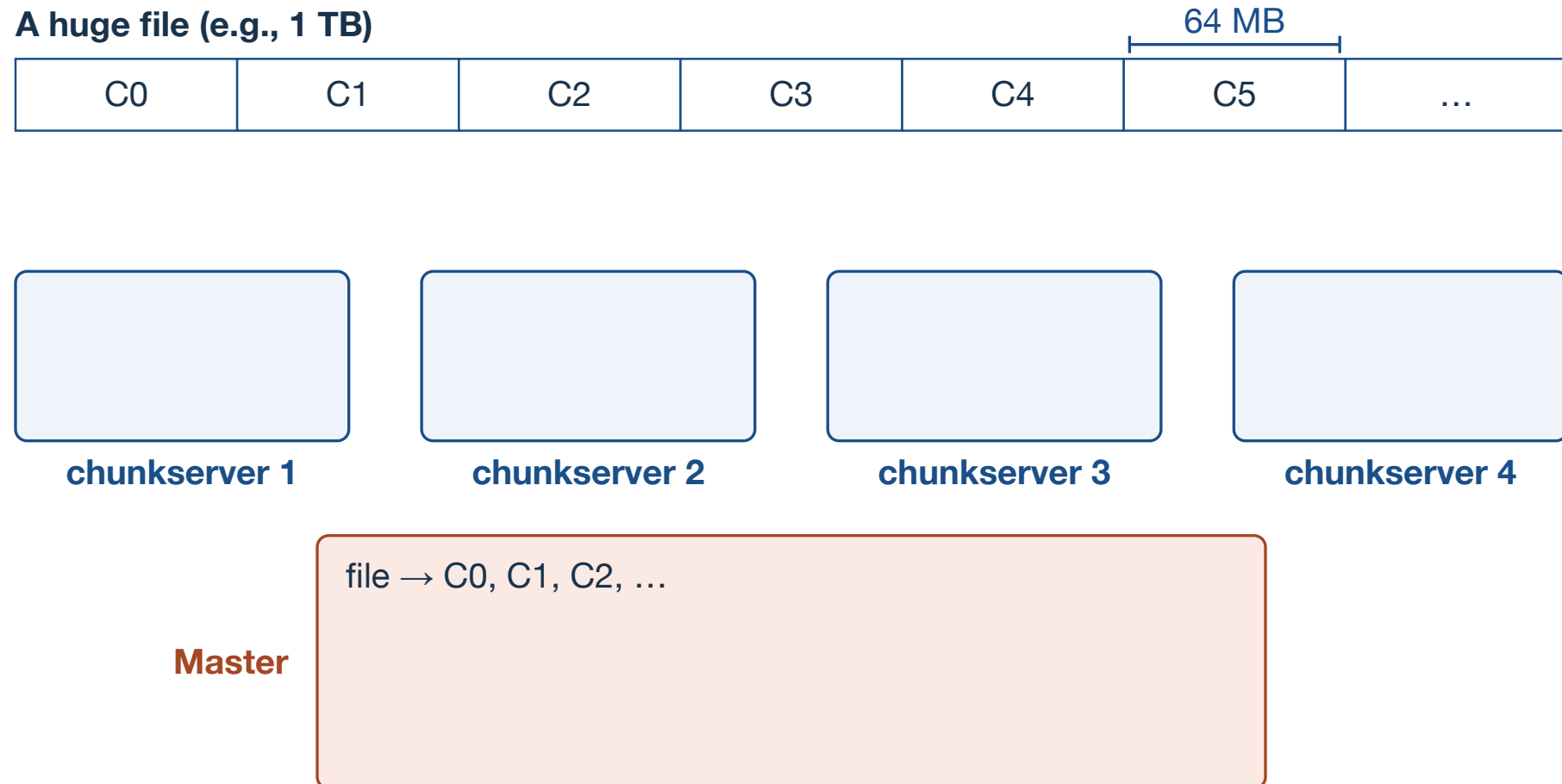
chunkserver 4

Master



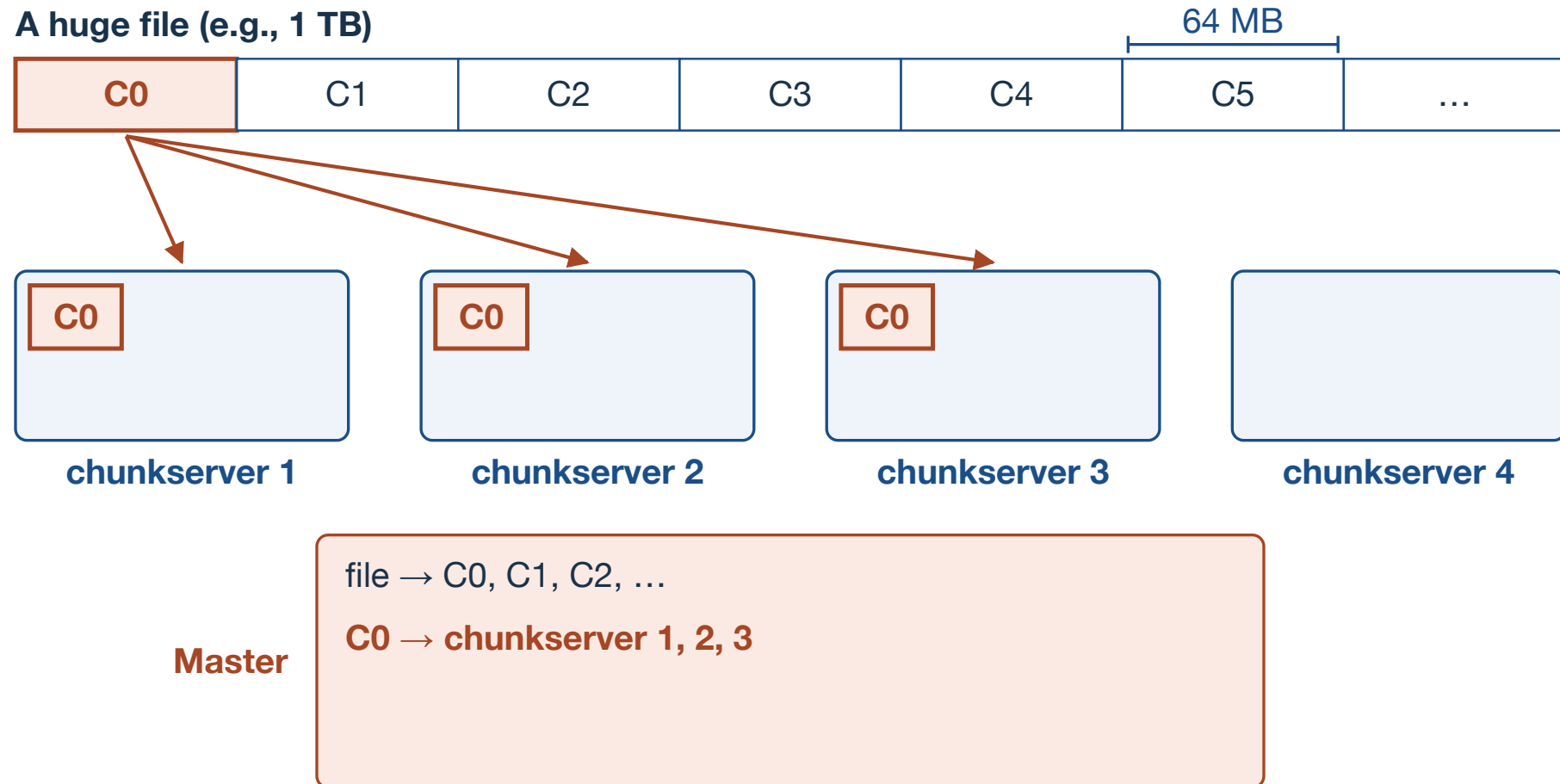
GFS files

A file is split into **64 MB chunks**; each chunk is **replicated** on several chunkservers, and the **master** records where each chunk lives



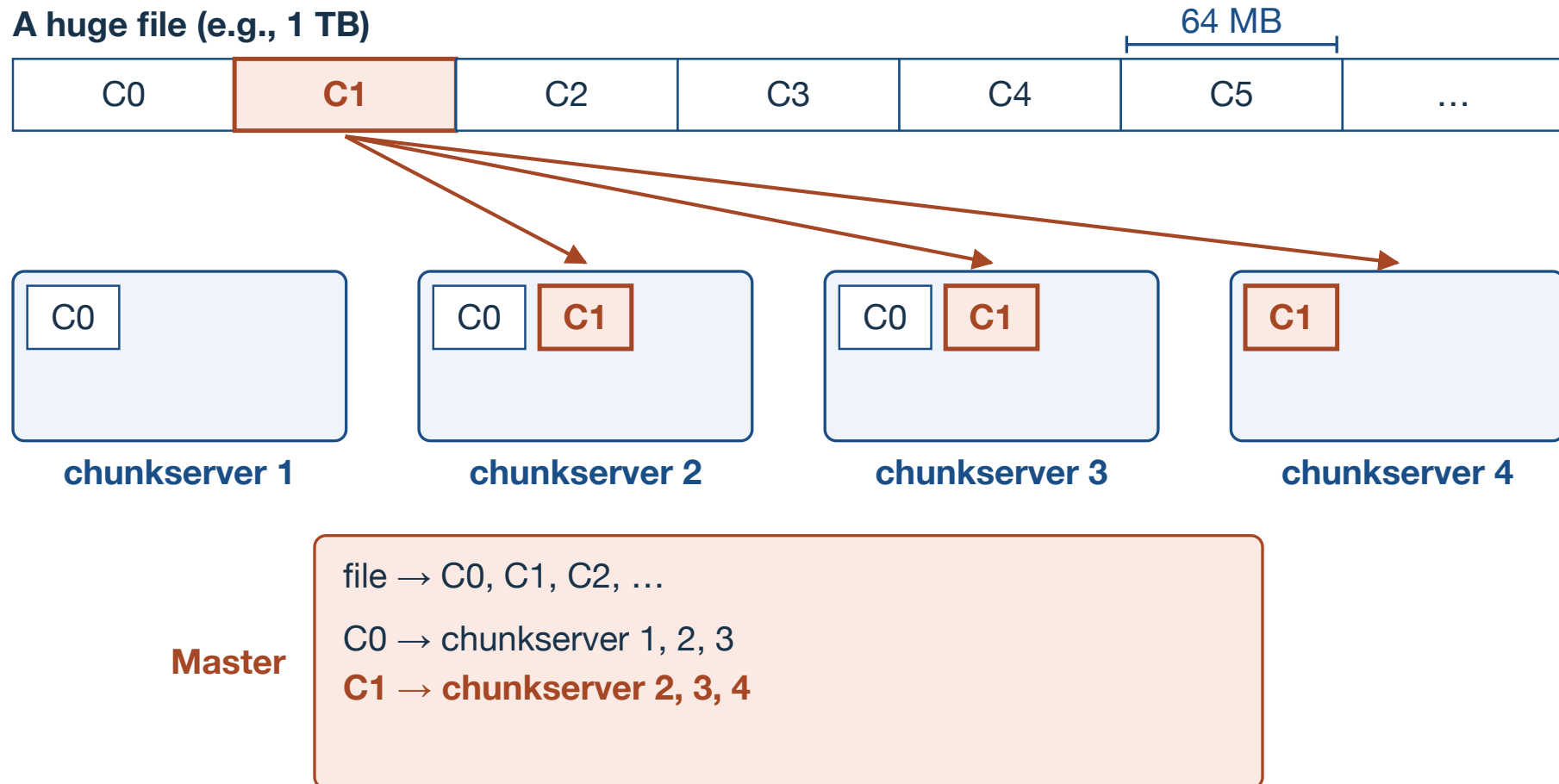
GFS files

A file is split into **64 MB chunks**; each chunk is **replicated** on several chunkservers, and the **master** records where each chunk lives



GFS files

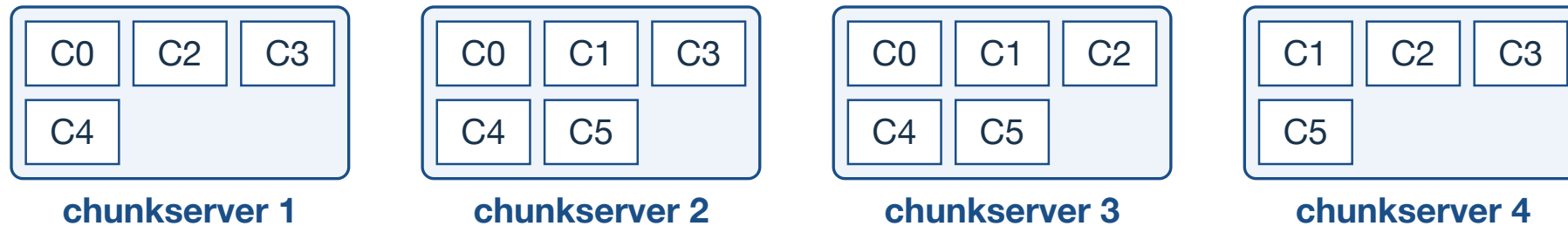
A file is split into **64 MB chunks**; each chunk is **replicated** on several chunkservers, and the **master** records where each chunk lives



GFS files

A file is split into **64 MB chunks**; each chunk is **replicated** on several chunkservers, and the **master** records where each chunk lives

A huge file (e.g., 1 TB)



Master

file → C0, C1, C2, ...

C0 → chunkserver 1, 2, 3

C1 → chunkserver 2, 3, 4

C2 → chunkserver 1, 3, 4

C3 → chunkserver 1, 2, 4

C4 → chunkserver 1, 2, 3

C5 → chunkserver 2, 3, 4

Why large chunks?

Default chunk size = 64 MB

- Compared to Linux ext4 block size: 1KB~64KB (typically 4KB)

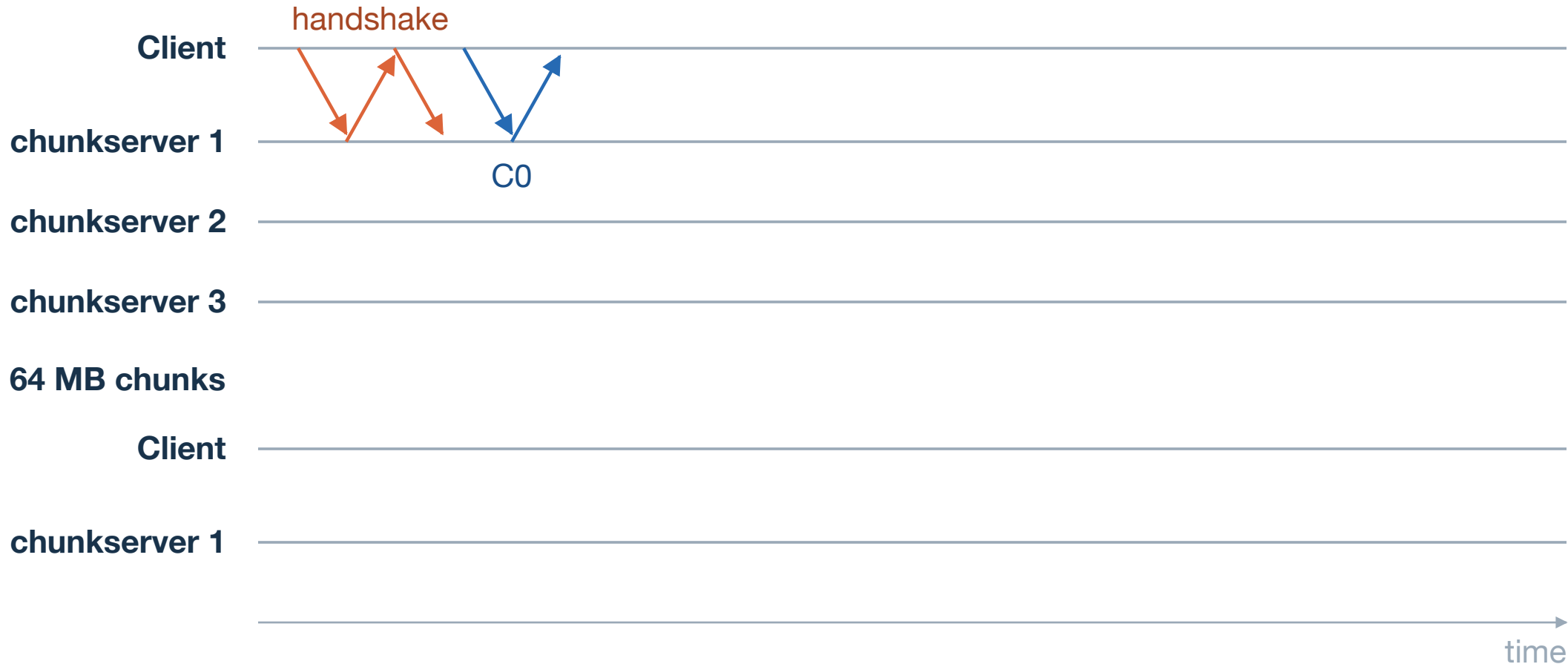
Benefits

- Reduce the need for frequent communication with the master for chunk location info
- Many operations hit the same chunk, so a client can reuse one persistent TCP connection to its chunkserver
 - Establishing a TCP connection can be costly (three-way handshake, see later lectures)
- The master can store all metadata in memory

Why large chunks? Reuse one TCP connection

A client reads 3 MB of a file, 1 MB per request

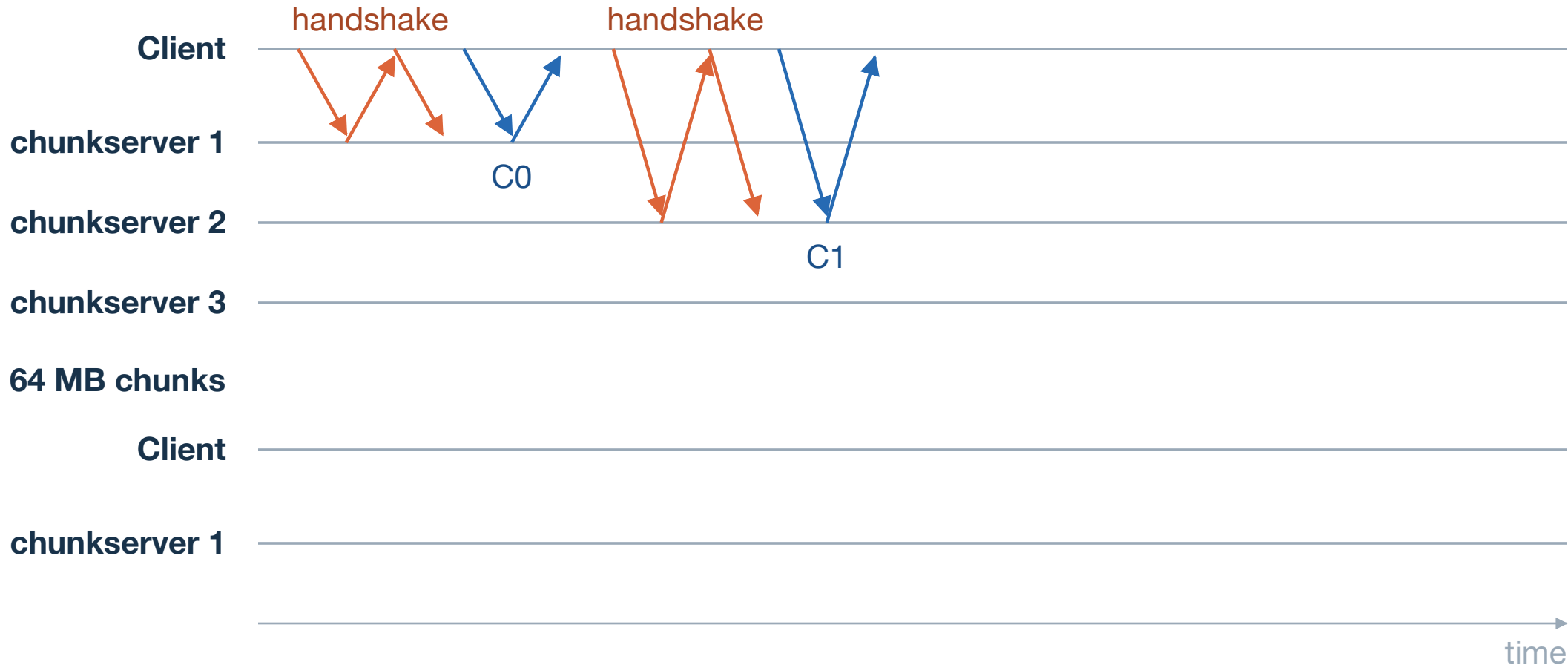
1 MB chunks



Why large chunks? Reuse one TCP connection

A client reads 3 MB of a file, 1 MB per request

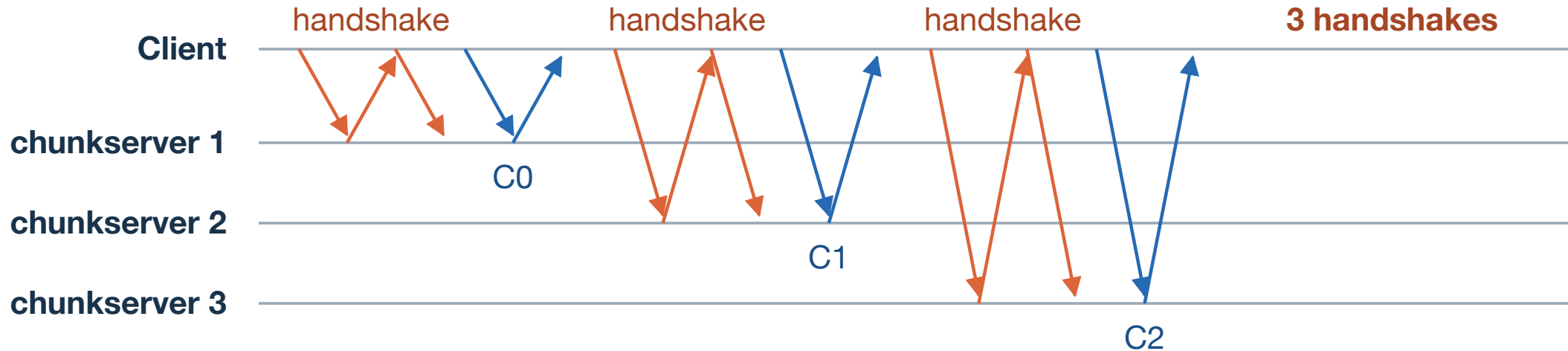
1 MB chunks



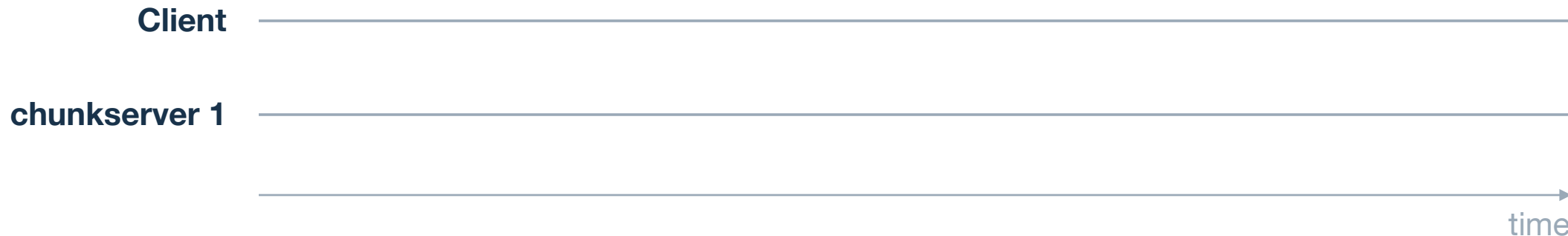
Why large chunks? Reuse one TCP connection

A client reads 3 MB of a file, 1 MB per request

1 MB chunks



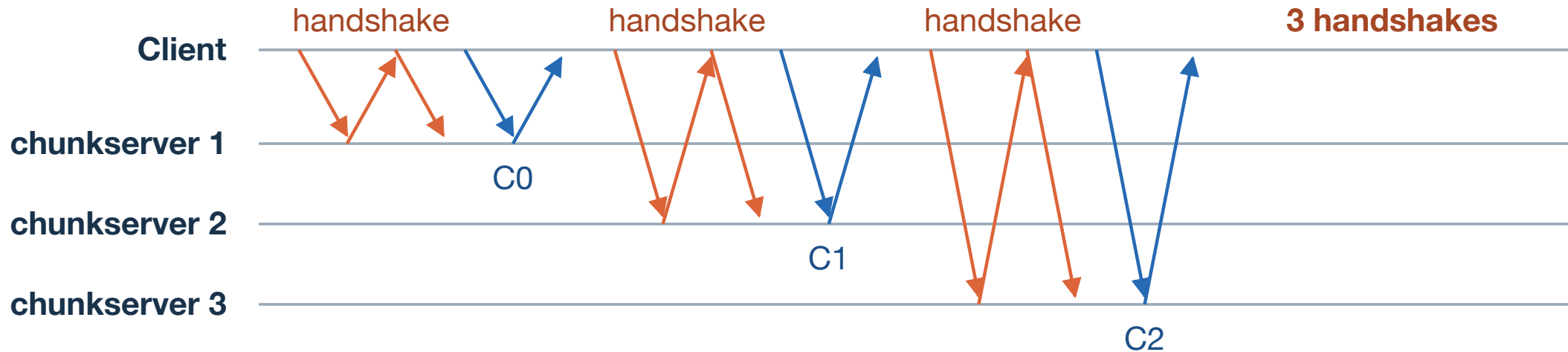
64 MB chunks



Why large chunks? Reuse one TCP connection

A client reads 3 MB of a file, 1 MB per request

1 MB chunks



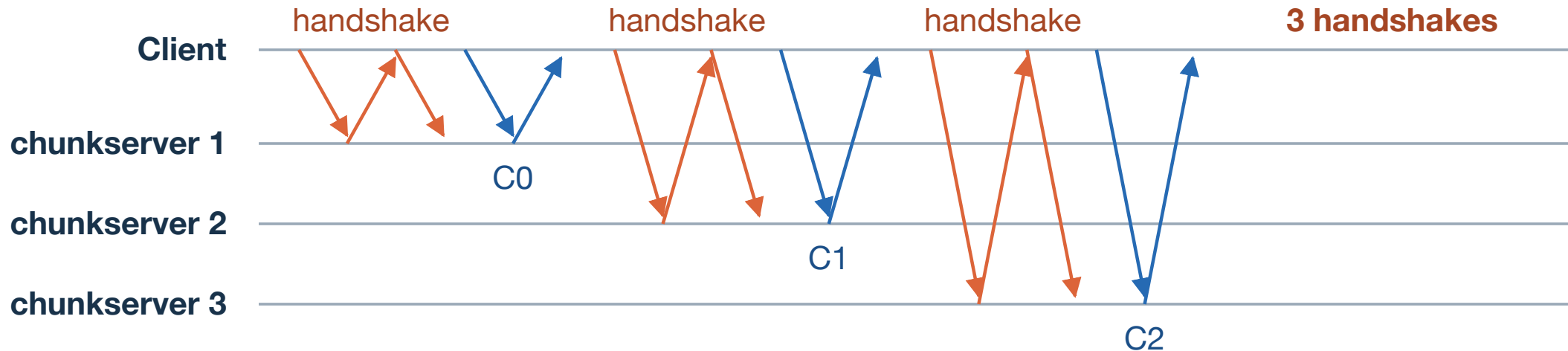
64 MB chunks



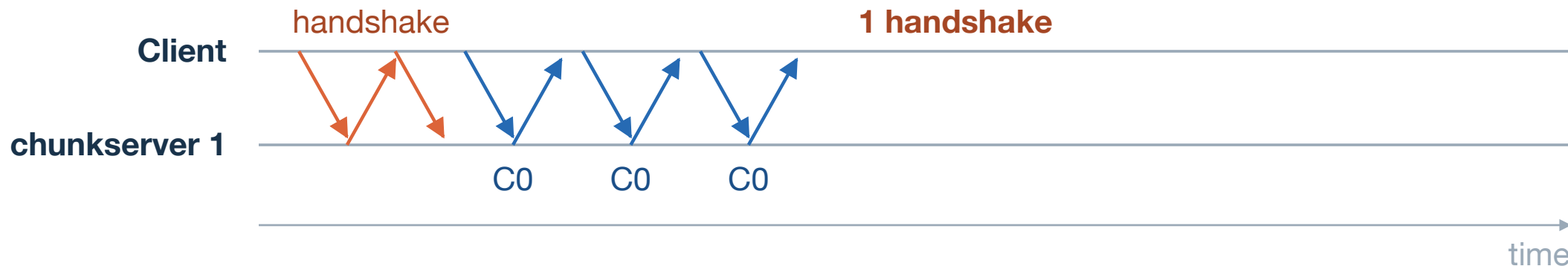
Why large chunks? Reuse one TCP connection

A client reads 3 MB of a file, 1 MB per request

1 MB chunks



64 MB chunks



Chunks (blocks) and chunkservers

Chunk size = 64 MB (default)

- Each 64 KB block of a chunk has a 32-bit checksum

Chunk handle

- Globally unique 64-bit number
- Assigned by the master at creation

Chunks are stored on local disks as Linux files (a.k.a. file system overlay)

Each chunk is replicated on multiple nodes

- Three replicas (default)
- More replicas for popular files to avoid hotspots

Short summary: GFS vs. naïve file system?

1. Master stores the current locations of chunks (blocks)
 1. Unlike a naïve distributed file system, which stores the locations in the inode block
 2. Why? For performance & simplified correctness management
2. Data are stored in large chunks
3. Chunks are replicated for fault tolerance & high performance
 - Our previous design (NFS) does not replicate the data

Why does GFS use one master? To keep the design simple

All **metadata** stored in the master's **memory**

- Super-fast access

Name-to-chunk maps (e.g., using an in-memory tree)

- Stored in **memory**
- Also persisted in **an operation log** on disk (see later lectures)

Stores **chunk-ID-to-chunk-location** maps in memory (and no need for a log)

- Queried from all the chunkservers at startup
- Kept up-to-date by periodic **HeartBeat** messages between the master and chunkservers
- Benefit: simpler consistency management

Client-GFS interaction model

GFS client code **linked** into each app

- No **OS-level** API
- Interacts with the **master** for metadata-related ops
- Interacts **directly with chunkservers** for data
 - The master is not a point of congestion

No caching: neither clients nor chunkservers cache data

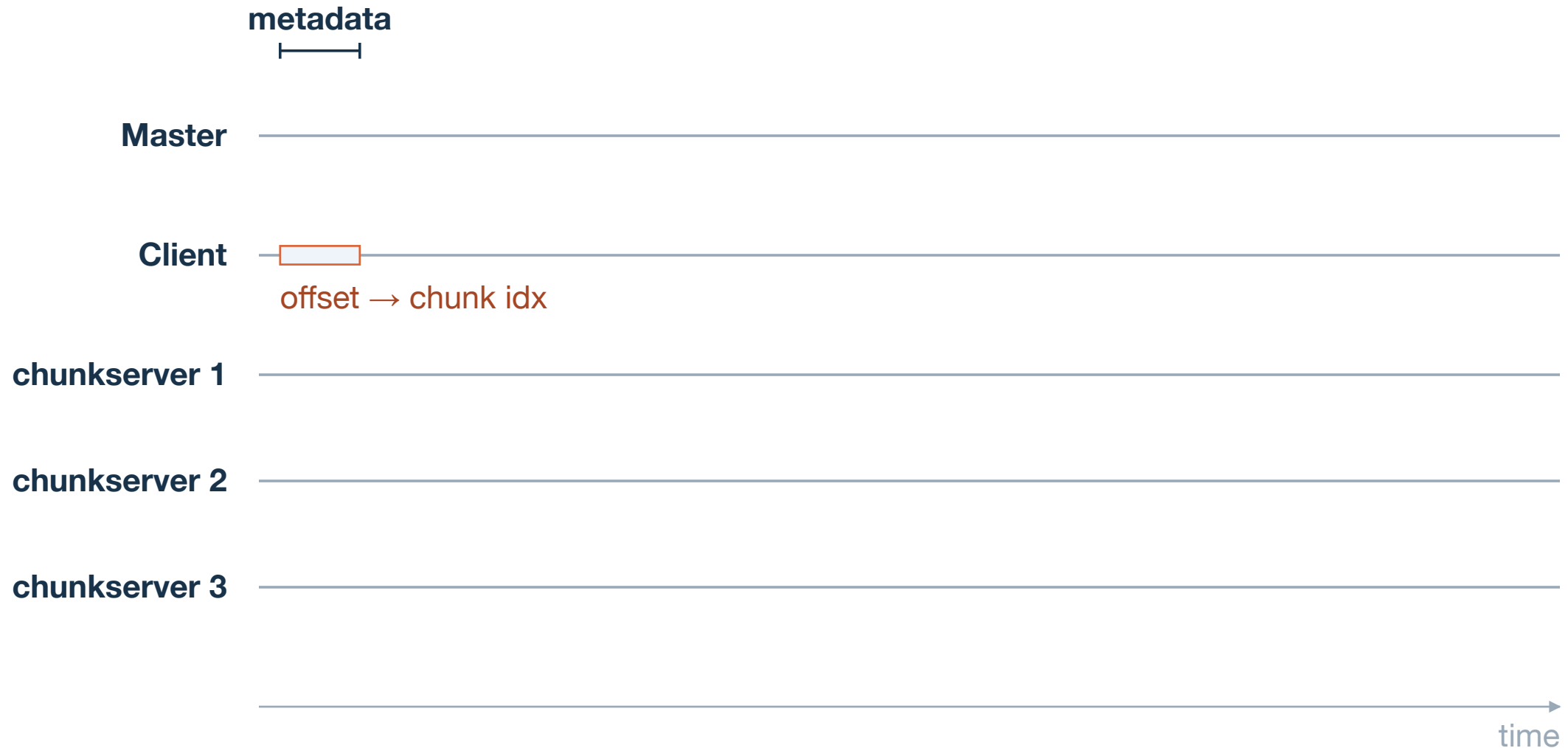
- Except for the system buffer cache
- Why? Large files offer little opportunity for caching

Clients cache metadata

- E.g., the locations of a file's chunks

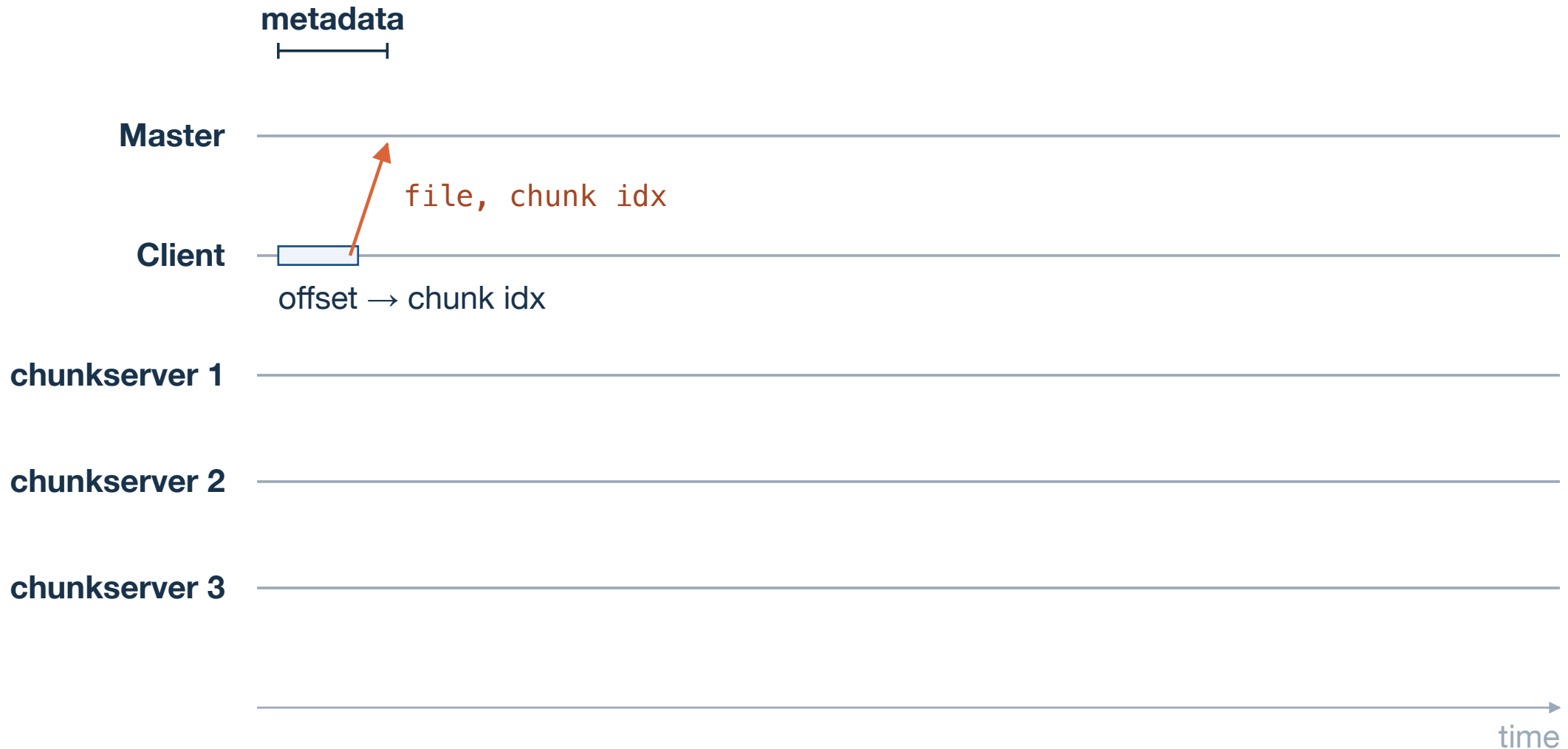
Reading a file in GFS

The client gets **metadata** from the **master**, then reads **data** directly from **any** replica



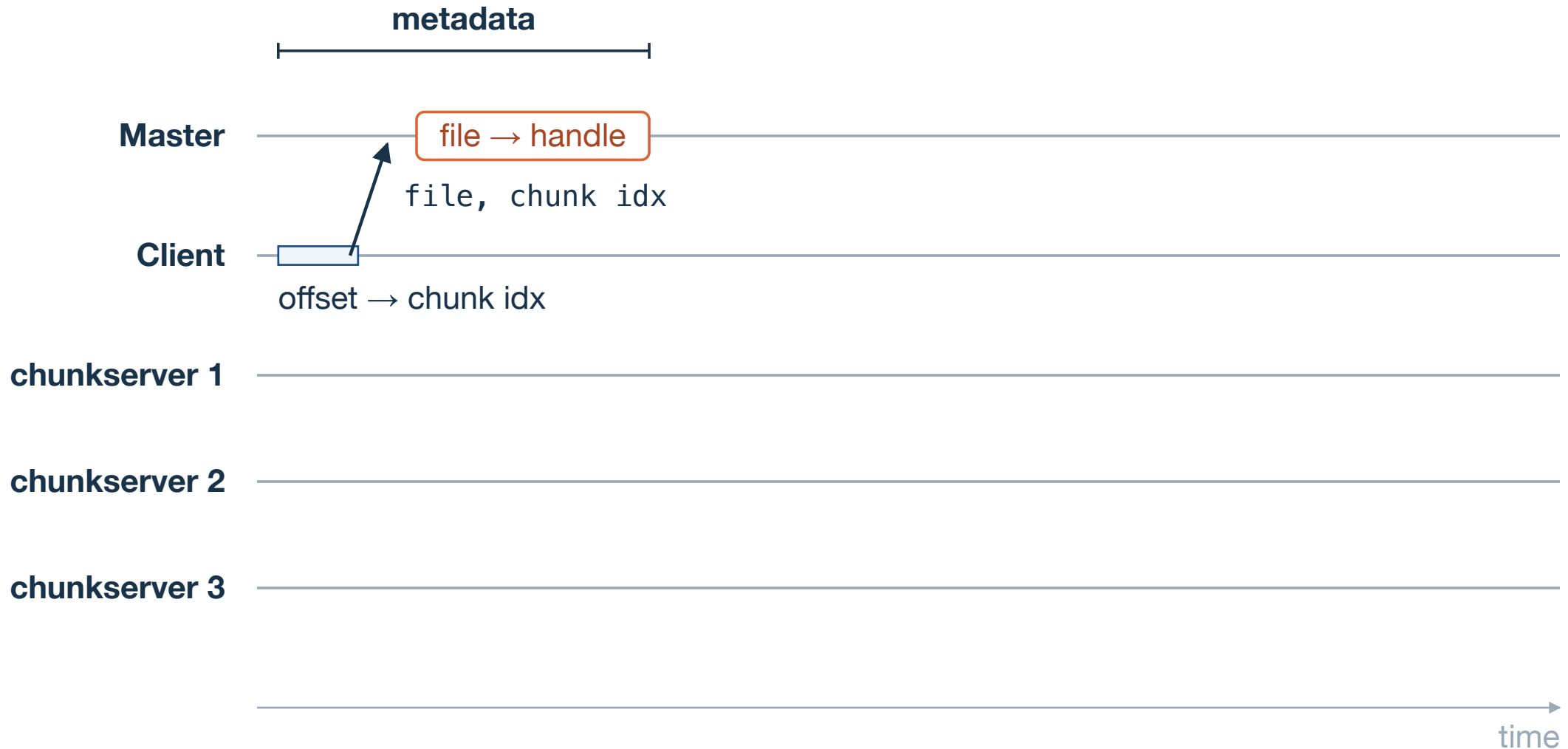
Reading a file in GFS

The client gets **metadata** from the **master**, then reads **data** directly from **any** replica



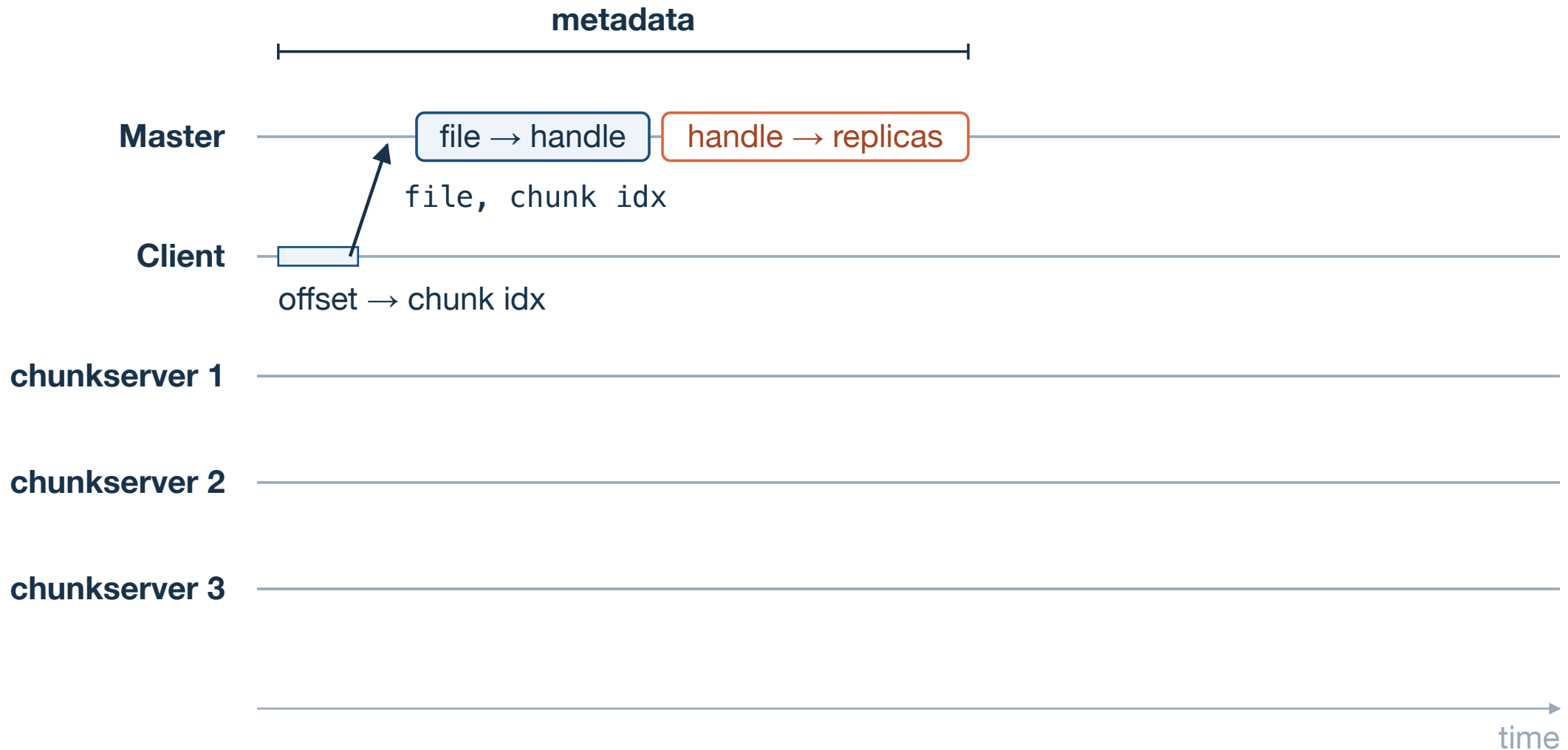
Reading a file in GFS

The client gets **metadata** from the **master**, then reads **data** directly from **any** replica



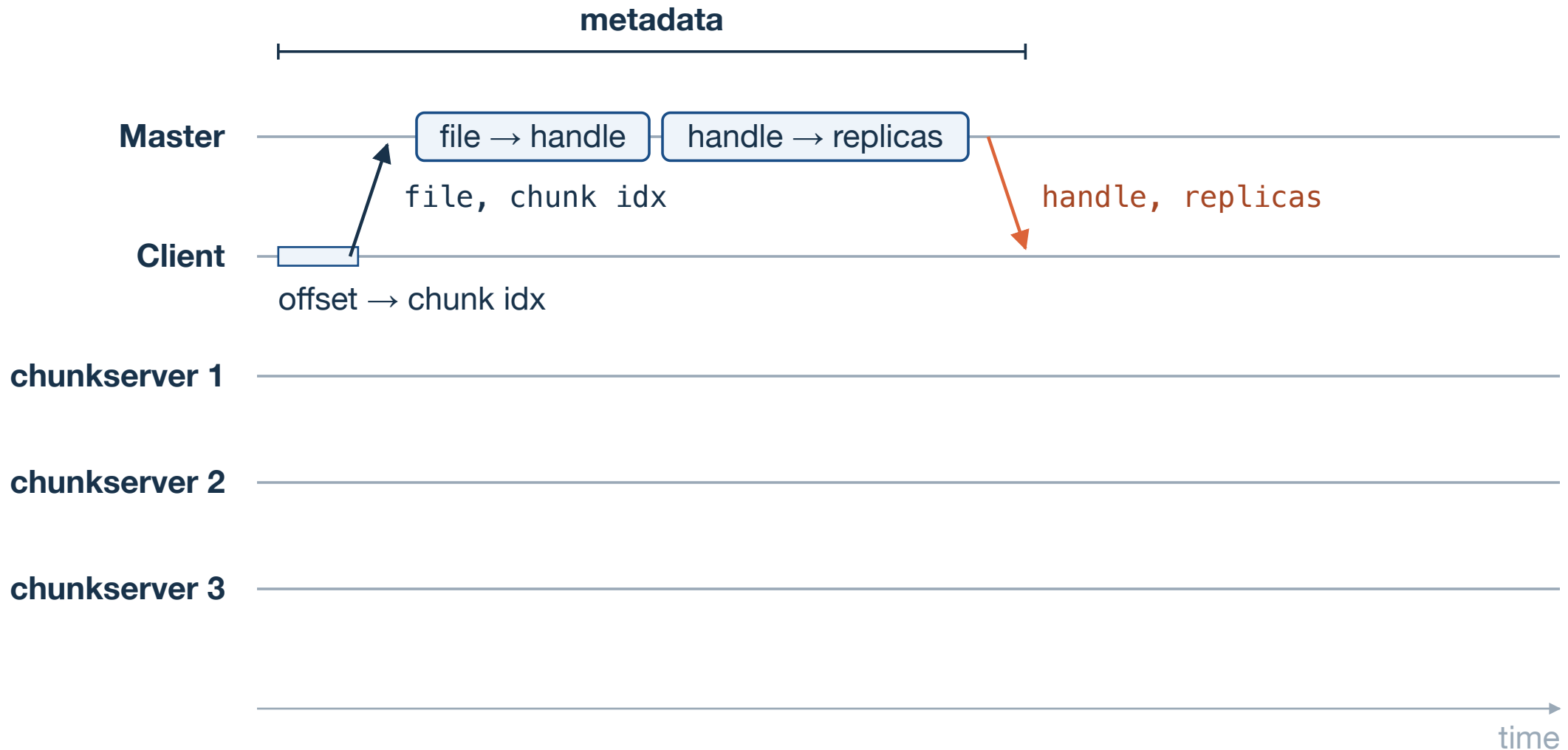
Reading a file in GFS

The client gets **metadata** from the **master**, then reads **data** directly from **any** replica



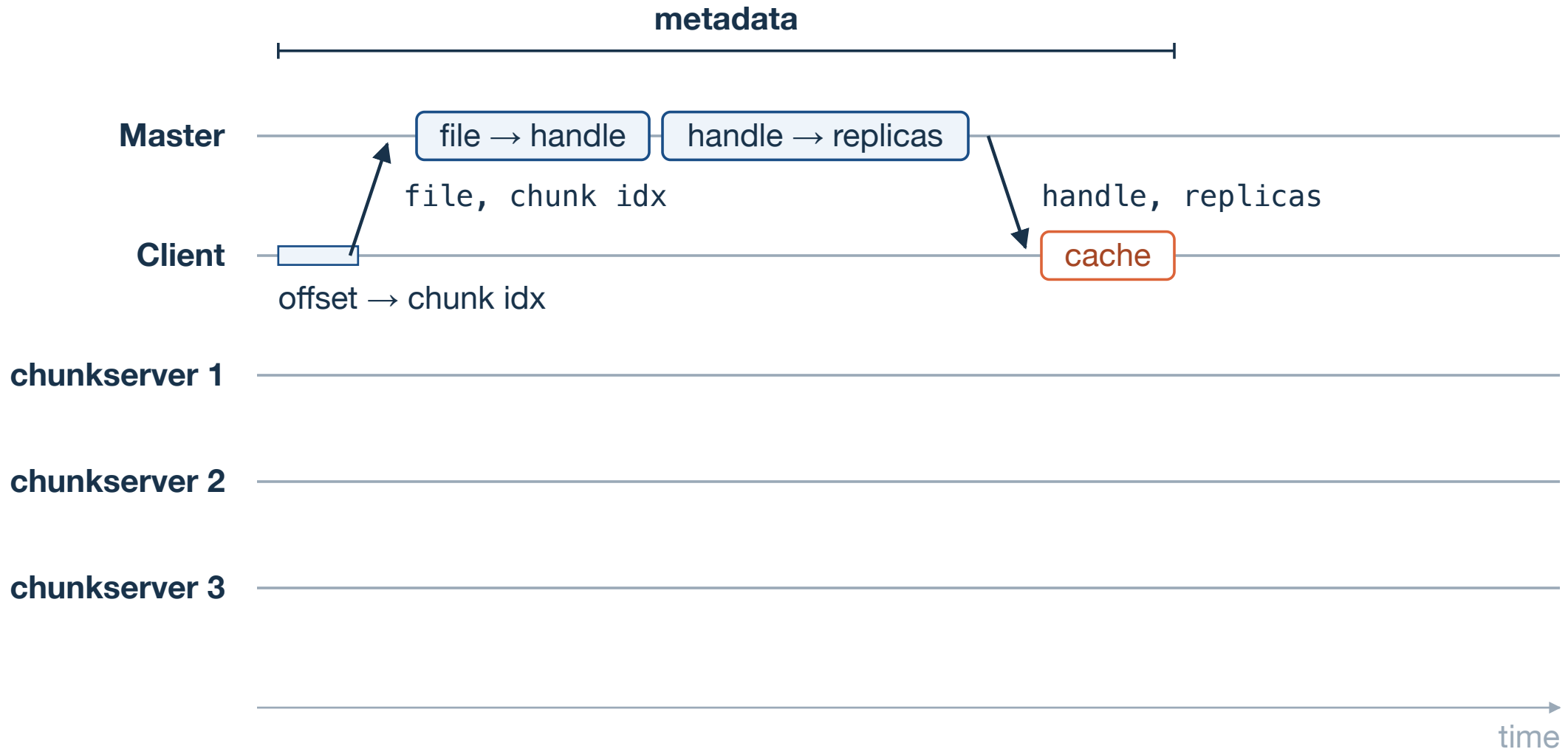
Reading a file in GFS

The client gets **metadata** from the **master**, then reads **data** directly from **any** replica



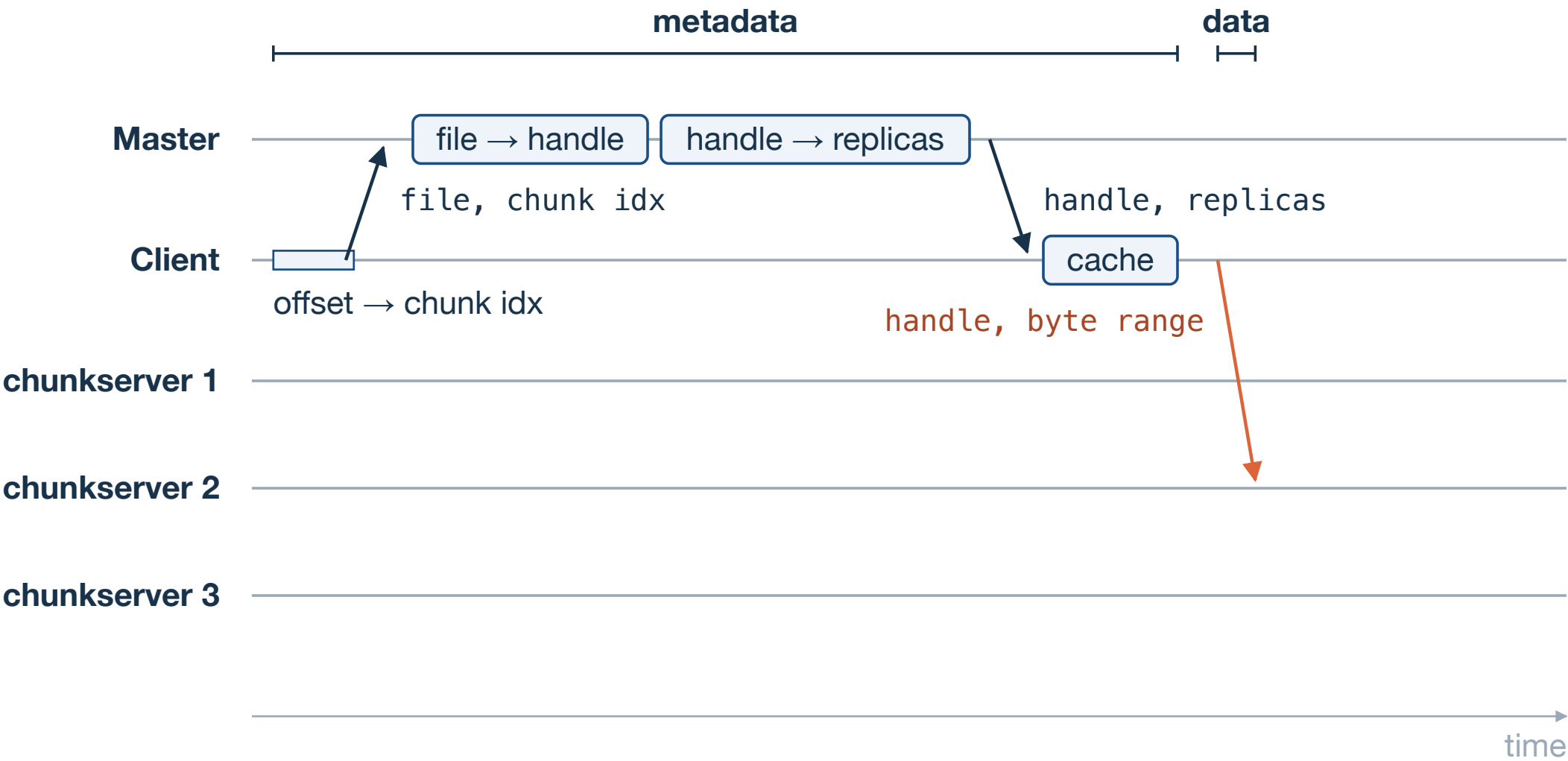
Reading a file in GFS

The client gets **metadata** from the **master**, then reads **data** directly from **any** replica



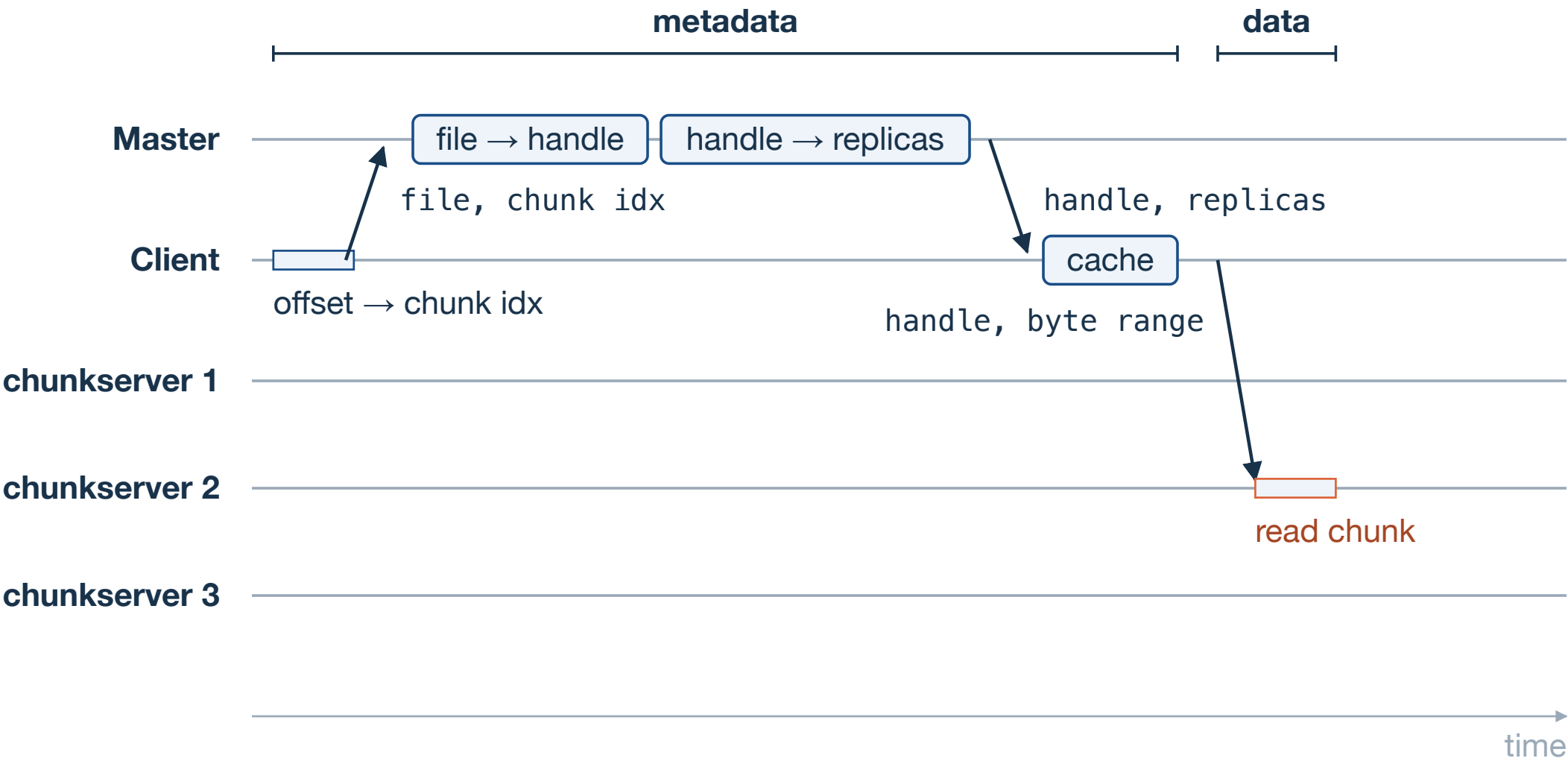
Reading a file in GFS

The client gets **metadata** from the **master**, then reads **data** directly from **any** replica



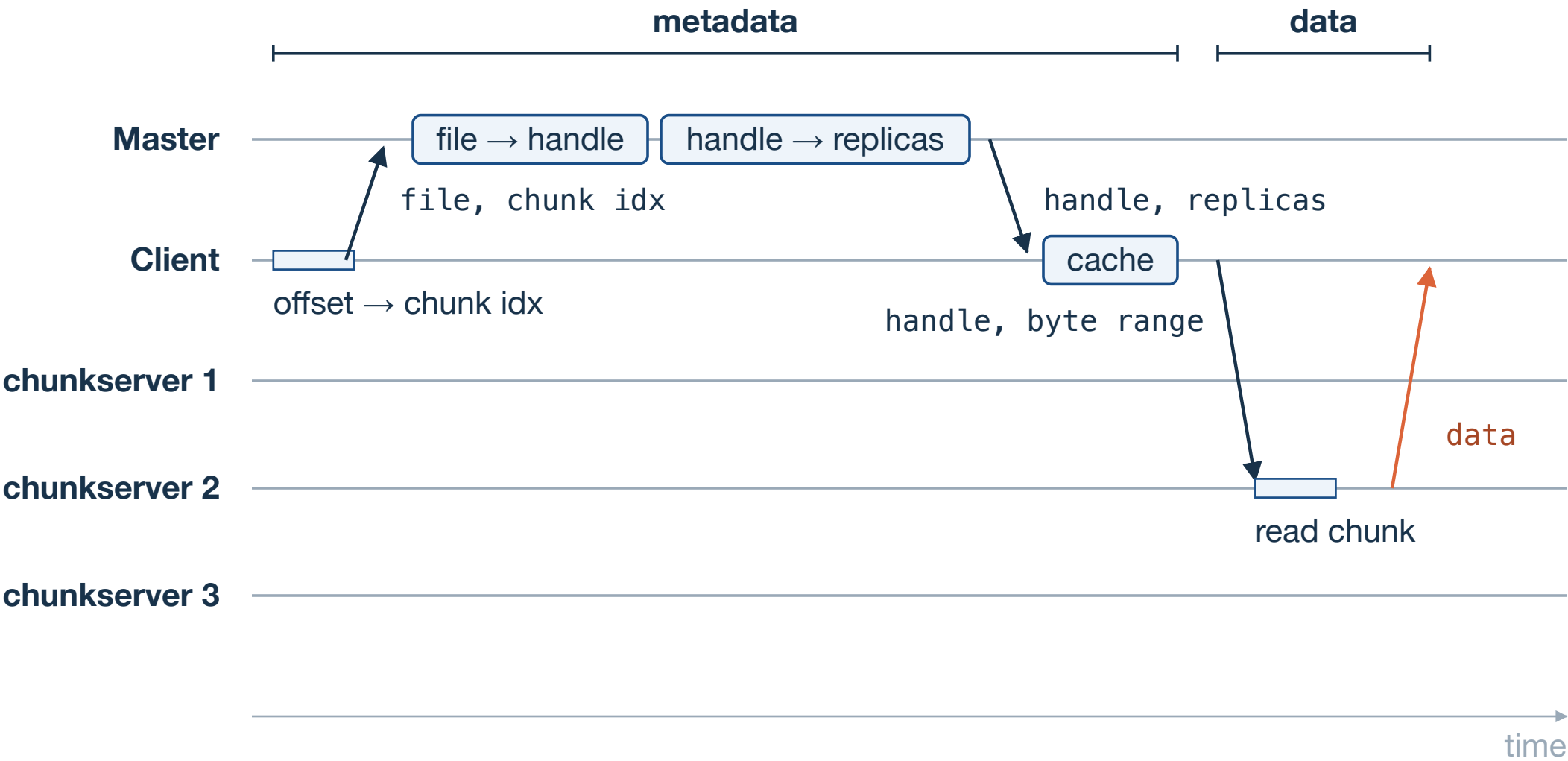
Reading a file in GFS

The client gets **metadata** from the **master**, then reads **data** directly from **any** replica



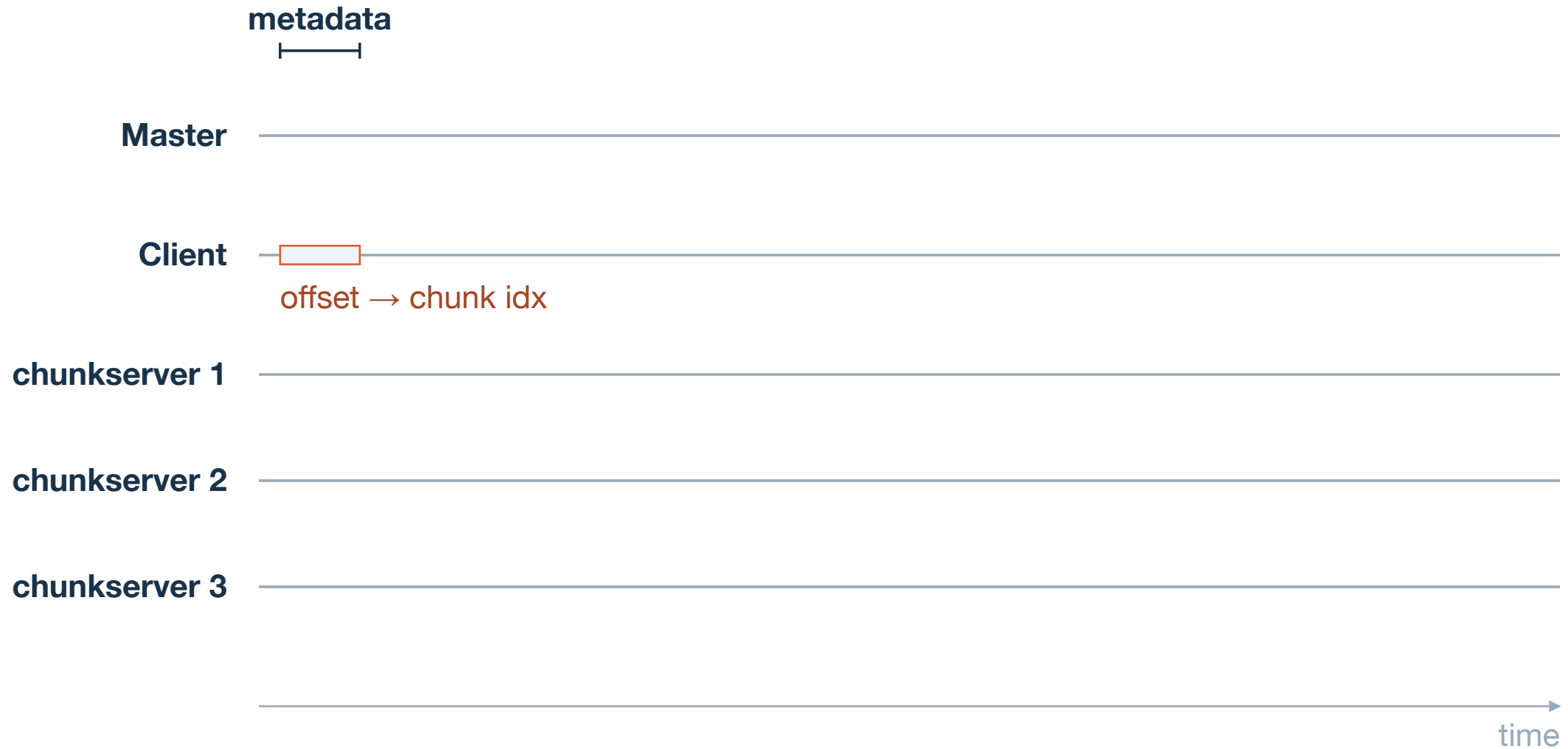
Reading a file in GFS

The client gets **metadata** from the **master**, then reads **data** directly from **any** replica



Reading a file in GFS

Clients **cache** metadata: a later read of the same chunk **skips the master**



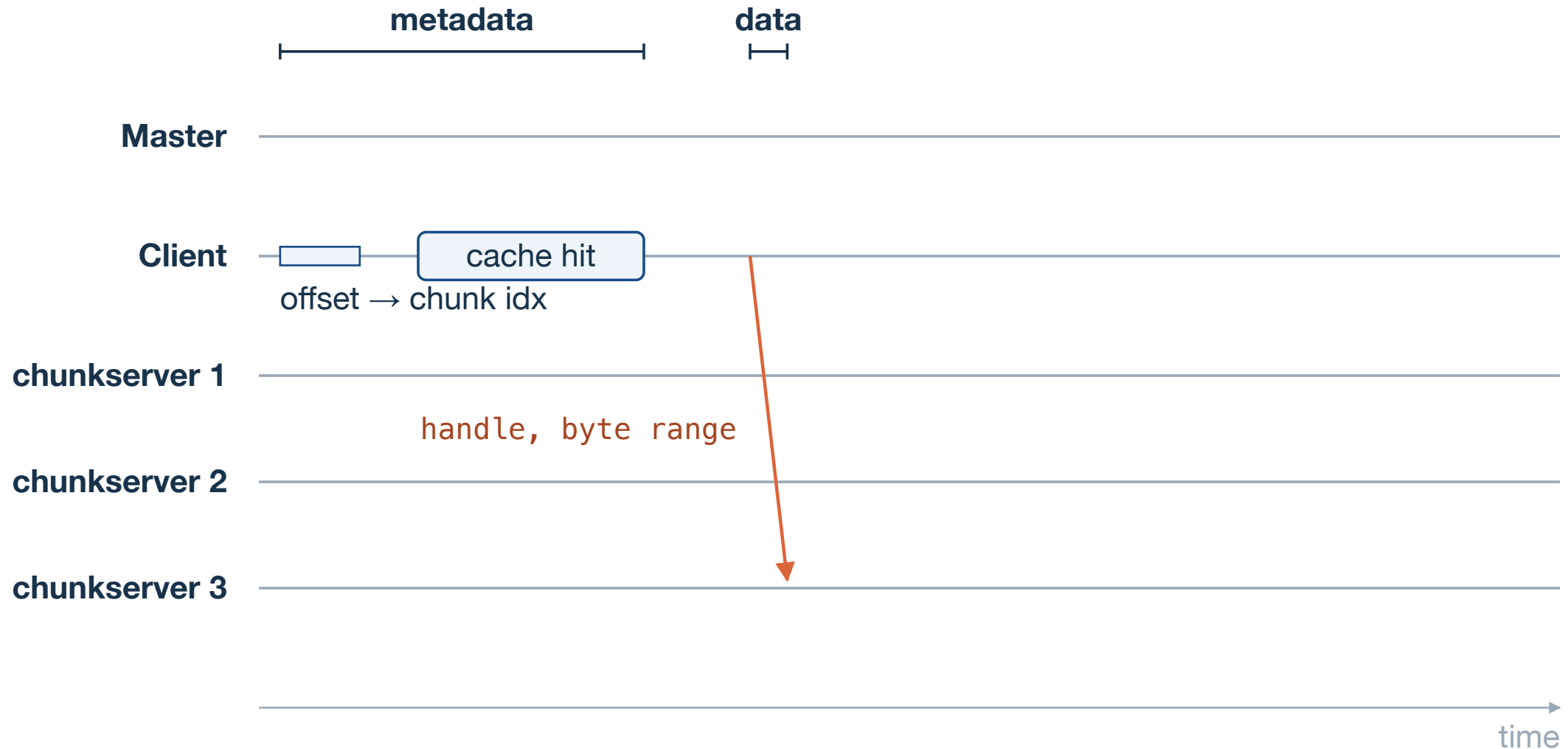
Reading a file in GFS

Clients **cache** metadata: a later read of the same chunk **skips the master**



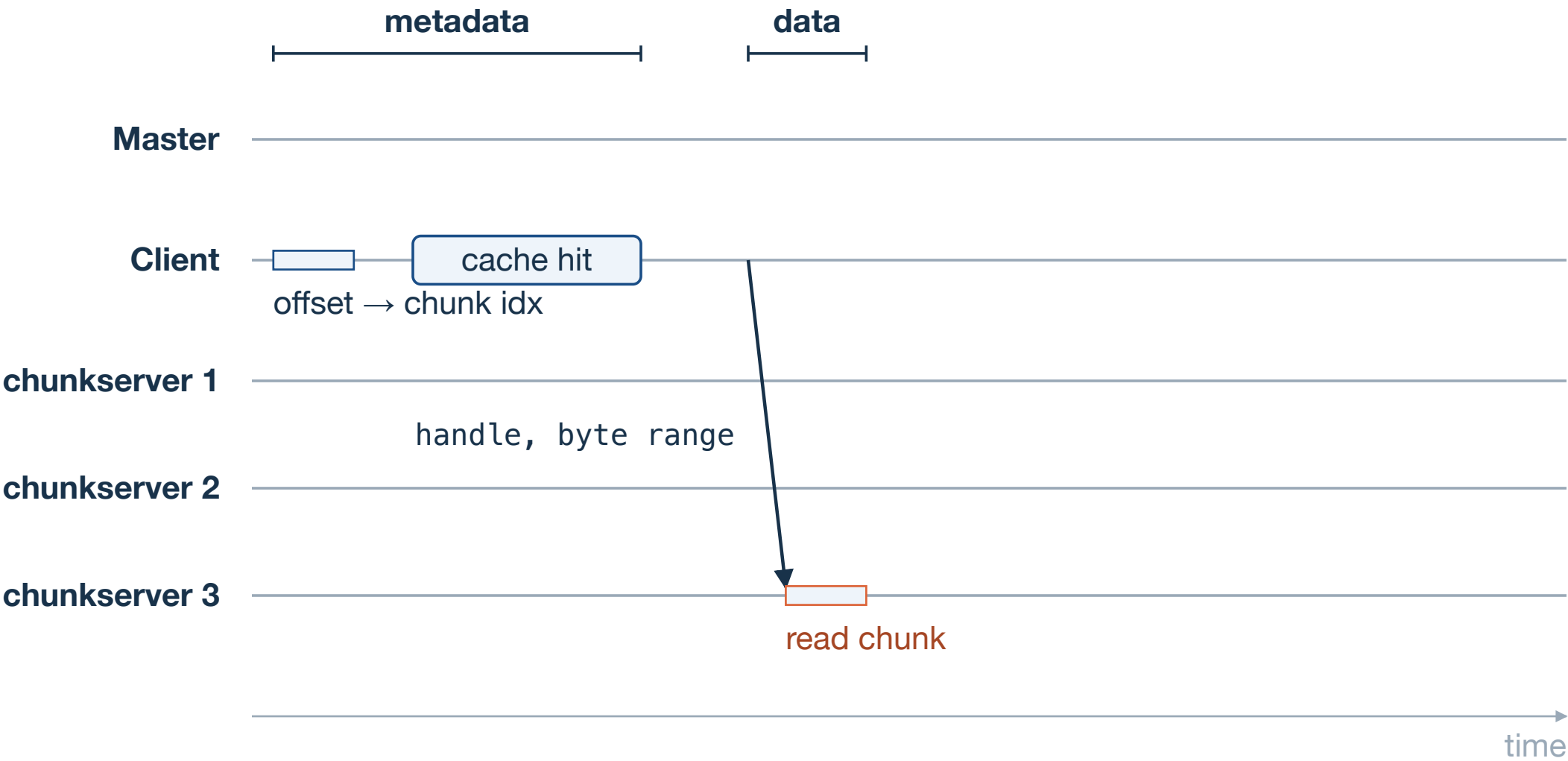
Reading a file in GFS

Clients **cache** metadata: a later read of the same chunk **skips the master**



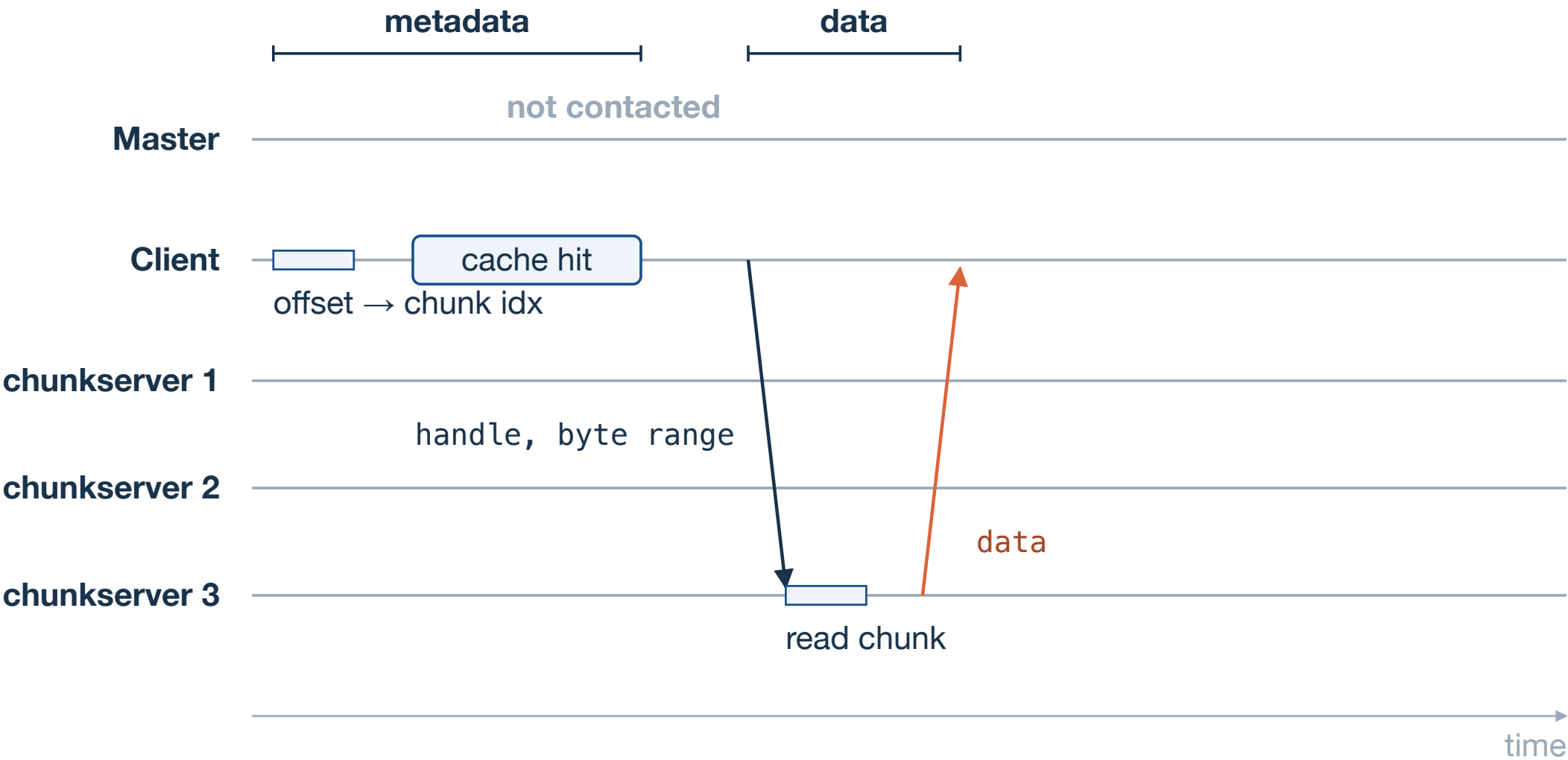
Reading a file in GFS

Clients **cache** metadata: a later read of the same chunk **skips the master**



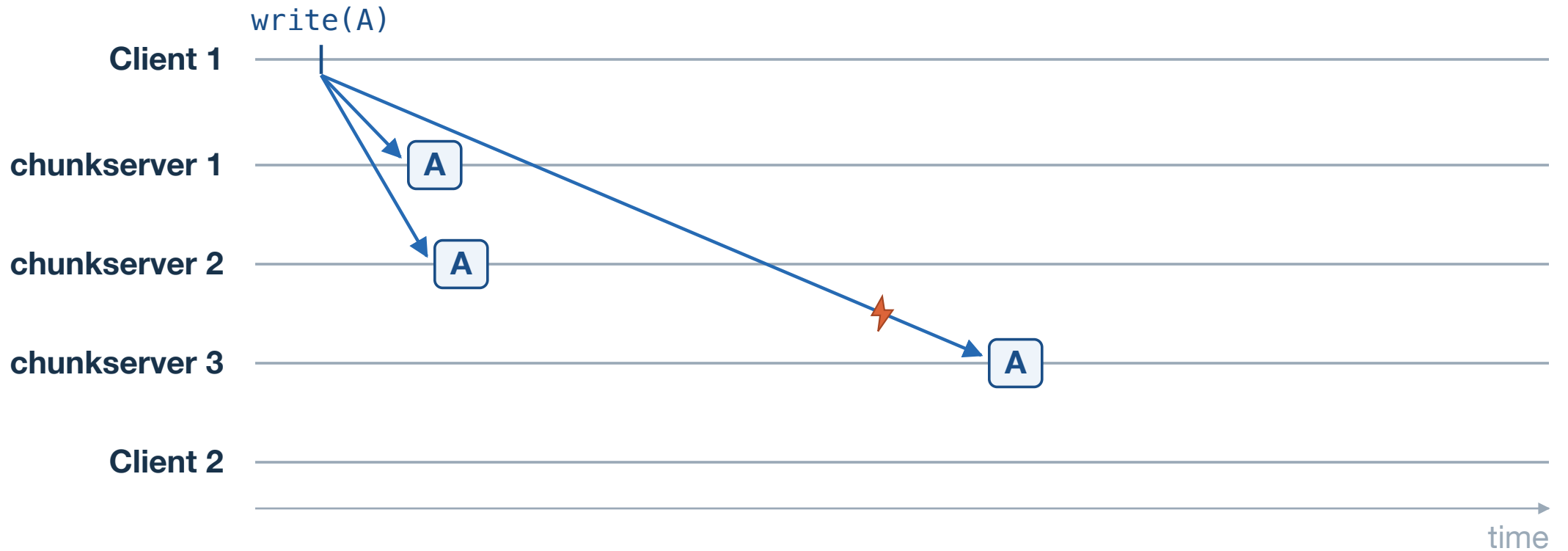
Reading a file in GFS

Clients **cache** metadata: a later read of the same chunk **skips the master**



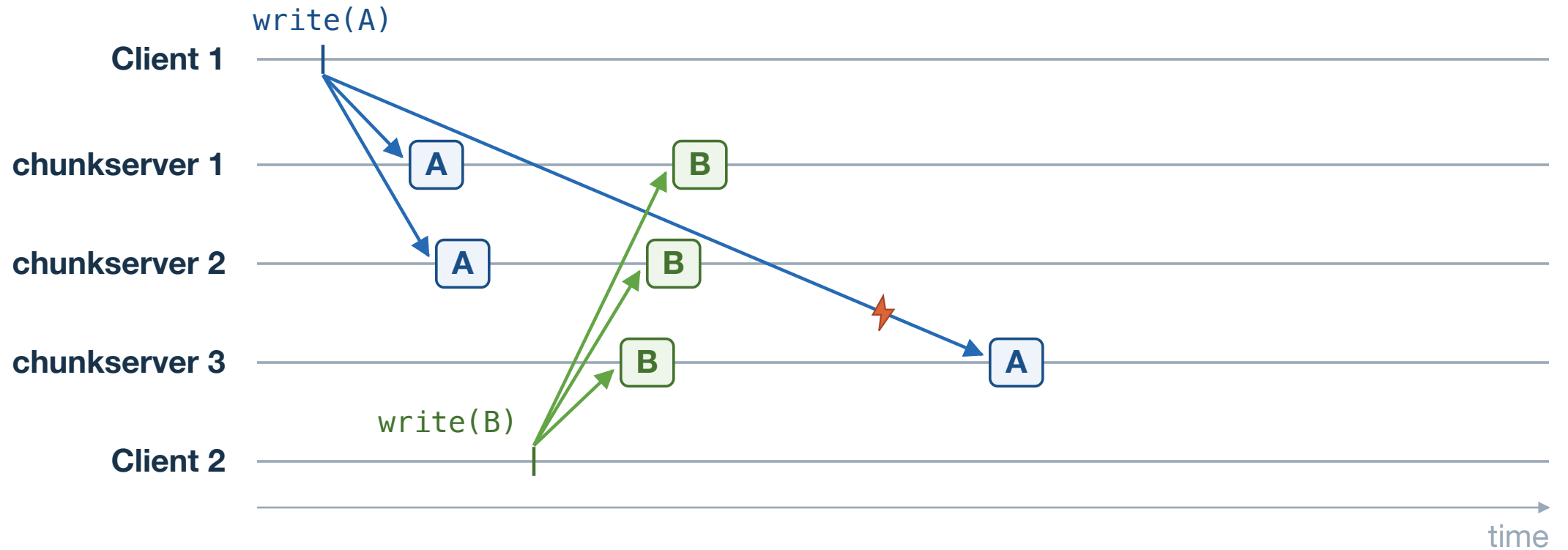
Challenge of writing: the order of writes

A write must reach **all three replicas**; two clients write the **same chunk** at the same time



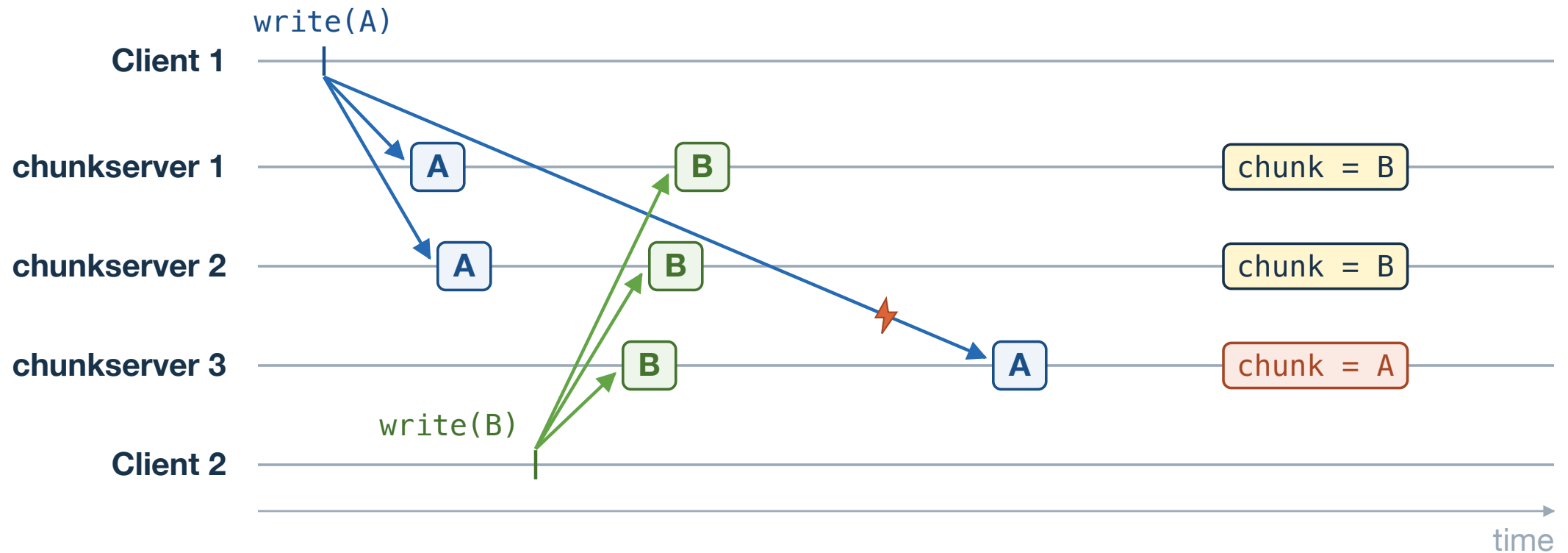
Challenge of writing: the order of writes

A write must reach **all three replicas**; two clients write the **same chunk** at the same time



Challenge of writing: the order of writes

A write must reach **all three replicas**; two clients write the **same chunk** at the same time



GFS: one replica, the primary, picks the order for all replicas

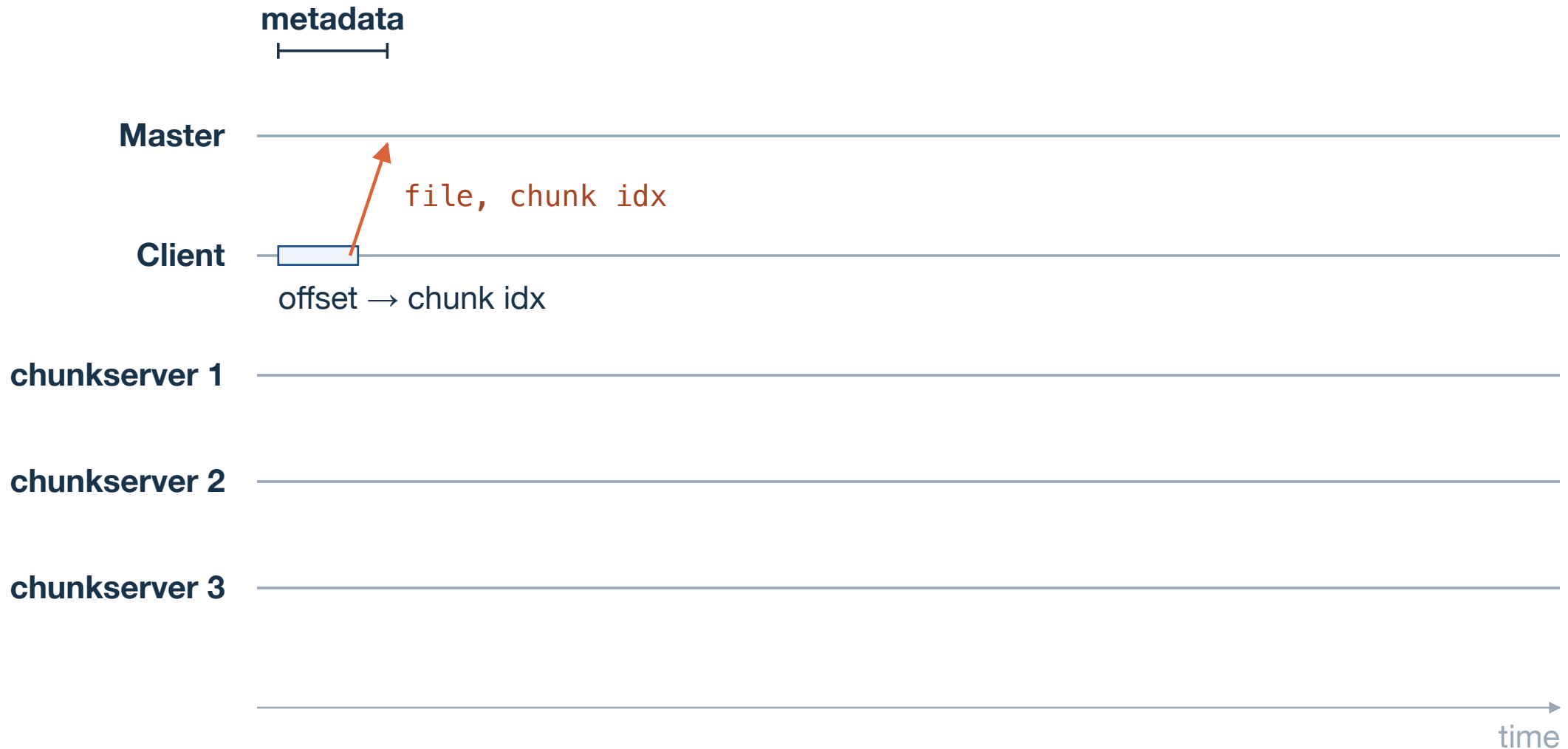
Writing a file in GFS: two phases

Phase 1: send data along a chain of replicas; chunkservers only buffer the data



Writing a file in GFS: two phases

Phase 1: send data along a chain of replicas; chunkservers only buffer the data



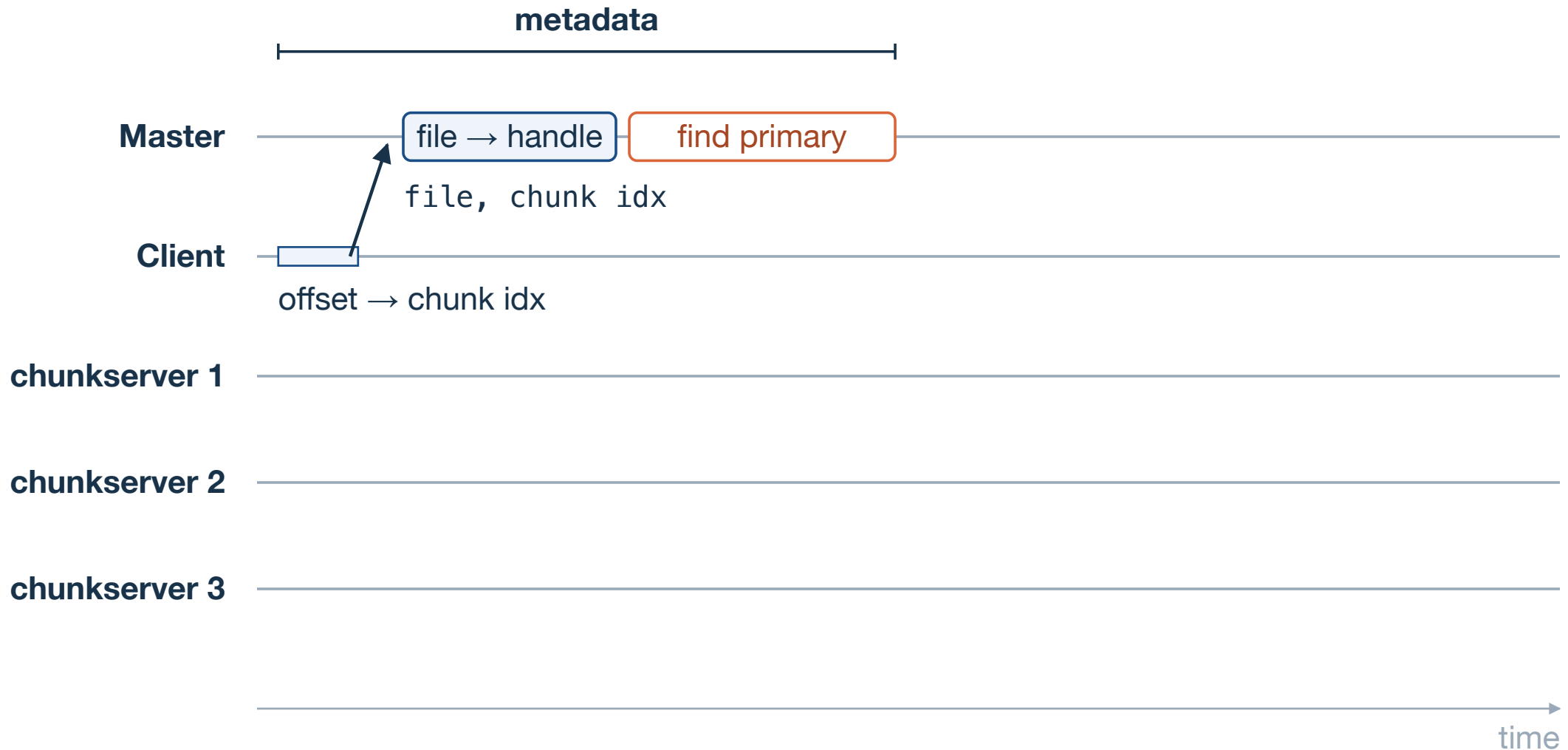
Writing a file in GFS: two phases

Phase 1: send data along a chain of replicas; chunkservers only buffer the data



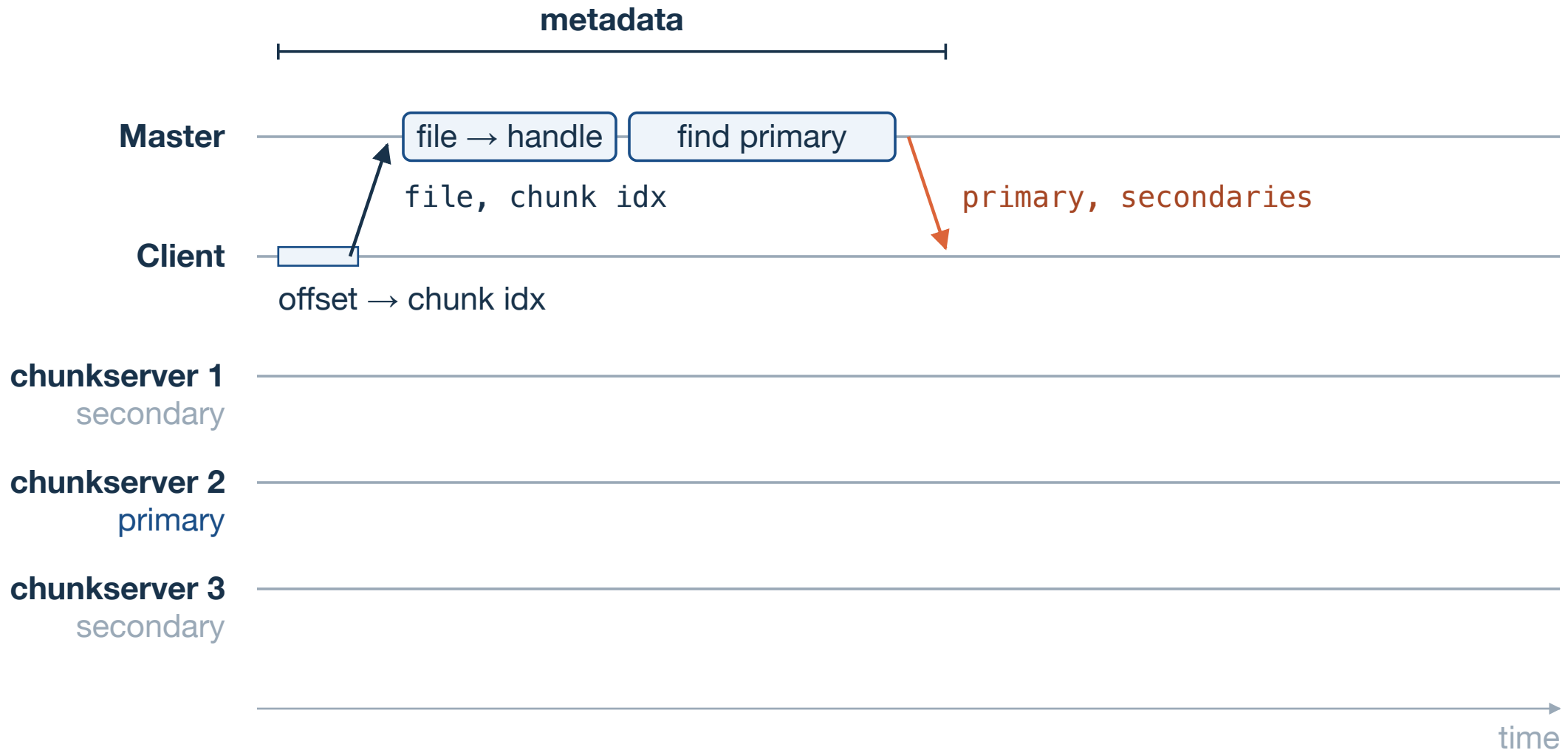
Writing a file in GFS: two phases

Phase 1: send data along a chain of replicas; chunkservers only buffer the data



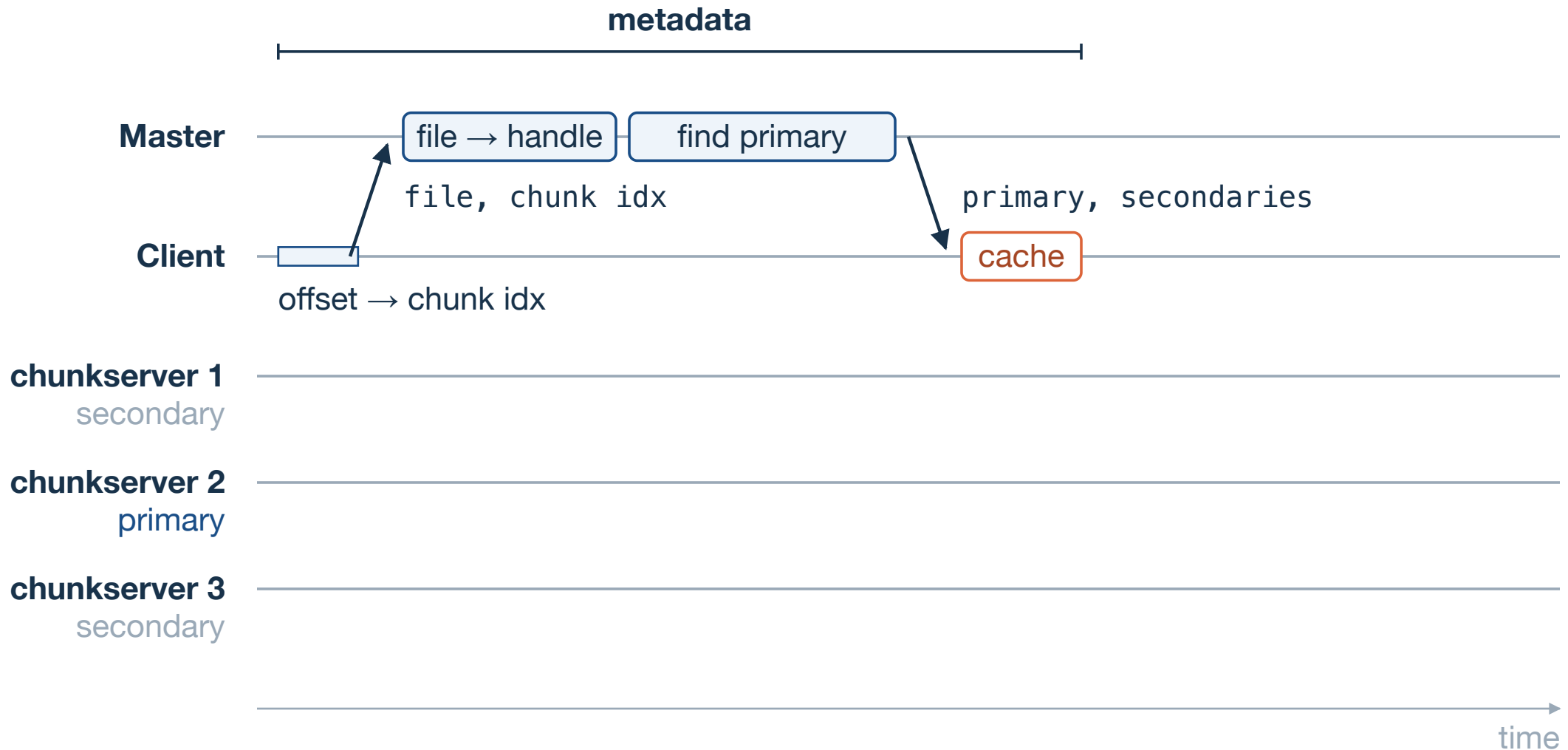
Writing a file in GFS: two phases

Phase 1: send data along a chain of replicas; chunkservers only buffer the data



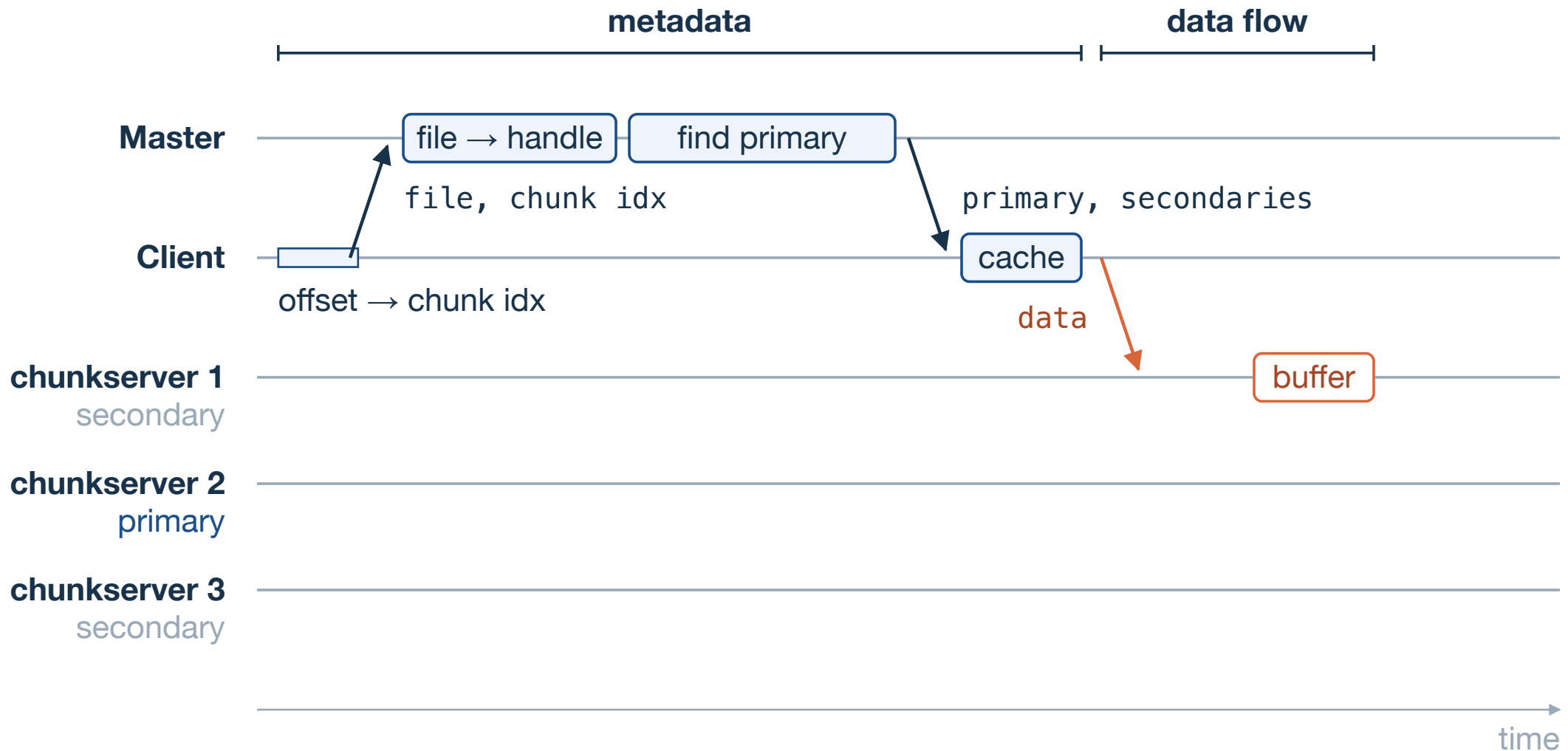
Writing a file in GFS: two phases

Phase 1: send data along a chain of replicas; chunkservers only buffer the data



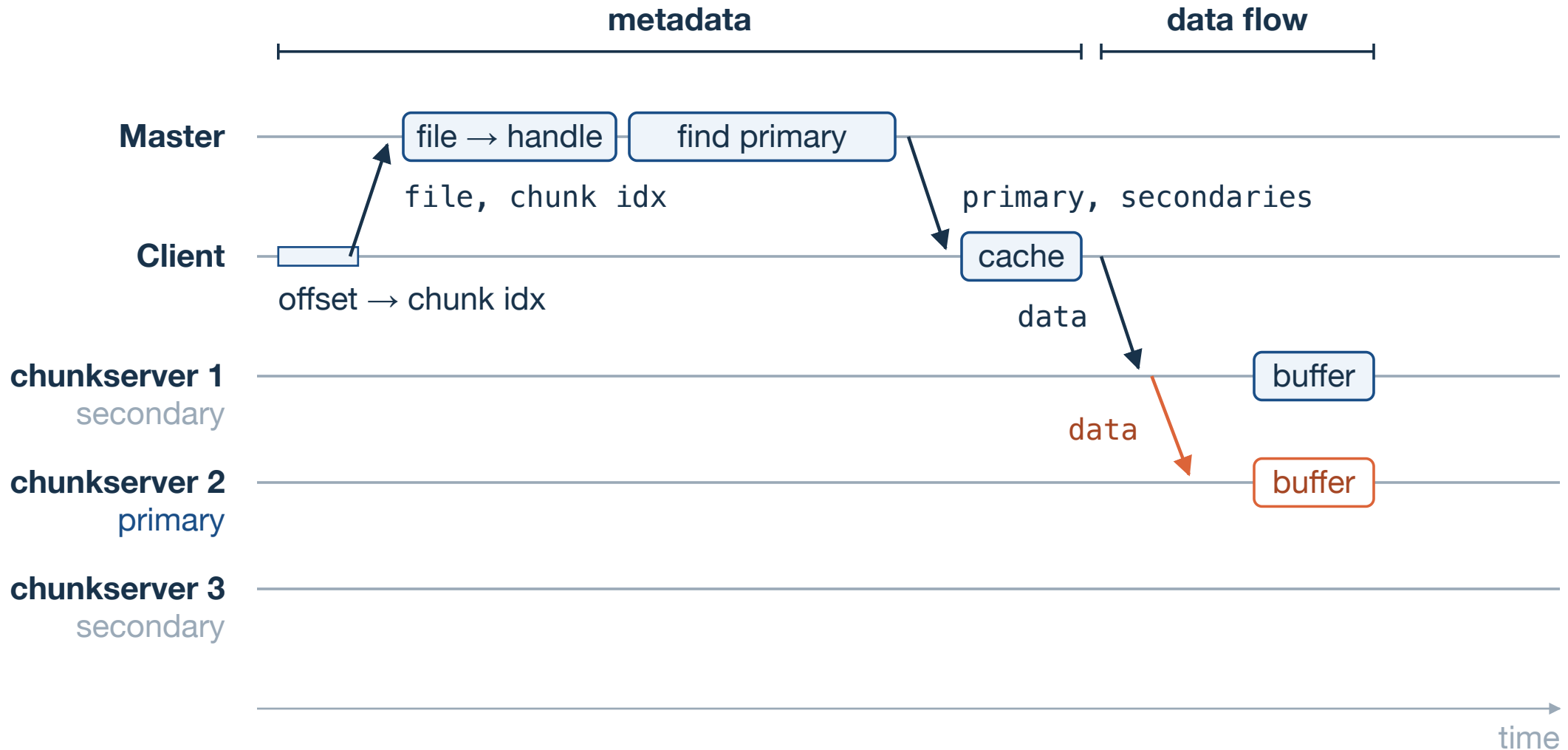
Writing a file in GFS: two phases

Phase 1: send data along a chain of replicas; chunkservers only buffer the data



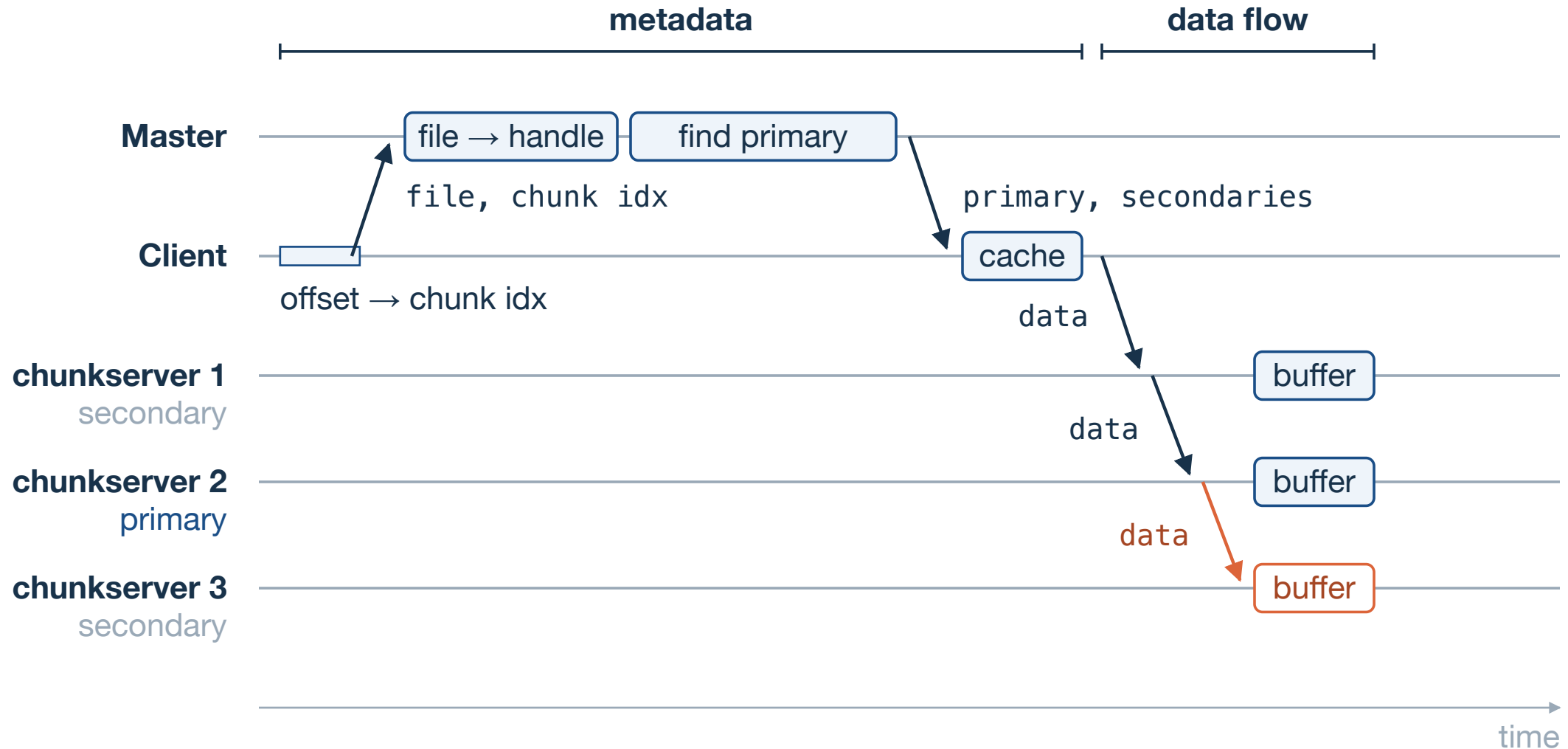
Writing a file in GFS: two phases

Phase 1: send data along a chain of replicas; chunkservers only buffer the data



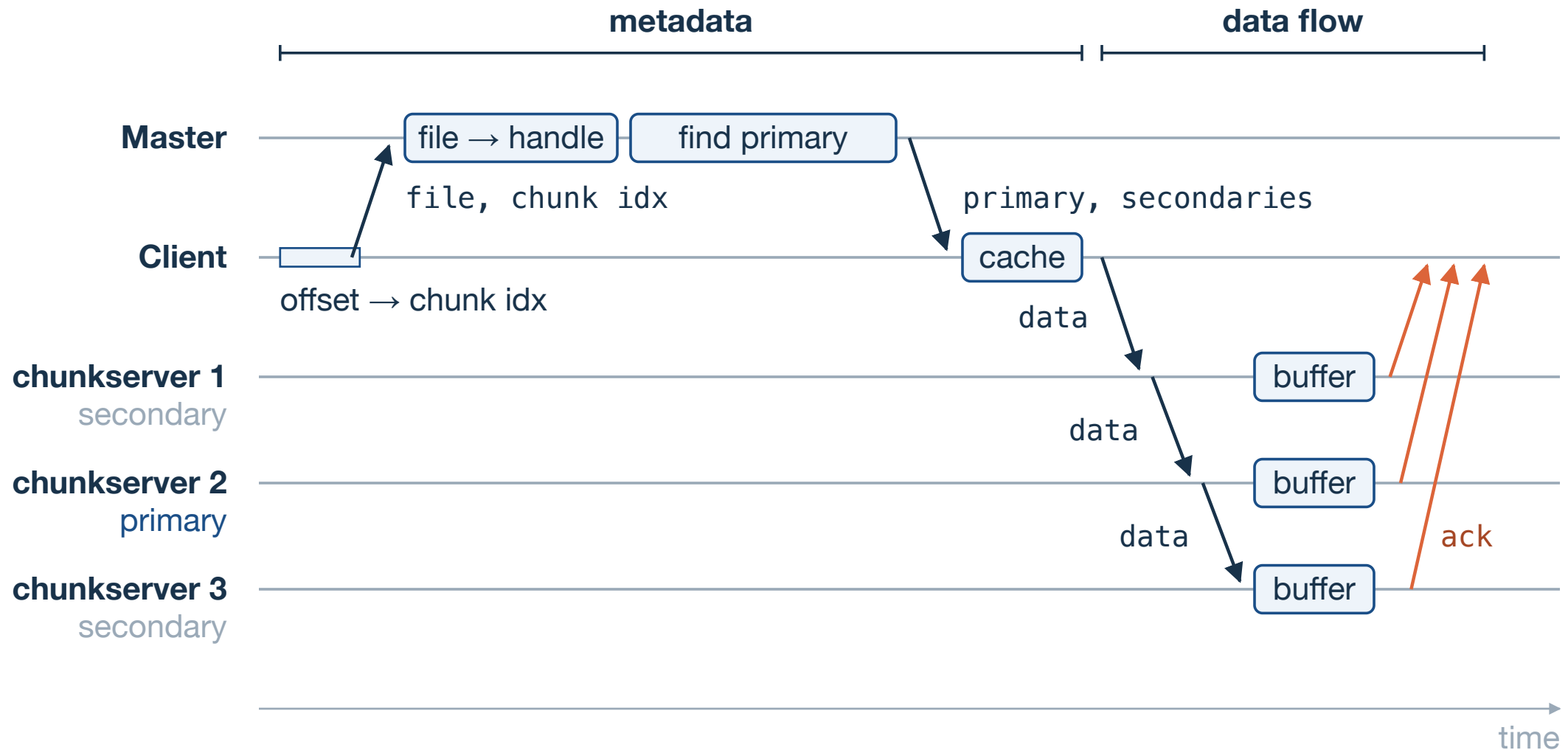
Writing a file in GFS: two phases

Phase 1: send data along a chain of replicas; chunkservers only buffer the data



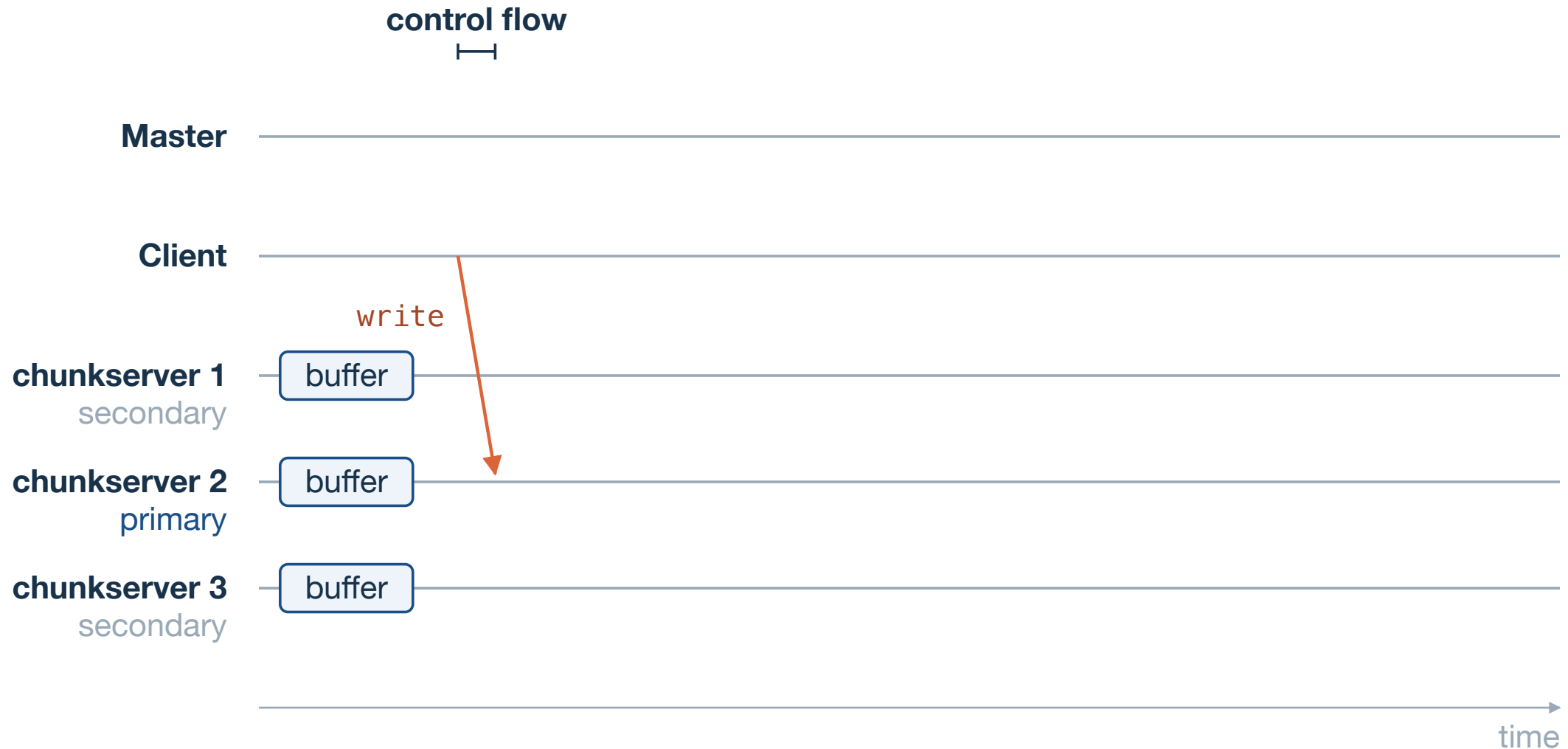
Writing a file in GFS: two phases

Phase 1: send data along a chain of replicas; chunkservers only buffer the data



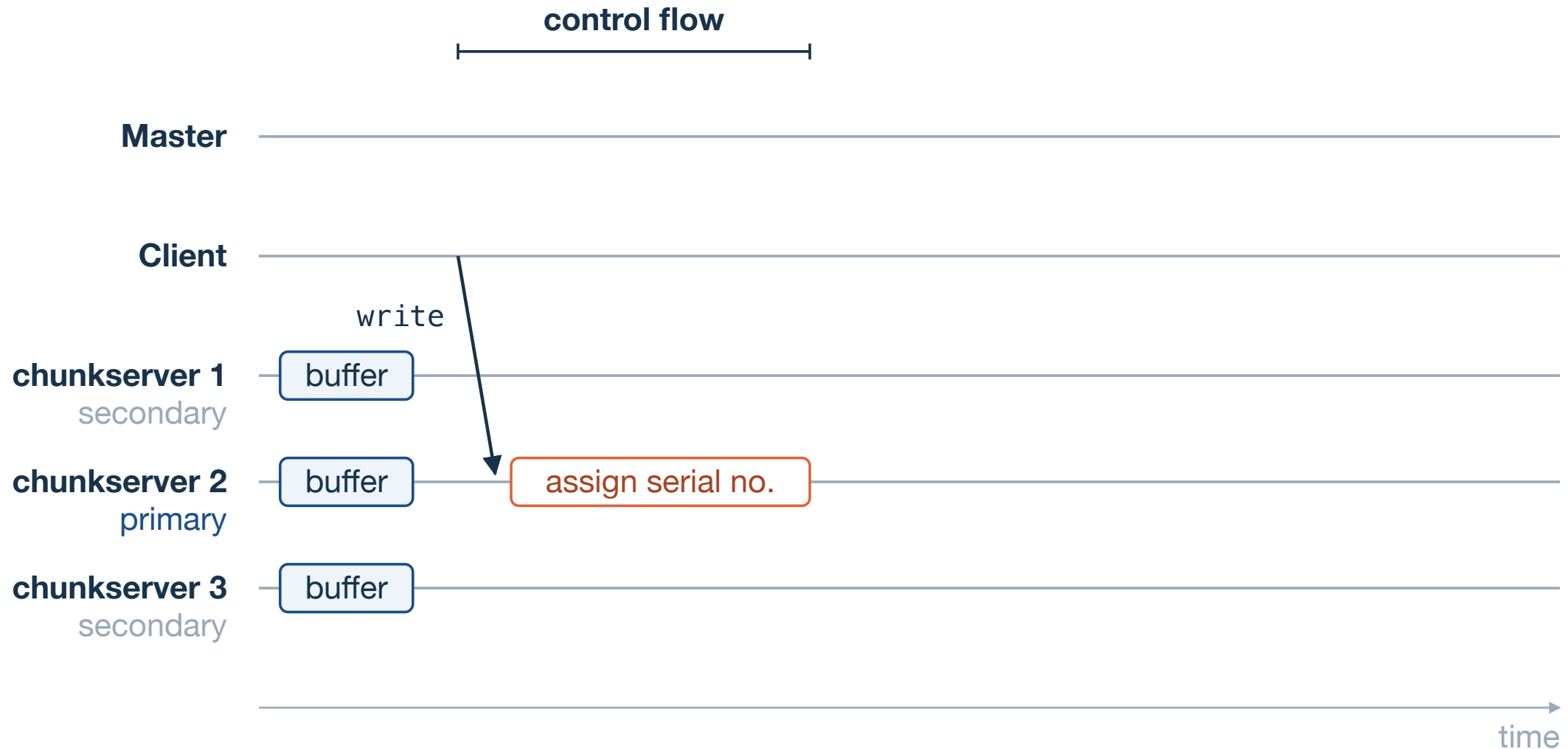
Writing a file in GFS: two phases

Phase 2: write. The **primary** assigns **serial numbers**; all replicas apply writes in that order



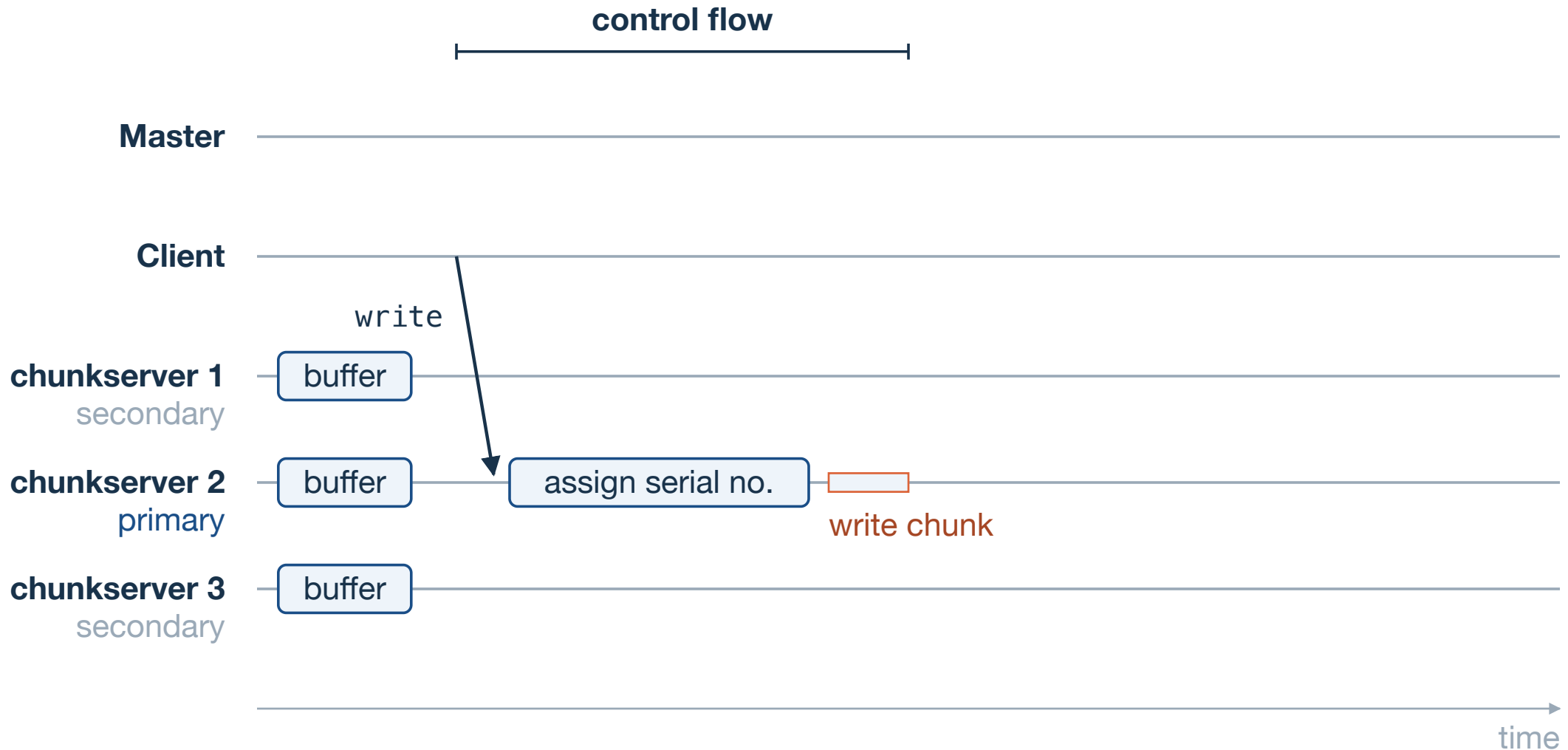
Writing a file in GFS: two phases

Phase 2: write. The **primary** assigns **serial numbers**; all replicas apply writes in that order



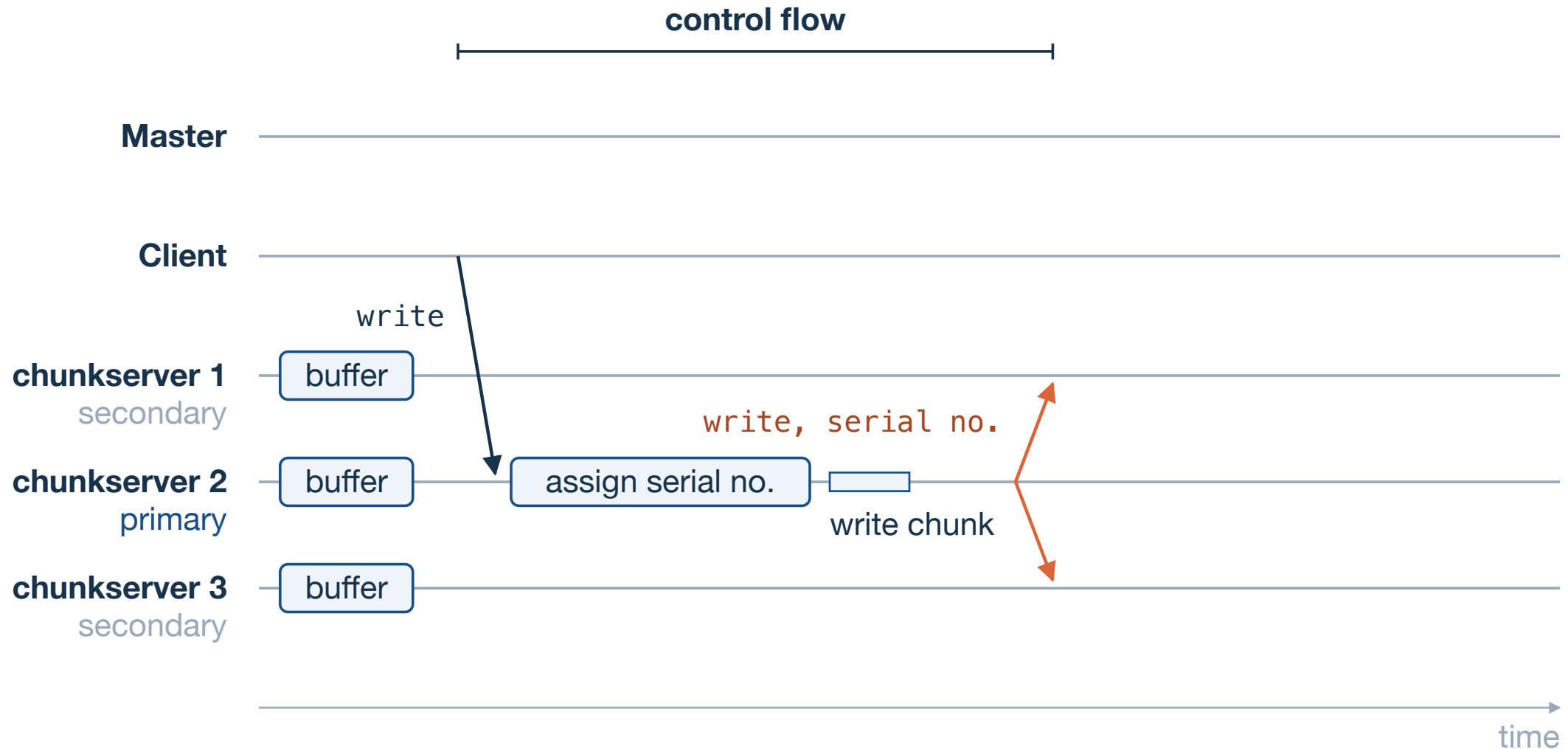
Writing a file in GFS: two phases

Phase 2: write. The **primary** assigns **serial numbers**; all replicas apply writes in that order



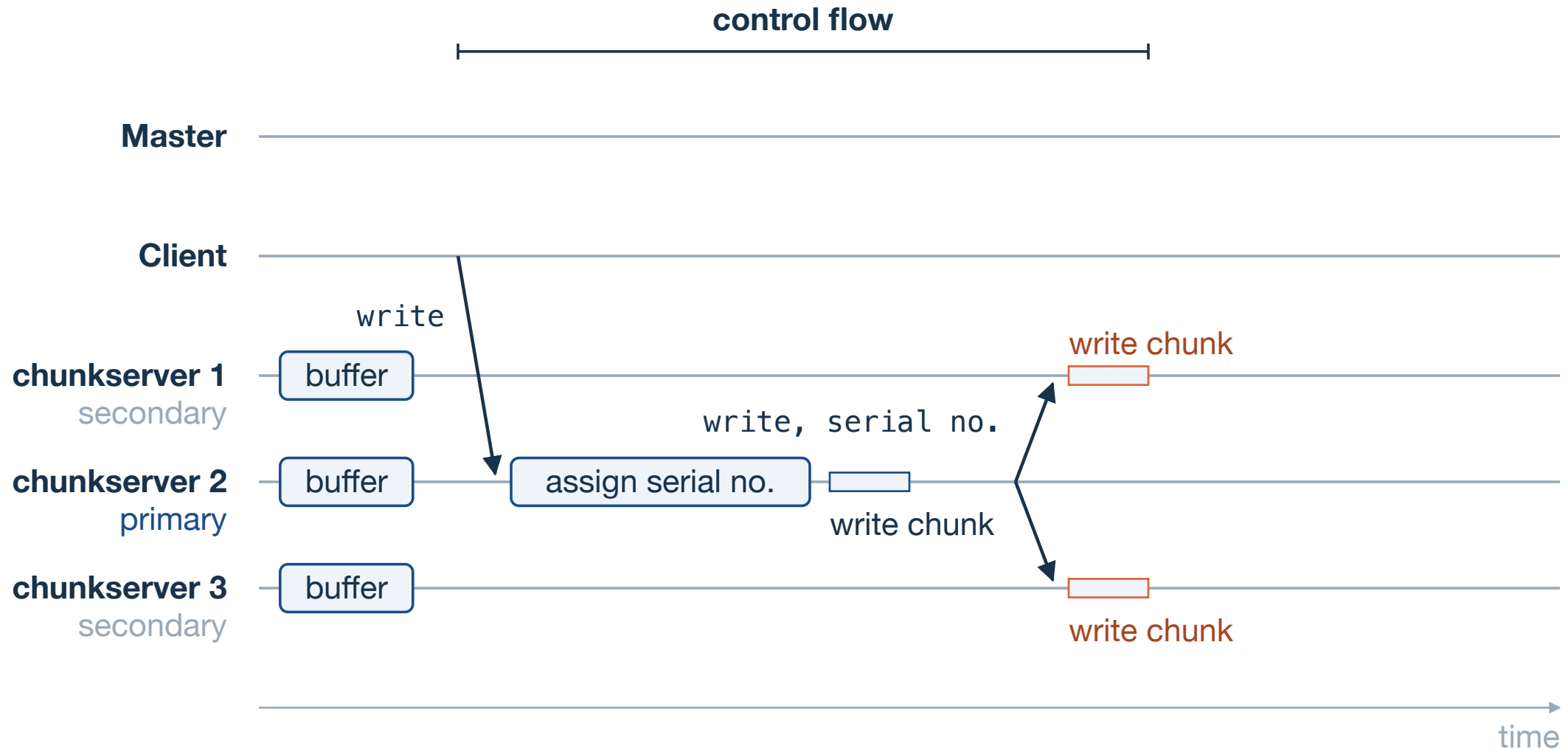
Writing a file in GFS: two phases

Phase 2: write. The **primary** assigns **serial numbers**; all replicas apply writes in that order



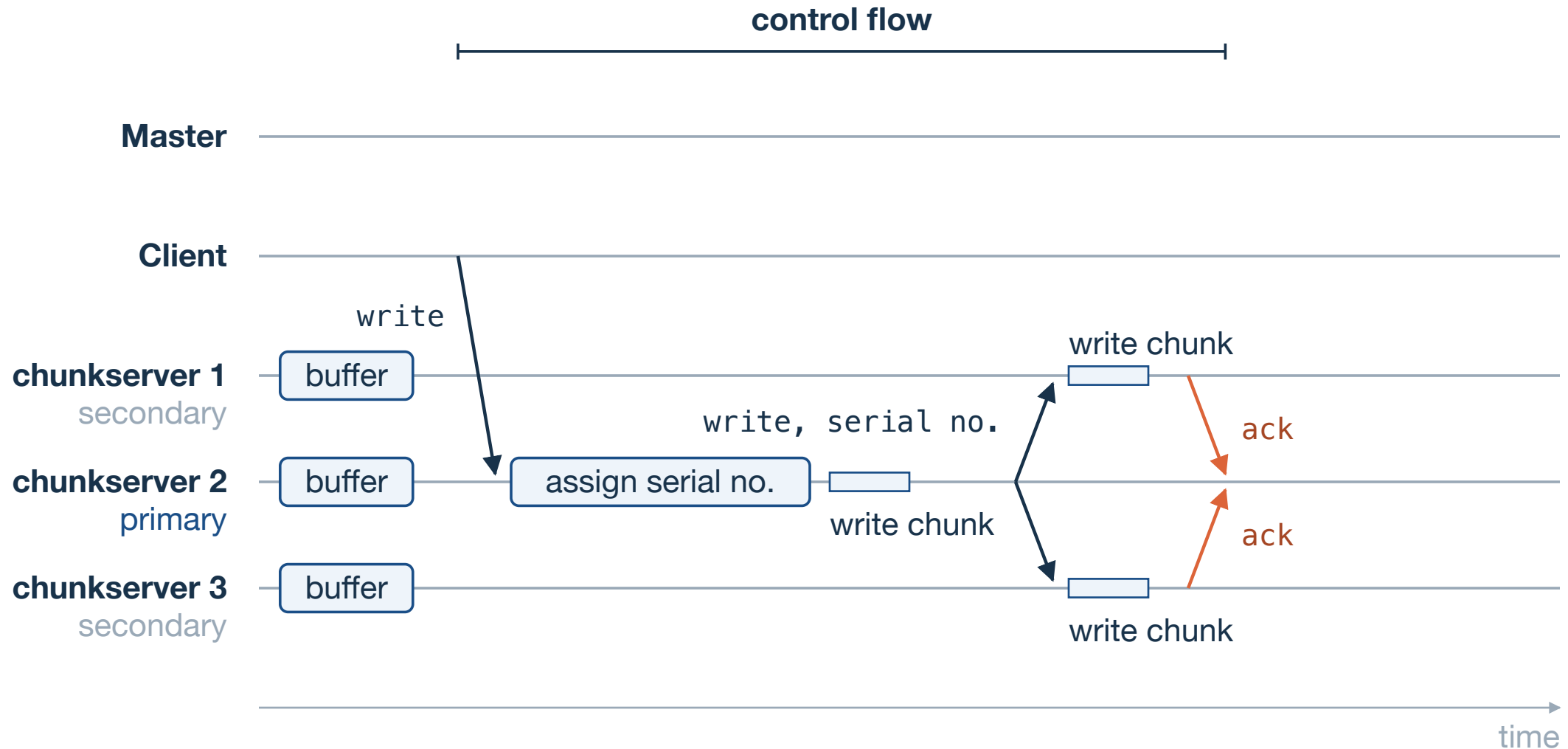
Writing a file in GFS: two phases

Phase 2: write. The **primary** assigns **serial numbers**; all replicas apply writes in that order



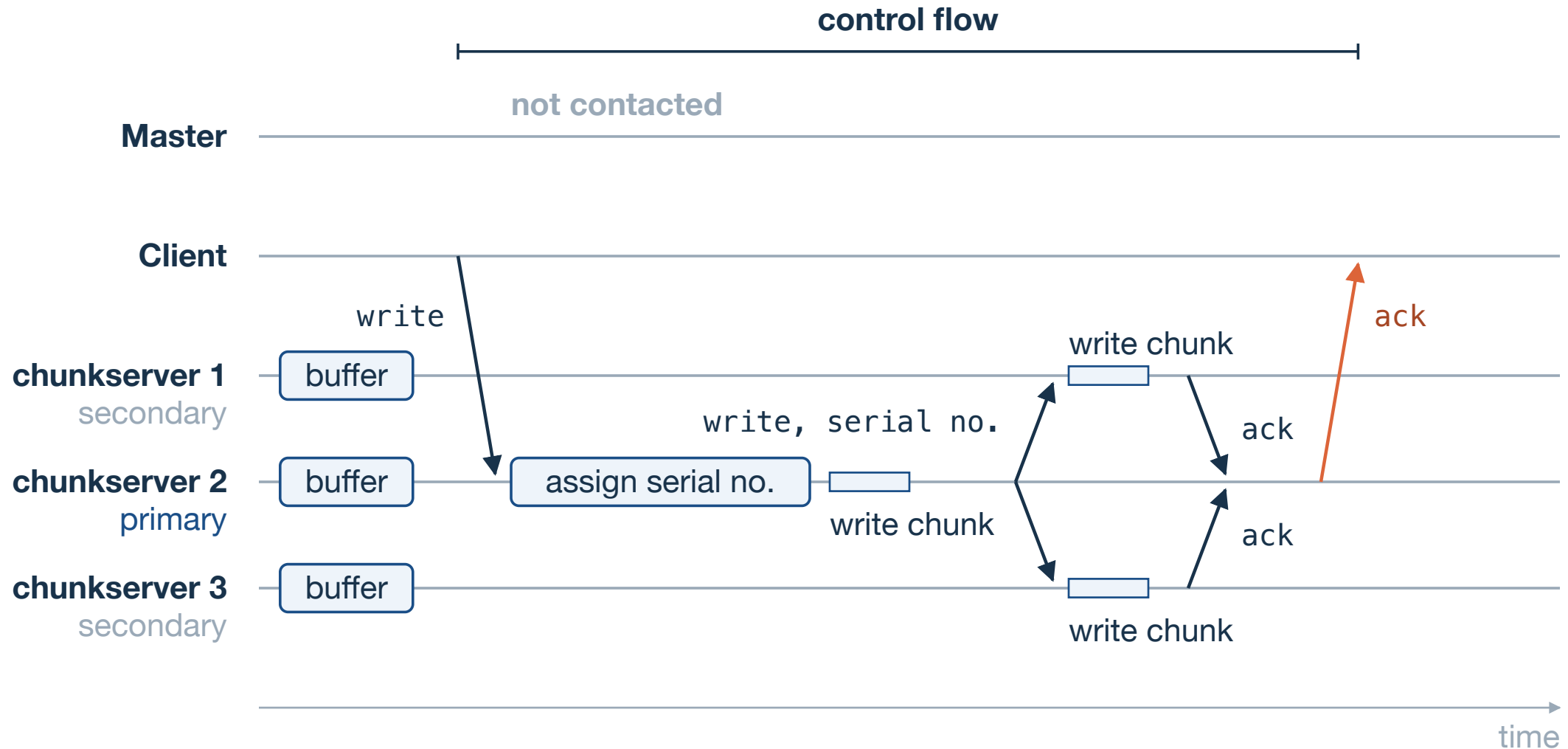
Writing a file in GFS: two phases

Phase 2: write. The **primary** assigns **serial numbers**; all replicas apply writes in that order



Writing a file in GFS: two phases

Phase 2: write. The **primary** assigns **serial numbers**; all replicas apply writes in that order



The primary tells every replica the write order

The primary **picks the order** and sends it in **one message**; all replicas apply writes in that order



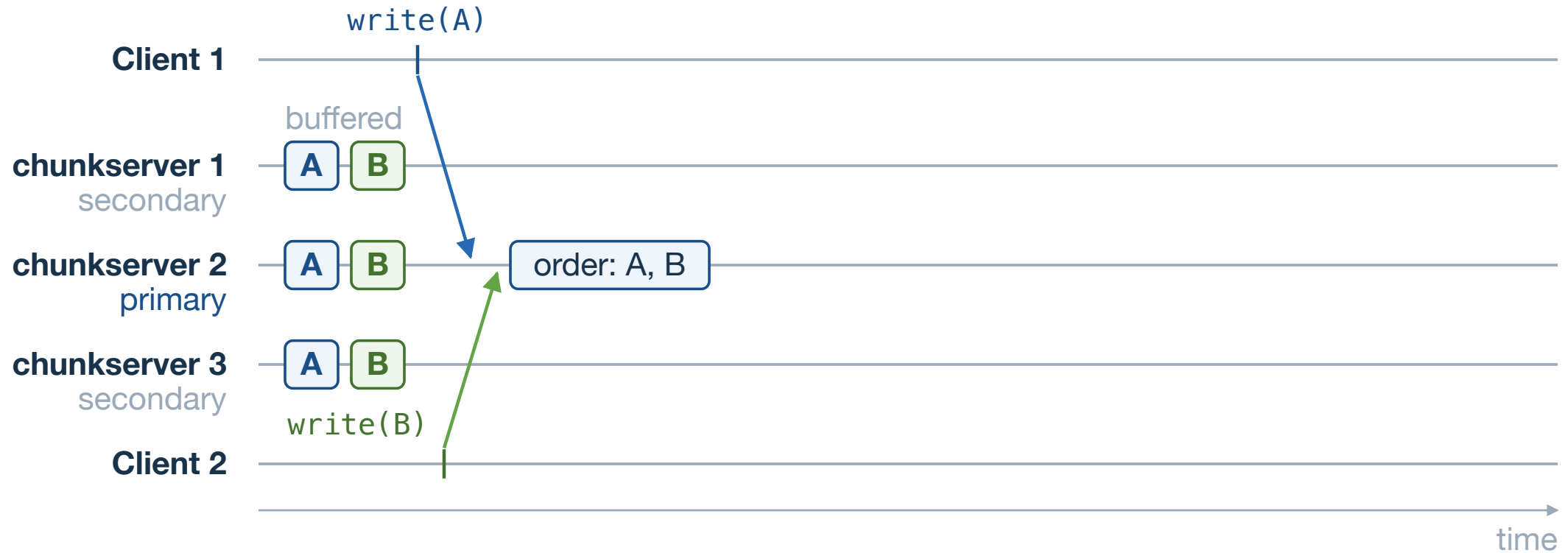
The primary tells every replica the write order

The primary **picks the order** and sends it in **one message**; all replicas apply writes in that order



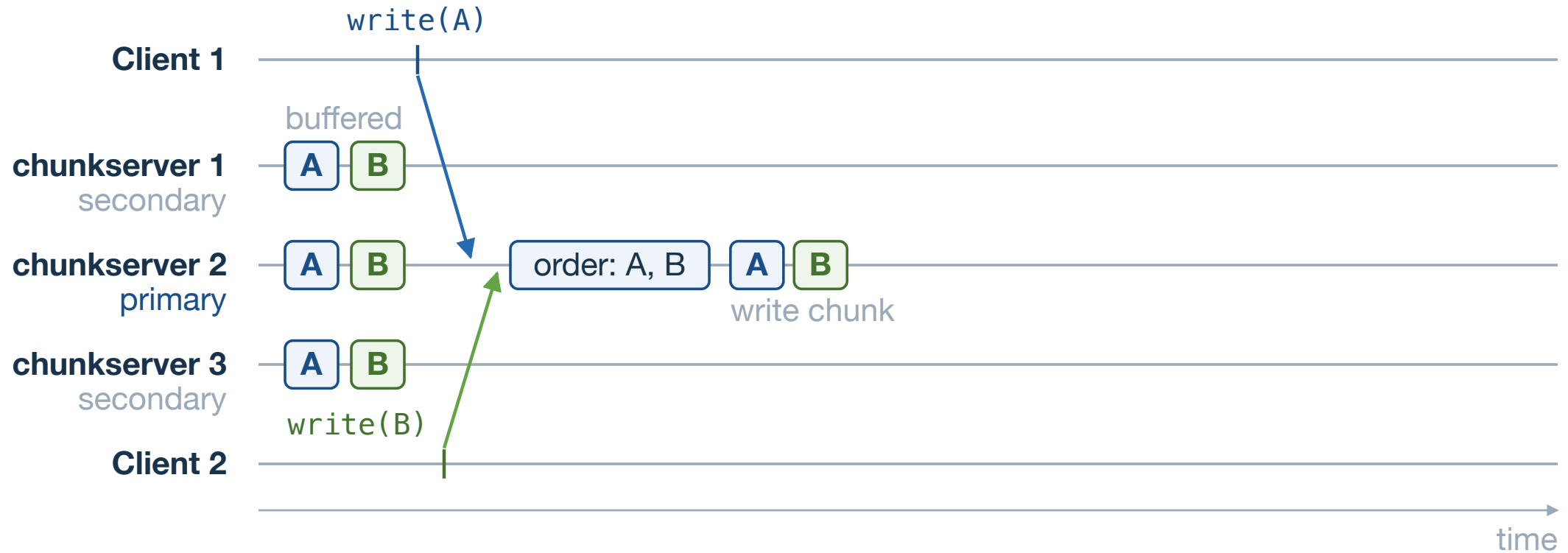
The primary tells every replica the write order

The primary **picks the order** and sends it in **one message**; all replicas apply writes in that order



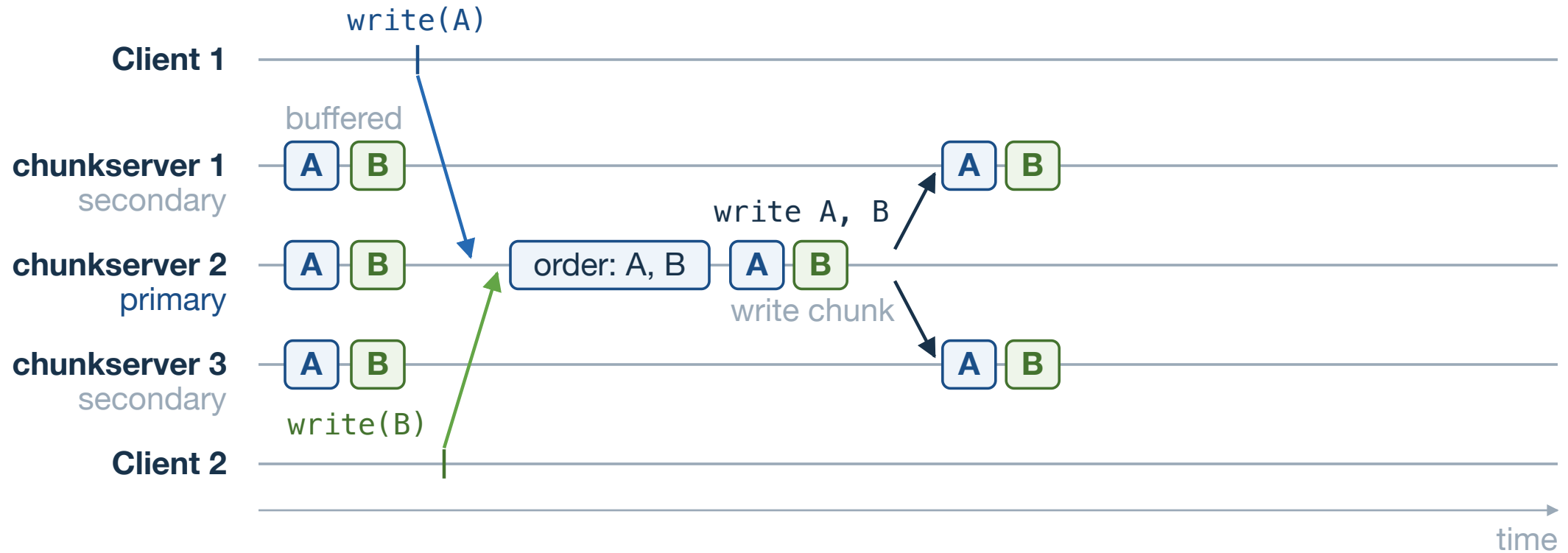
The primary tells every replica the write order

The primary **picks the order** and sends it in **one message**; all replicas apply writes in that order



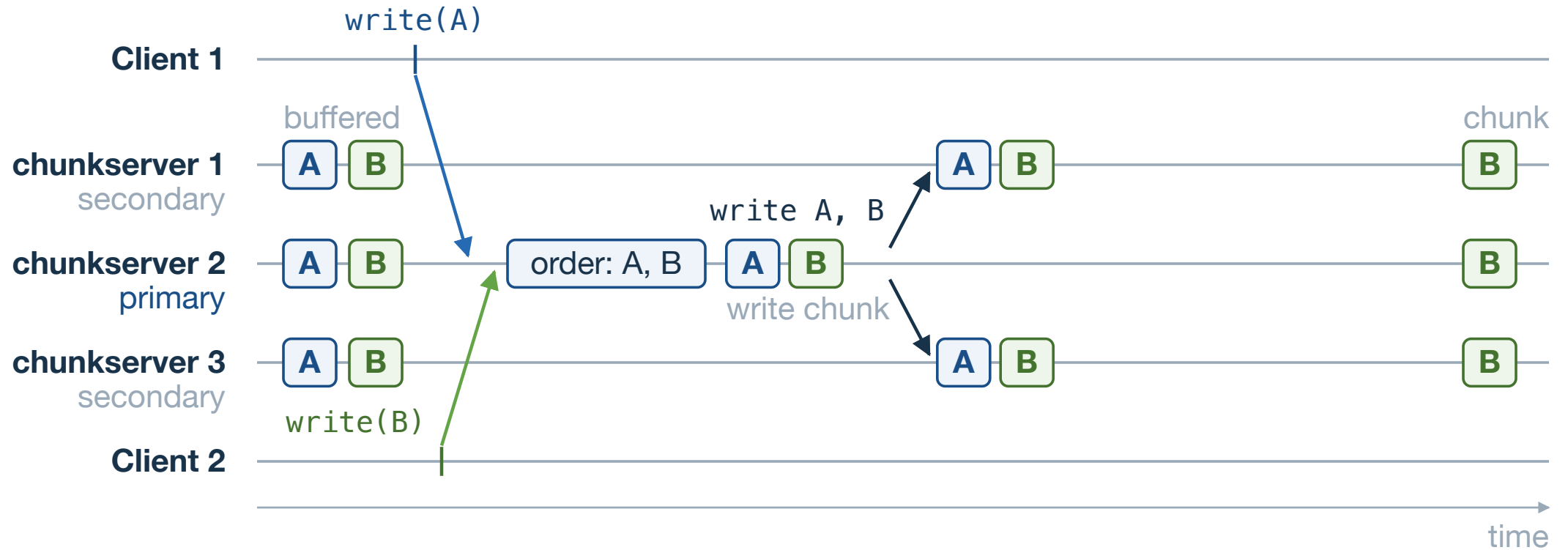
The primary tells every replica the write order

The primary **picks the order** and sends it in **one message**; all replicas apply writes in that order



The primary tells every replica the write order

The primary **picks the order** and sends it in **one message**; all replicas apply writes in that order



Writing a file in GFS: two phases

Data flow (phase 1) is different from **control flow** (phase 2)

Data flow

- Client → chunkserver → chunkserver → ...
- Order does not matter

Control flow

- Client → primary → all secondaries
- Primary: the replica holding a **lease** from the master (60 s, renewed via HeartBeat)
- Order maintained (also for concurrent writes from multiple clients)

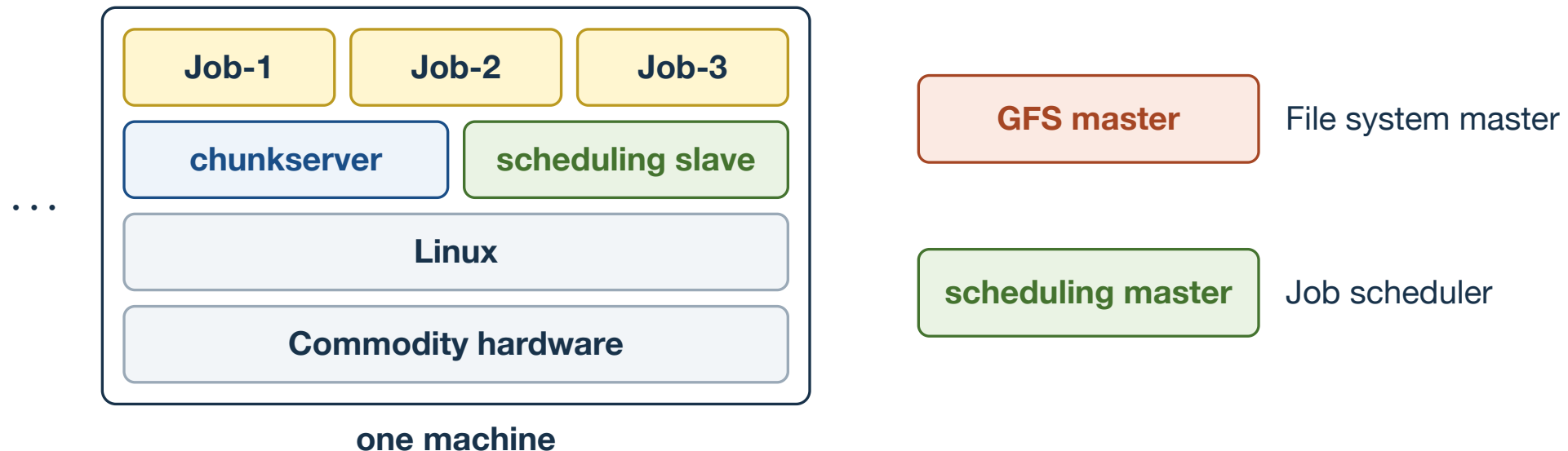
Chunk version numbers are used to detect if any replica has stale data

- Maintained by the master
- If a replica has stale data, it shall be replaced

Summary: GFS in Google clusters

Google cluster environment

- Core services: GFS + cluster scheduling system
- Typically, 100s to 1000s of active jobs
- 200+ clusters, many with 1000s of machines
- Pools of 1000s of clients
- 4+ PB file systems, 40 GB/s read/write loads



HDFS: another popular (open-source) DFS

Hadoop Distributed FS

Primary storage system for Hadoop apps



A **framework** that allows for the **distributed** processing of **large** data sets across clusters of computers



Design goals & assumptions of HDFS

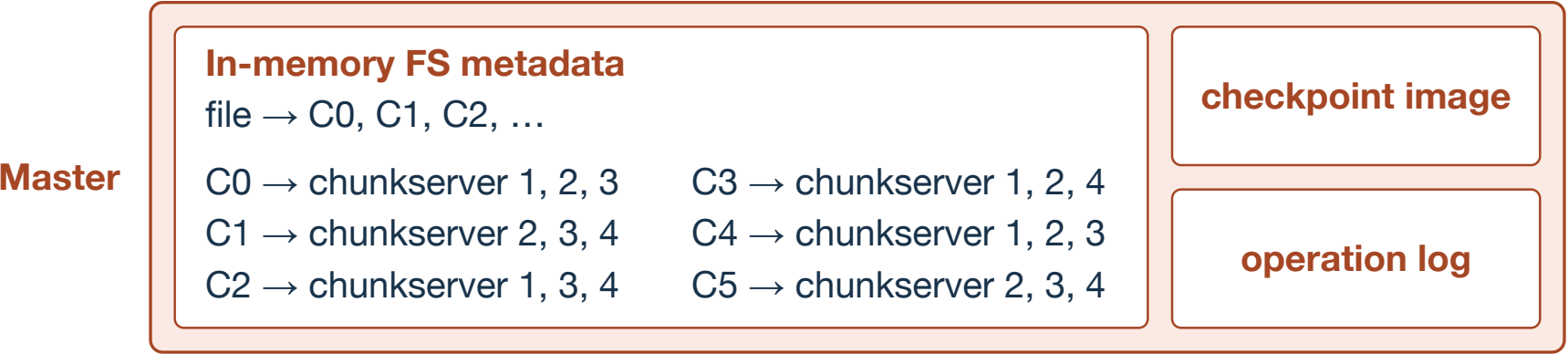
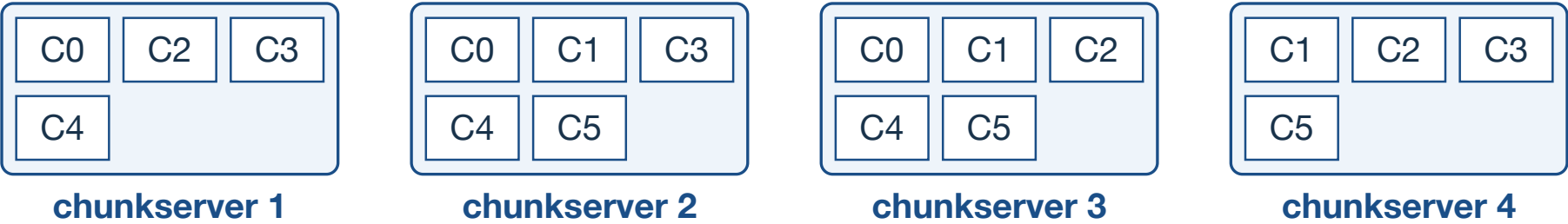
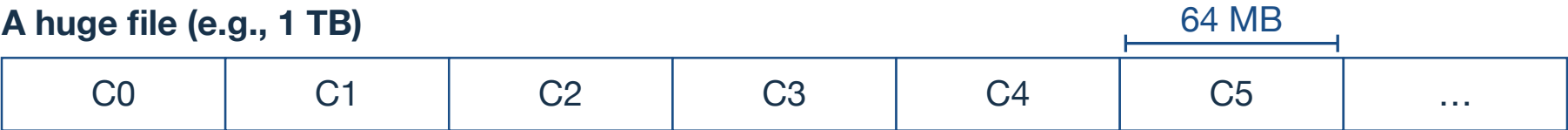
HDFS is an **open-source** (Apache) implementation **inspired by the GFS** design

Similar goals to GFS

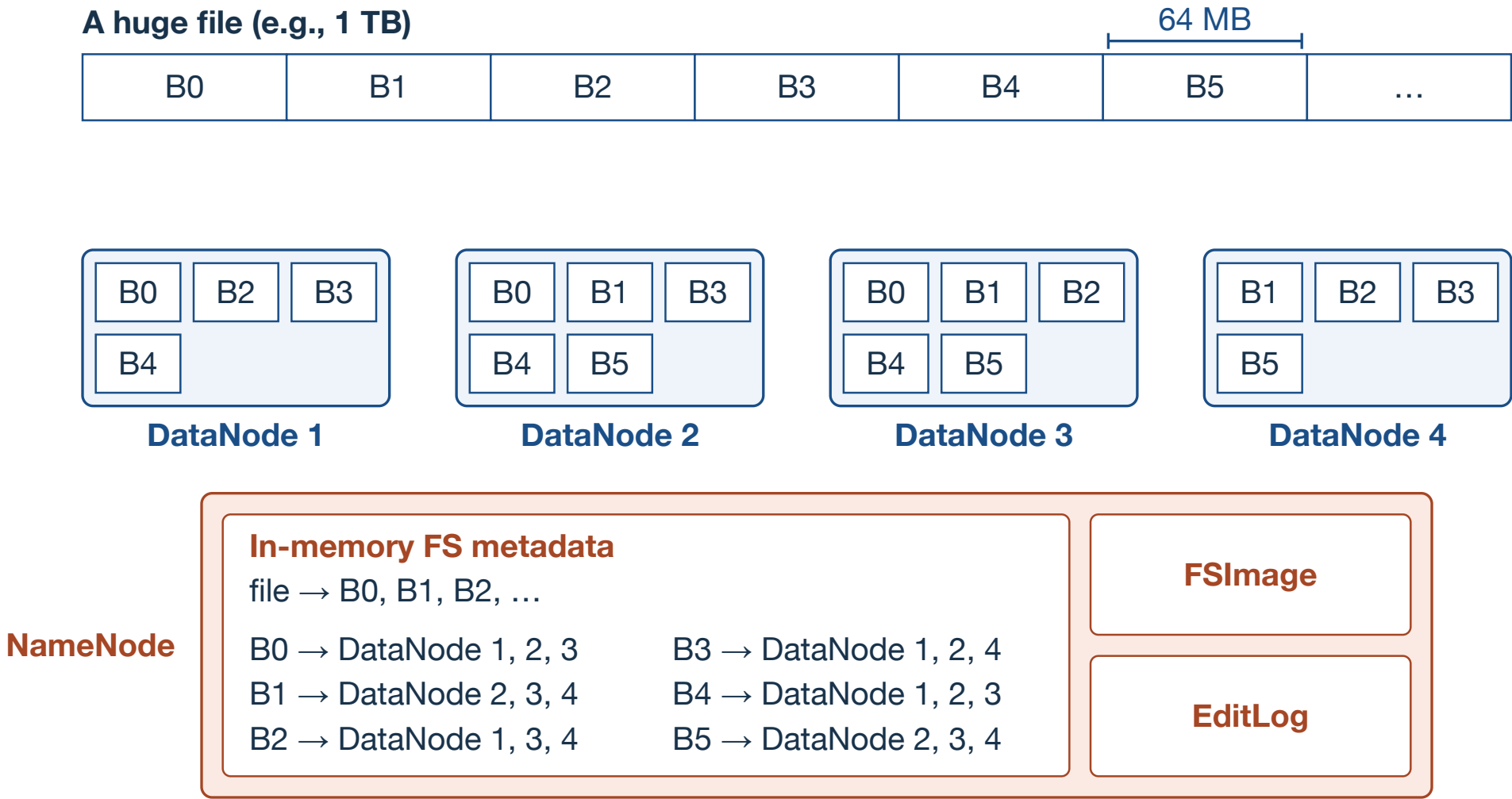
- Run on commodity hardware
- Highly fault tolerant
- High throughput & large-scale deployments
- ...

Recall GFS files

A huge file (e.g., 1 TB)



HDFS files

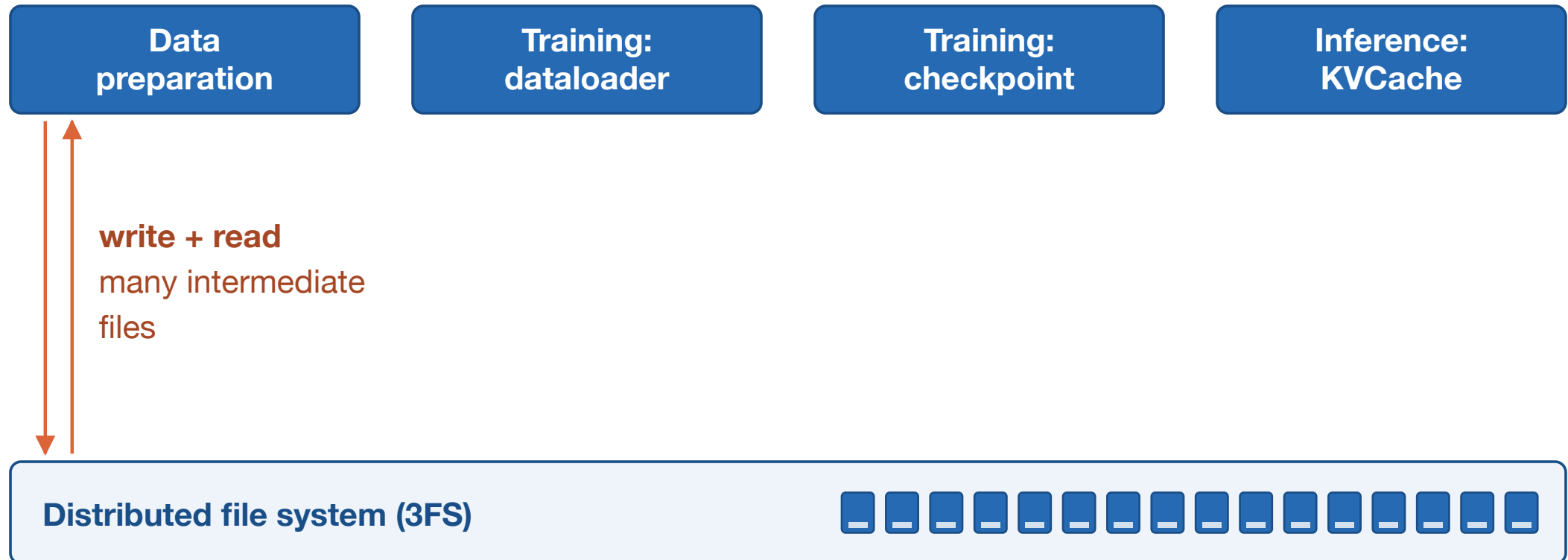


Case study: 3FS

The Fire-Flyer File System (DeepSeek)

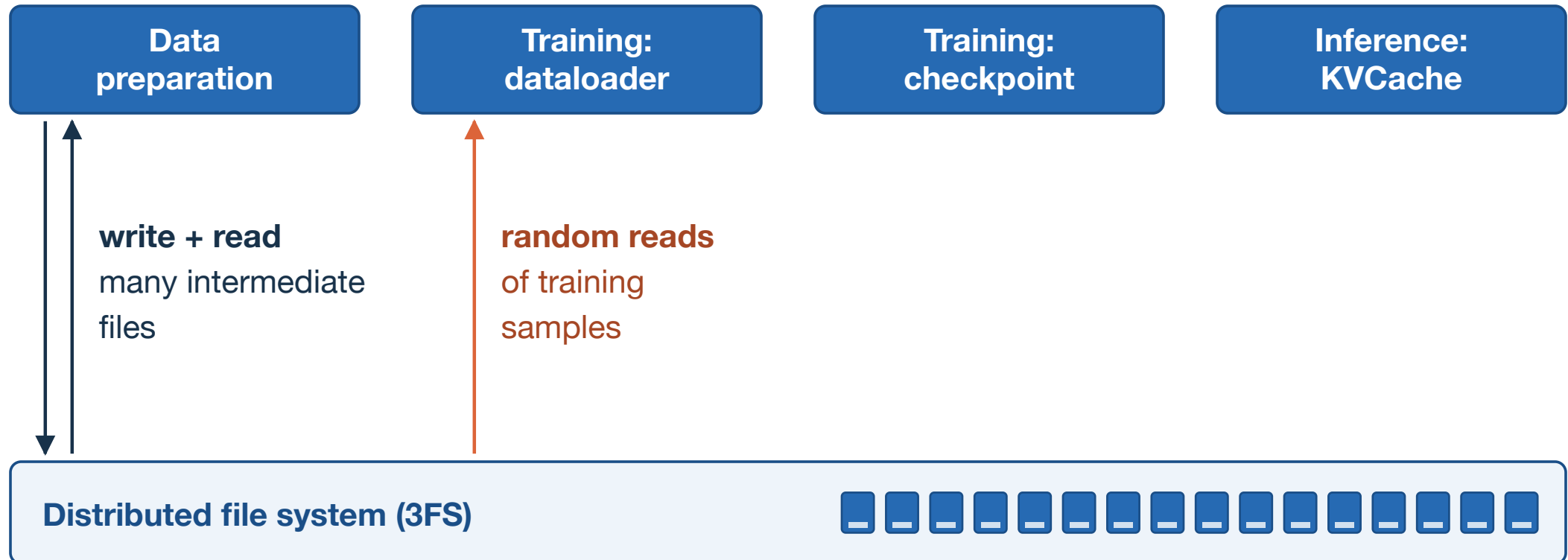
Files behind AI workloads

Training and inference keep their data as **files** on one **shared** distributed file system



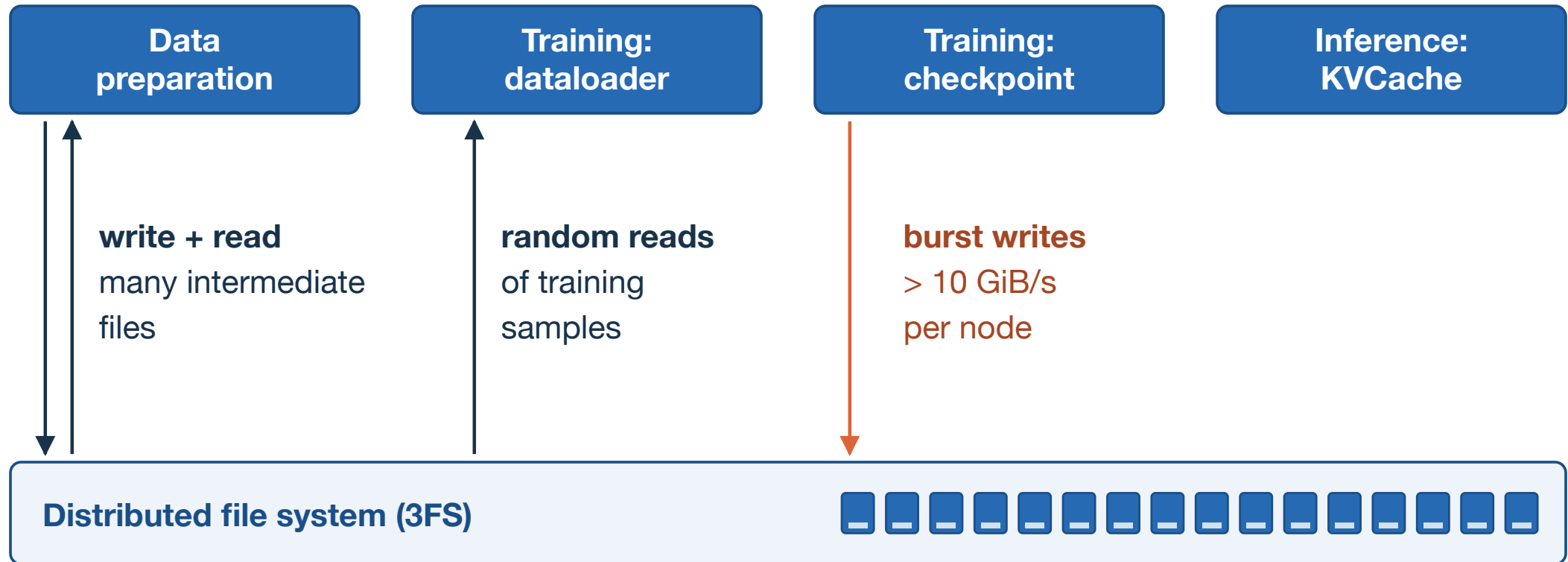
Files behind AI workloads

Training and inference keep their data as **files** on one **shared** distributed file system



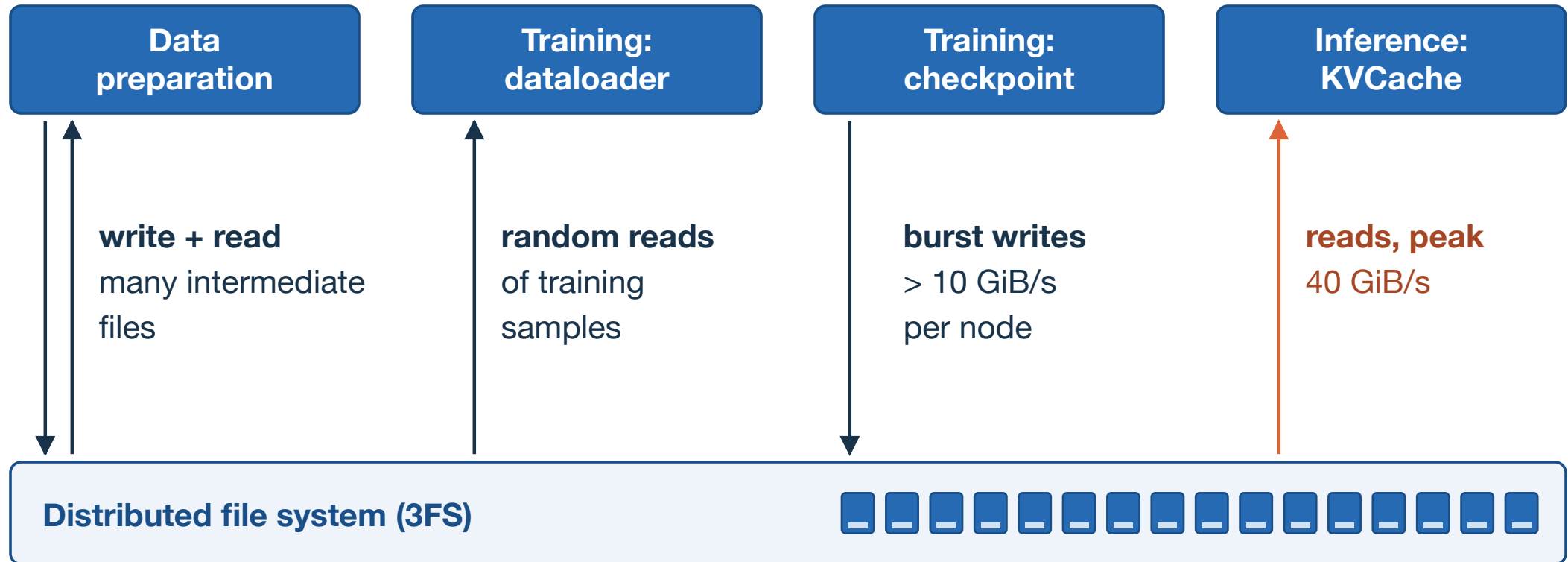
Files behind AI workloads

Training and inference keep their data as **files** on one **shared** distributed file system



Files behind AI workloads

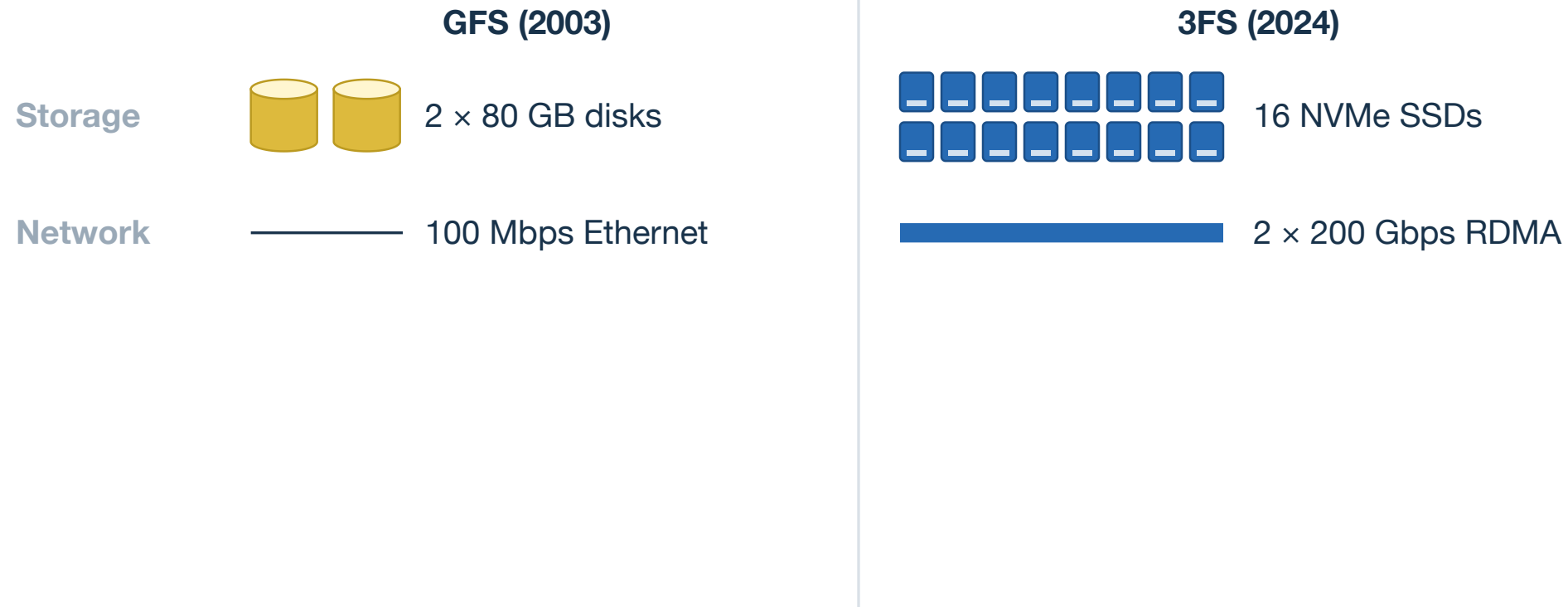
Training and inference keep their data as **files** on one **shared** distributed file system



What has changed since GFS?

Hardware per storage node: disks → NVMe SSDs; network 100 Mbps → 2 × 200 Gbps (4000×)

Workload: a few huge files read sequentially → many small files read randomly¹

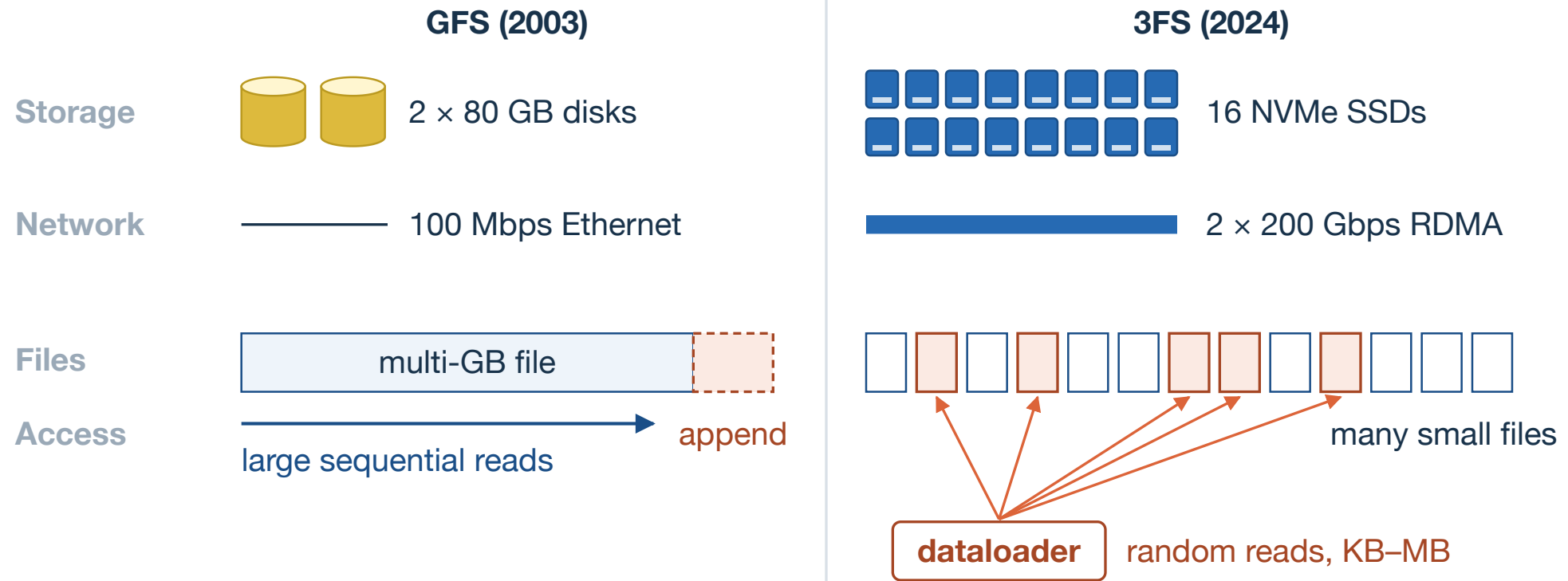


¹ 3FS design notes: "a large number of small files"; "random access to dataset samples ... a few kilobytes to several megabytes"

What has changed since GFS?

Hardware per storage node: disks → NVMe SSDs; network 100 Mbps → 2 × 200 Gbps (4000×)

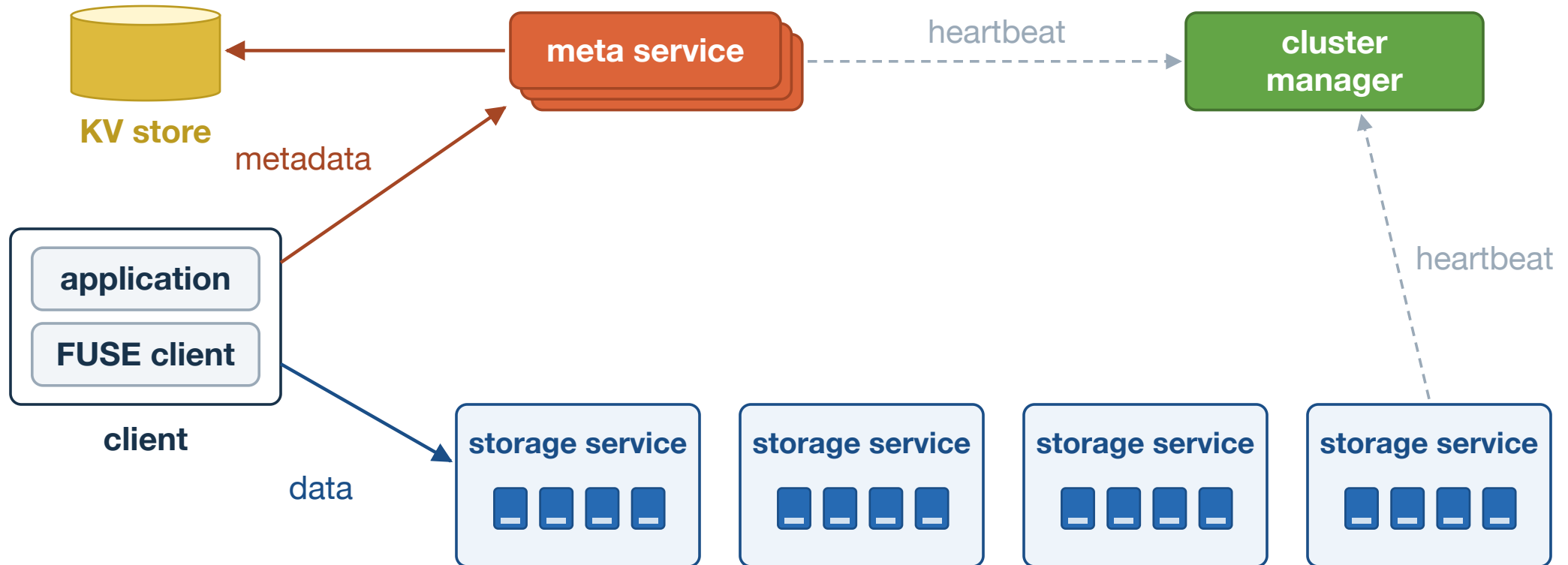
Workload: a few huge files read sequentially → many small files read randomly¹



¹ 3FS design notes: "a large number of small files"; "random access to dataset samples ... a few kilobytes to several megabytes"

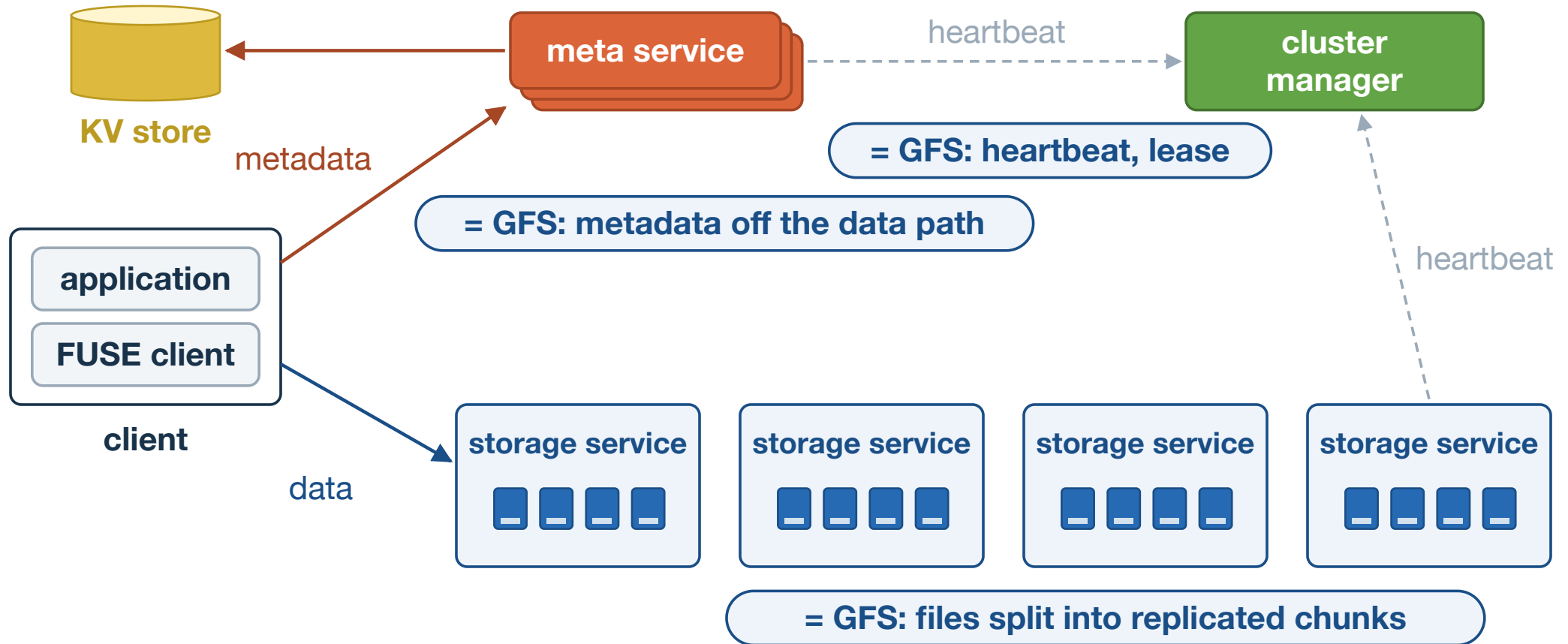
3FS keeps many designs of GFS and NFS

Four roles: **cluster manager**, **meta service**, **storage service** and **client**



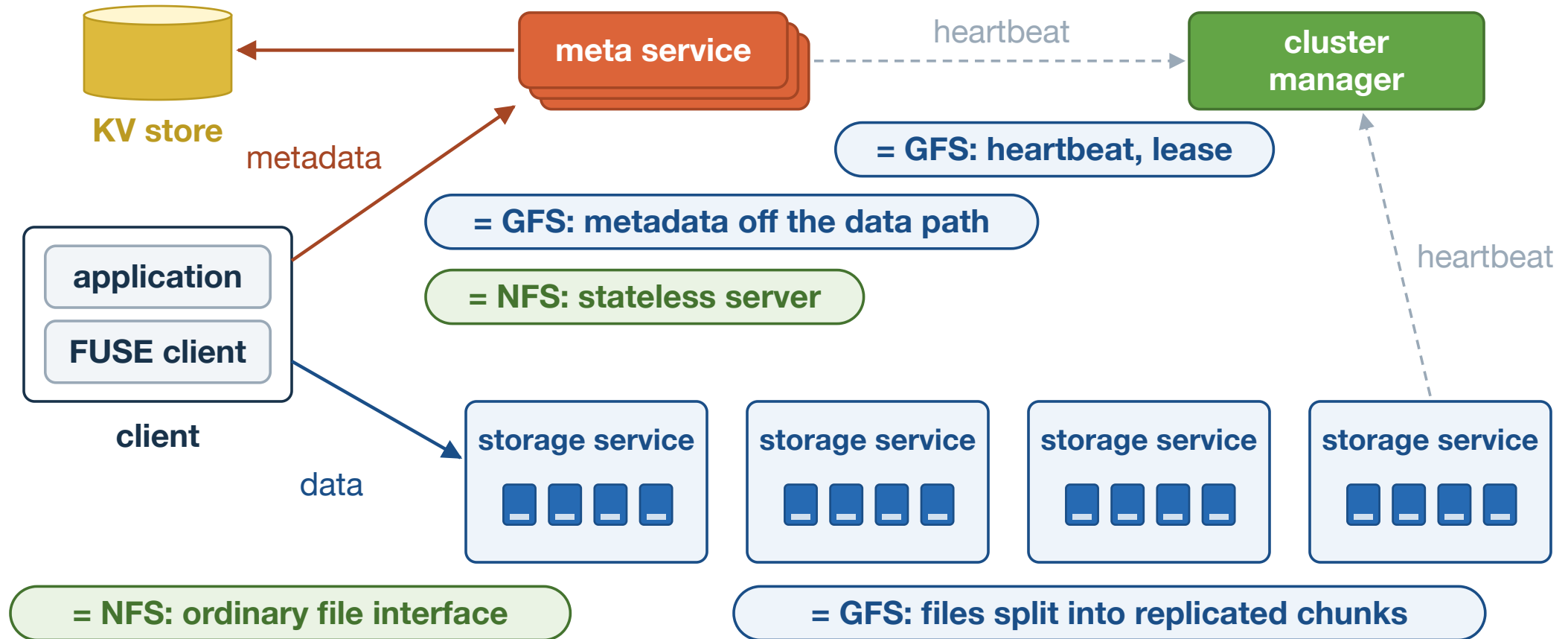
3FS keeps many designs of GFS and NFS

Four roles: **cluster manager**, **meta service**, **storage service** and **client**



3FS keeps many designs of GFS and NFS

Four roles: **cluster manager**, **meta service**, **storage service** and **client**



Where does 3FS keep metadata?

The Fire-Flyer paper: metadata lives in a **distributed key-value store**; meta services keep **no state** of their own

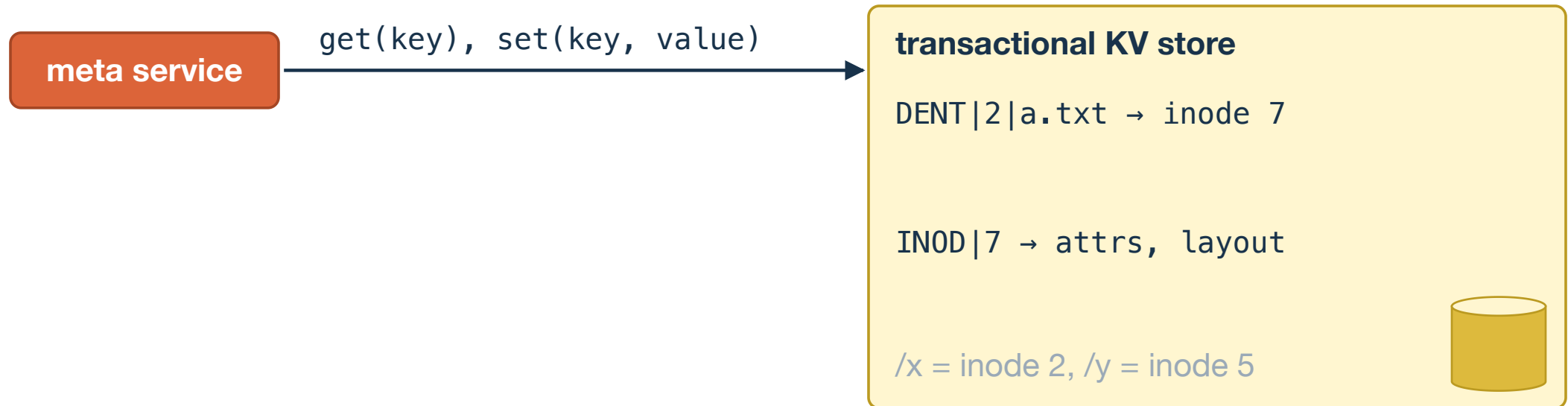
File system meta data are stored in tables of a distributed key-value storage system. Each file or directory has a unique inode ID. The File inode/directory ID and meta data, such as file size and location information of the file content data, are stored as key-value pairs in the inode table. A separate directory entry table stores key-value pairs of $(parent_dir_inode_id, entry_name) : (entry_inode_id, \dots)$ to support iterating entries in a directory and resolving file/directory paths. All states of meta services are persisted on the distributed key-value storage system. Several meta services run concurrently to handle meta requests from clients.

The store is **FoundationDB**, a **transactional** key-value store

- Whose design is quite complicated, and will be covered in later lectures

What is a transactional key-value store?

Transaction: several reads and writes, applied **all or nothing** (see later lectures)



What is a transactional key-value store?

Transaction: several reads and writes, applied **all or nothing** (see later lectures)

meta service

rename /x/a.txt → /y/a.txt

begin

clear DENT|2|a.txt

set DENT|5|a.txt → inode 7

commit

example, simplified from 3FS

transactional KV store

~~DENT|2|a.txt → inode 7~~

DENT|5|a.txt → inode 7

INOD|7 → attrs, layout

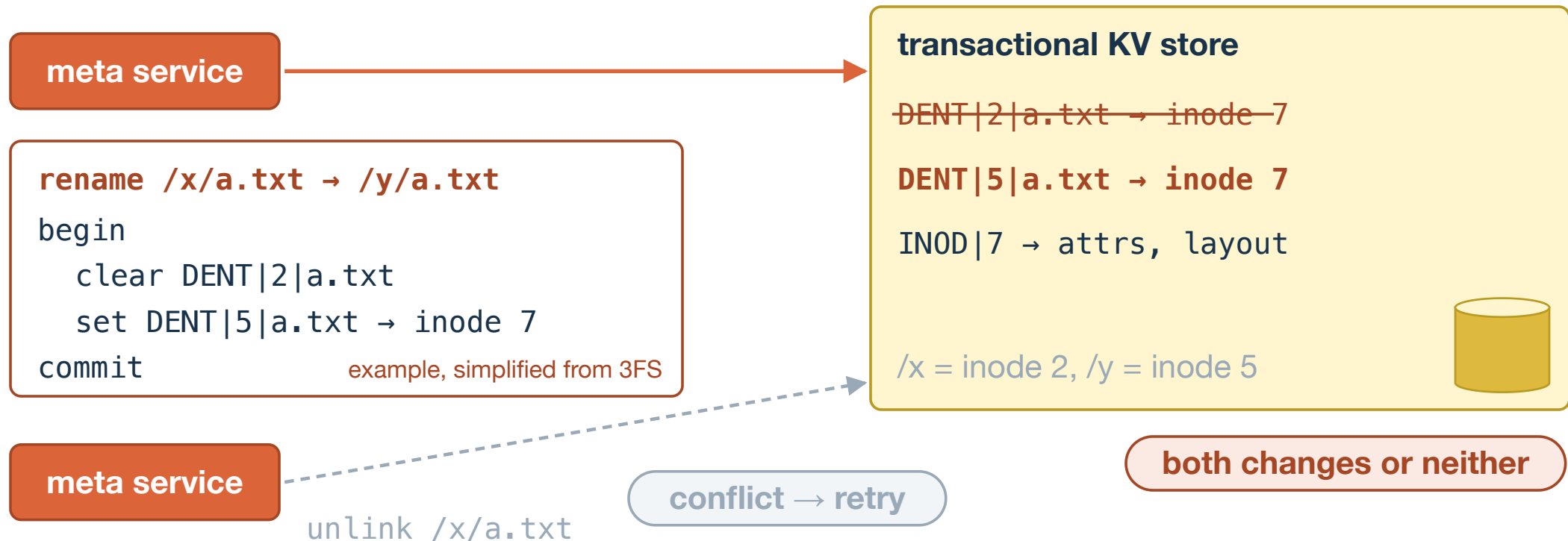
/x = inode 2, /y = inode 5



both changes or neither

What is a transactional key-value store?

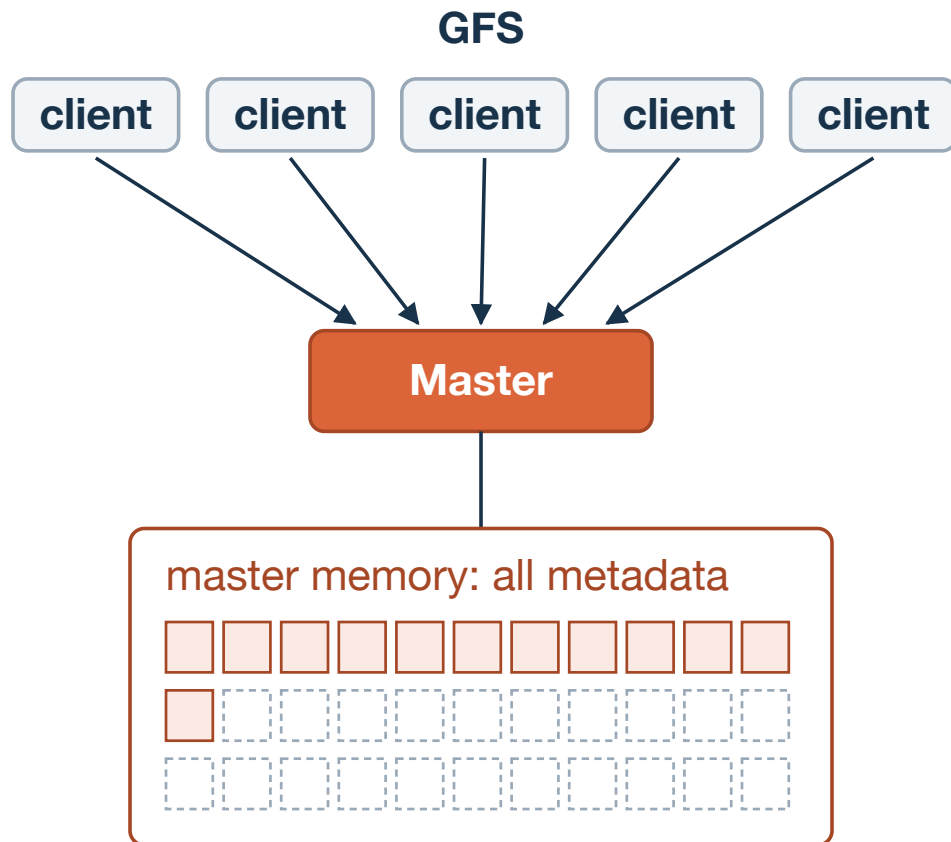
Transaction: several reads and writes, applied **all or nothing** (see later lectures)



3FS keeps inodes and directory entries as key-value pairs in FoundationDB

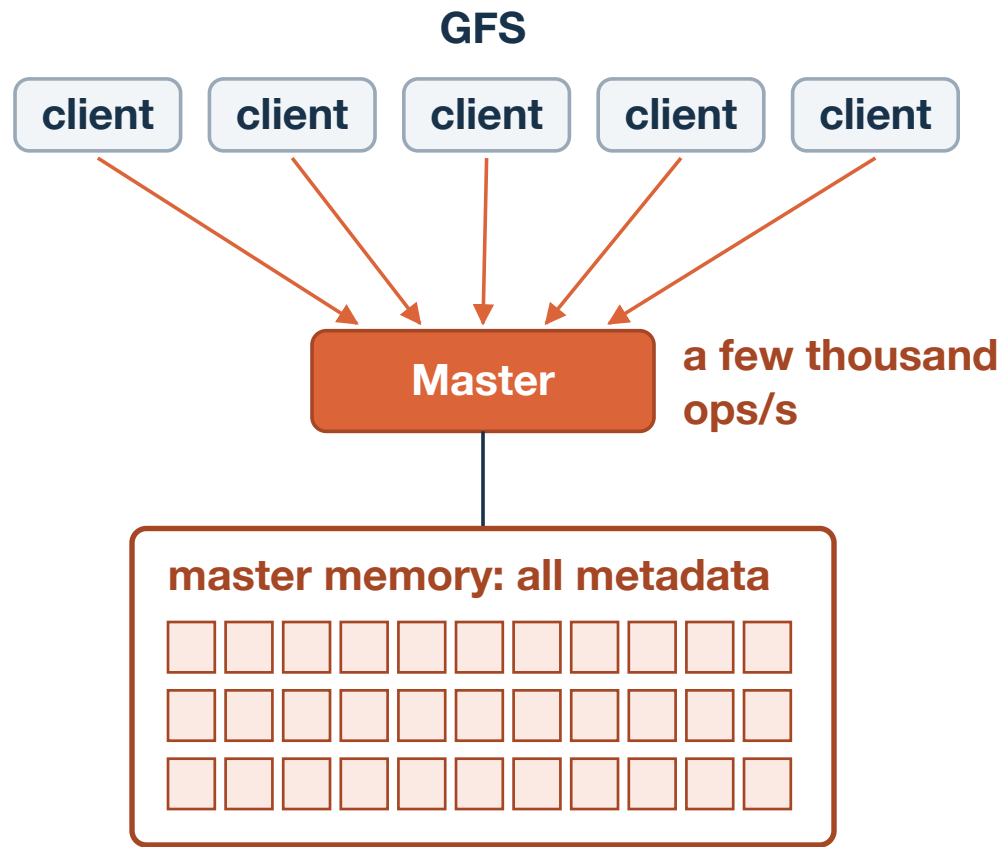
Why not a single master?

GFS: one master keeps **all metadata in memory**; many files and many clients make it the **limit**



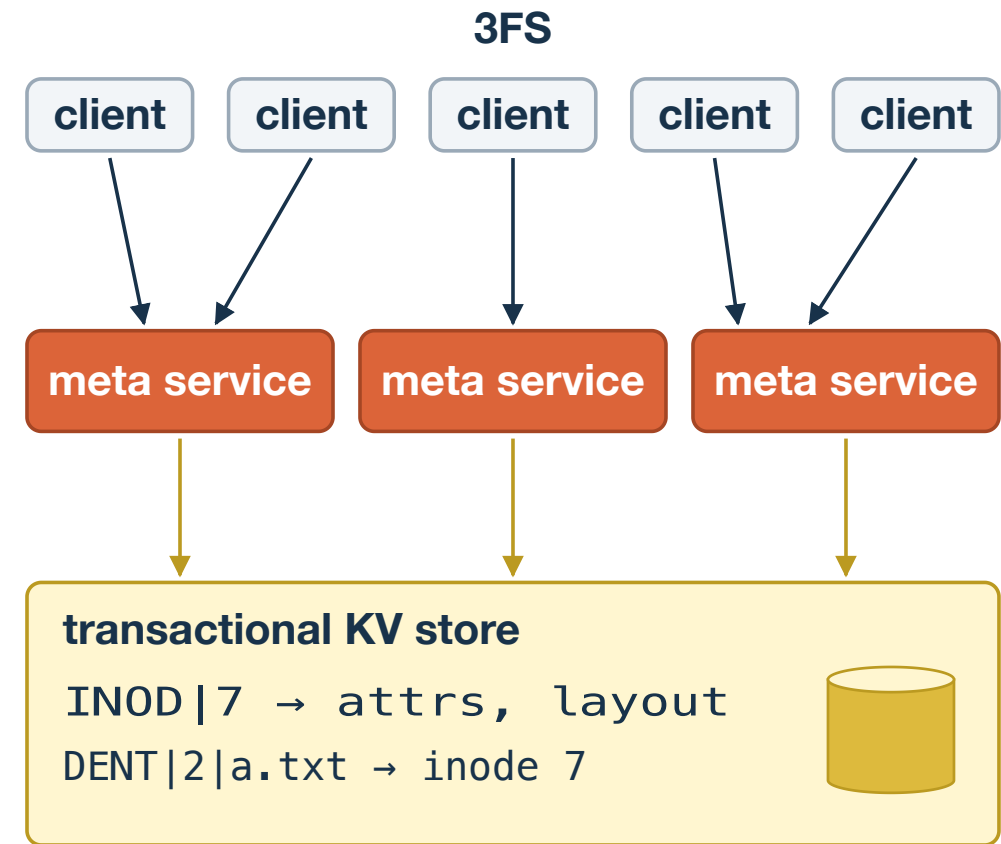
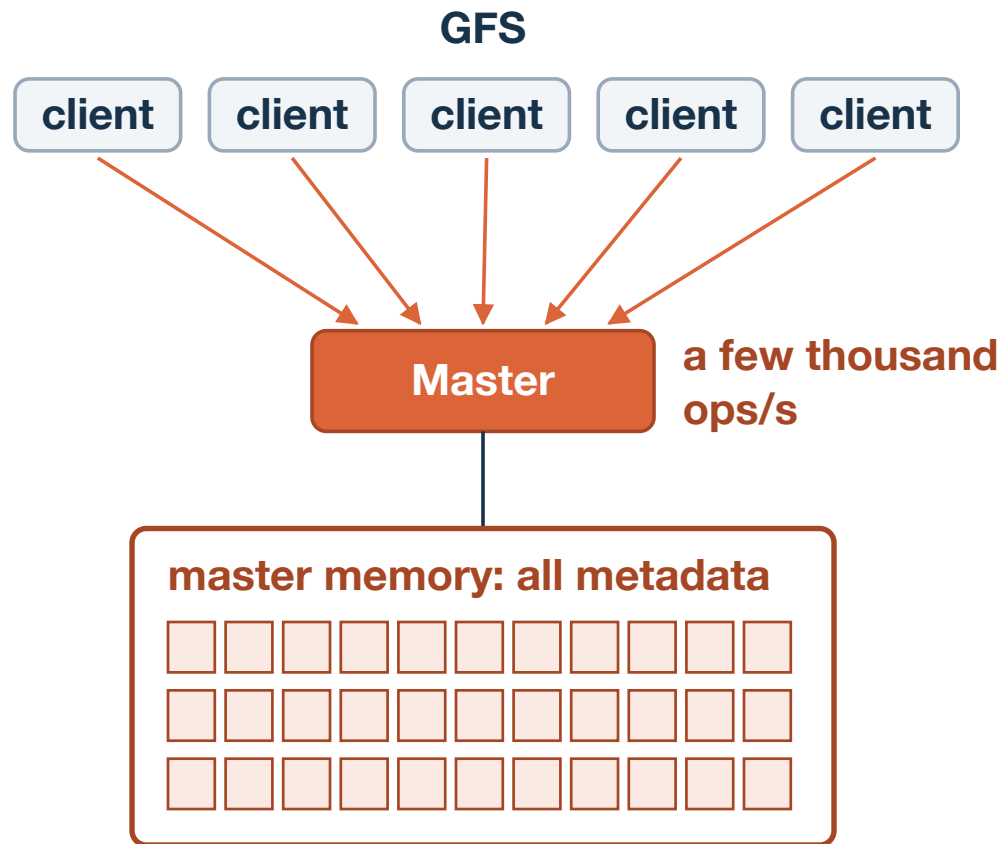
Why not a single master?

GFS: one master keeps **all metadata in memory**; many files and many clients make it the **limit**



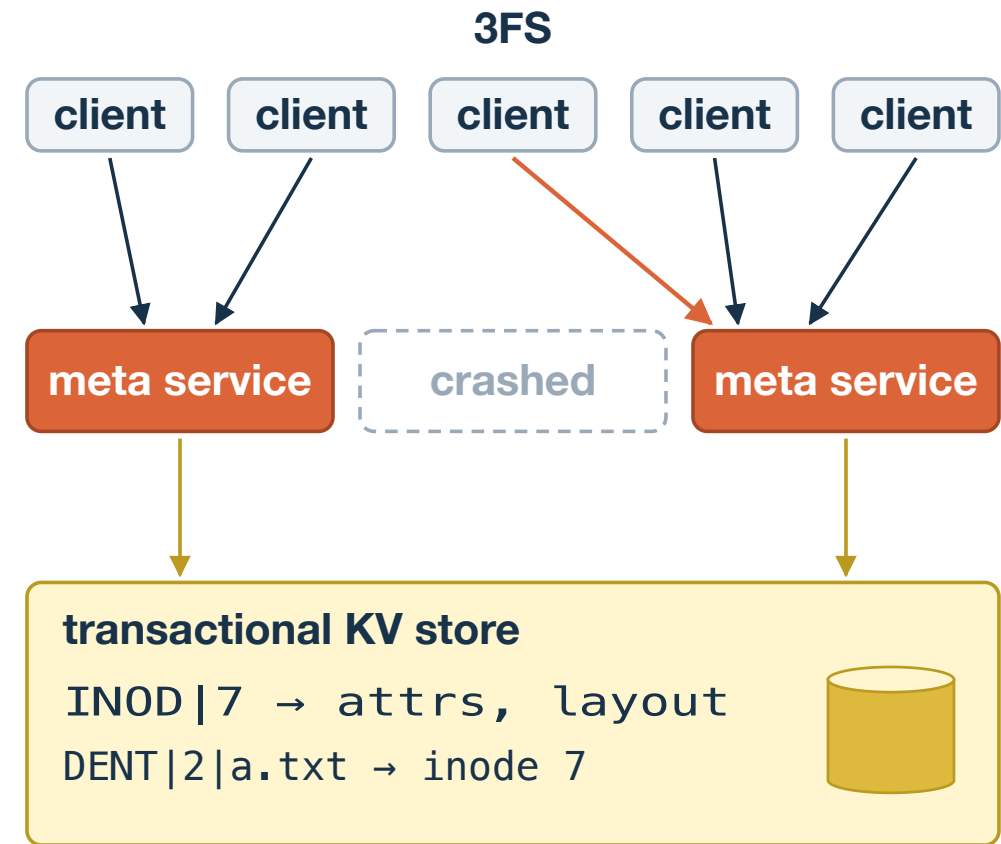
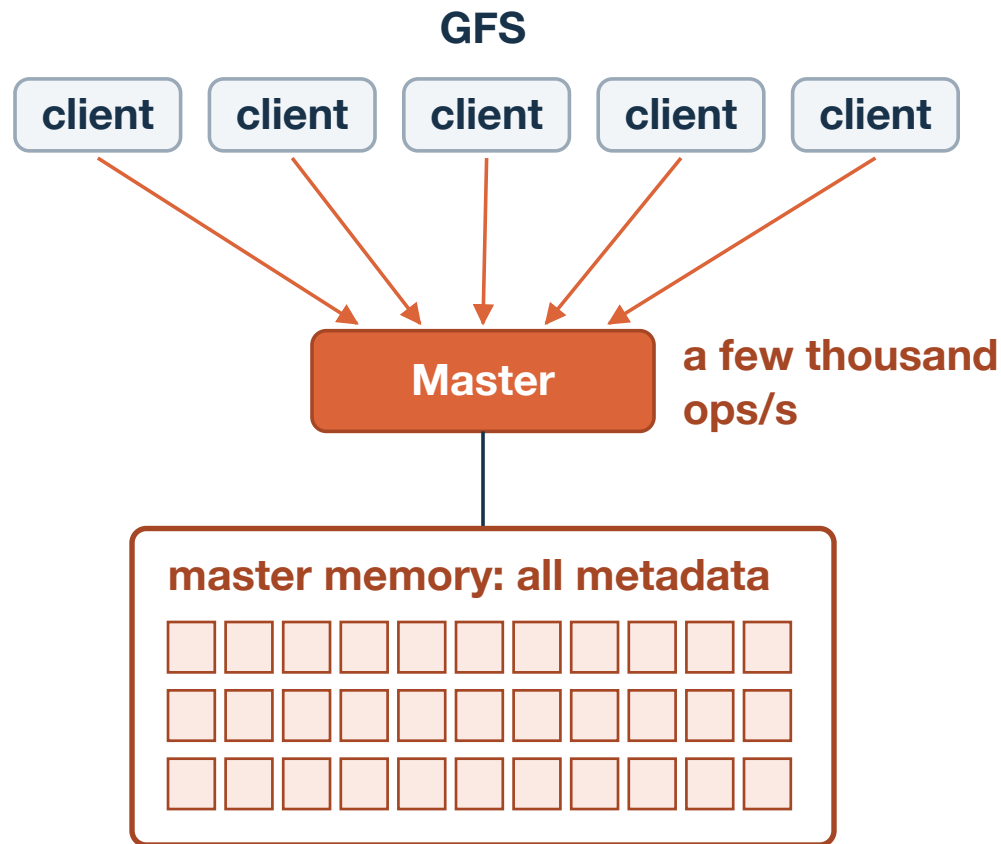
Why not a single master?

GFS: one master keeps **all metadata in memory**; many files and many clients make it the **limit**



Why not a single master?

GFS: one master keeps **all metadata in memory**; many files and many clients make it the **limit**



rename = **one KV transaction**; meta services keep **no state**

Which opened files does 3FS track?

The 3FS design notes: only files opened for **writing**

3FS / docs / design_notes.md 

↑ Top

Preview Code Blame 290 lines (171 loc) · 31.1 KB

Raw     

Dynamic file attributes

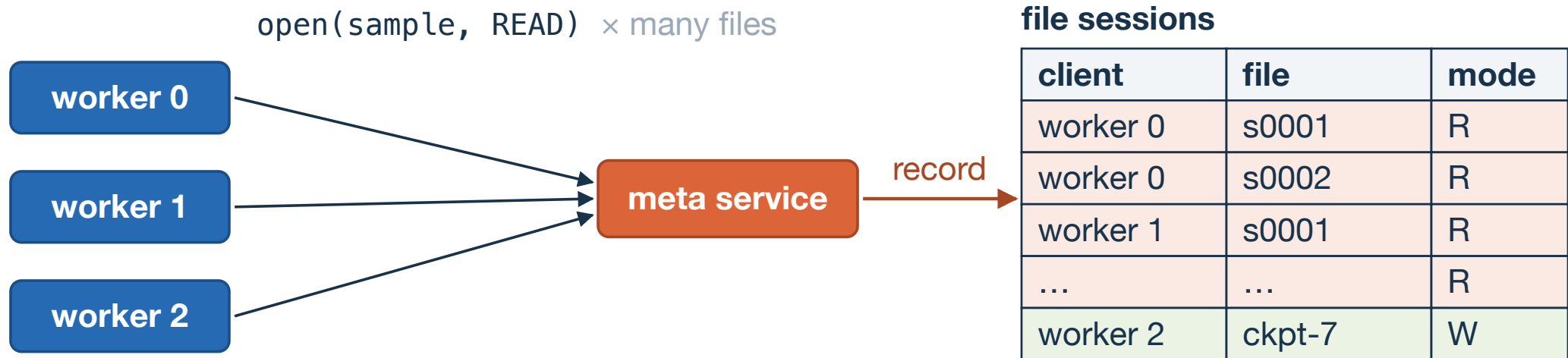
On most local file systems, deleting an opened file is deferred until all associated file descriptors are closed. Consequently, it is necessary to track all file descriptors of the file. Training jobs open a large number of files during startup. Storing all file descriptors would impose heavy load on meta service and FoundationDB. Since training jobs do not depend on this feature, 3FS does not track file descriptors opened in read-only mode.

3FS maintains a file session for each file descriptor (fd) opened in write mode since deleting write opened files may lead to unreclaimable garbage chunks from concurrent writes. When a file with active write sessions is deleted, meta service delays the deletion until all its fds are closed. To prevent lingering sessions from offline clients, the 3FS meta service periodically checks client liveness and cleans up sessions of offline clients.

¹ Source: https://github.com/deepseek-ai/3FS/blob/main/docs/design_notes.md

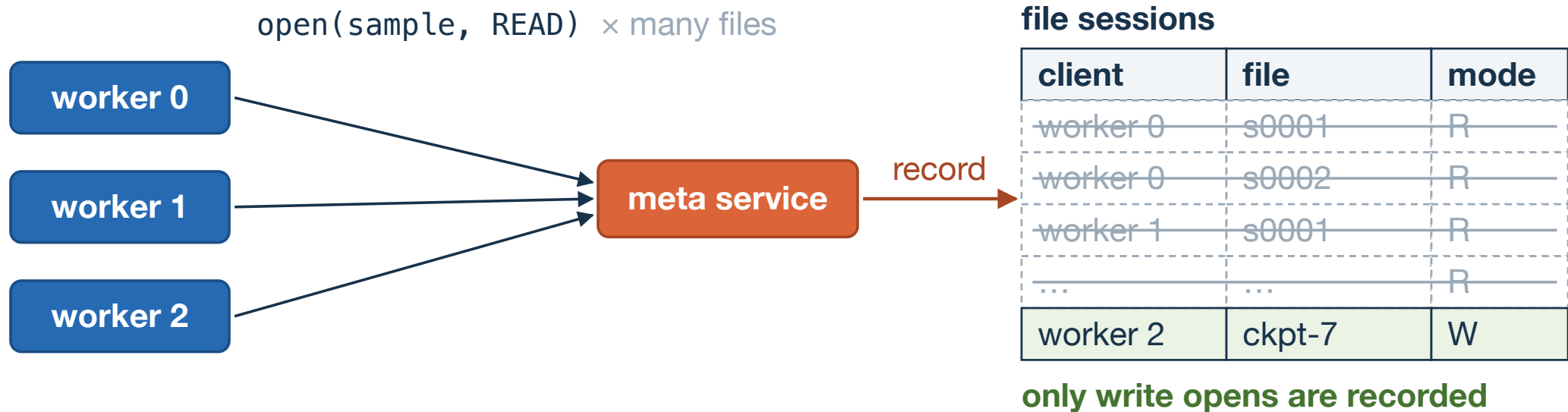
Why not track opened files?

A training job **opens a large number of files** at startup; recording every open is costly



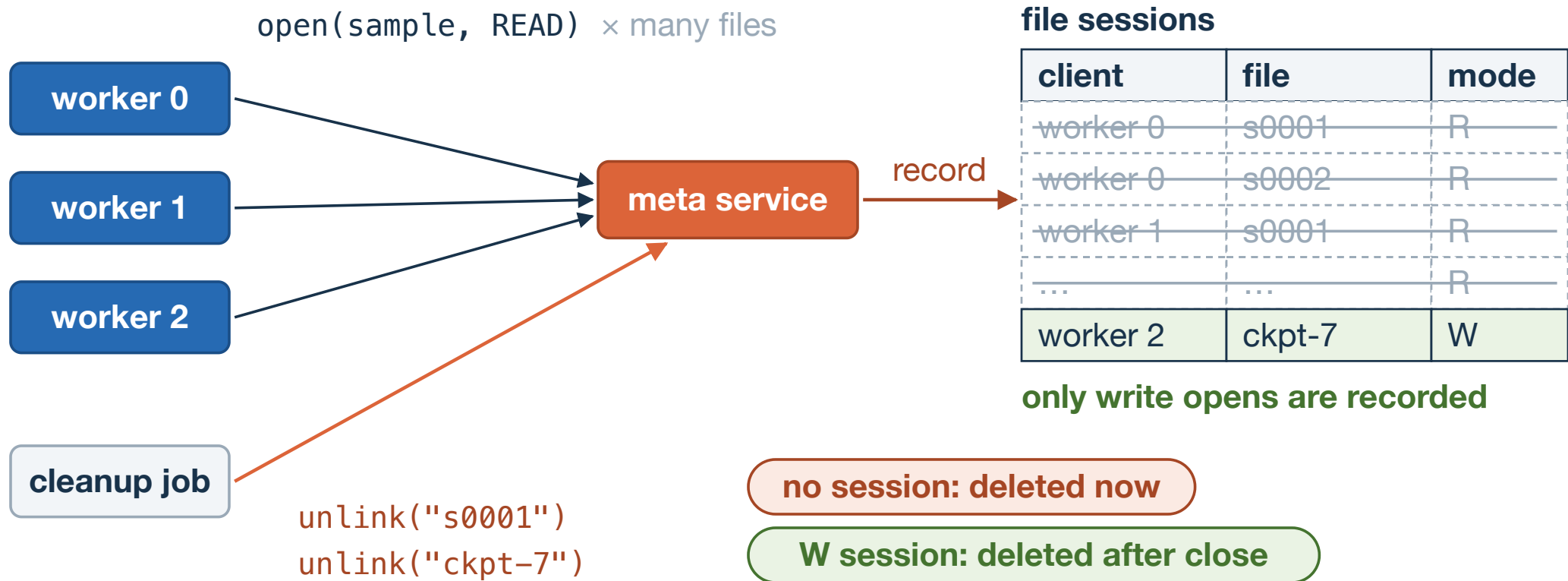
Why not track opened files?

A training job **opens a large number of files** at startup; recording every open is costly



Why not track opened files?

A training job **opens a large number of files** at startup; recording every open is costly



Same choice as NFS (recall: delete after open); NFS: simple crash recovery, 3FS: **cheap opens**

How does a 3FS client find a chunk?

The 3FS design notes: chunk ID = **inode id + chunk index**, so the client **computes** chunk locations itself

3FS / docs / design_notes.md 

↑ Top

Preview Code Blame 290 lines (171 loc) · 31.1 KB

Raw     

Location of file chunks

3FS divides file data into equally sized chunks and stripes them across multiple replication chains (replication chains and chain tables are defined in Section [Data placement](#)). Users can specify the chain table, chunk size, and stripe size for files on a per-directory basis. Each chunk is independently stored on multiple storage services, with its chunk ID generated by concatenating the file's inode id and chunk index.

When creating a new file, the metadata service employs a round-robin strategy to select consecutive replication chains from the designated chain table, based on the stripe size. Next, a random seed is generated to shuffle the selected chains. This allocation strategy ensures balanced data distribution across chains and SSDs.

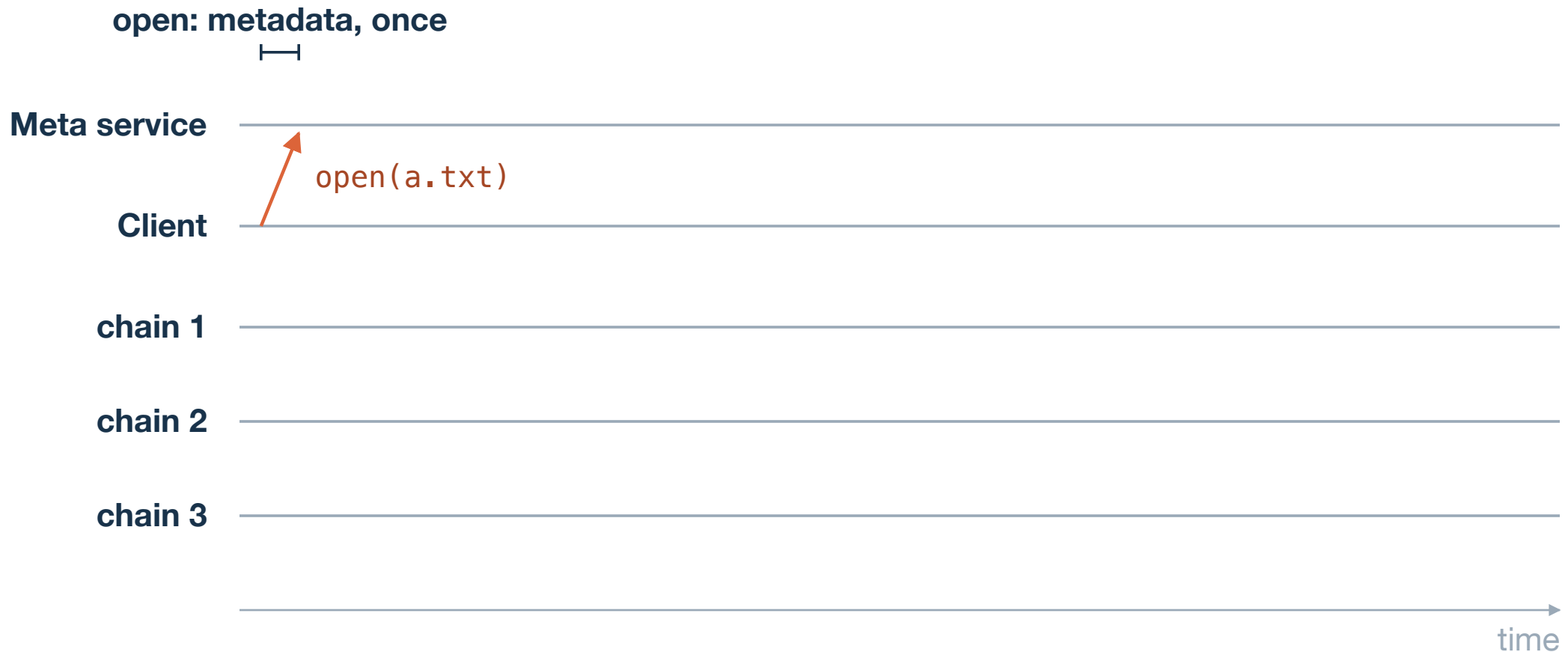
When an application opens a file, the client contacts the meta service to obtain the file's data layout information. Then the client can independently compute chunk IDs and chains for data operations, minimizing the involvement of the meta service in the critical path.

¹ Source: https://github.com/deepseek-ai/3FS/blob/main/docs/design_notes.md

Why compute chunk locations?

GFS keeps master lookups rare with 64 MB chunks; random reads touch many chunks

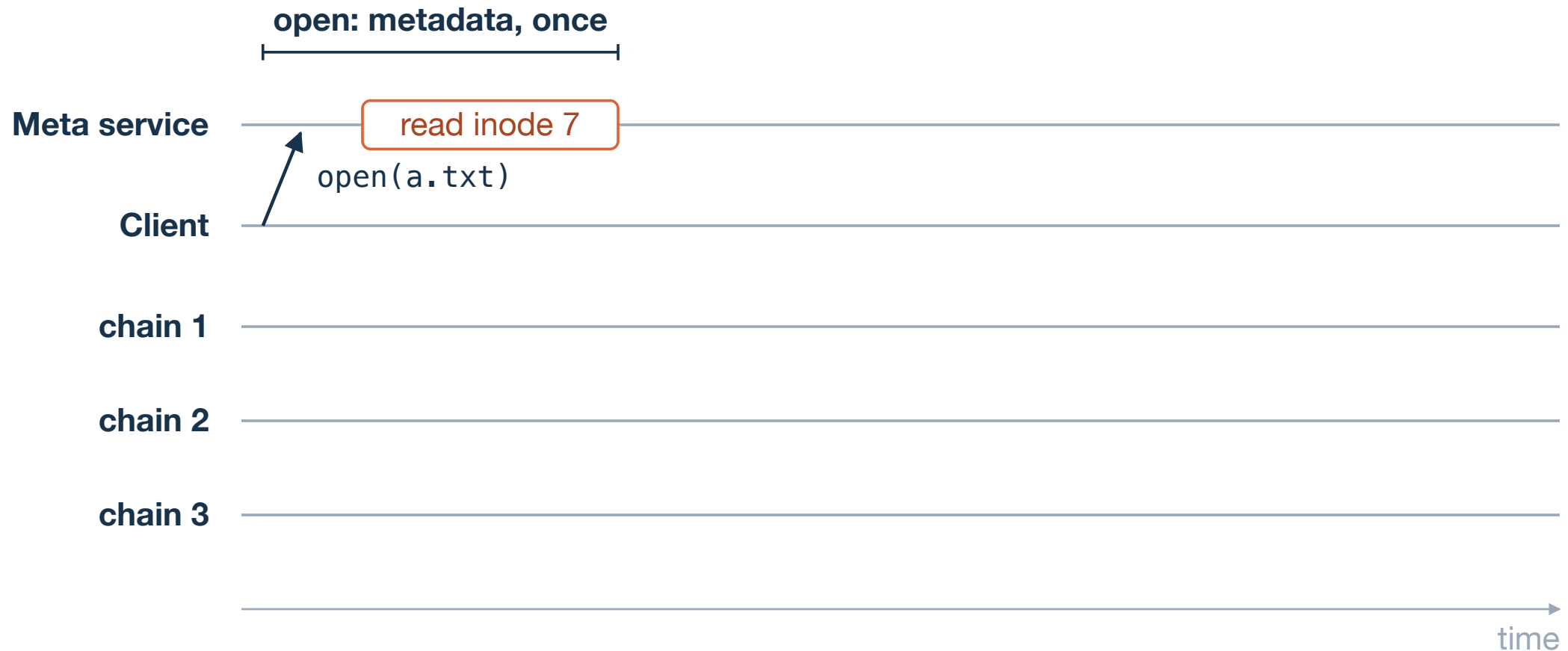
3FS: a formula, not a table: `chunk id = (inode, index)`, `chain = chains[index % stripe]`



Why compute chunk locations?

GFS keeps master lookups rare with 64 MB chunks; random reads touch many chunks

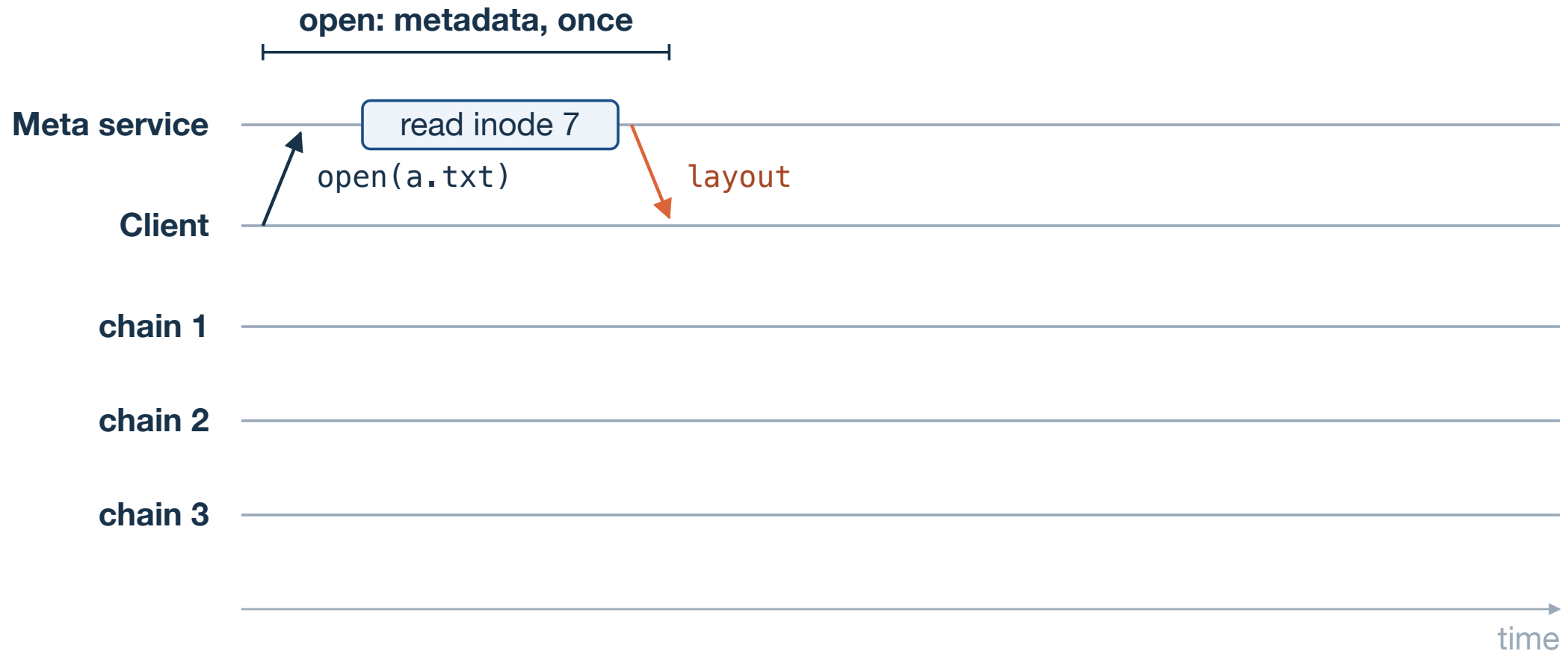
3FS: a formula, not a table: `chunk id = (inode, index)`, `chain = chains[index % stripe]`



Why compute chunk locations?

GFS keeps master lookups rare with 64 MB chunks; random reads touch many chunks

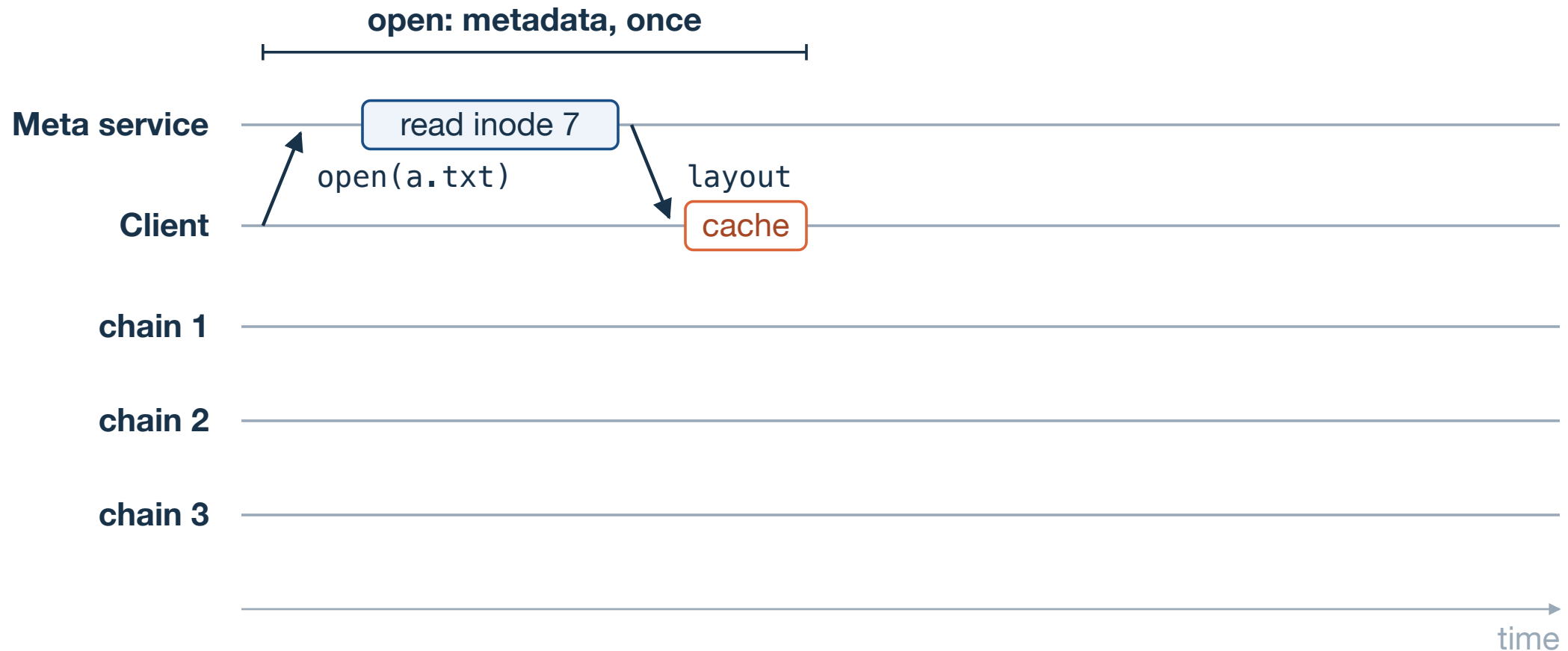
3FS: a formula, not a table: `chunk id = (inode, index)`, `chain = chains[index % stripe]`



Why compute chunk locations?

GFS keeps master lookups rare with 64 MB chunks; random reads touch many chunks

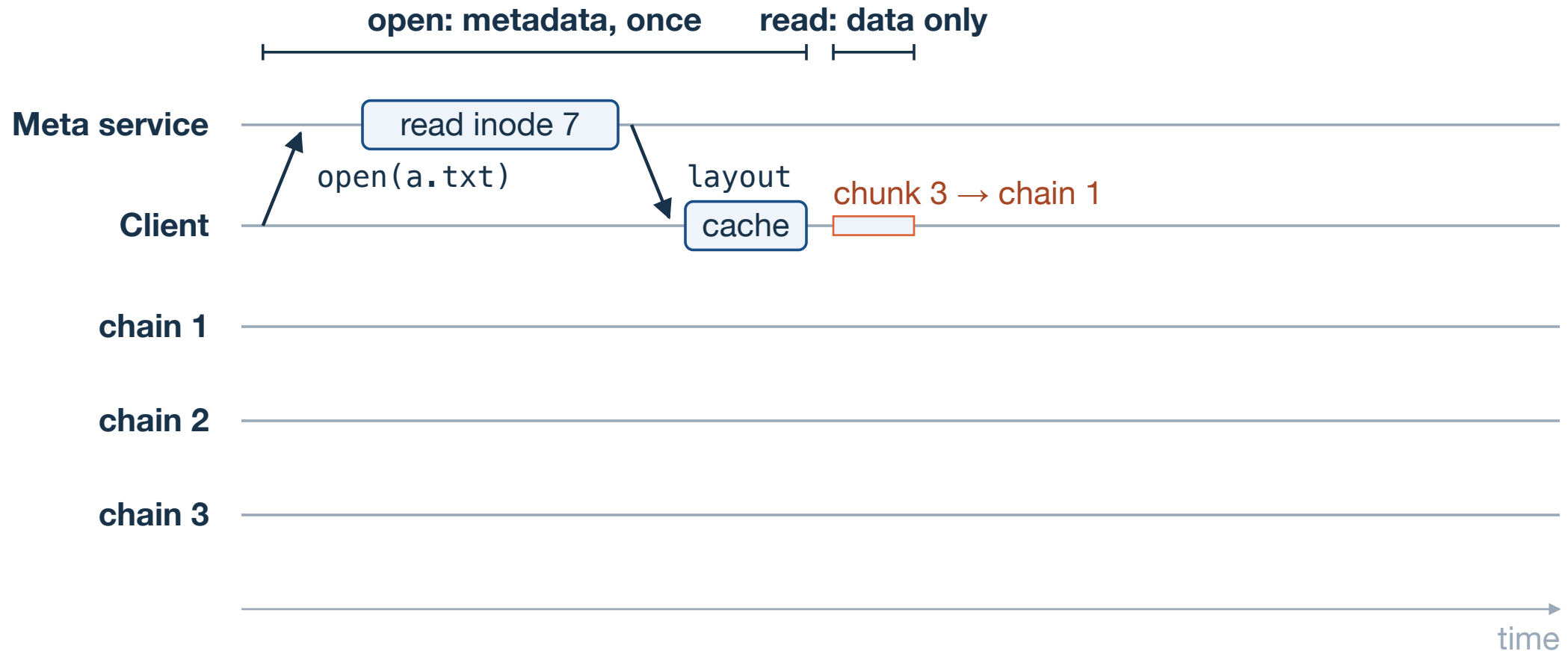
3FS: a formula, not a table: `chunk id = (inode, index)`, `chain = chains[index % stripe]`



Why compute chunk locations?

GFS keeps master lookups rare with 64 MB chunks; random reads touch many chunks

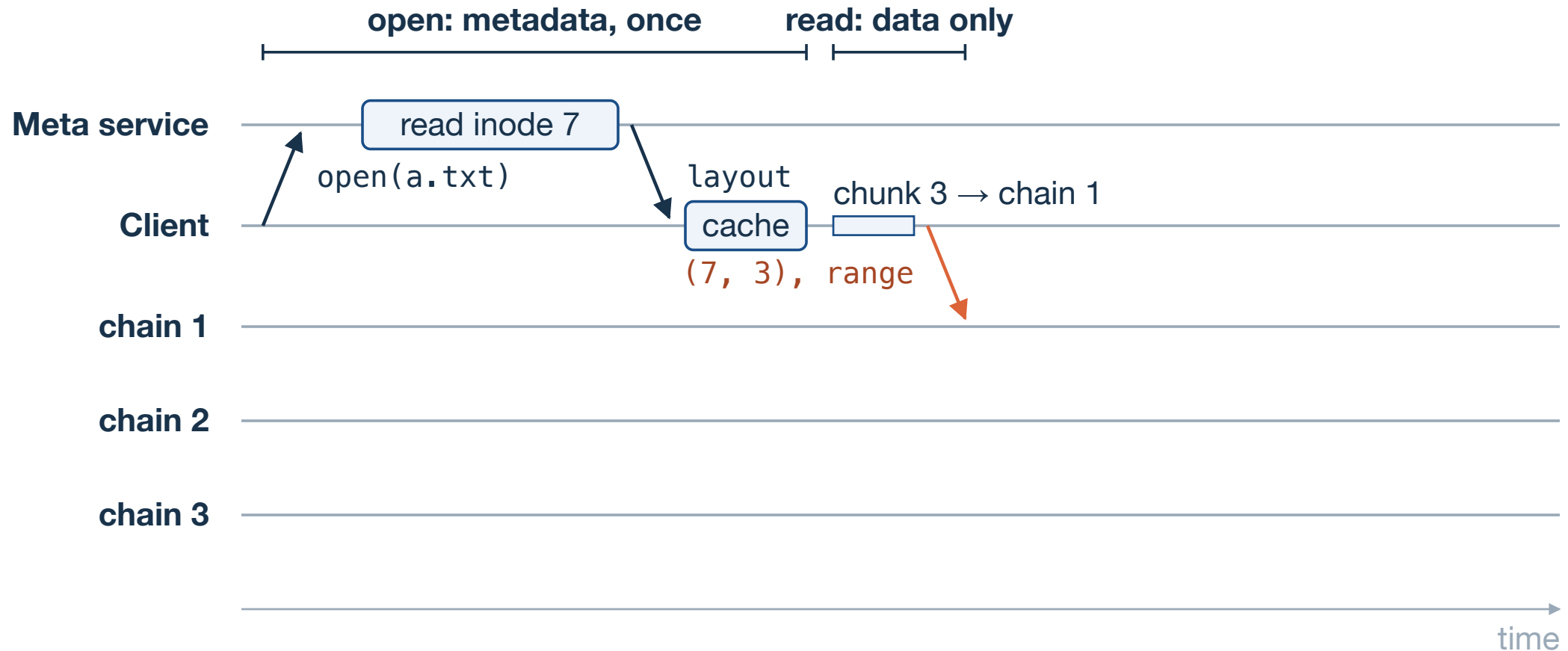
3FS: a formula, not a table: $\text{chunk id} = (\text{inode}, \text{index})$, $\text{chain} = \text{chains}[\text{index} \% \text{stripe}]$



Why compute chunk locations?

GFS keeps master lookups rare with **64 MB chunks**; random reads touch many chunks

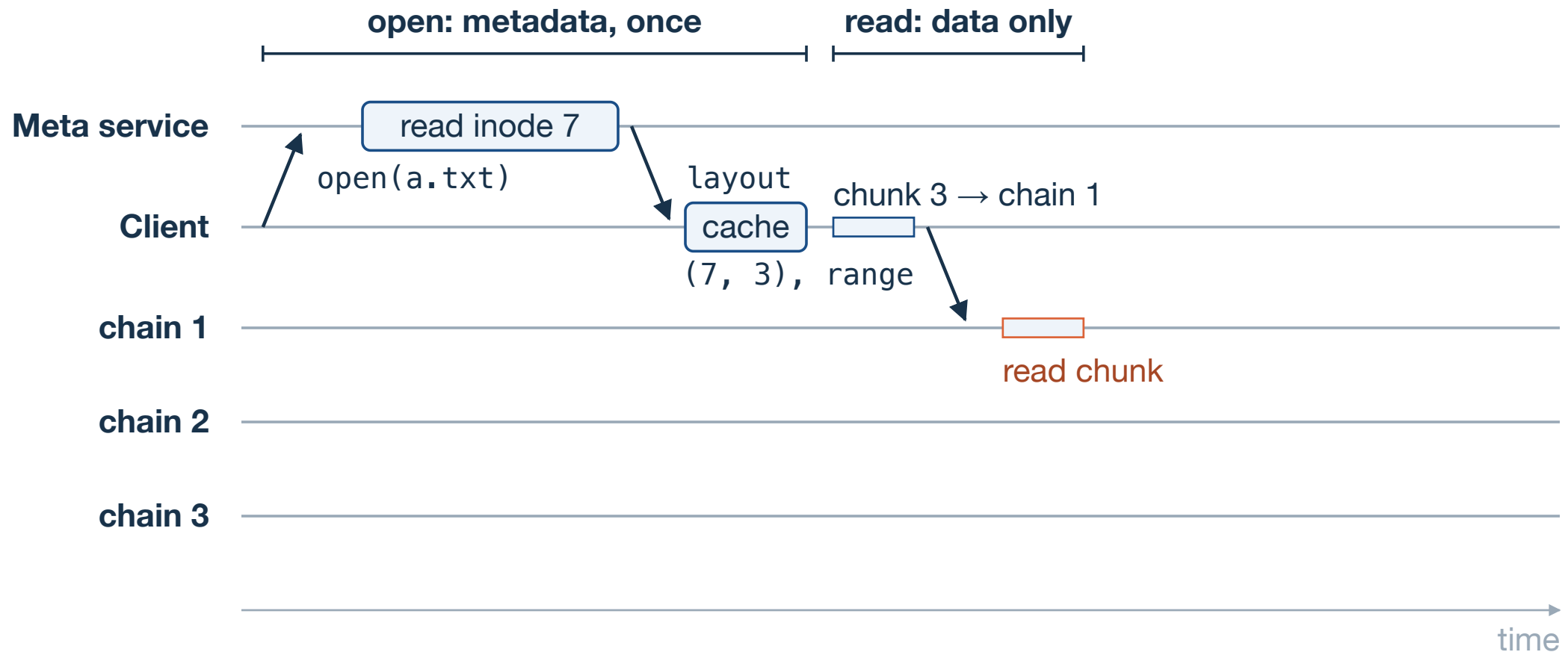
3FS: a **formula**, not a table: `chunk id = (inode, index)`, `chain = chains[index % stripe]`



Why compute chunk locations?

GFS keeps master lookups rare with **64 MB chunks**; random reads touch many chunks

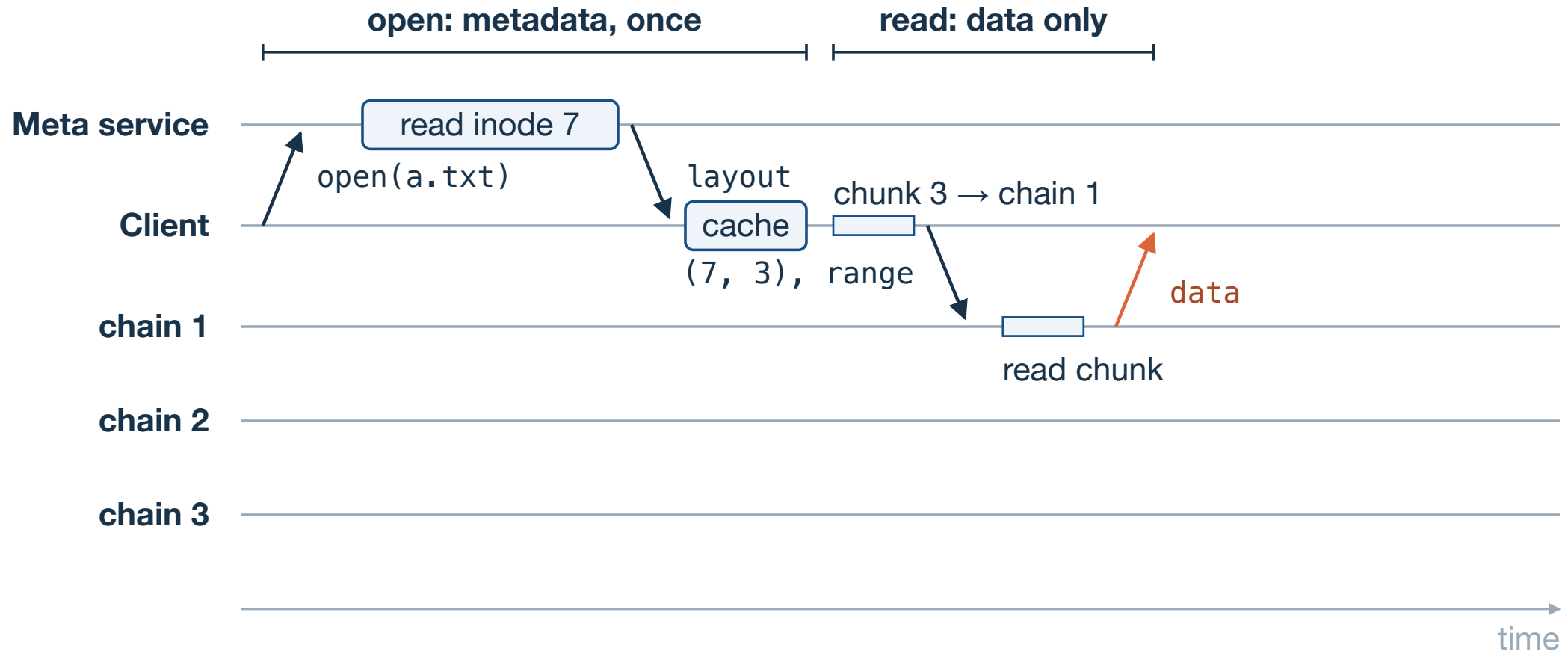
3FS: a **formula**, not a table: `chunk id = (inode, index)`, `chain = chains[index % stripe]`



Why compute chunk locations?

GFS keeps master lookups rare with **64 MB chunks**; random reads touch many chunks

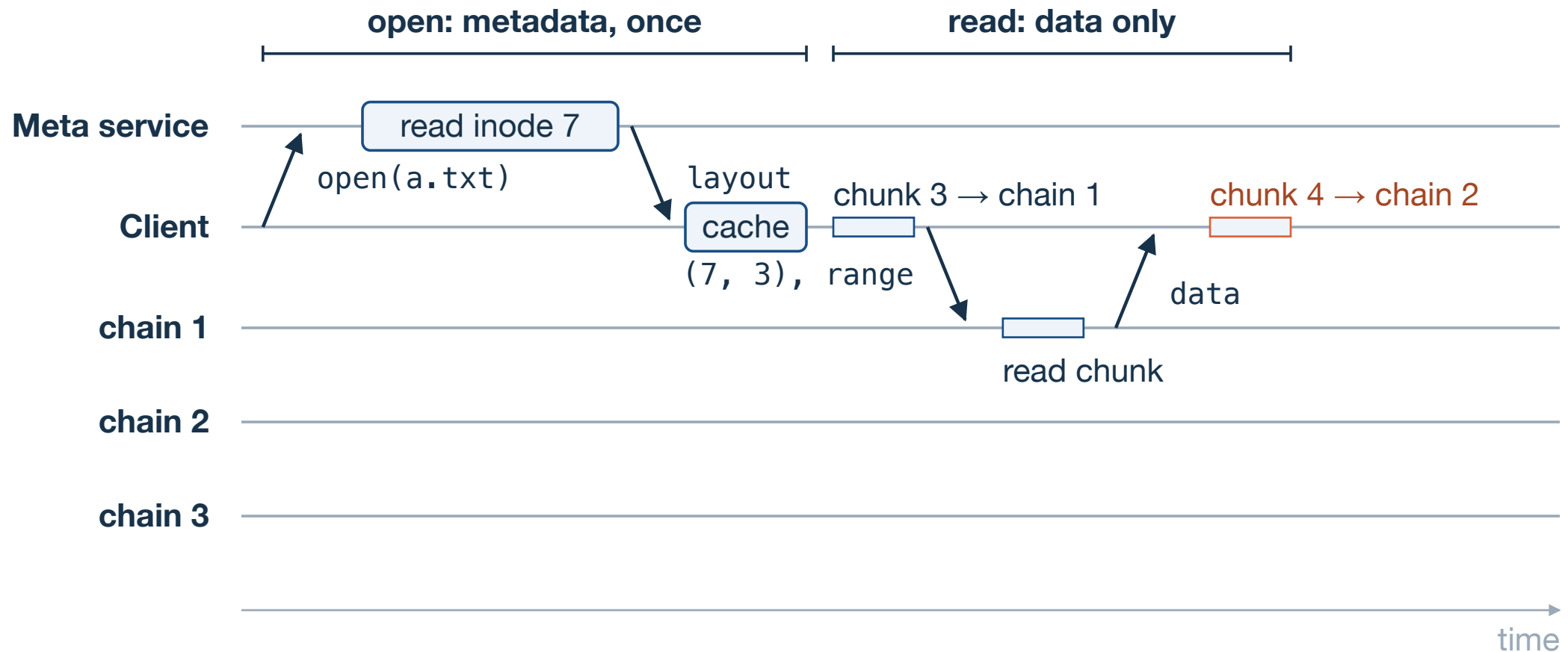
3FS: a **formula**, not a table: `chunk id = (inode, index)`, `chain = chains[index % stripe]`



Why compute chunk locations?

GFS keeps master lookups rare with **64 MB chunks**; random reads touch many chunks

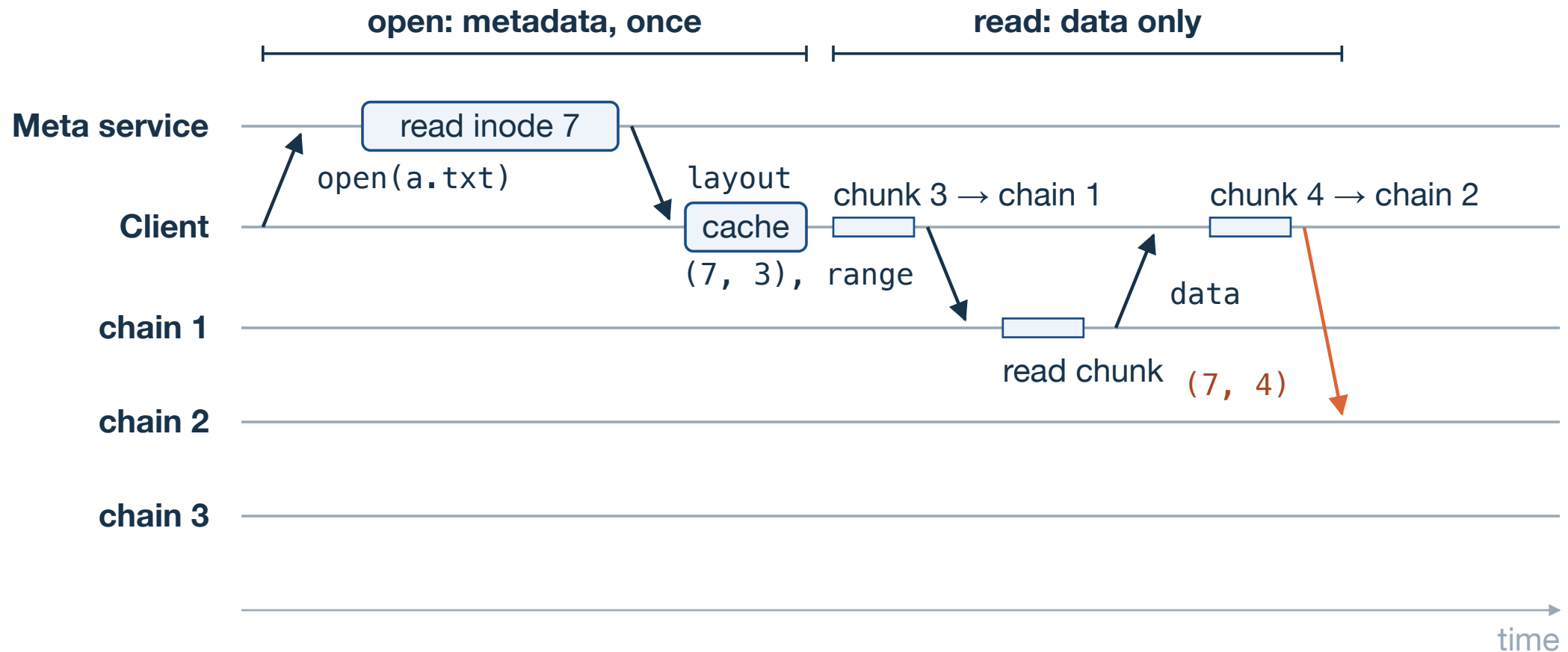
3FS: a **formula**, not a table: `chunk id = (inode, index)`, `chain = chains[index % stripe]`



Why compute chunk locations?

GFS keeps master lookups rare with **64 MB chunks**; random reads touch many chunks

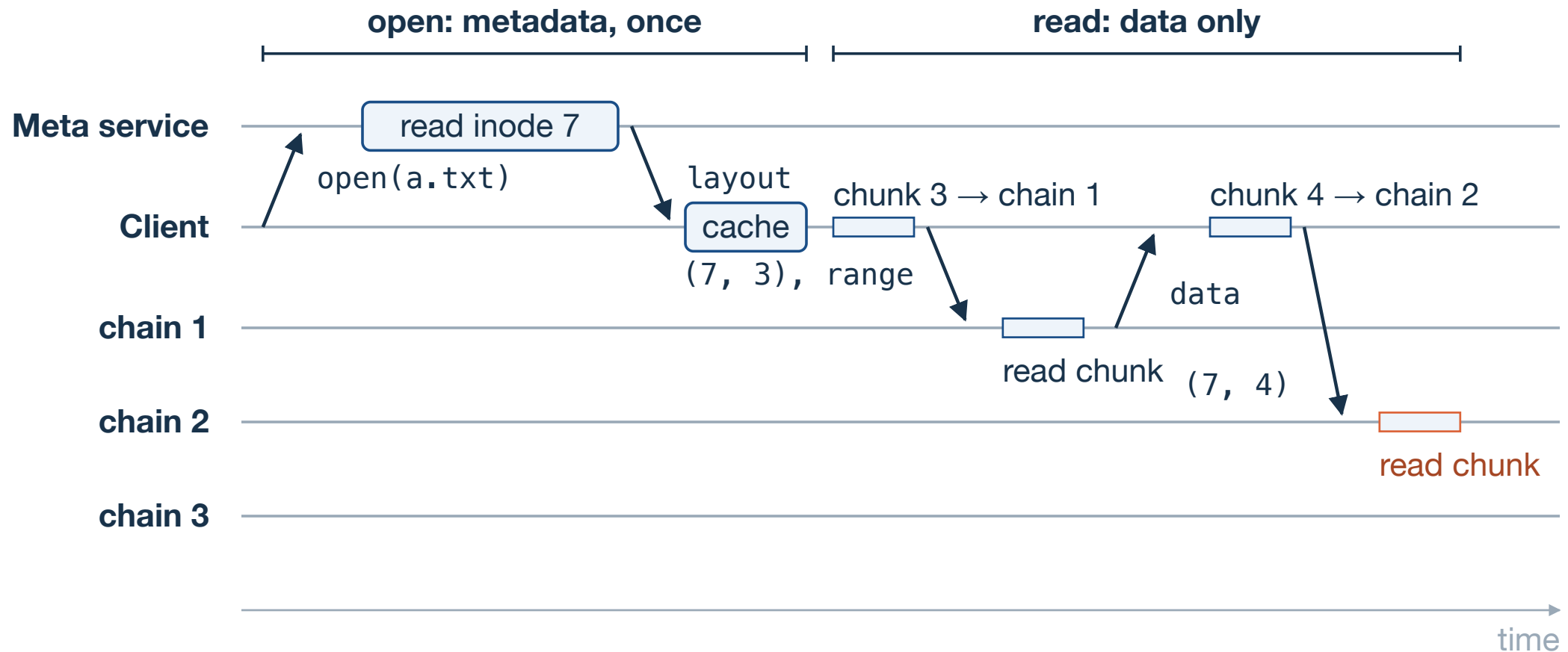
3FS: a **formula**, not a table: `chunk id = (inode, index)`, `chain = chains[index % stripe]`



Why compute chunk locations?

GFS keeps master lookups rare with **64 MB chunks**; random reads touch many chunks

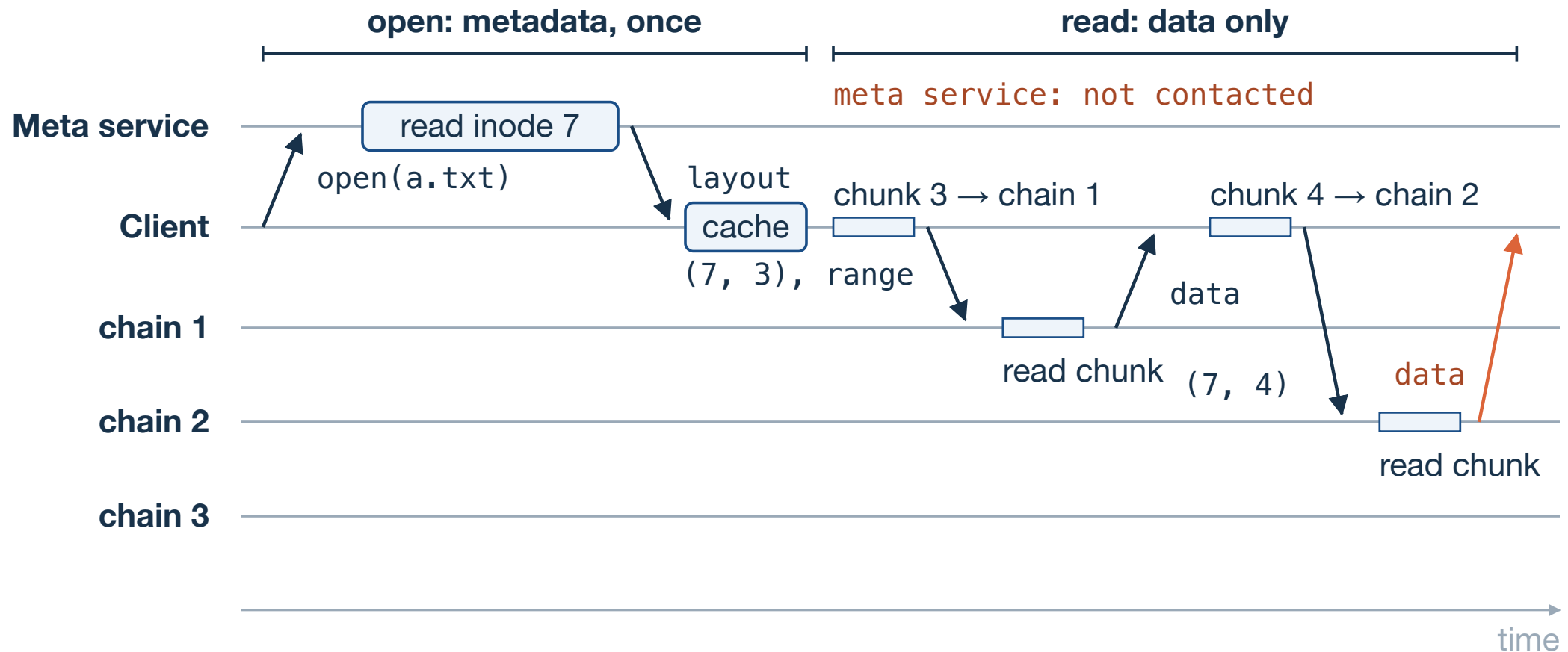
3FS: a **formula**, not a table: `chunk id = (inode, index)`, `chain = chains[index % stripe]`



Why compute chunk locations?

GFS keeps master lookups rare with **64 MB chunks**; random reads touch many chunks

3FS: a **formula**, not a table: `chunk id = (inode, index)`, `chain = chains[index % stripe]`



What does locality-oblivious mean?

The 3FS README: a **disaggregated** architecture, accessed in a **locality-oblivious** manner

3FS / README.md

↑ Top

Preview

Code

Blame

129 lines (88 loc) · 7.48 KB

Raw

The Fire-Flyer File System (3FS) is a high-performance distributed file system designed to address the challenges of AI training and inference workloads. It leverages modern SSDs and RDMA networks to provide a shared storage layer that simplifies development of distributed applications. Key features and benefits of 3FS include:

- Performance and Usability
 - **Disaggregated Architecture** Combines the throughput of thousands of SSDs and the network bandwidth of hundreds of storage nodes, enabling applications to access storage resource in a locality-oblivious manner.

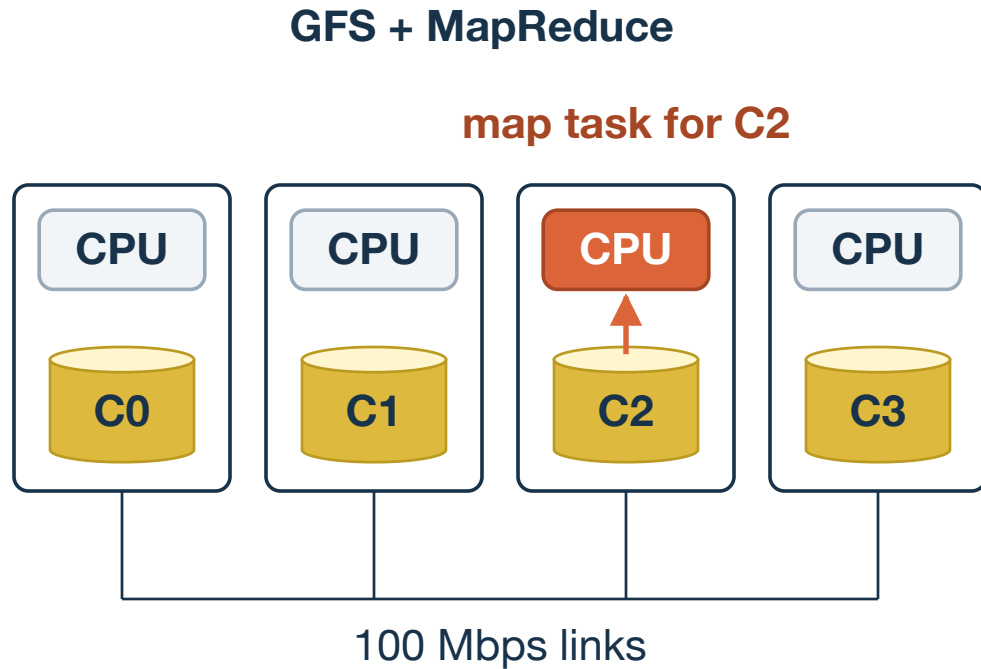
Disaggregated: GPU nodes and storage nodes are **separate machines**

Locality-oblivious: a client **ignores where data lives**; no task is placed near its data

¹ Source: <https://github.com/deepseek-ai/3FS>

Why locality-oblivious?

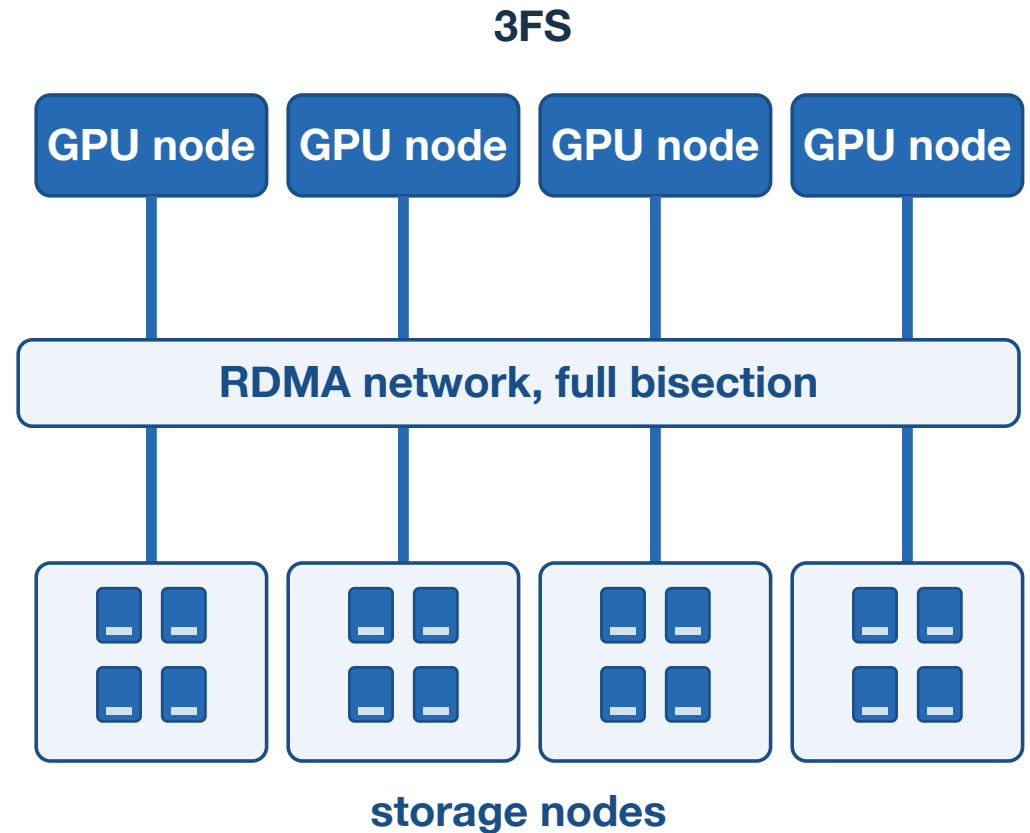
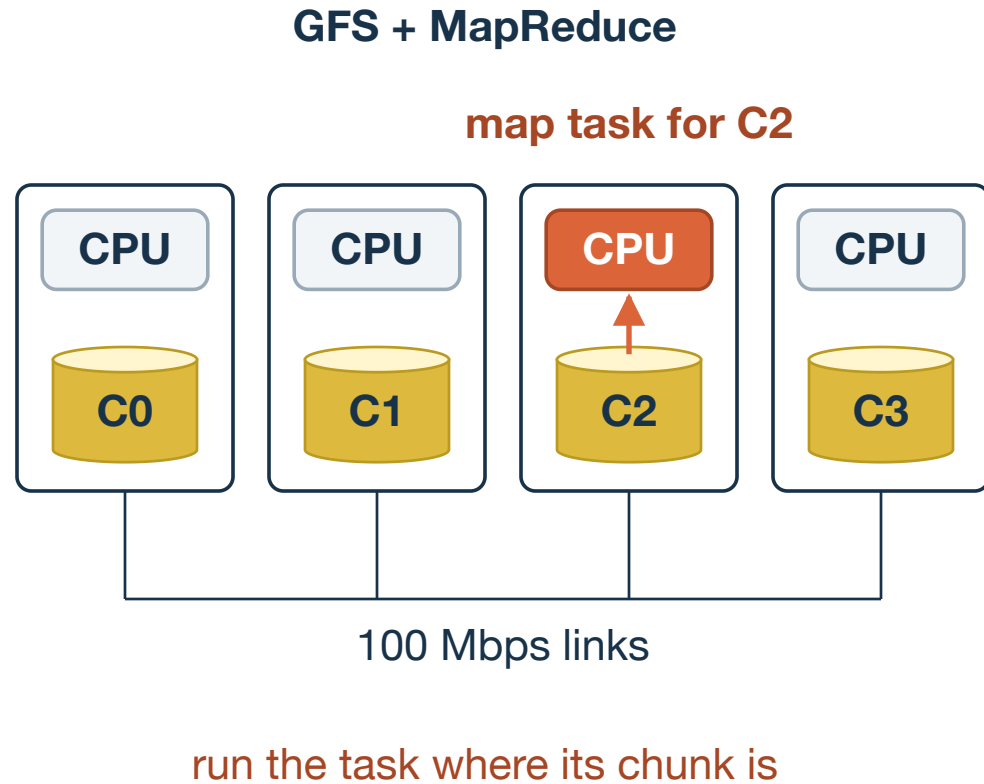
MapReduce: “network bandwidth is a relatively scarce resource” → run a task where its data is



run the task where its chunk is

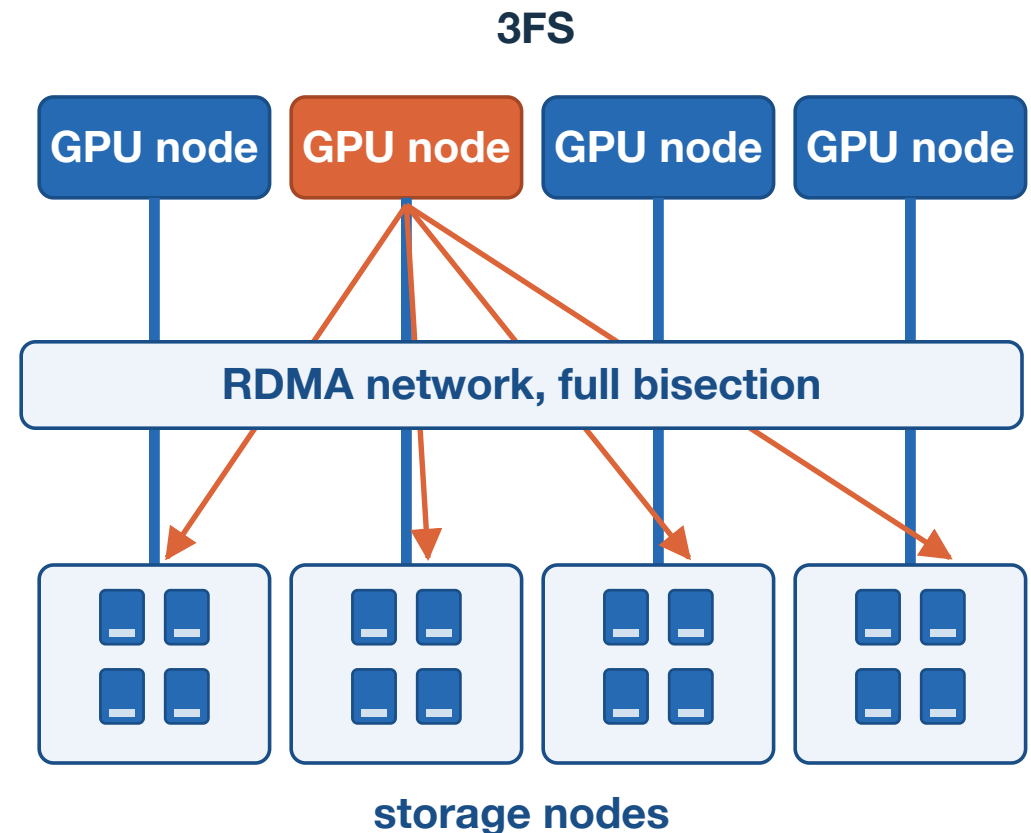
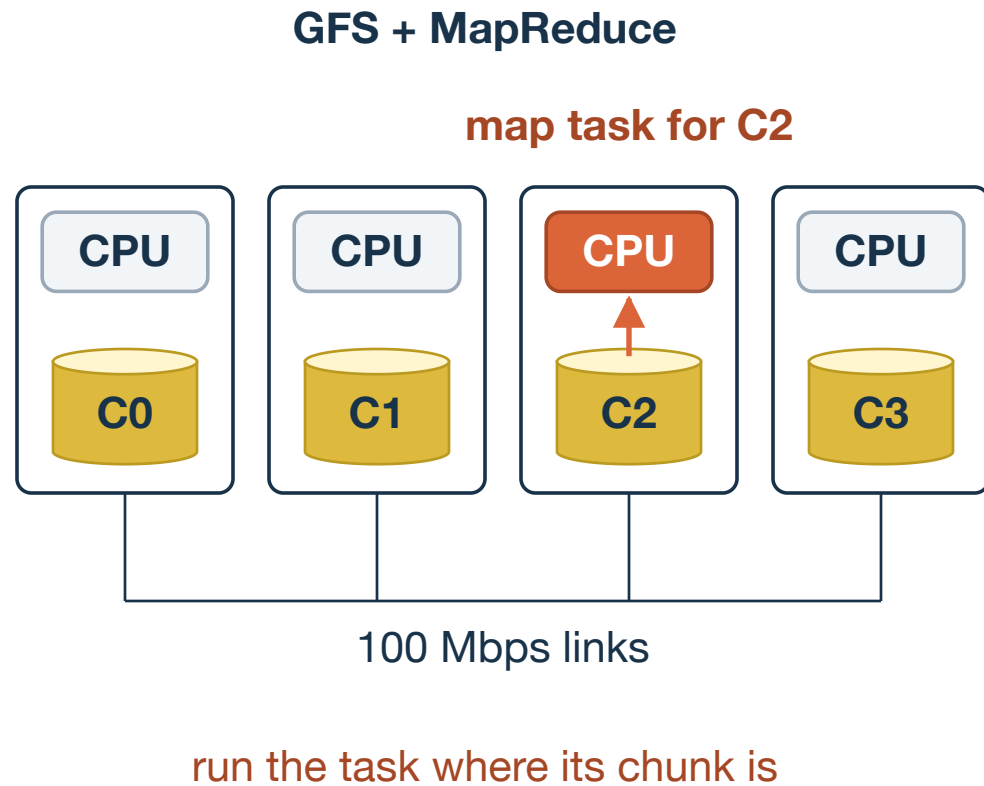
Why locality-oblivious?

MapReduce: “network bandwidth is a relatively **scarce** resource” → run a task **where its data is**



Why locality-oblivious?

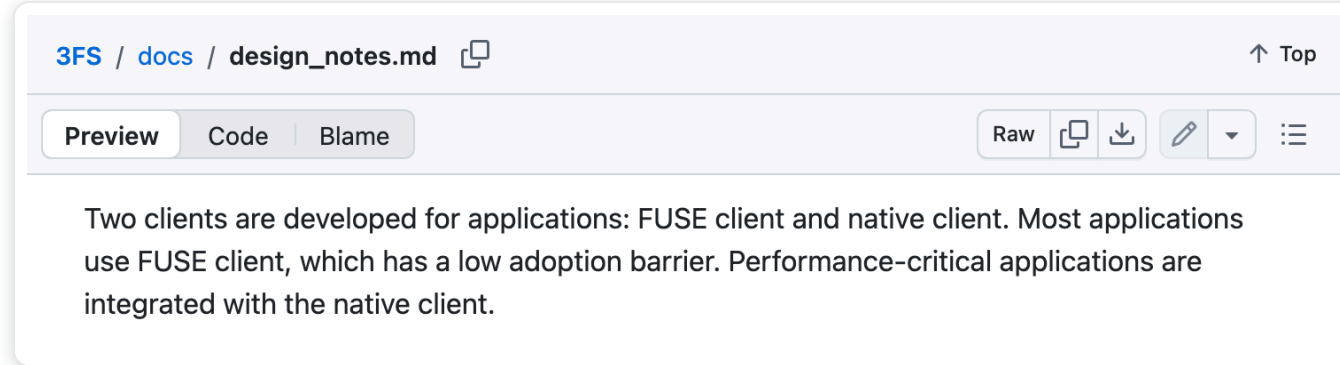
MapReduce: “network bandwidth is a relatively **scarce** resource” → run a task **where its data is**



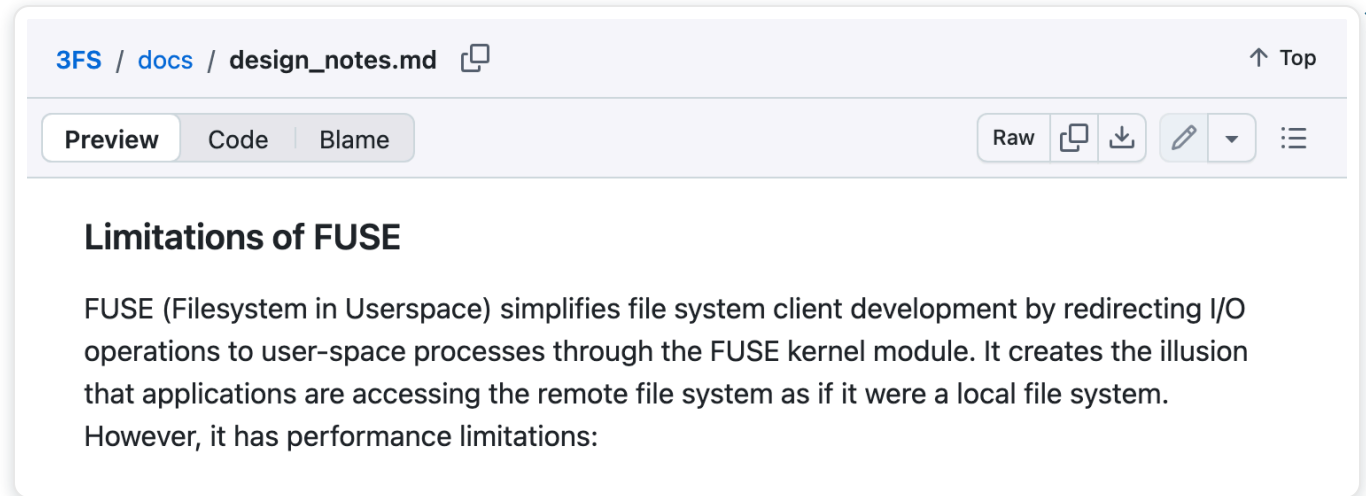
3FS: any client reads **any SSD at full speed** → GPUs and storage **scale separately**

How do applications access 3FS?

The 3FS design notes describe **two clients**



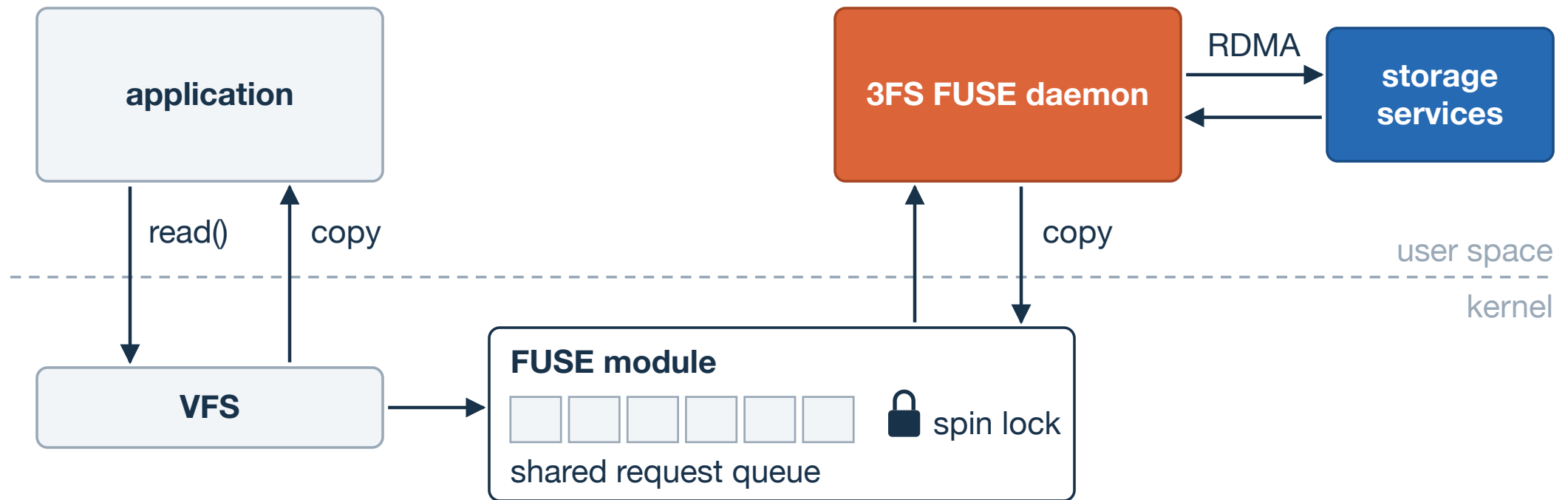
and explain why the **FUSE client alone is not enough**



¹ Source: https://github.com/deepseek-ai/3FS/blob/main/docs/design_notes.md

What is FUSE?

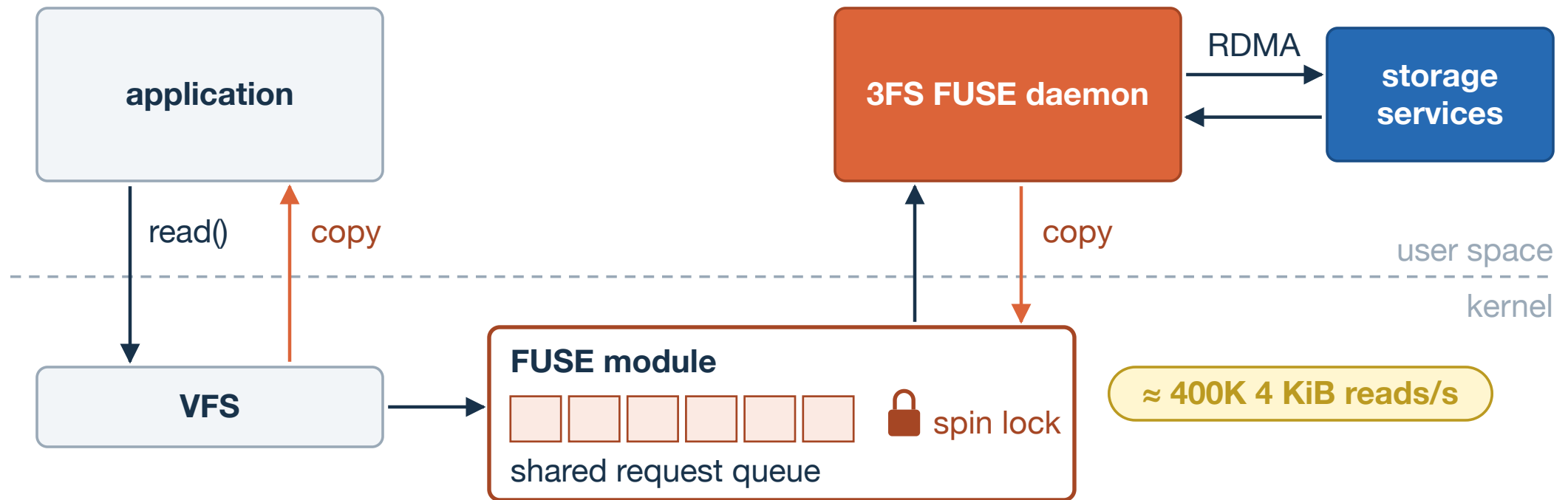
FUSE (Filesystem in Userspace): the kernel forwards file system calls to a **user-space daemon**, which talks to the storage services



No new API to learn, and no kernel module whose bugs crash the machine

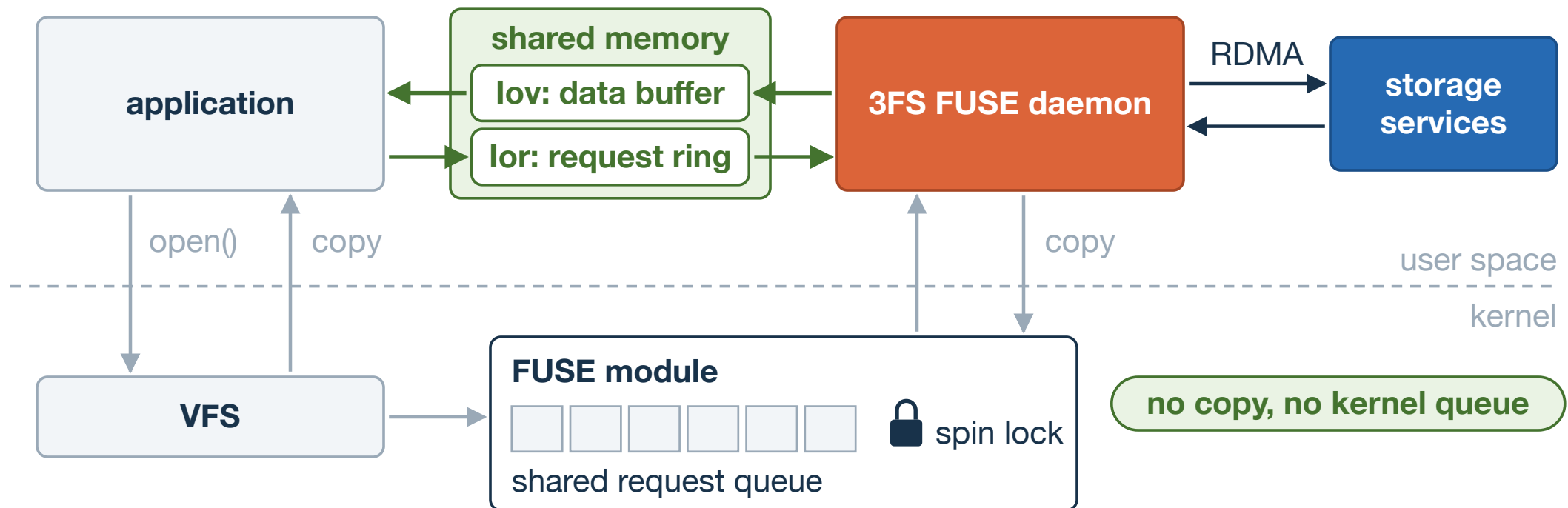
Why is FUSE not enough?

With fast SSDs and RDMA, the FUSE **software path** becomes the bottleneck: every request is **copied** across the kernel boundary and waits on one **spin lock**



Why is FUSE not enough?

With fast SSDs and RDMA, the FUSE **software path** becomes the bottleneck: every request is **copied** across the kernel boundary and waits on one **spin lock**



Native API: `open` via FUSE, then I/O through a **shared-memory ring** (modeled after `io_uring`)¹

¹ [3FS design notes](#): "The user-space file system daemon cannot access application memory. Data transfer between kernel and user spaces consumes memory bandwidth and increases end-to-end latency"; "FUSE places these requests into a multi-threaded shared queue, protected by a spin lock ... FUSE only handles approximately 400K 4KiB reads per second"; "The asynchronous, zero-copy API is inspired by Linux `io_uring`"

NFS vs. GFS vs. 3FS

	NFS	GFS	3FS
Workload	share files in a LAN	huge files, sequential reads, appends	many files, random reads, burst writes
Hardware	one server	disks, 100 Mbps Ethernet	NVMe SSDs, RDMA (fast networking)
Interface	transparent (<code>mount</code>)	own library, no <code>rename</code>	transparent (FUSE) + native API
Metadata	on the server	one master, in memory	stateless services + transactional KV store
Chunk location	n/a	ask the master, then cache	compute from the file layout

Different workloads and hardware lead to different designs

Summary

Designing a distributed file system (DFS) is not simply

- Single-node file system + RPC

Many design choices for performance, consistency model & failure handling

Three case studies

- **NFS**: transparent access to files on a remote server
- **GFS**: highly scalable & fault-tolerant DFS optimized for Google's workload
- **3FS**: DFS for AI workloads on new hardware

References

- [1] [Principles of Computer System Design: An Introduction](#) — Saltzer & Kaashoek, 2009. Morgan Kaufmann (P6, P8, P14, P16, P18, P20, P22)
- [2] [Conversation state](#) — OpenAI, 2026. OpenAI API docs (P13)
- [3] [The Google File System](#) — Ghemawat, Gobioff & Leung, 2003. SOSP (P27, P33, P35, P43, P50)
- [4] [Fire-Flyer File System \(3FS\)](#) — DeepSeek, 2025 (P49, P59, P60)
- [5] [Fire-Flyer AI-HPC: A Cost-Effective Software-Hardware Co-Design for Deep Learning](#) — An et al., 2024. SC (P49, P50, P52)
- [6] [3FS design notes](#) — DeepSeek, 2025 (P50, P51, P52, P53, P54, P55, P56, P57, P58, P61, P62, P63)
- [7] [GFS: Evolution on Fast-forward](#) — McKusick & Quinlan, 2009. ACM Queue (P54)
- [8] [MapReduce: Simplified Data Processing on Large Clusters](#) — Dean & Ghemawat, 2004. OSDI (P60)